Designing Data-Intensive Applications — Vol 1 (Preface, Fundamentals primer, Ch 1–6)

Martin Kleppmann: beginner-friendly study notes

8 parts · every chapter explained in plain English, with redrawn figures

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes written from the book. Figure numbers refer to the book. Not for redistribution.

Contents

1. **Preface — How to Read This Book**
    - What this book is about
    - What kind of book it is (and is not)
    - Who should read it
    - Scope: what is covered and what is deliberately left out
    - Map of the book
    - How to use these notes

2. **Chapter 0a — System Design Fundamentals**
    - Module Introduction
    - Part 1 — Foundations and performance
    - What system design is
    - Clients, servers and the request/response cycle
    - The building blocks of a web application
    - Latency, throughput and bandwidth
    - Percentiles, tail latency and why averages lie
    - Little's law
    - Latency numbers every engineer should know
    - Units, powers of two and data sizes
    - Back-of-the-envelope estimation
    - Availability and the nines
    - SLA, SLO, SLI and error budgets
    - MTBF, MTTR and measuring failure
    - Reliability, availability and durability
    - Faults, failures and fault tolerance
    - Redundancy, single points of failure and failover
    - Scalability
    - Vertical and horizontal scaling
    - Elasticity
    - Stateless and stateful services
    - Performance vs scalability
    - Concurrency vs parallelism
    - Bottlenecks: CPU, memory, disk and network
    - How to approach a system design interview
    - Part 1 glossary
    - Part 2 — Networking and communication
    - IP addresses, NAT, ports and sockets
    - DNS — the internet's phone book
    - Network layers — OSI and TCP/IP models
    - TCP, UDP and QUIC
    - HTTP — the language of the web
    - HTTPS and TLS
    - API styles — REST, RPC and gRPC, GraphQL
    - Real-time communication — polling, long polling, WebSockets, SSE, webhooks
    - API design basics
    - Proxies and API gateways
    - Load balancers
    - CDNs and anycast
    - Bandwidth vs latency, and network partitions
    - Part 2 glossary
    - Part 3 — Data storage
    - What a database is
    - Relational databases and SQL
    - Transactions and ACID
    - Isolation levels and anomalies
    - NoSQL database families
    - SQL vs NoSQL and BASE
    - Indexes
    - Normalization and denormalization
    - Replication
    - Partitioning and sharding
    - Consistent hashing and virtual nodes
    - Storage types: block, file and object
    - OLTP vs OLAP, row vs column storage, lakes and warehouses
    - Durability: WAL, fsync, backups and recovery
    - Connection pooling, ORMs and N+1 queries
    - Choosing a database
    - Part 3 glossary
    - Part 4 — Caching, async processing and consistency
    - Caching basics
    - Where caches live
    - Caching strategies
    - Eviction policies
    - Cache invalidation, stale data and stampedes
    - Message queues
    - Event streaming: topics, partitions, consumer groups and offsets
    - Delivery guarantees, idempotency and retries
    - Batch vs stream processing
    - Event-driven architecture, event sourcing and CQRS
    - Consistency models
    - CAP theorem and PACELC
    - Quorums
    - Consensus, leader election and distributed locks
    - Clocks and ordering
    - Distributed transactions: two-phase commit and sagas
    - Heartbeats, gossip and failure detection
    - Probabilistic data structures and Merkle trees
    - Unique ID generation
    - Geospatial indexing: geohash and quadtree
    - Part 4 glossary
    - Part 5 — Architecture, reliability, operations and security
    - Architectural styles: monolith, modular monolith, SOA, microservices and serverless
    - Synchronous vs asynchronous communication
    - Service discovery, sidecars and service mesh
    - Timeouts, retries, exponential backoff and jitter
    - Circuit breakers, bulkheads and fallbacks
    - Rate limiting and throttling
    - Load shedding and graceful degradation
    - Autoscaling
    - Virtual machines, containers and Docker
    - Kubernetes basics

- Infrastructure as code
- CI/CD
- Deployment strategies and rollback
- Observability: logs, metrics and traces
- Dashboards, alerting, on-call and SLO-based alerts
- Disaster recovery
- Authentication and authorization
- Sessions, cookies, tokens and JWT
- OAuth 2.0, OpenID Connect, SSO and API keys
- Encryption, hashing, passwords and secrets
- Network security: firewalls, WAF and DDoS
- Common web vulnerabilities: CORS, CSRF, XSS and SQL injection
- Least privilege and zero trust
- Cost awareness
- Part 5 glossary
- Part 6 — Putting it together
- A request's journey
- Worked example: designing a URL shortener
- Decision cheat sheet
- Top 60 interview questions
- Master A–Z glossary
- Part 6 glossary

3. **Chapter 1 — Reliable, Scalable, and Maintainable Applications**
   - Overview
   - Key terms
   - Thinking About Data Systems
   - Reliability
   - Scalability
   - Maintainability
   - Summary
   - Interview angle
   - Self-check
4. **Chapter 2 — Data Models and Query Languages**
   - Overview
   - Key terms
   - Relational Model Versus Document Model
   - Query Languages for Data
   - Graph-Like Data Models
   - Summary
   - Interview angle
   - Self-check
5. **Chapter 3 — Storage and Retrieval**
   - Overview
   - Key terms
   - Data Structures That Power Your Database
   - Transaction Processing or Analytics?
   - Column-Oriented Storage
   - Summary
   - Interview angle
   - Self-check
6. **Chapter 4 — Encoding and Evolution**
   - Overview
   - Key terms
   - Formats for Encoding Data
   - Modes of Dataflow
   - Summary
   - Looking ahead
   - Interview angle
   - Self-check
7. **Chapter 5 — Replication**
   - Overview
   - Key terms
   - Leaders and Followers
   - Problems with Replication Lag
   - Multi-Leader Replication
   - Leaderless Replication
   - Summary
   - Interview angle
   - Self-check
8. **Chapter 6 — Partitioning**
   - Overview
   - Key terms
   - Partitioning and Replication
   - Partitioning of Key-Value Data
   - Partitioning and Secondary Indexes
   - Rebalancing Partitions
   - Request Routing
   - Summary
   - Interview angle
   - Self-check

Preface — How to Read This Book

Book: Preface, pp. xiii–xix

What this book is about

Backend engineering is full of buzzwords: NoSQL, Big Data, web-scale, sharding, ACID, eventual consistency, CAP theorem, MapReduce, cloud services, real-time. The preface promises to show the lasting ideas underneath them.

> **Key term — the buzzwords, in one line each: NoSQL** = databases that do not use the classic table-and-SQL model (document, key-value, graph stores). **Big Data** = a loose label for datasets too large for one ordinary machine. **Web-scale** = able to serve internet-sized traffic. **Sharding** = splitting one dataset into pieces on different machines. **ACID** = the four safety promises of a transaction (Atomicity, Consistency, Isolation, Durability; Chapter 7). **Eventual consistency** = copies may disagree briefly but match once updates stop. **CAP theorem** = the claim that, during a network failure, a distributed system must choose between consistency and availability. **MapReduce** = a model for processing huge datasets in parallel across many machines. **Cloud services** = computing rented over the internet. **Real-time** = processed within moments of arrival. All are explained in later chapters; see also System Design Fundamentals.

Forces that reshaped databases and distributed systems in the preceding decade:

- **Huge internet companies** (Google, Yahoo!, Amazon, Facebook, LinkedIn, Microsoft, Twitter) had to invent new tools to cope with their data and traffic.
- **Businesses want agility**: short cycles, cheap experiments, flexible data models.
- **Open source won** over commercial and home-grown software.
- **Hardware shifted**: single CPU cores (independent processing units) barely get faster, but machines have many cores and networks keep speeding up, so parallelism will only grow.
- **Distribution became cheap**: with infrastructure as a service (IaaS: renting VMs, storage and networks by the hour, e.g. Amazon Web Services), even a tiny team can run across many machines and regions.
- **Downtime is less acceptable**: users expect services to be highly available, so long outages (planned or not) are increasingly unacceptable.

> **Definition: data-intensive vs compute-intensive.** An application is *data-intensive* when its hardest problem is the data itself: its amount, complexity, or speed of change. It is *compute-intensive* when the bottleneck is raw CPU. This book is about the first kind.

Data-intensive applications combine several tools: databases (including NoSQL ones), message queues, caches, search indexes, and batch and stream processing frameworks.

> **Key term — the usual building blocks:** a **message queue** holds messages from one program until another is ready, so the two need not run at the same speed. A **cache** keeps a copy of hot data somewhere faster (usually memory). A **search index** lets you find records by words or fields they contain. **Batch processing** works through a large fixed pile of data in one go (e.g. a nightly report); **stream processing** handles data continuously as it arrives.

The central promise: tools change quickly, but the **principles and trade-offs** behind them change slowly. Knowing those lets you place any new product, use it well, and predict where it will hurt.

What kind of book it is (and is not)

- **Not a tutorial** for one product and **not a dry theory textbook**.
- It studies **real, successful production systems** under demanding scale, performance and reliability requirements, opening up their **internals and key algorithms** and asking *why* they are built that way.

Afterwards you should be able to pick the right technology and combine tools into a sound architecture. You will **not** be ready to write a storage engine (the part of a database that writes data to disk and reads it back) from scratch (seldom needed), but you will have a solid mental model of what your systems do under the hood, which helps design and debugging.

Who should read it

Engineers, architects, and technical managers building software with a server or backend that stores or processes data and talks over the internet (web apps, mobile apps, connected sensors), especially whoever chooses tools and designs architectures. Even if you cannot choose your tools, knowing their strengths and weak spots helps.

Assumed background:

You should know	Nice to have	Does not matter
Building web apps or network services; relational databases (tables of rows and columns) and SQL	Non-relational databases and other data tools; a feel for TCP (reliable byte delivery between machines) and HTTP (the web's request/response protocol)	Your programming language or framework

Especially valuable if you want to make a system scale (e.g. to millions of users), keep it highly available and robust, keep it maintainable over years, or just understand what happens inside large online services.

> **"You're not Google or Amazon, so forget scale and use a relational database."** Partly true: building for scale you do not need is premature optimization and can lock you into a rigid design. But choosing the right tool still matters, every technology has trade-offs, and relational databases are not the last word on data.

Scope: what is covered and what is deliberately left out

Covered: the architecture of data systems and how they fit into data-intensive applications; fundamental principles and trade-offs; the design choices of different products.

Deliberately left out:

- Installation or usage guides for specific packages or APIs (an API is the set of functions or requests software offers to other programs); their own documentation does that.
- Deployment, operations, security, and management: too important for footnotes, they deserve their own books.
- "Big Data" as an organising idea: too vague. The book instead contrasts **single-node vs distributed** systems (one machine vs many cooperating over a network; a *node* is one machine) and **online/interactive vs offline/batch** processing (answering a waiting user vs crunching data in the background).

Bias toward open source. The book favours free and open source software (FOSS) because you can read and run the code, and open platforms reduce vendor lock-in (being stuck with a supplier because switching costs too much). Proprietary systems (closed source, SaaS: software the provider runs and you use over the internet, or in-house systems known only from papers) are still discussed where relevant.

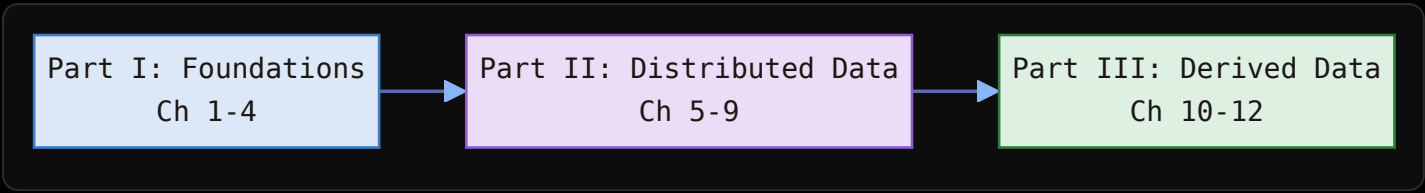
References. Almost every idea in the book appeared earlier in talks, papers, blogs, source code, bug trackers, mailing lists and engineering folklore; the book collects and cites them (800+ references,

mostly free online). Up-to-date links: `github.com/ept/ddia-references`; for papers, Google Scholar usually finds an open-access PDF.

Acknowledgments (in brief). The author calls the book a systematic gathering of others' ideas from research and industry, and argues engineers should learn from older work rather than chase only the new. It took almost four years; the chapter-opening maps were illustrated by Shabbir Diwan and Edie Freedman. (Publisher contact details and an O'Reilly Safari notice are irrelevant to the content.)

Map of the book

Part I builds single-machine foundations, Part II spreads data over many machines, Part III connects many systems through derived data.



Part I — Foundations of Data Systems

Ideas that apply to every data system, on one machine or a cluster (networked machines working as one system).

Ch	Title	One-line summary
1	Reliable, Scalable, and Maintainable Applications	What these three goals mean, how to describe load and performance, and how to reason about them.
2	Data Models and Query Languages	Relational vs document vs graph models, and their query languages: declarative ones (SQL, Cypher, SPARQL, Datalog) versus MapReduce, which sits between declarative and imperative.
3	Storage and Retrieval	How storage engines lay data out on disk: hash indexes, LSM-trees, B-trees, and column-oriented storage for analytics.
4	Encoding and Evolution	Formats for turning data into bytes (JSON, Thrift, Protocol Buffers, Avro) and how schemas change safely over time.

Part II — Distributed Data

Moving from one machine to many, which scalability often requires, and the problems that follow.

Ch	Title	One-line summary
5	Replication	Keeping copies on several machines: leaders and followers, multi-leader, leaderless quorums, replication lag.
6	Partitioning	Splitting a dataset into shards: by key range or hash, secondary indexes, rebalancing, request routing.
7	Transactions	What ACID means, weak isolation levels and their anomalies, and ways to get serializability (serial execution, 2PL, SSI).
8	The Trouble with Distributed Systems	Partial failures, unreliable networks and clocks, and reasoning about truth when nodes can be wrong.
9	Consistency and Consensus	Linearizability, ordering guarantees, two-phase commit, and fault-tolerant consensus and coordination services.

Part III — Derived Data

Systems where one dataset is computed from another, common in heterogeneous architectures where no single database does everything, so an application combines databases, caches, indexes and so on.

Ch	Title	One-line summary
10	Batch Processing	From Unix pipelines to MapReduce and beyond: joins, workflows, and dataflow engines over large fixed datasets.
11	Stream Processing	Event streams, message brokers and partitioned logs, change data capture, event sourcing, and processing unbounded data.
12	The Future of Data Systems	Data integration, "unbundling" databases, end-to-end correctness, and doing the right thing ethically.

The book ends with a **Glossary** of key terms (p. 553).

How to use these notes

1. Read Chapter 1 first; its vocabulary (faults, load parameters, percentiles, maintainability) is used everywhere. Terms in the tables above (LSM-trees, quorums, 2PL, SSI, linearizability) are defined in their chapters.
2. Part I is prerequisite for Part II, Part II for Part III; chapters within a Part build on each other too.
3. Each chapter README has the same layout: overview, key terms, the book's sections in order, a summary, an interview angle, and a self-check. Try the self-check before rereading.
4. For any new tool, ask: what model does it use, what does it guarantee, and what does it give up to get that?

Chapter 0a — System Design Fundamentals

> **Condensed primer.** This chapter is a ~21k-word primer that defines every system-design fundamental in a few precise sentences; the full ~100k-word treatment (worked examples, diagrams, interview Q&A) lives in System Design Patterns, Module 0, and every section below ends with a link to its full explanation there.

Module Introduction

Every other chapter assumes the basic vocabulary of system design: *latency*, *load balancer*, *cache*, *sharding*, *eventual consistency*. This primer defines all of it from zero, in six parts:

Part	Covers
1 — Foundations and performance	Clients/servers, latency, percentiles, estimation, availability, SLAs, scaling, bottlenecks, the interview method
2 — Networking and communication	IP, DNS, TCP/UDP/QUIC, HTTP, TLS, API styles, real-time push, proxies, load balancers, CDNs
3 — Data storage	Relational databases, ACID, isolation, NoSQL, indexes, replication, sharding, storage types, OLTP/OLAP, durability
4 — Caching, async and consistency	Caches, queues, streams, delivery guarantees, consistency models, CAP, quorums, consensus, clocks, sagas, sketches, IDs
5 — Architecture, operations and security	Architectural styles, resilience patterns, rate limiting, containers, Kubernetes, CI/CD, observability, DR, auth, security
6 — Putting it together	A request's journey, a worked URL shortener, a decision cheat sheet, 60 interview questions, an A–Z index

Read Parts 1–5 in order once; then use Part 6 as a reference. Terms are **bold** where they are defined.

→ Full explanation

Part 1 — Foundations and performance

The ground floor: what system design is, the standard building blocks, and the vocabulary for speed (latency, throughput, percentiles), size (units, estimation), staying up (availability, reliability, durability) and growing (scalability), ending with a method for the design interview.

> **Part 1 TL;DR:** Describe speed with percentiles, not averages; estimate load from DAU (1 million/day ≈ 12/s); each extra nine of availability is 10× less downtime; series dependencies multiply availability down, redundancy multiplies it up; keep app servers stateless and scale out; find the bottleneck by measuring; in interviews go requirements → estimates → API → data model → design → deep dives → trade-offs.

→ Full explanation

What system design is

System design is deciding which parts a piece of software needs, what each does, how they talk and where data lives, so the whole meets its goals: number of users, response time, staying up when machines break, and cost. Coding asks "how does this function work?"; design asks "how do twenty programs on two hundred machines cooperate without losing data?". The working loop is: understand the goal → estimate the load → sketch the parts → find weak spots (what runs out first, what takes

everything down) → fix and justify, repeated as the system grows. Every choice is a **trade-off** (faster reads may mean slower writes; more copies mean safety but cost and disagreement), and in interviews the reasoning is graded, not one "right" answer.

→ Full explanation

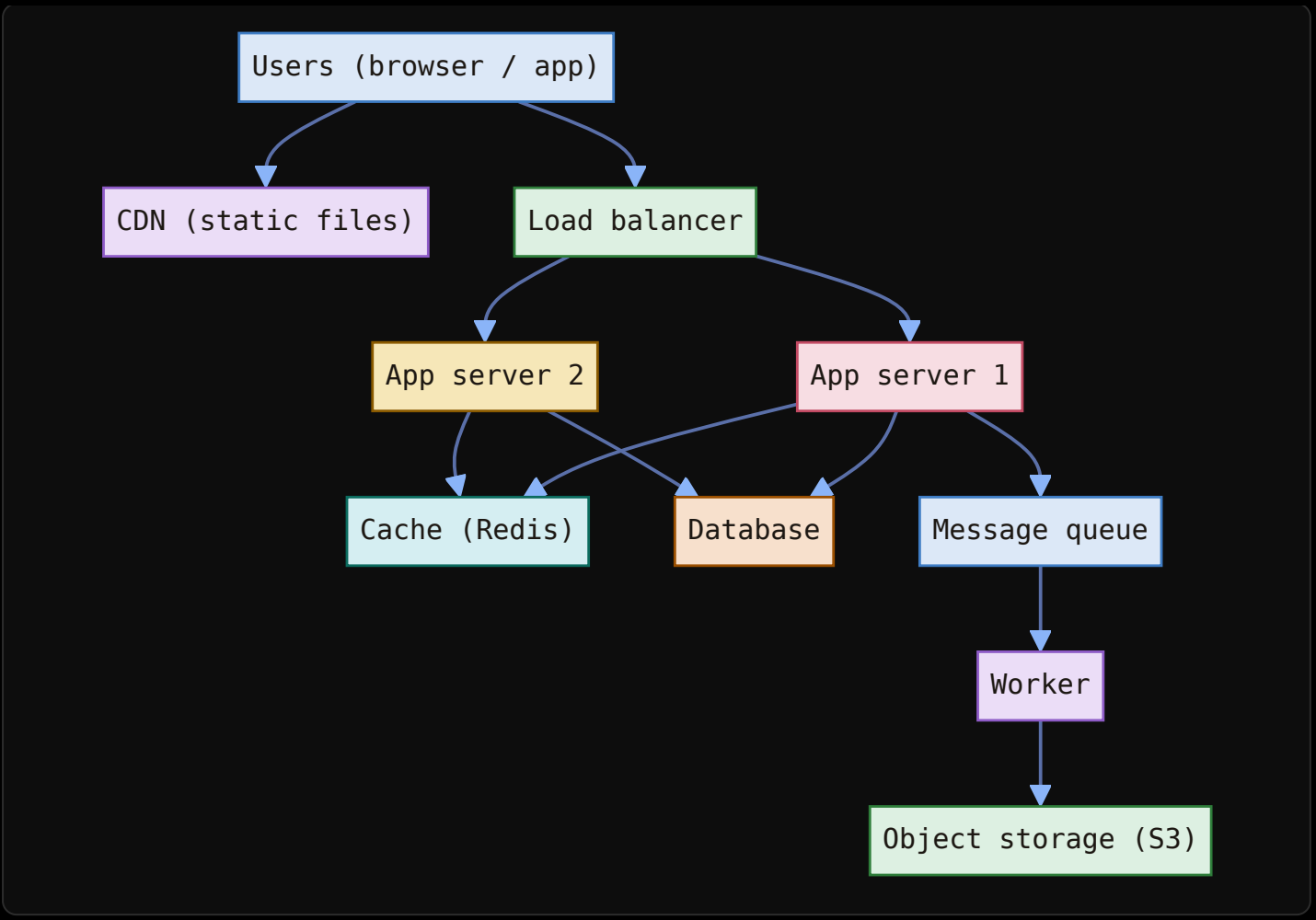
Clients, servers and the request/response cycle

A **client** is any program that asks for something (browser, app, or another server); a **server** is a program (and loosely its machine) that waits for requests and answers them — a *role*, so one logical server may be hundreds of machines and most microservices are both. They follow **request/response**: the client sends a request (`GET /profile`), the server does work and returns a response (`200 OK` + body), usually over **HTTP**. A **thin client** relies on the server; a **thick client** does much work locally. Normally the client **pulls**; features like chat need the server to **push** (WebSockets, SSE — Part 2). Remember the network between them is slow and unreliable compared with work inside one machine.

→ Full explanation

The building blocks of a web application

Part	Job
Client	Renders UI, calls the backend over HTTP
Load balancer	Spreads requests over servers, skips dead ones
App server	Runs business logic; run several identical copies
Database	Stores data durably and lets you query it; holds **state**, so hardest to scale
Cache	Fast in-memory copy of hot data (Redis, Memcached); RAM reads are well under 1 ms
Message queue + **workers**	Holds jobs for background programs so the request returns immediately
Object storage	Cheap storage for large files under keys (Amazon S3)
CDN	Servers worldwide keeping static files near users



Add each box only when you can name the problem it solves ("reads outnumber writes 100:1, so a cache"). Classic mistakes: images stored in the database (put them in object storage, keep the key), slow work (emails, encoding) inside the web request.

→ Full explanation

Latency, throughput and bandwidth

Latency is how long one thing takes (ms = thousandths, µs = millionths of a second); **throughput** is work done per unit time (requests/s, MB/s); **bandwidth** is a link's maximum capacity in bits per second (Mbps, Gbps).

Response time vs latency

Response time is what the client sees end to end; strictly, **latency** is only the delay before work starts (queueing, network hops) and **service time** is the work itself: response time = network delay + queueing delay + service time. In interviews "latency" usually means response time — say which you mean.

Latency comes from **propagation** (light in fibre ≈ 200 km/ms; London–New York round trip ≈ 70 ms), **transmission** (1 MB over 100 Mbps = 80 ms), **queueing** and **processing**. Latency and bandwidth are independent: more bandwidth does not speed up a small API call. Near full utilisation, queues grow and latency explodes while throughput flattens, so teams target ~50–70% peak utilisation. **Batching** raises throughput at a latency cost. 1 byte = 8 bits, so 100 Mbps ≈ 12.5 MB/s.

Low-latency checklist

Start with a **latency budget**: split the target (e.g. p99 < 200 ms) across hops (network, gateway, service + DB, slack) and work first on the hop that overspends. Levers, roughly by payoff: remove round trips (batch, parallelise, no N+1); reuse connections (pools, keep-alive, HTTP/2–3, TLS 1.3 resumption); move bytes closer (CDN, edge TLS, regional replicas); serve from memory (caches,

precomputed feeds and counts); do less while the user waits (queue side effects); shrink payloads (gzip/Brotli, Protobuf, pagination, only needed fields); make each query cheap (index, denormalise the hot read); keep headroom (50–70% utilisation, shed rather than queue); cut the tail (deadlines, hedging, bounded fan-out, fewer GC pauses and shared locks); measure per-hop percentiles with histograms and tracing. Rules: **fix the biggest term first** (a 70 ms ocean crossing dwarfs a 2 ms parse) and **every lever has a price** (staleness, money, load, delayed results) — say which you pay.

→ Full explanation

Percentiles, tail latency and why averages lie

A **percentile** is the value below which that share of measurements fall: **p50** (median), p90, p95, **p99** (1 in 100 slower), **p999**. **Tail latency** is the slow end (p99, p999, max).

Why averages lie

The **mean** blends fast and slow requests into a number no request had: nine 10 ms requests and one 1,000 ms give mean 109 ms, median 10 ms, p95/p99 1,000 ms. Real latency is skewed (GC pauses, cache misses, retries), so use percentiles.

Why the tail matters more than you think

Heavy users hit the tail most, and **fan-out** amplifies it: if a request waits on 100 servers each slow 1% of the time, 1 − 0.99^100 ≈ 63% of requests are slow ("The Tail at Scale", Dean and Barroso, 2013). Compute percentiles from **histograms** or sketches (HDRHistogram, t-digest); never average percentiles — merge histograms. Reduce tails with **hedged requests** (send a duplicate after ~p95), timeouts, less fan-out and spare capacity. Beware **coordinated omission**: a load tester that waits for slow replies under-reports the tail.

→ Full explanation

Little's law

For any stable system, **L = λ × W**: average items inside (L) = arrival rate (λ, equal to throughput when stable) × average time inside (W). 1,000 req/s × 0.2 s = 200 requests in flight, so you need ≥ 200 threads/connections; a pool of 20 connections with 5 ms queries caps at 20 / 0.005 = 4,000 queries/s; if latency doubles with fixed concurrency, max throughput halves. Convert ms to seconds, and it does not apply when the queue grows without bound.

→ Full explanation

Latency numbers every engineer should know

Orders of magnitude (rounded for current hardware; ratios matter, not exact values): CPU **L1 cache** 1 ns; **RAM** read 100 ns; compress 1 KB 2 µs; send 1 KB over 1 Gbps 10 µs; random 4 KB read from NVMe **SSD** 20–100 µs; read 1 MB sequentially from RAM 50–250 µs; round trip inside a data centre 0.5 ms; 1 MB from SSD 0.2–1 ms; **HDD** seek 10 ms; 1 MB from HDD 5–20 ms; London–New York round trip 70 ms; California–Europe 150 ms. Consequences: RAM is ~100–1,000× faster than SSD for random reads (caches live in RAM); a data-centre round trip costs thousands of memory reads (batch calls); cross-ocean latency only falls by moving data closer; sequential beats random I/O (why LSM-trees and logs append). A remote cache still pays the ~0.5 ms network hop.

→ Full explanation

Units, powers of two and data sizes

A **bit** is 0/1; a **byte** is 8 bits. KB, MB, GB, TB, PB each step ×1,000 in estimates (exactly 2^10 = 1,024; strictly KiB/MiB/GiB). Remember 2^10 ≈ 10^3, 2^20 ≈ 10^6, 2^30 ≈ 10^9, 2^40 ≈ 10^12, 2^32 ≈ 4.3 billion (IPv4 addresses; unsigned 32-bit limit, ~2 billion signed), 2^64 ≈ 1.8 × 10^19. Typical sizes: integer 4–8 B, timestamp 8 B, UUID 16 B, short post with metadata 0.5–1 KB, compressed photo 200 KB–2 MB, a

minute of HD video ~25–60 MB. A day ≈ 86,400 s ≈ **10^5 s**, a month ≈ 2.6 × 10^6 s, a year ≈ 3 × 10^7 s, so **1 million/day ≈ 12/s**. A 1 Gbps card moves at most 125 MB/s.

→ Full explanation

Back-of-the-envelope estimation

A **back-of-the-envelope estimate** is a quick rough calculation whose goal is the right **order of magnitude**, because that decides the architecture (100 QPS fits one server; 100,000 needs caching and sharding). Key terms: **DAU/MAU** (daily/monthly active users), **QPS/RPS** (queries/requests per second), average QPS = requests per day / ~10^5, **peak factor** (peak ÷ average, commonly 2–3×, 5–10× for spiky products), **read/write ratio**, storage/day = items × size, bandwidth = QPS × response size. Rough per-machine capacities: app server 1,000–10,000 req/s; relational DB 1,000–10,000 simple queries/s; Redis/Memcached 100,000+ ops/s; Kafka broker hundreds of MB/s. Don't forget peak, replication (×3 storage), metadata and indexes (+20–50%); round aggressively and say every assumption aloud.

Worked example: how many servers do we need?

Correct the naive "peak ÷ capacity" with measured cost, a utilisation ceiling, a concurrency check and failure headroom. Assume 30,000 req/s peak, 4 ms CPU and 50 ms end-to-end per request, 16-core machines, 20 KB responses, 3 AZs. One core: 1,000 ÷ 4 = 250 req/s; one server: 4,000 req/s; at a 60% ceiling 2,400 → 30,000 ÷ 2,400 = 12.5 → **13 servers**. Little's law: 30,000 × 0.05 s = 1,500 in flight, ~115 per server (size threads and DB pools to match). Survive losing one of 3 zones: ⌈13 ÷ 2⌉ = 7 per zone → **21**; +10% rolling-deploy surge → **~23**. Network: 600 MB/s ≈ 4.8 Gbps total, ~0.23 Gbps each — CPU is the limit. The naive 8 was ~3× low. **Repeat per tier** with its own bottleneck: cache by memory (working set ÷ RAM × replicas), database by IOPS or write throughput per primary (forcing sharding), streaming by network and disk bandwidth per broker.

→ Full explanation

Availability and the nines

Availability is the fraction of time (or of valid requests) the system works: uptime / (uptime + downtime).

The nines table

Availability	Per year	Per month (30 d)	Per day
99% (two nines)	3.65 days	7.2 h	14.4 min
99.9% (three)	8.76 h	43.2 min	1.44 min
99.95%	4.38 h	21.6 min	43 s
99.99% (four)	52.6 min	4.32 min	8.6 s
99.999% (five)	5.26 min	26 s	0.86 s

Each nine is 10× less downtime and usually far more than 10× the effort; at 99.99% recovery must be automatic. Parts in **series** multiply (99.99% × 99.9% × 99.9% ≈ 99.79%); independent parts in **parallel** combine as 1 − (failure probabilities multiplied) (two 99% servers → 99.99%), assuming failures are independent. Cloud single-service SLAs are usually 99.9–99.99%.

→ Full explanation

SLA, SLO, SLI and error budgets

An **SLI** (Service Level Indicator) is a measurement ("% of requests succeeding under 300 ms over 30 days"); an **SLO** (Objective) is the internal target for it ("99.9%"); an **SLA** (Agreement) is a customer

contract with credits or refunds if missed, normally looser than the SLO so the team is warned first. Choose SLIs users feel (failed checkouts, not CPU) and never set an SLO of 100%.

Error budgets

The **error budget** is 1 − SLO: at 99.9%, 0.1% of requests (100,000 of 100 million a month, ≈ 43 minutes of full outage) may fail. While budget remains, ship fast; when spent, slow releases and fix reliability. The idea comes from Google's **SRE** (Site Reliability Engineering, running production with software-engineering methods); Part 5 covers burn-rate alerts.

→ Full explanation

MTBF, MTTR and measuring failure

MTBF (mean time between failures — higher is better) and **MTTR** (mean time to repair/recover — lower is better); related: **MTTD** (mean time to detect) and **MTTF** (mean time to failure, for replaced parts like disks). Availability ≈ MTBF / (MTBF + MTTR): 1,000 h / 1,001 h ≈ 99.9%. At scale failures are constant, so cutting MTTR (monitoring, automatic failover, quick rollback) is usually the cheaper lever — the same failures with 10-second automatic recovery give ≈ 99.9997%. Detection time counts toward MTTR.

→ Full explanation

Reliability, availability and durability

Reliability: keeps doing the right thing correctly over time despite faults (measured by failure rate, MTBF). **Availability**: up and responding right now (% uptime). **Durability**: saved data stays saved (probability of losing an object per year). They are separate promises: Amazon S3 Standard is designed for 99.999999999% ("eleven nines") durability but 99.99% availability (SLA credits below 99.9%). A system can be available but wrong, or down but durable. Availability comes from redundancy and failover; durability from copies on separate disks/sites, writing to disk before acknowledging, and backups — replication is *not* backup, because a bad delete replicates too.

→ Full explanation

Faults, failures and fault tolerance

A **fault** is one component going wrong; a **failure** is the whole system no longer serving users; **fault tolerance** (resilience) keeps faults from becoming failures, perhaps in degraded form.

Fault type	Example	Defence
Hardware	Disk dies	Redundant machines, replicated data
Software	Bug, leak	Testing, gradual rollouts, restarts
Network	Packet loss, partition	Timeouts, retries, replicas in other zones
Human	Bad config, deleted table	Reviews, automation, backups, rollback
Dependency	Third-party API down	Timeouts, fallbacks, circuit breakers

Failure models are formal assumptions about how a faulty part behaves, each harder to tolerate than the last: **fail-stop** (halts and everyone reliably learns it; f+1 copies tolerate f), **crash (crash-stop)** (halts silently — looks like slowness, so consensus needs 2f+1 nodes), **crash-recovery** (restarts with only what it saved to disk — write durably before acknowledging), **omission** (drops some messages — acks and idempotent retries), **timing (performance)** (correct but too late or with drifting clocks — timeouts, leases, hedging), **Byzantine (arbitrary)** (wrong or malicious answers — 3f+1 replicas, signatures, checksums). Company data centres usually assume **crash-recovery plus omission plus timing faults**, plus checksums against corrupted data; blockchains design for Byzantine faults. State your model: it sets the replica count (Raft: 3 nodes survive 1 crash; Byzantine tolerance needs 4). The pattern is detect →

isolate → recover. Beware **correlated failures** (a shared power supply or bad deploy breaking "independent" copies).

→ Full explanation

Redundancy, single points of failure and failover

Redundancy is having spares; a **single point of failure (SPOF)** is any part whose loss alone stops everything (one DB, one LB, one DNS provider); **failover** switches work to a standby. Remove SPOFs by running ≥ 2 copies in separate **failure domains** (racks, **availability zones** — separate data centres in a region with own power and network — or regions), routing around dead copies, **health checks**, and automated failover. **Active-passive** (primary-standby) keeps an idle up-to-date standby; **active-active** serves from all copies (harder for writable state). Standbys are **cold** (must be started, minutes–hours), **warm** (running, partly current) or **hot** (in sync, seconds). **N+1/N+2** means one or two units beyond peak need. Risk: wrongly declaring a live primary dead gives **split brain** (two primaries accepting writes), prevented with quorums and fencing (Parts 3–4); test failover regularly.

→ Full explanation

Scalability

Scalability is handling more load by adding resources, without redesign or performance collapse, with cost growing roughly in proportion. Load is described by **load parameters** (QPS, concurrent users, read/write ratio, data size, followers per user); "scalable" only means something relative to one of them. Dimensions: **load** (more requests/s), **data** (more stored data), **geographic** (users worldwide needing low latency everywhere) and **organisational** scalability (more engineers without stepping on each other). Core techniques: scale out stateless servers, cache hot data, read replicas, sharding, queues for slow work, CDN for static content. Scale the actual bottleneck, and design so the next 10× is easy rather than 1,000× on day one.

→ Full explanation

Vertical and horizontal scaling

Vertical scaling (scale up) makes one machine bigger (cloud instances reach hundreds of cores and terabytes of RAM); **horizontal scaling (scale out)** adds machines and spreads work.

	Vertical	Horizontal
Simplicity	No code change	Needs LB, stateless design or sharding
Limit	Biggest machine	Practically unlimited
Fault tolerance	Still a SPOF	Losing a node loses a fraction
Cost	Big machines cost disproportionately	Commodity machines
Consistency	Easy, one copy	Harder

Scaling up first is often right (Stack Overflow in 2016 ran on 11 web servers and 4 SQL Server machines). What makes scale-out hard is **state**: stateless servers are trivial; data must be replicated or partitioned. Most systems do both.

→ Full explanation

Elasticity

Elasticity is adding *and removing* resources automatically and quickly as load changes, so you pay for roughly what you use (scalability = *can* grow; elasticity = grows and shrinks by itself). Mechanism: watch a metric (CPU, RPS, queue length), add servers above a threshold, remove below, LB follows — **autoscaling** (Part 5), also provided by **serverless** platforms that scale to zero. Limits: boot time

(seconds for containers, minutes for VMs), mostly stateless parts only, and **flapping** if scale-in is too eager.

→ Full explanation

Stateless and stateful services

State is data remembered between requests. A **stateless service** keeps no client-specific data in its memory, so any copy can serve any request and copies can be added, killed or replaced freely; a **stateful service** ties a client or data to a particular server. Make app servers stateless by putting sessions in a shared store (Redis) or a signed token (JWT), files in object storage, carts in a DB or cache. Stateful parts that remain (databases, game servers, WebSocket servers) need **sticky sessions** and a plan for server loss. Cost of statelessness: an extra hop (usually < 1 ms) — and the state store itself now needs scaling and redundancy.

→ Full explanation

Performance vs scalability

Performance is how fast the system is for one request at current load; **scalability** is how well performance holds as load grows. Diagnostic: one request to an idle system is slow → performance problem (profile it; more servers won't help); fast alone but slow under traffic → scalability problem. Some changes trade one for the other (a shared-cache hop costs a little per-request speed but enables scale-out).

→ Full explanation

Concurrency vs parallelism

Concurrency is many tasks in progress over the same period, interleaved whenever one waits (how work is *organised*); **parallelism** is tasks running at the same instant on multiple cores or machines (how it is *executed*). Web servers mostly wait on I/O, so concurrency lets one server juggle thousands of requests; parallelism speeds CPU-heavy work. Mechanisms: **processes** (isolated memory), **threads** (shared memory, need locks), **event loop / async I/O** (one thread never blocks: Node.js, NGINX, asyncio), **lightweight threads/coroutines** (goroutines, Java virtual threads). Hazards: **race condition** (result depends on timing, e.g. a lost withdrawal), **deadlock** (tasks wait forever on each other's locks), contention. **Amdahl's law**: speed-up is limited by the serial part (10% serial → at most 10×). A single-threaded Node.js server is concurrent but not parallel.

→ Full explanation

Bottlenecks: CPU, memory, disk and network

A **bottleneck** is the part limiting overall capacity; improving anything else doesn't help. Signs: **CPU** near 100%; **memory** swapping or OOM; **disk** high wait time or IOPS/MB/s at limit; **network** saturated link, loss, high RTT. Non-hardware bottlenecks are as common: one hot lock or row, a too-small connection pool, a rate-limited external API. Measure under load, find what is at its limit or where queues form, fix, re-measure (the bottleneck moves). **CPU-bound** work is limited by computation, **I/O-bound** by waiting; most web services are I/O-bound.

IOPS and disk performance

IOPS (input/output operations per second) limits small random access; **throughput** (MB/s) limits large sequential transfers — a disk can hit one without the other. Rough figures: HDD (7,200 rpm) 75–150 IOPS, 100–250 MB/s; SATA SSD tens of thousands, ~500 MB/s; NVMe SSD hundreds of thousands to ~1 million, several GB/s; AWS gp3 baseline 3,000 IOPS and 125 MB/s. HDD random reads are slow because the arm must move (5–10 ms). Cloud disks have provisioned limits that can throttle a DB while CPU looks idle.

→ Full explanation

How to approach a system design interview

A 45–60 minute open-ended design conversation; use a repeatable structure (rough timings for 45 min):

1. **Requirements (5 min)** — **functional** (what it does: features) and **non-functional** (how well: scale, latency, availability, durability, consistency, security, cost); agree what is out of scope.
2. **Estimates (3–5 min)** — QPS average and peak, read/write ratio, storage, bandwidth, and what they imply.
3. **API (5 min)** — main endpoints with inputs/outputs.
4. **Data model (5 min)** — entities, keys, access patterns, which database fits.
5. **High-level design (10 min)** — boxes and arrows; walk one read and one write end to end.
6. **Deep dives (10–15 min)** — the hardest parts given the numbers (ID generation, sharding, hot keys, failures).
7. **Trade-offs and wrap-up (3–5 min)** — weaknesses, SPOFs, monitoring, what changes at 10×.

Non-functional requirements and the levers that meet them

A non-functional requirement helps only if **measurable** ("p99 < 200 ms", not "fast") and tied to a **design lever**:

Requirement	Measure	Levers	Cost
Availability	Success-rate SLO, allowed downtime	Multi-zone redundancy, health checks, failover, degradation	Machines, copy consistency
Latency	p50/p95/p99 per endpoint	Caching, CDN, fewer round trips, async, headroom	Staleness, money
Scalability	Peak QPS, data growth	Stateless + LB, sharding, queues	Cross-shard work
Consistency	Who must see a write, how soon	Single leader + sync replicas, quorums, consensus, or eventual + conflict handling	Write latency, partition availability
Durability	No lost acknowledged write; RPO	WAL, replicate before ack, backups, cross-region	Latency, storage
Reliability	Error rate, MTTR	Timeouts, backoff retries, idempotency, breakers, bulkheads	Retry load, complexity
Security and privacy	Who may do what, sensitive data, compliance	AuthN/AuthZ at the edge, encryption, least privilege, audit logs	Check latency, ops
Maintainability	Time to diagnose, time to ship, deploys/day	See below	Up-front effort
Cost	Per request, user or GB	Right-size, autoscale, storage tiers, caching	Less headroom

Maintainability (most lifetime cost is operating and changing a system) splits into **operability** (easy to keep healthy: telemetry, SLO alerts, runbooks, automated deploys/rollbacks), **simplicity** (easy to understand: few component types, clear ownership, no **accidental complexity**) and **evolvability** (easy to change: versioned APIs, backward-compatible schemas, loose coupling, feature flags, online migrations, tests) — after Kleppmann's DDIA ch. 1 and ISO/IEC 25010.

Think aloud, justify every component with a requirement or number, give a complete simple design before depth, and never name a technology without the reason.

→ Full explanation

Part 1 glossary

Every Part 1 term is defined in bold in the sections above. A few extra one-liners:

Term	Meaning
Order of magnitude	Rough size as a power of ten
Fan-out	One request triggering many parallel downstream requests
Round trip time (RTT)	Time for a message to reach a destination and the reply to return
Transmission delay	Time to push a message's bits onto a link
NVMe	Fast SSD interface with very high IOPS
Thread	Independent flow of execution inside a process

→ Full explanation

Part 2 — Networking and communication

Part 1 was the boxes; this part is the arrows: how one computer finds another, how bytes travel, which **protocols** (agreed rules for how two programs talk) they speak, and which middlemen (proxies, load balancers, CDNs) sit in between.

> **Part 2 TL;DR:** IP finds the machine, the port finds the program; DNS maps names to IPs through cached root/TLD/authoritative servers; TCP is reliable and ordered but costs a handshake and head-of-line blocking, UDP is fire-and-forget, QUIC (HTTP/3) is reliable multi-stream encrypted transport over UDP; a fresh HTTPS connection costs ~2 RTTs before the request; REST for public APIs, gRPC inside, GraphQL for flexible clients; load balancers (L4 vs L7, algorithms, health checks, draining) make scale-out work; CDNs cache near users; round trips, not bandwidth, usually dominate.

→ Full explanation

IP addresses, NAT, ports and sockets

An **IP address** identifies a machine's network connection; routers forward each **packet** (a chunk of up to ~1,500 bytes on Ethernet) by destination IP. **IPv4** is 32 bits (~4.3 billion, exhausted in the 2010s); **IPv6** is 128 bits (~3.4 × 10^38). A **port** (0–65535) picks the program (22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 6379 Redis). A **socket** is IP + port; a TCP connection is identified by the **5-tuple** (protocol, source IP/port, destination IP/port), which is why one server port holds many connections. **Private IPs** (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` — **CIDR** notation giving the fixed leading bits) live inside a network such as a cloud **VPC** (your isolated cloud network); **NAT** rewrites them to a shared **public IP** (a **NAT gateway** in the cloud). Consequences: servers can't call clients behind NAT (so push uses client-opened connections); a proxy to one backend has ~64,000 source ports (~28,000 with Linux defaults), so **ephemeral port exhaustion** is real; **carrier-grade NAT** makes IP-based rate limits hit many users at once; IPs change, so use DNS and service discovery.

→ Full explanation

DNS — the internet's phone book

DNS turns a **domain name** into IPs. A name is a hierarchy read right to left: root `.` → **top-level domain (TLD)** (`com`) → domain (`example`) → **subdomain** (`www`). Players: the OS **stub resolver**; the **recursive resolver** (ISP, company, 8.8.8.8, 1.1.1.1) that does the work and caches; **root servers** (13 named identities, ~2,000 anycast instances); **TLD servers**; the **authoritative server** holding the real records.

On a miss the resolver walks root → TLD → authoritative (the stub sends one **recursive query** — "give me the final answer"; the resolver then follows **iterative queries**, each server returning a **referral** to the next) and caches each answer for its **TTL**; busy resolvers usually answer from cache. Records: **A/AAAA** (name → IPv4/IPv6), **CNAME** (alias; not allowed at the bare **apex** domain, hence ALIAS/ANAME/CNAME flattening), **MX** (mail), **NS** (authoritative servers), **TXT** (verification, SPF), plus SOA, SRV, PTR, CAA; a **zone** is one server's set of records. Long TTLs mean fewer lookups but slow changes ("propagation" is just caches expiring); short TTLs (30–60 s) allow fast failover — lower the TTL before a migration. DNS also steers traffic: **round-robin DNS** (several A records, blind to health), **GeoDNS** (answer by location, helped by **EDNS Client Subnet**), **latency-based routing**. A cold lookup costs 20–120 ms; DNS outages (Dyn 2016, Facebook 2021) make healthy sites look down, so many use two providers.

→ Full explanation

Network layers — OSI and TCP/IP models

Networking is split into **layers**, each wrapping the data with its own header (**encapsulation**): HTTP request → TCP segment (ports) → IP packet (addresses) → Ethernet frame (**MAC address**, a network card's hardware address). The 7-layer **OSI model** is teaching vocabulary (7 application, 6 presentation, 5 session, 4 transport, 3 network, 2 data link, 1 physical); the 4-layer **TCP/IP model** (application, transport, internet, link) is what runs. "L3/L4/L7" decides what a device can see: L3 (routers, IP firewall rules) sees IPs; **L4** also sees ports and TCP/UDP state but not content; **L7** reads the HTTP request (path, headers, cookies) and can route by it, at the cost of CPU and decrypting TLS.

→ Full explanation

TCP, UDP and QUIC

These are **transport-layer** protocols. **TCP** is a connection-based, reliable, ordered byte stream: a **three-way handshake** (SYN, SYN-ACK, ACK) costs one **RTT** before data; **sequence numbers**, **ACKs** and **retransmission** give reliability; **flow control** (receive window) protects the receiver; **congestion control** (slow start; Reno, CUBIC — Linux default — and BBR) protects the network, so new connections start slow; either side closes with **FIN**, and the side that closes first holds **TIME_WAIT** (60 s on Linux). **Head-of-line (HOL) blocking**: one lost packet stalls all later bytes, including other requests sharing the connection. **UDP** sends independent **datagrams** with no handshake, ACKs, ordering or congestion control (8-byte header vs TCP's 20+) — used for DNS, video calls, games, where late data is useless. **QUIC** (IETF standard 2021, from Google) runs over UDP and combines transport + TLS 1.3 into 1 RTT (**0-RTT** on resume), has independent streams (loss stalls only its stream), **connection migration** via a **connection ID** (survives Wi-Fi → 4G), is always encrypted and lives in user space (more CPU; some firewalls block UDP, so clients fall back). Every new TCP + TLS connection costs 2–3 RTTs, so reuse connections.

→ Full explanation

HTTP — the language of the web

HTTP is a request/response protocol: method, path, headers, optional body → status code, headers, body. Methods: GET (read), HEAD, POST (create/action), PUT (replace), PATCH (partial update), DELETE, OPTIONS. **Safe** = changes nothing (GET, HEAD, OPTIONS); **idempotent** = repeating leaves the same state (GET, HEAD, PUT, DELETE, OPTIONS), so it can be retried after a timeout; POST needs an **idempotency key**. Status classes: 1xx (101 WebSocket upgrade), 2xx (200, 201 Created, 202 Accepted = queued, 204), 3xx (301, 302, 304 Not Modified), 4xx client error (400, 401 = not authenticated, 403 = authenticated but not allowed, 404, 409, 429), 5xx server error (500, 502 bad gateway answer, 503, 504 gateway timeout). Key headers: `Host`, `Content-Type`, `Authorization`, `Cache-Control`, `ETag` / `If-None-Match` (→ 304), `Accept-Encoding`, `X-Forwarded-For` (original client IP). HTTP is **stateless**; **cookies** (`Set-Cookie` with `HttpOnly; Secure; SameSite`) carry a session ID whose data lives in a shared store or signed token. **Keep-alive** reuses connections (default since HTTP/1.1).

	HTTP/1.1 (1997)	HTTP/2 (2015)	HTTP/3 (2022)
Transport	TCP	TCP	QUIC (UDP)
Format	Text	Binary frames, **HPACK** header compression	Binary, **QPACK**
Parallelism	One request at a time per connection (~6 connections)	**Multiplexing**: many interleaved streams on one connection	Streams on one connection
HOL blocking	HTTP and TCP level	TCP level	Per stream only

HTTP/3 is discovered via `Alt-Svc` or a DNS HTTPS record, falling back to HTTP/2. Don't use GET for state changes, and don't return 200 with an error body.

→ Full explanation

HTTPS and TLS

HTTPS is HTTP inside **TLS** (formerly **SSL**), giving **confidentiality**, **integrity** and **authentication** — **encryption in transit** (vs **at rest** for stored data). Current version TLS 1.3 (2018); 1.2 still common. A server proves identity with a **certificate** (domain, **public key**, expiry, signature by a **Certificate Authority**), checked along the **chain of trust** to a pre-installed **root CA**; Let's Encrypt issues free 90-day certificates renewed by ACME (planned 64 days in 2027, 45 in 2028), so automate renewal. **Asymmetric** crypto (key pair) is used only in the handshake to authenticate and agree a secret via **Diffie–Hellman** (each side sends a **key share** and both derive the same key without ever sending it); bulk data uses fast **symmetric** ciphers (AES-GCM, ChaCha20). TLS 1.3 takes **1 RTT** (1.2 took 2), so a fresh HTTPS-over-TCP request costs ~3 RTTs including the request; **session resumption** and **0-RTT** (replayable, so idempotent requests only) help returning clients. Fresh keys per session give **forward secrecy**. **SNI** puts the hostname in the ClientHello so one IP serves many certificates; **mTLS** has both sides present certificates (service-to-service).

→ Full explanation

API styles — REST, RPC and gRPC, GraphQL

An **API** is the set of operations a service offers to other programs. **REST** models **resources** (plural nouns: `/users/42/orders`) manipulated by HTTP methods, stateless and cacheable, usually JSON. **RPC** models remote functions (`chargeCard(order)`); **gRPC** (Google) runs on HTTP/2 with **Protocol Buffers** — a compact binary schema format whose numbered field tags allow backward-compatible evolution (add numbers, never reuse) — generates typed clients, supports unary, server, client and bidirectional **streaming**, and needs gRPC-Web or a gateway for browsers. **GraphQL** (Facebook, 2015) exposes one endpoint with a typed **schema**; clients query exactly the fields they want, writes are **mutations**, live updates **subscriptions**, each field filled by a **resolver**; risks are weak HTTP caching, expensive queries (limit depth/cost) and the **N+1 query problem** (fixed by DataLoader batching). REST suffers **over-fetching/under-fetching**.

RPC internals: stubs, marshalling, binding and call semantics

RPC (Birrell and Nelson, Xerox PARC, 1984; later ONC RPC, Java RMI, Thrift, gRPC) has five parts: an **interface definition (IDL)** both sides compile against (gRPC: `.proto`); a **client stub** with the remote procedure's signature that packs arguments and unpacks results; a **server stub (skeleton)** that unpacks, calls the real procedure and packs the result; **marshalling/unmarshalling** to and from bytes (fixing byte order, sizes, encoding; no pointers, structures copied by value); and the **RPC runtime** (connections, matching replies by **call ID**, timeouts, retransmission). **Binding** finds a server: static config, a directory (Grapevine; ONC's `rpcbind`), or DNS/service discovery with client-side balancing. A remote call is thousands of times slower and can **fail partially** — with no reply the caller **cannot know** whether it ran — so systems state **call semantics**: **maybe** (once or not, no retry), **at-least-once** (retry until reply; idempotent operations only), **at-most-once** (server dedups by call ID; caller may get "unknown outcome"), **exactly-once** (not achievable in general; approximated by retries plus durable dedup, i.e.

idempotency keys). gRPC has **no deadline by default** (always set one) and retries only when the server never saw the call or by an explicit per-method policy.

	REST	gRPC	GraphQL
Best for	Public APIs, CRUD, caching	Internal calls, streaming	Many client types, complex screens
Weak spot	Over/under-fetching	Browser support, debugging	Caching, query cost

→ Full explanation

Real-time communication — polling, long polling, WebSockets, SSE, webhooks

Plain HTTP only lets the client ask, so server push means a client-opened channel or, between servers, a callback. **Short polling** asks every few seconds (wasteful). **Long polling** holds the request open until news or timeout (~30 s), then re-asks. **WebSockets** upgrade an HTTP/1.1 request (`Upgrade: websocket` → `101 Switching Protocols`) into a two-way framed channel (`ws://` / `wss://`); send **heartbeats** (ping/pong every 20–60 s) so idle connections aren't cut (AWS ALB idle timeout 60 s by default) and reconnect with backoff. **Server-Sent Events (SSE)** stream one-way events in a never-ending `text/event-stream` response; the browser's `EventSource` reconnects with `Last-Event-ID` (used for LLM token streaming). **Webhooks** are server-to-server HTTP POSTs to a registered URL — sign them (HMAC), retry with backoff, and make receivers idempotent since events repeat and arrive out of order. Persistent connections make servers **stateful** (capacity = open connections; route messages to the right server via pub/sub; drain on deploy).

→ Full explanation

API design basics

Versioning: publish a new version only for breaking changes (URL `/v1/`, header, or Stripe-style dated versions); prefer **additive** changes and tell clients to ignore unknown fields. **Pagination**: never return unbounded lists. **Offset pagination** (`limit=20&offset=40`) allows page jumps but slows deep in the list and shifts under inserts; **cursor (keyset) pagination** (`after=<opaque last-key token>` → `WHERE id < 1234 ORDER BY id DESC LIMIT 20`) is constant-speed and stable — used by feeds. **Rate-limit headers** (`X-RateLimit-Limit/Remaining/Reset`; IETF standardising `RateLimit`) plus **429 Too Many Requests** with `Retry-After`. Also: consistent error bodies, idempotency keys on POST, filtering/sorting via query parameters, and an **OpenAPI** (formerly Swagger) specification.

→ Full explanation

Proxies and API gateways

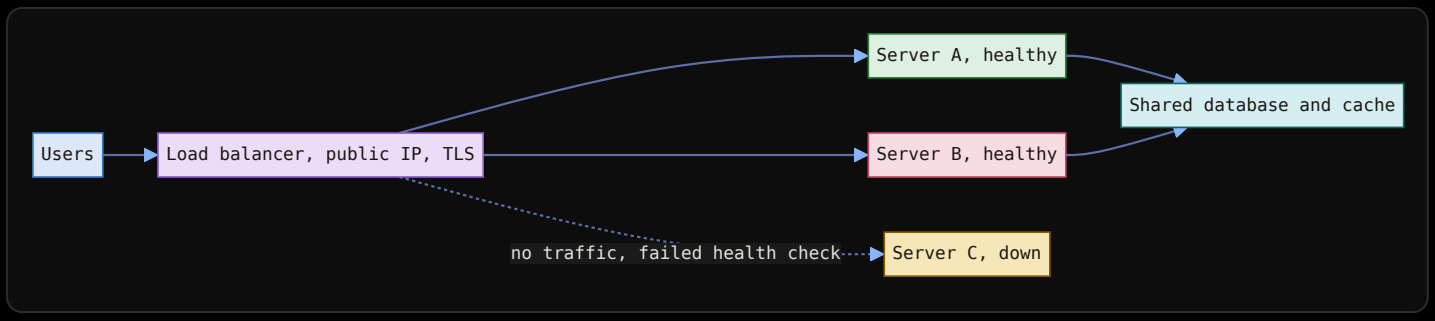
A **proxy** forwards requests on someone's behalf. A **forward proxy** acts for **clients** (corporate filtering, hiding client IPs, caching downloads); a **reverse proxy** acts for **servers** (load balancing, TLS termination, caching, compression, hiding internals — NGINX, HAProxy, Envoy). An **API gateway** is a reverse proxy specialised for APIs, the single front door that terminates TLS, authenticates (API key, JWT, OAuth token), rate-limits, routes by path/host/header, transforms or **aggregates** calls, and records logs/metrics/traces (AWS API Gateway, Kong, Apigee). **BFF (Backend for Frontend)** is one gateway per client type. Each proxy is an extra hop and potential SPOF (make it redundant); keep business logic out of the gateway; forward the real client IP via `X-Forwarded-For` (or the **PROXY protocol** at L4).

→ Full explanation

Load balancers

A **load balancer (LB)** spreads incoming traffic across a **pool** (target group) of servers, sending only to healthy ones; clients see one **virtual IP (VIP)**. It enables scale-out, survives server loss, allows

zero-downtime deploys, centralises TLS and logging and hides backends; servers behind it should be stateless.



Types

L4 balances TCP/UDP connections by IP/port without reading content: microseconds of overhead, any protocol, but per-connection (a long HTTP/2 or gRPC connection sticks to one backend) — AWS **NLB**. **L7** terminates the connection and parses HTTP: routes by path/host/header/cookie, balances per request, does TLS termination, compression, retries, redirects, WAF features — more CPU (~1 ms) — AWS **ALB**. Common layering: L4 in front of a fleet of L7 proxies. Forms: **hardware** appliances (F5 BIG-IP, NetScaler; fast, expensive), **software** (NGINX, HAProxy, Envoy, Traefik; Google **Maglev**, Meta **Katran** on commodity Linux), **managed cloud** (ALB, NLB, Gateway LB, Google Cloud LB, Azure), **DNS load balancing**, and **GSLB** (global server load balancing across regions via DNS steering and/or **anycast**). **Client-side load balancing** lets each client pick from a registry's healthy list (gRPC, Netflix Ribbon, Envoy sidecars), saving a hop.

Load-balancing algorithms

Static: **round robin** (in turn; blind to load), **weighted round robin** (by capacity; also for **canary releases**), **IP hash** (`hash(client IP) mod N`; sticky but remaps almost everyone when N changes), **consistent hashing** (ring with **virtual nodes**; only ~1/N keys move — for cache affinity; HAProxy/Envoy ring hash and Maglev), **random**. **Dynamic**: **least connections** (long or uneven requests, WebSockets), **least response time** (smoothed, e.g. **EWMA**; risks **herding**), **power of two choices (P2C)** — pick two at random, send to the less loaded; avoids herding across many LB instances (Envoy's least-request uses P2C; NGINX `random two least_conn`). Others: least bandwidth, resource-based, URL hash, locality-aware.

Health checks

Active checks probe each backend (TCP connect or `GET /health`), e.g. every 10 s, unhealthy after 2–3 failures, healthy after 2–5 successes. **Passive** checks (**outlier detection**) eject backends producing errors in real traffic (Envoy; NGINX `max_fails`). **Liveness** = process running; **readiness** = able to serve now. Avoid deep checks that fail the whole fleet on a shared-dependency blip; LBs **fail open** (send to all if all look unhealthy — AWS target groups, Envoy panic threshold).

Sticky sessions (session affinity)

Route one user to the same backend via an LB cookie (AWS `AWSALB`) or IP hash. Needed for unavoidable in-memory state (WebSockets); otherwise avoid — uneven load, lost sessions on scale-in. Prefer a shared session store or signed token.

SSL/TLS termination

The LB decrypts HTTPS and forwards plain HTTP inside the private network: one place for certificates, saves backend CPU, enables L7 routing/WAF — but internal traffic is unencrypted. **Re-encryption (bridging)** decrypts, inspects, re-encrypts; **passthrough** leaves decryption to backends (routing only by SNI).

Connection draining (deregistration delay)

On removal, stop sending *new* requests but let in-flight ones finish up to a timeout (AWS default 300 s, often 30–60 s); tell WebSocket clients to reconnect. Draining + health checks make **rolling deployments** invisible.

Making the load balancer itself highly available

Active–passive LB pair sharing a VIP via **VRRP** (`keepalived`); **active–active** nodes behind DNS, anycast or **ECMP** (routers spreading flows across LB machines announcing one IP) — how Maglev, Katran and cloud LBs work; managed ALB/NLB run nodes in every enabled AZ.

Direct server return (bonus detail)

In **DSR** an L4 LB forwards inbound packets but servers reply straight to clients, so the LB carries only the small inbound half (Maglev, Katran, LVS).

Mistakes: a single LB box, round robin for WebSockets, sticky sessions instead of a session store, L4 in front of gRPC (needs per-request balancing), forgetting `X-Forwarded-For`.

→ Full explanation

CDNs and anycast

A **CDN** is a worldwide network of caching **edge servers** grouped in **PoPs** (points of presence), serving content near users (Sydney fetches from Sydney, not Virginia). The **origin** holds the master copy; **cache hit/miss** and **hit ratio** (95% → origin sees 1 in 20) measure effectiveness; the **cache key** is host + path + selected parameters/headers. The origin controls caching with `Cache-Control` (`max-age`, `s-maxage` for shared caches, `no-store`, `private`) and validators (`ETag`, `Last-Modified` → 304). Benefits: lower latency (TLS terminates nearby), origin offload, absorbing spikes and **DDoS**. **Pull CDNs** fetch on first miss (most websites); **push CDNs** are uploaded in advance (large known files). Freshness: **purge** (seconds–minutes), **cache busting** with versioned names (`app.3f9a1c.js`, allows year-long TTLs), short TTLs. An **origin shield** is a middle tier collapsing edge misses into ~one origin request. Dynamic content still benefits (near TLS, warm connections, **edge computing** like Cloudflare Workers, Lambda@Edge). **Anycast** announces one IP from many sites via **BGP** (how networks advertise reachable IP ranges), routing users to the "nearest" site by routing policy — automatic proximity, DDoS spreading, fast failover (Cloudflare, 8.8.8.8, 1.1.1.1, DNS roots), versus **unicast** (one IP, one place). Never cache personalised responses under a shared key. Netflix runs its own CDN, Open Connect.

→ Full explanation

Bandwidth vs latency, and network partitions

Round trips usually dominate: 10 sequential trans-Atlantic round trips take ≥ 700 ms on any link speed — fix with fewer round trips (batching, HTTP/2, reuse) and moving data closer. Bulk transfers are bandwidth-bound: 10 TB over 1 Gbps ≈ 22 hours (hence AWS Snowball). Latency has a speed-of-light floor. The **bandwidth-delay product** (bandwidth × RTT) is the data in flight needed to fill a link (1 Gbps × 100 ms = 12.5 MB). A **network partition** splits healthy nodes into groups that can't talk; a node can't distinguish "crashed" from "unreachable", so systems use timeouts and heartbeats and then either stay consistent (minority side refuses) or stay available (reconcile later) — the CAP choice (Part 4); without quorums you risk split brain. The **fallacies of distributed computing** start with "the network is reliable", "latency is zero", "bandwidth is infinite". A timeout doesn't prove the remote operation failed — hence idempotent retries.

→ Full explanation

Part 2 glossary

Every Part 2 term is defined in bold above. Extra one-liners:

Term	Meaning
Router	Device forwarding packets toward their destination IP
Recursive vs iterative query	"Give me the final answer" vs "point me to who knows"
Session	Server-side record of a logged-in user, referenced by a cookie or token
Pool / target group	The set of backends behind a load balancer
Herding	Many balancers sending traffic to the same seemingly best server

→ Full explanation

Part 3 — Data storage

Code can be redeployed in minutes; data cannot be recreated. This part covers where data lives, how it is organised, how it survives crashes, and how it spreads over many machines once one is not enough.

> **Part 3 TL;DR:** Start with a relational database (ACID transactions, SQL, indexes) and know that most default to an isolation level weaker than serializable; NoSQL families each trade general querying for one access pattern; indexes turn scans into a few page reads but slow writes (B-tree for reads/ranges, LSM-tree for heavy writes); **copy for reads, split for writes** — replication for availability and read scaling, sharding by a good shard key for data size and write throughput, consistent hashing to move only ~1/N keys; put files in object storage; separate OLTP from OLAP; durability is layered: WAL + fsync, replicas, other regions, tested backups.

→ Full explanation

What a database is

A **database** (strictly a **database management system, DBMS**) stores data safely, finds it quickly and coordinates many concurrent users — things plain files can't do: indexed lookup among millions of rows in well under a millisecond, safe **concurrent** access, crash safety (a write fully happens or not at all) and a query language. Because app servers are stateless, the database holds the important **state**, making it the hardest part to scale and the focus of deep interview questions. Per request it parses, **plans** (the **query planner**/optimizer picks e.g. an index over a scan), executes (from the in-memory **buffer cache** or disk), **logs** writes durably first, and returns results. A **read** fetches without changing; a **write** inserts, updates or deletes; consumer apps often read 10–100× more than they write.

→ Full explanation

Relational databases and SQL

A **relational database** stores **tables** of **rows** (records) and **columns** (fields) under a **schema** declaring column **data types**; data that doesn't fit is rejected. You query it with **SQL**, which describes *what* you want (PostgreSQL, MySQL, Oracle, SQL Server, SQLite; cloud RDS, Aurora, Cloud SQL). A **primary key** uniquely identifies each row; a **foreign key** holds another table's primary key, and enforcing it is **referential integrity**; a **unique constraint** forbids duplicate values. A **join** combines matching rows: **inner join** (only matches), **left join** (all left rows, blanks where no match), **right join** (the mirror), **full outer join** (all rows from both). Schema-changing statements (`CREATE/ALTER/DROP TABLE`) are **DDL**. Strengths: ad hoc queries, transactions, enforced structure, mature tooling. Costs: schema changes on huge tables need online tools (`gh-ost`, `pt-online-schema-change`); scaling writes past one machine means sharding or **distributed SQL** (Spanner, CockroachDB, YugabyteDB, TiDB). One modern server (64 cores, 512 GB) handles tens of thousands of simple transactions/s — don't pick NoSQL "for scale" for a few hundred GB. Use surrogate IDs, not emails, as keys; no comma-separated lists in a column.

→ Full explanation

Transactions and ACID

A **transaction** groups reads and writes into one unit: `BEGIN … COMMIT` makes it permanent, `ROLLBACK` discards it. **ACID**: **Atomicity** — all or nothing, undone on failure or crash; **Consistency** — moves data between valid states per your declared rules (largely the application's job; not CAP's C); **Isolation** — concurrent transactions don't see each other's unfinished work (ideally as if serial; usually weaker levels for speed); **Durability** — once committed, it survives crashes (log flushed to disk before replying). Costs: locks or version tracking, and an fsync per commit (≈ 0.1–2 ms on SSD). A transaction across machines (**distributed transaction**, Part 4) is far slower. Never hold a transaction open across a slow external call.

→ Full explanation

Isolation levels and anomalies

Concurrent transactions touching the same data can produce **anomalies**: **dirty read** (reading uncommitted data), **dirty write** (overwriting uncommitted data; prevented almost everywhere), **non-repeatable read / read skew** (a re-read row changed), **phantom read** (a re-run query returns new rows), **lost update** (two read-modify-writes, one lost: 10 → 11 instead of 12), **write skew** (two transactions read the same rows, decide, and write *different* rows, together breaking a rule — e.g. both on-call doctors go off call).

Level	Dirty read	Non-repeatable	Phantom
Read uncommitted	Possible	Possible	Possible
Read committed	Prevented	Possible	Possible
Repeatable read	Prevented	Prevented	Possible (per standard)
Serializable	Prevented	Prevented	Prevented

Snapshot isolation (not in the original standard) reads a frozen snapshot via **MVCC** (multiple row versions, so readers and writers don't block); it still allows write skew. **Serializable** equals some one-at-a-time order: by actual serial execution (Redis, VoltDB), **two-phase locking**, or **serializable snapshot isolation (SSI)** (PostgreSQL). Defaults: PostgreSQL read committed (its repeatable read is snapshot isolation); MySQL InnoDB repeatable read; Oracle read committed (its "serializable" is snapshot). Without full serializability use atomic updates (`SET n = n + 1`), `SELECT … FOR UPDATE`, **optimistic concurrency control / compare-and-set** (version column, retry if 0 rows updated) or constraints. Last-ticket fix: `UPDATE … SET left = left - 1 WHERE id = 9 AND left > 0` and check one row changed.

→ Full explanation

NoSQL database families

NoSQL ("not only SQL", popular from ~2009 after Bigtable, Dynamo, Cassandra) is an umbrella for non-relational stores, each shaped around one access pattern and giving up joins, ad hoc queries or strong transactions.

Family	Model	Examples	Typical use
Key-value store	`put/get` by key, opaque value	Redis, Memcached, DynamoDB, etcd, Riak	Sessions, carts, counters, flags
Document store (document database)	JSON-like nested **documents** in **collections**, indexable fields	MongoDB (16 MB max doc), Couchbase, Firestore	Self-contained records read whole
Wide-column store	Rows grouped by **partition key**, sorted by **clustering key**, one table per query	Cassandra, ScyllaDB, HBase, Bigtable	Huge write volume read by key and time
Graph database	**Nodes** and **edges** with properties	Neo4j (Cypher), Neptune	Social graphs, fraud rings
Time-series database	Timestamped points, compression, retention	InfluxDB, TimescaleDB, Prometheus	Metrics, sensors
Search engine	**Inverted index** (word → documents), **tokenising**, stemming, **relevance ranking** (BM25)	Elasticsearch, OpenSearch (Lucene), Solr	Full-text search, as a secondary copy
Vector database	**Embeddings** (number vectors where similar items are close), **nearest-neighbour search** via **approximate nearest neighbour (ANN)** indexes such as HNSW	Pinecone, Milvus, pgvector	Semantic search, RAG

"Wide-column" is not "columnar" (analytics storage, below). Lines have blurred: MongoDB has multi-document transactions since 4.0, DynamoDB has transactions, PostgreSQL indexes JSON (`jsonb`).

→ Full explanation

SQL vs NoSQL and BASE

Choose from access patterns and correctness needs, not fashion. **BASE** describes many early NoSQL systems: **Basically Available** (keeps answering during failures, maybe with old data), **Soft state** (copies change while catching up), **Eventually consistent** (copies converge once writes stop). Lean SQL for money, inventory, bookings, complex queries and reporting, and when data fits one machine; document stores for varying nested shapes; wide-column/key-value for massive writes, key lookups and multi-region; graph for many-hop relationships. Using several types in one system is **polyglot persistence**. "NoSQL has no schema" is wrong: it is **schema-on-read** (in code) versus SQL's **schema-on-write**.

→ Full explanation

Indexes

An **index** is an extra structure, maintained on every write, that finds rows by value without a **full table scan**; check usage with `EXPLAIN`.

- **B-tree** — default: a shallow balanced tree of fixed-size **pages** (PostgreSQL 8 KB, InnoDB 16 KB) with high **branching factor** (~4 levels for billions of rows; a lookup walks root → **leaf**, one page per level); supports `=` and **range queries**/ordering; in-place updates (random writes).
- **LSM-tree** (Cassandra, ScyllaDB, HBase, RocksDB, LevelDB) — writes go to a log and an in-memory sorted **memtable**, flushed as immutable **SSTables** (sequential writes); background **compaction** merges files and drops old versions and deletion markers (**tombstones**); reads check memtable then SSTables, skipping files via **Bloom filters**. Faster writes, slower reads: the **read vs write amplification** trade-off.
- **Hash index** — O(1) exact lookups, no ranges (Bitcask/Riak; PostgreSQL `HASH`).
- **Composite index** `(user_id, created_at)` — helps queries on its **leftmost prefix** only; equality column first, range column after.
- **Covering index** — holds every needed column, enabling an **index-only scan** (`INCLUDE`).

- **Clustered** (rows stored in index order; InnoDB clusters by primary key) vs **secondary** indexes; also unique, **partial**, full-text, GIN/GiST, geospatial.

Costs: slower writes, space, memory; low-**selectivity** columns rarely benefit. Functions on columns (`LOWER(email)`) and leading wildcards (`LIKE '%shoe'`) defeat B-trees; build big indexes with `CREATE INDEX CONCURRENTLY`.

→ Full explanation

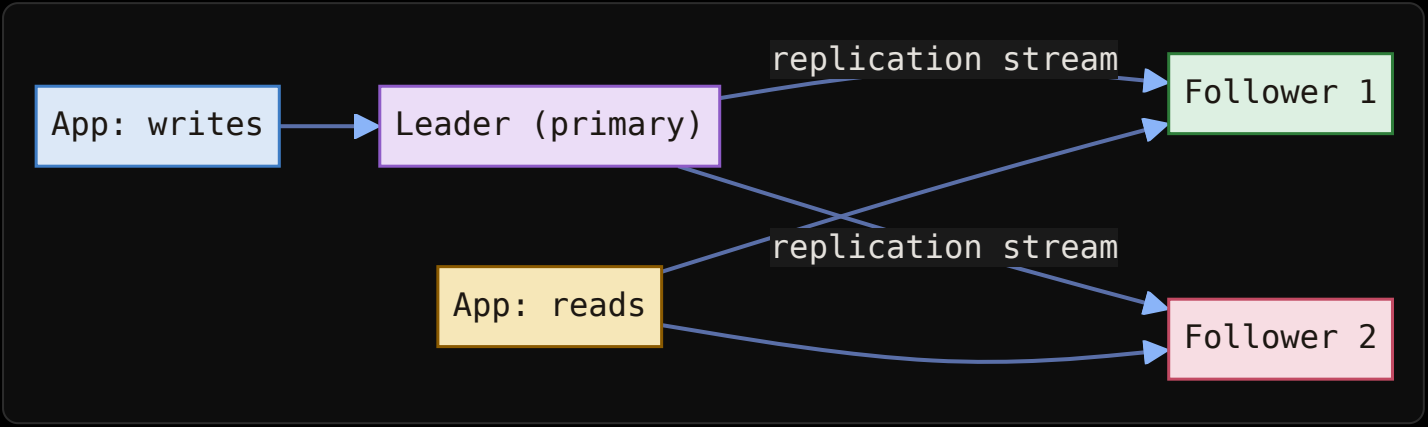
Normalization and denormalization

Normalization stores each fact once; **denormalization** duplicates facts to speed reads. Normal forms: **1NF** one value per cell; **2NF** non-key columns depend on the whole key; **3NF** they depend only on the key ("the key, the whole key, and nothing but the key", William Kent, 1983). Normalization prevents **update anomalies** (10,000 copies to rename). Denormalization techniques: copied columns, stored counts (`comment_count`, updated atomically), nested documents, **materialized views** (stored, refreshed query results). Start normalized; denormalize measured hot paths and decide how copies stay in sync; at scale and in join-less NoSQL (one Cassandra table per query) denormalization is the norm.

→ Full explanation

Replication

Replication keeps copies (**replicas**) of the same data on several machines for availability, read scaling and locality — but not write throughput: **copy for reads, split for writes**.



- **Leader-follower replication** (primary-replica): one **leader** takes writes and streams its log to **followers**; followers serving reads are **read replicas**. Default in PostgreSQL, MySQL, MongoDB, Redis.
- **Multi-leader replication**: several leaders (usually one per region) accept writes, so **write conflicts** must be resolved — **last write wins (LWW)** (loses data, clock-dependent), merge logic, or **CRDTs** (types whose concurrent updates always merge identically).
- **Leaderless replication** (Dynamo, Cassandra, Riak): write to W and read from R of N replicas, with **R + W > N** for overlap (a **quorum**, e.g. N=3, W=2, R=2); fix stale copies by **read repair**, **anti-entropy** (often Merkle trees) and **hinted handoff**.

Synchronous replication waits for a follower (no committed data lost, but slow followers stall writes); **asynchronous** replies immediately (fast, may lose the last writes on leader loss); **semi-synchronous** waits for one. **Replication lag** breaks **read-your-own-writes** (read own data from the leader briefly or track log position) and **monotonic reads** (pin users to a replica). **Failover** = detect (missed **heartbeats**, 10–30 s), choose the most up-to-date follower, reconfigure clients (Patroni, Orchestrator, managed DBs). **Split brain** is prevented by majority quorums, **fencing** (STONITH) and fencing tokens. Replicas are not backups. Examples: GitHub's 2018 43-second partition caused a cross-coast failover and a 24-hour degradation; Aurora keeps six copies across three AZs, writes need 4 of 6, reads 3 of 6.

→ Full explanation

Partitioning and sharding

Partitioning splits a dataset into pieces; **sharding** places the pieces (**shards**) on different machines, dividing both data and write load (each shard is still replicated). **Horizontal partitioning** splits rows; **vertical partitioning** splits columns (hot small vs cold large). The **shard key** decides placement:

Strategy	How	Strength	Weakness
Range	Shard owns a key range	Range queries hit few shards	Hot newest shard with timestamp keys
Hash	`hash(key)` picks shard	Even spread	Range queries scatter; `mod N` remaps on resize
Directory	Lookup service maps key → shard	Fully flexible	Extra lookup, must be HA
Geo	By user region	Latency, data residency (GDPR)	Uneven sizes

Routing happens in the app, a router (Vitess, `mongos`, Citus) or any node (Cassandra). **Federation** (functional partitioning) splits databases by feature, matching microservices. A good shard key has high **cardinality**, spreads load evenly, appears in the main query (single-shard; otherwise **scatter-gather**) and isn't monotonic under range sharding. A **hot spot** shard or **hot key** (celebrity, flash sale) needs **key salting**, caching, dedicated shards or write buffering. **Rebalancing** uses many fixed logical partitions moved whole (Elasticsearch, Couchbase, Riak), dynamic splitting (CockroachDB 512 MB ranges, MongoDB 128 MB chunks, HBase 10 GB regions) or consistent hashing. Costs: cross-shard joins and transactions, global IDs, **local vs global secondary indexes**, multiplied operations — so shard late, after indexes, caching, replicas and scaling up. Instagram and Notion (480 logical shards on 32, later 96, machines) map many logical shards to fewer servers.

→ Full explanation

Consistent hashing and virtual nodes

Consistent hashing assigns keys so adding or removing one of N servers moves only ~1/N of keys (plain `mod N` moves ~80% going from 4 to 5 servers). Servers and keys are hashed onto a **hash ring** (e.g. 0 to 2^32 − 1); each key belongs to the first server clockwise; replicas go to the next distinct servers. **Virtual nodes** give each server many ring positions (100–256; Cassandra moved its default from 256 to 16 in 4.0) for even load, spread-out takeover when a node leaves, and weighting by machine size. Alternatives: **rendezvous hashing** (highest score per key-server pair), **jump consistent hash** (fast, add/remove only at the end), **Maglev hashing**. Like all hashing it destroys key order. Used by Dynamo, Cassandra, Riak, Memcached clients (ketama), Discord and LBs.

→ Full explanation

Storage types: block, file and object

	Block	File	Object
Unit	Fixed-size blocks under a file system	Files in folders	Whole objects (**blob** + metadata) under a key in a **bucket**
Access	Attached disk (EBS, Persistent Disk)	Network protocol (**NFS**, **SMB**; EFS, NAS)	HTTP `PUT/GET/DELETE` (S3, GCS, Azure Blob, MinIO)
Shared	Usually one server	Many	Many
Partial edit	Yes	Yes	No, replace whole
Use	Databases, VM disks	Shared files	Media, backups, data lakes

Block storage is low-latency (local NVMe well under 1 ms; network block ~1–few ms) and measured in IOPS; object storage is cheapest per GB, practically unlimited and very durable but tens of ms to first byte. Upload pattern: the app issues a **pre-signed URL** (time-limited permission), the client uploads directly to object storage, the DB stores only key and metadata, downloads go through a CDN. S3: eleven nines durability, ≥ 3 AZs, objects up to 50 TB (since December 2025), strong read-after-write consistency since December 2020.

→ Full explanation

OLTP vs OLAP, row vs column storage, lakes and warehouses

OLTP is many small, fast transactional reads/writes from the application (ms latency; PostgreSQL, MySQL, DynamoDB); **OLAP** is few big analytical scans over many rows but few columns (seconds). Don't run OLAP on the OLTP database. A **row store** keeps each row together (good for OLTP); a **column store** keeps each column together, reading only needed columns, compressing well and processing in batches (ClickHouse, Druid, Pinot, Redshift, BigQuery, Snowflake, DuckDB; file formats **Parquet**, **ORC**). A **data warehouse** holds curated, structured, schema-on-write data, often as a **star schema** (a **fact table** of events plus **dimension tables**); a **data lake** holds raw data of any format cheaply in object storage, schema-on-read (risk: "data swamp"; queried by Spark, Trino/Presto, Athena); a **lakehouse** adds transactional table formats (Delta Lake, Iceberg, Hudi). Pipelines: **ETL** transforms before loading, **ELT** loads then transforms in the warehouse, **change data capture (CDC)** streams changes from the DB log (Debezium into Kafka).

→ Full explanation

Durability: WAL, fsync, backups and recovery

Durability is layered: WAL + fsync survive process crashes and power cuts; replication/RAID survive disk or machine loss; other zones/regions survive site loss; backups and point-in-time recovery survive bugs and humans; immutable separate-account backups survive ransomware. The **write-ahead log (WAL)** (redo log in InnoDB, commit log in Cassandra) appends each change sequentially and **fsyncs** it (forces it from the OS **page cache** to the device) before acknowledging; data pages are written later, a **checkpoint** marks how far they are current, and recovery replays the WAL. The same log feeds replication and CDC. fsync costs tens of µs to several ms, so databases use **group commit**; relaxed settings (`synchronous_commit = off`, `innodb_flush_log_at_trx_commit = 2`, Redis AOF fsync every second) can lose recent commits. **RAID** (1 mirror, 5/6 parity, 10 striped mirrors) only covers disk failure. Backups: **full**, **incremental**, **logical** (`pg_dump`), **physical** (`pg_basebackup`, XtraBackup), **snapshots** (copy-on-write). **Point-in-time recovery (PITR)** restores a base backup and replays archived WAL to any moment (RDS: up to 35 days). Follow **3-2-1** (3 copies, 2 media, 1 off-site) and test restores — GitLab (January 2017) found several backup mechanisms broken and lost ~6 hours of data.

→ Full explanation

Connection pooling, ORMs and N+1 queries

Opening a DB connection costs TCP + TLS + auth + session setup (several to tens of ms; PostgreSQL forks a process per connection, `max_connections` defaults to 100). A **connection pool** keeps open connections to borrow and return — in-app (HikariCP, driver pools) or as a proxy (**PgBouncer**, Pgpool-II, ProxySQL, RDS Proxy), where **transaction pooling** lends a real connection per transaction so dozens serve thousands of clients (vital for many instances or serverless). Size pools to a few times the DB's cores and remember instances × pool size (50 × 20 = 1,000). An **ORM** maps rows to objects (Hibernate, Django ORM, SQLAlchemy, ActiveRecord, Entity Framework, Prisma), parameterises queries and manages schema **migrations**, but hides the SQL. The **N+1 query problem**: 1 query for a list plus 1 per item for related data (51 round trips for 50 posts); fix with **eager loading** (`select_related`/`prefetch_related`, `includes`, `JOIN FETCH`) or batching one `WHERE id IN (...)` query (**DataLoader**), and detect it by logging query counts (Bullet).

→ Full explanation

Choosing a database

Justify from the **access pattern**, consistency and scale, in order: main queries; data size and growth; read/write ratio and peak writes; consistency needs (money vs likes); multi-region writes; what the team can operate (managed services often win).

Need	Choice	Examples
General data, transactions	Relational	PostgreSQL, MySQL, Aurora
Relational at global scale	Distributed SQL	Spanner, CockroachDB, YugabyteDB
Cache, sessions, counters	In-memory key-value	Redis, Memcached
Key access at huge scale	Managed key-value	DynamoDB, Bigtable
Nested records	Document	MongoDB, Firestore
Heavy writes by key and time	Wide-column	Cassandra, ScyllaDB, HBase
Many-hop relationships	Graph	Neo4j, Neptune
Metrics	Time-series	Prometheus, InfluxDB
Full-text, logs	Search engine	Elasticsearch, OpenSearch
Similarity	Vector	pgvector, Pinecone
Analytics	Columnar warehouse	Snowflake, BigQuery, ClickHouse
Files, media, backups	Object storage	S3, GCS
Small strongly consistent config	Coordination store	etcd, ZooKeeper

Default to PostgreSQL or MySQL and add specialist stores beside it as needs appear.

→ Full explanation

Part 3 glossary

Every Part 3 term is defined in bold above. Extra one-liners:

Term	Meaning
Commit / rollback	Make a transaction permanent / discard its changes
Concurrency	Several operations running at overlapping times
Selectivity	How few rows a filter value matches
Access pattern	How an application reads and writes its data
Hash function	Turns any input into a fixed-size, random-looking number

→ Full explanation

Part 4 — Caching, async processing and consistency

Three ideas keep a growing system fast and correct: **caching** (keep copies of popular data somewhere quicker), **asynchronous processing** (hand slow work to a queue and answer now) and **consistency in distributed systems** (decide what "the current value" means when data lives on many machines, and how machines agree).

> **Part 4 TL;DR:** Cache-aside with delete-on-write plus TTLs is the default; protect hot keys from stampedes (single flight, stale-while-revalidate, jitter); queues decouple and absorb spikes,

> Kafka-style logs keep a replayable, per-partition-ordered history; assume at-least-once delivery and make consumers idempotent, with backoff, jitter and a DLQ; consistency models range from linearizable to eventual; CAP = during a partition choose consistency or availability, PACELC adds latency vs consistency otherwise; R + W > N quorums, Raft/Paxos majorities and fencing tokens prevent split brain; don't order events by wall clocks; sagas, not 2PC, across services.

→ Full explanation

Caching basics

A **cache** is a small fast store holding copies of data whose real home (the **source of truth** or origin) is slow or expensive. **Cache hit** = found; **cache miss** = go to the source and usually fill the cache; **hit ratio** = hits ÷ lookups; **TTL** = how long an entry may live; **stale data** = a copy that no longer matches the source; **warm/cold cache** = already holding / not yet holding the popular data. Wins: latency (in-process < 1 µs, Redis 0.2–1 ms, indexed DB query 1–10 ms), database offload (95% hits → DB sees 5% of reads) and cost. Average read time = hit ratio × cache time + miss ratio × (cache + DB time): with 1 ms cache, 20 ms DB, 90% hits ≈ 3 ms; 99% ≈ 1.2 ms and 10× fewer DB reads. Caches work because traffic is **skewed** (a small hot fraction gets most requests). Costs: staleness, RAM price, an extra moving part, and a cold restart sending everything to a DB sized for 5%. Never treat the cache as the source of truth; always set a TTL.

→ Full explanation

Where caches live

From the user inward — closer is faster but harder to invalidate:

Layer	What it is	Notes
Browser/client cache	Files and responses on the device, controlled by `Cache-Control: max-age`, `ETag` → 304	Fastest; can't be deleted remotely, so use TTLs and **cache busting**
CDN cache	Edge servers caching public content (Part 2)	Purge API, seconds
Reverse proxy cache	NGINX, Varnish storing whole responses	Skips the app entirely
In-process cache	A map inside the app (Caffeine, `lru_cache`)	Nanoseconds; one copy per server, lost on restart
Distributed cache	Shared cluster: **Memcached** (simple, multi-threaded, no persistence) or **Redis** (rich types, persistence, replication, cluster mode; **Valkey** is its 2024 fork)	Shared, survives deploys, < 1 ms hop
Database buffer cache	PostgreSQL **shared buffers**, InnoDB **buffer pool**	Free, automatic

Also DNS caches, CPU caches and the OS **page cache**. Use in-process for small rarely changing data (config, flags), Redis for user data. Facebook's Memcache paper (2013) describes billions of requests/s; Netflix runs EVCache.

→ Full explanation

Caching strategies

A **caching strategy** sets who reads/writes the cache relative to the database:

Strategy	Mechanism	Main risk / fit
Cache-aside (lazy loading)	App checks cache; on miss reads DB and fills; on write updates DB then **deletes** the key	Brief stale window; the default
Read-through	Cache itself loads from DB on miss (Caffeine loader, DAX)	Simpler app code
Write-through	Write goes to cache, which synchronously writes DB	Fresh reads, slower writes, caches unread data
Write-behind (write-back)	Write to cache only, flushed to DB later in batches	Fast; loses data if the node dies — counters only
Write-around	Writes skip the cache	For data rarely read soon after
Refresh-ahead	Reload hot entries before expiry	No miss waits; wasted refreshes

Delete rather than update on write because racing writers can leave the older value cached; keep a TTL as a safety net. **Negative caching** stores short-lived "not found" markers so lookups for missing keys don't all hit the DB.

→ Full explanation

Eviction policies

When full, a cache **evicts** by policy (distinct from TTL **expiration**): **LRU** (least recently used; hash map + doubly linked list, O(1); the default), **LFU** (least frequently used; good for steady favourites, needs count decay), **FIFO**, **random** (cheap, decent), TTL-based, **MRU**, and hybrids like **W-TinyLFU** (Caffeine). Redis `maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction` (rejects writes — wrong for a cache); Redis approximates LRU by sampling keys. A big scan can flush an LRU cache; LFU or scan-resistant policies avoid that.

→ Full explanation

Cache invalidation, stale data and stampedes

Cache invalidation removes or updates copies when the source changes ("one of the two hard things", Phil Karlton). Options: TTL only; delete on write; **event-driven** invalidation from a CDC stream (works with many writers); **versioned keys** (`user:42:v7` → `v8`); CDN purge.

Stale data

Comes from TTLs, lost deletes, reader/writer races and filling the cache from a lagging replica; decide per feature how stale is acceptable (follower count yes, transfer-screen balance no).

The thundering herd / cache stampede

A **cache stampede** (thundering herd, dog-piling): a hot key expires and thousands of concurrent misses run the same expensive query, overloading the DB, which delays the refill.

Fixes for stampedes

Request coalescing / single flight (one rebuild per key; CDNs call it **request collapsing**); a **lock or lease** (`SET key:lock 1 NX PX 5000`; Facebook's lease); **serve stale while revalidating** (`stale-while-revalidate`); **probabilistic early refresh**; **TTL jitter** (also prevents a **cache avalanche** of mass expiry); warm-up after restarts. Related: **cache penetration** (keys that exist nowhere — negative caching or a Bloom filter) and **hot key** (replicate it or cache in-process).

→ Full explanation

Message queues

A **message queue** holds messages until a consumer is ready. **Producer** sends; **broker** stores (RabbitMQ, SQS, Kafka); **consumer**/worker processes. It enables **asynchronous** processing (user gets a fast reply), **decoupling**, **load levelling** of spikes, reliability (a crashed worker's message is redelivered) and independent scaling. Lifecycle: broker persists the message; a consumer receives it and it is hidden (SQS **visibility timeout**, 30 s default); the consumer **acks** after work; no ack → redelivered (hence duplicates). **Point-to-point** queues give each message to one of many **competing consumers**; **pub/sub** gives every subscriber of a **topic** its own copy; they combine (SNS → SQS, RabbitMQ fanout). Monitor **queue depth** and **consumer lag**; put big payloads in object storage and send a reference (**claim check**); don't queue what the user must see immediately.

→ Full explanation

Event streaming: topics, partitions, consumer groups and offsets

An **event streaming platform** (log-based broker: Kafka, Kinesis, Pulsar, Redpanda) keeps messages in an append-only **log** for a retention period (Kafka default 7 days) whether read or not; consumers track their position and can replay. A **topic** is split into **partitions** (ordered logs, the unit of parallelism); a message **key** is hashed to a partition so a key's events stay in order; the **offset** is a message's position. A **consumer group** shares a topic's partitions (each partition to one member; extra consumers beyond partitions idle); each group records **committed offsets**; membership changes cause a **rebalance**; lag = newest − committed offset. Ordering holds only within a partition. Classic queues (RabbitMQ with **exchanges**; SQS standard = at-least-once, best-effort order; SQS FIFO = ordered with de-duplication) suit one-off jobs; streams suit pipelines, multiple independent readers, replay and CDC. Kafka now uses its Raft-based KRaft metadata quorum; Kafka 4.0 (2025) removed ZooKeeper.

→ Full explanation

Delivery guarantees, idempotency and retries

A **delivery guarantee**: **at-most-once** (ack before processing; may lose), **at-least-once** (ack after; may duplicate — the usual default), **exactly-once** — impossible as *delivery* over an unreliable network; systems offer **exactly-once / effectively-once processing** by dropping duplicates. **Idempotency** means doing it twice equals once ("set to 100" yes, "add 20" no). Implement with an **idempotency key** checked against a processed-keys table and recorded in the same transaction as the work (Stripe's `Idempotency-Key`), **upserts**, unique constraints or conditional updates. Kafka's exactly-once (since 0.11, 2017) combines an **idempotent producer** with transactions writing output and offsets atomically — side effects outside Kafka still need idempotency. SQS FIFO de-duplicates within 5 minutes.

Ordering

Global order means one lane, so order per key (partition or FIFO group); retries can reorder, so carry versions and ignore older ones.

Retries, dead-letter queues and backpressure

Retry transient failures with **exponential backoff** and **jitter**, capped. A message that keeps failing (**poison message**) moves after N attempts to a **dead-letter queue (DLQ)** for alerting, fixing and **redrive**. **Backpressure** is a slow consumer signalling "slow down": buffer, block/throttle producers (429/503), drop low-priority work (load shedding) or scale consumers; TCP flow control is a low-level form.

→ Full explanation

Batch vs stream processing

Batch processing runs a job over a bounded, already-collected dataset on a schedule (minutes–hours latency, easy to re-run); **stream processing** handles unbounded events continuously (ms–seconds, harder: state, late events). **MapReduce** (Google 2004, Hadoop): **map** emits key-value pairs, **shuffle** groups by key,

reduce combines; failed pieces re-run. **Spark** keeps intermediates in memory; warehouses run batch SQL. Stream engines (**Flink**, **Kafka Streams**, Spark Structured Streaming) keep state and group events into **windows** (tumbling, sliding), distinguishing **event time** from **processing time** and using **watermarks** to close windows. **Lambda architecture** runs batch + speed layers; **Kappa architecture** uses only a replayable stream.

→ Full explanation

Event-driven architecture, event sourcing and CQRS

Event-driven architecture (EDA): services publish **events** (past-tense facts, `OrderPlaced`, which listeners can't refuse) rather than sending **commands** (`ShipOrder`, which can be rejected); new consumers need no publisher change. The **dual write** problem (DB updated, event not published) is solved by the **transactional outbox**: insert the event into an outbox table in the same transaction; a relay or CDC (Debezium) publishes with retries; consumers are idempotent. **Event sourcing** stores the event sequence itself as the source of truth, rebuilding state by replay with periodic **snapshots** — full audit and time travel, but harder schema evolution and separate read models. **CQRS** separates the write model from read-optimised views (denormalized tables, search indexes) updated by events — reads scale independently, at the cost of lag (eventual consistency). EDA trades traceability for loose coupling.

→ Full explanation

Consistency models

A **consistency model** is the contract about what a read may return after a write (not ACID's C). From strongest:

- **Linearizability** (strong consistency): behaves like one copy; once any read sees a new value, no later read sees the old. Needed for locks, uniqueness, balances; costs latency and availability under partition.
- **Sequential consistency**: everyone sees one order consistent with each client's order, not necessarily real time.
- **Causal consistency**: causally related operations seen in order (reply after post); about the strongest model that stays available under partition.
- **Session guarantees**: **read-your-writes**, **monotonic reads**, **monotonic writes**, **writes-follow-reads** — cheap per-user promises.
- **Eventual consistency**: replicas converge if writes stop, with no time bound (DNS).

Many stores choose per request: DynamoDB strongly consistent reads at double the read cost; Cassandra levels ONE/QUORUM/ALL; Spanner externally consistent transactions.

→ Full explanation

CAP theorem and PACELC

The **CAP theorem** (Brewer 2000; proved by Gilbert and Lynch 2002): with a **network partition**, a replicated store must choose **C** (linearizability; refuse or wait — **CP**) or **A** (every non-crashed node answers, maybe stale — **AP**); **P** (partition tolerance) isn't optional, so the choice applies only during partitions. CP: ZooKeeper, etcd, Consul, Spanner, HBase, MongoDB with majority reads/writes. AP: Cassandra, Riak, Dynamo-style stores at low levels, CouchDB, DNS; many are tunable.

PACELC

Abadi (2010): if **P**artition, choose **A** or **C**; **E**lse choose **L**atency or **C**onsistency. PA/EL: Cassandra, DynamoDB defaults; PC/EC: Spanner, synchronous single-leader; PA/EC (MongoDB is often described so under some settings) and PC/EL also exist — labels vary with configuration.

Common mistakes (misreadings of CAP)

"Pick two of three" (CA isn't available to distributed systems); AP means stale only during partitions; CP fails only unsafe requests (minority side); CAP's C isn't ACID's C; choose per feature (cart AP, checkout CP).

→ Full explanation

Quorums

A **quorum** is the minimum number of replicas that must take part. With **N** replicas, **W** write acknowledgements and **R** read responses, **R + W > N** makes every read overlap the latest successful write (newest version wins; **read repair** and **anti-entropy** fix laggards). For N = 3: W2/R2 balanced and tolerates one down; W3/R1 fast reads; W1/R3 fast writes; W1/R1 may be stale. A **majority quorum** (> half) tolerates ⌊(N − 1)/2⌋ failures, so use odd sizes (4 nodes tolerate 1, like 3). A **sloppy quorum** with **hinted handoff** writes to stand-in nodes, raising availability but temporarily breaking the overlap guarantee.

Trade-offs and caveats

R + W > N alone isn't linearizability (concurrent writes, sloppy quorums, partial failures); larger W or R makes that operation slower and less available.

→ Full explanation

Consensus, leader election and distributed locks

Consensus gets machines to agree on a value or log despite crashes and lost messages — preventing split brain; it underpins **etcd** (Kubernetes state), **ZooKeeper**, **Consul**, CockroachDB, TiDB, Spanner. **Paxos** (Lamport, ~1989–90, published 1998) is proven but hard; **Raft** (Ongaro and Ousterhout 2014) is designed to be understandable (etcd, Consul, CockroachDB, KRaft). Raft: nodes are **follower/candidate/leader** in numbered **terms** (epochs); a follower missing **heartbeats** for its randomised **election timeout** (e.g. 150–300 ms) becomes a candidate, increments the term and asks for votes; a **majority** (voting only for candidates with up-to-date logs) elects a leader; entries are **committed** once a majority stores them; stale leaders are rejected by term. A 5-node cluster survives 2 failures (CP). Use a coordination service for **leader election** rather than writing your own.

Distributed locks and fencing tokens

A **distributed lock** lets one process across machines act at a time, with a **lease** (expiry). A process can pause (GC, VM migration) past its lease while another takes the lock; a **fencing token** — an increasing number issued with each grant, checked by storage, which rejects older tokens — stops the stale holder (ZooKeeper zxid, etcd revision). A Redis `SET NX PX` lock is fine for efficiency, not correctness.

→ Full explanation

Clocks and ordering

Machine clocks disagree (**clock skew**) and run fast or slow (**clock drift**); **NTP** corrects them to a few-tens of ms over the internet (< 1 ms in a good data centre) but can jump backwards. The **wall clock** gives time of day; the **monotonic clock** only moves forward (use it for timeouts). So **last-write-wins** by timestamp can silently drop a later write. **Logical clocks**: a **Lamport clock** (increment per event, send counter, on receive take max + 1) guarantees cause has a smaller number but can't detect concurrency; a **vector clock** (one counter per process, element-wise max) tells before/after/**concurrent**, which Dynamo-style **version vectors** use to keep conflicting "siblings" (size grows with participants). **Hybrid logical clocks (HLC)** (CockroachDB, YugabyteDB) mix wall time and a counter; **Google TrueTime** (Spanner) uses GPS and atomic clocks to give an uncertainty interval of a few ms, and Spanner waits it out before committing.

→ Full explanation

Distributed transactions: two-phase commit and sagas

A **distributed transaction** must commit all-or-nothing across several databases or services. **Two-phase commit (2PC)**: a **coordinator** asks **participants** to **prepare** (do the work durably, lock, vote yes/no), then tells all to **commit** or abort. True atomicity and isolation, but **blocking** (if the coordinator dies after yes votes, participants hold locks and wait), slow and availability-coupled; used inside Spanner (over Paxos-replicated groups) and via **XA**, rarely across microservices (**three-phase commit** is little used). A **saga** chains **local transactions**, with **compensating transactions** undoing earlier steps in reverse on failure (refund, release seat); coordinated by **choreography** (services react to events) or **orchestration** (a central orchestrator such as Temporal, Step Functions, Camunda). Sagas lack isolation (use `PENDING` states), compensations must be idempotent, and irreversible actions (emails) go last. Sagas are the microservices norm.

→ Full explanation

Heartbeats, gossip and failure detection

Failure detection guesses which nodes are alive, since dead, slow and unreachable look alike. A **heartbeat** is a periodic "I'm alive"; missing several makes a node **suspected dead**. Short timeouts react fast but cause **false positives** and **flapping**; long ones react slowly. Central monitoring (ZooKeeper sessions) must itself be HA. A **gossip protocol** has each node periodically exchange membership lists with heartbeat counters with a few random peers, spreading news to N nodes in ~log N rounds with no SPOF (Cassandra, original Dynamo, Consul's memberlist, Redis Cluster). The **phi accrual detector** (Cassandra, Akka) outputs a suspicion level from arrival history; **SWIM** asks other nodes to ping a suspect indirectly. A node declared dead may still be running — use fencing.

→ Full explanation

Probabilistic data structures and Merkle trees

Structures that trade a small controlled error for far less memory.

Bloom filter

Answers "is X in the set?" with "definitely not" or "probably yes" — false positives, never false negatives. An m-bit array plus k hash functions: adding sets k bits; any 0 bit on lookup means absent. ~10 bits per item with 7 hashes ≈ 1% false positives; no deletion (a **counting Bloom filter** allows it). Used per SSTable in LSM stores to skip disk reads, against cache penetration, by crawlers.

HyperLogLog

Estimates **cardinality** (distinct count) in fixed memory by tracking, across many buckets, the longest run of leading zeros in item hashes. Redis HLL: ≤ 12 KB, 0.81% standard error, billions of items, mergeable (`PFADD`, `PFCOUNT`, `PFMERGE`); approximate `COUNT(DISTINCT)` in warehouses.

Count-min sketch

Estimates item **frequency** with a d × w counter table and one hash per row; query takes the minimum, so it may overestimate, never underestimate. For heavy hitters, trending topics, hot-key detection, top-K.

Merkle tree

A tree of hashes up to a **root hash**; any data change alters the path to the root. Replicas compare roots and descend only into differing children, finding differences in ~log(blocks) comparisons — Cassandra/Dynamo anti-entropy, Git, blockchains, certificate transparency.

Summary table

Structure	Answers	Error	Memory
Bloom filter	Membership	False positives only	~10 bits/item for 1%
HyperLogLog	Distinct count	~0.81% (Redis)	12 KB
Count-min sketch	Frequency	Overestimates only	Fixed table
Merkle tree	Where datasets differ	Exact	One hash per block

→ Full explanation

Unique ID generation

IDs must be unique across machines without coordination, ideally time-sortable (index-friendly), compact (64-bit) and not leaking volume. **Auto-increment** is simple and ordered but centralised, collides after sharding (step by shard count, or a ticket server like Flickr's) and is guessable. **UUID** (128 bits, 36 chars): **UUIDv4** is 122 random bits (collision-negligible, but random order fragments B-trees); **UUIDv7** (RFC 9562, 2024) prefixes a 48-bit ms timestamp so it sorts by time — a good default (**ULID** is similar). **Snowflake ID** (Twitter 2010, 64 bits): 1 sign bit, 41 bits ms timestamp (~69 years), 10 bits machine ID (1,024 generators), 12 bits sequence (4,096/ms per machine, ~4 million/s); needs unique machine IDs and non-backward clocks (Discord, Instagram variant, Sonyflake). Also **segment/range allocation** (blocks of IDs handed out) and content hashing with collision checks.

→ Full explanation

Geospatial indexing: geohash and quadtree

B-trees sort one value, so "nearby" needs a **geospatial index**. **Geohash** repeatedly halves longitude/latitude, encoding bits as base-32 characters; a shared prefix means nearby (5 chars ≈ 5 × 5 km, 6 chars ≈ 1.2 × 0.6 km), so proximity becomes a prefix search — query the cell plus its 8 neighbours (Redis `GEOADD` / `GEOSEARCH`, Elasticsearch). A **quadtree** splits a map cell into four when it holds too many points, adapting to density (usually in memory). Others: **R-tree** (PostGIS), Google **S2** (sphere cells), Uber **H3** (hexagons with equidistant neighbours).

→ Full explanation

Part 4 glossary

Every Part 4 term is defined in bold above. Extra one-liners:

Term	Meaning
Skewed traffic	A small fraction of items receives most requests
Competing consumers	Several workers taking messages from one queue
Version vector	Per-replica counters used to detect concurrent writes
Lease	A lock or permission that expires automatically

→ Full explanation

Part 5 — Architecture, reliability, operations and security

How to arrange the building blocks into a system, keep it running when pieces break, ship changes safely, see what it is doing, and keep the wrong people out.

> **Part 5 TL;DR:** Start with a modular monolith, split into microservices for team independence or independent scaling; prefer async messaging off the critical path; always set timeouts, retry only transient failures on idempotent calls with backoff + jitter at one layer, and wrap fragile dependencies in circuit breakers, bulkheads and fallbacks; rate limit (token bucket, 429) and shed load before collapse; autoscale with headroom and bounds; containers + Kubernetes reconcile desired state; ship through CI/CD with canaries, feature flags and fast rollback; observe with metrics, logs and traces and page on SLO burn rate; set RPO/RTO and test restores; AuthN ≠ AuthZ, keep tokens short-lived, hash passwords slowly, apply least privilege and zero trust.

→ Full explanation

Architectural styles: monolith, modular monolith, SOA, microservices and serverless

An **architectural style** answers how code is split into running programs and how they talk.

Style	What it is	Gains	Costs
Monolith	One codebase, one deployable, one DB; scaled by copies behind an LB	Simple build/test/debug, easy transactions	Slow builds and coupled releases as it grows; scale all of it
Modular monolith	One deployable with strictly bounded modules	Tidiness without the network (Shopify)	Discipline/tooling needed
SOA	2000s coarse services joined by an **enterprise service bus (ESB)**	Enterprise integration	Shared models, central bus
Microservices	Small services per business capability, **own database** (**database per service**: independent schemas and scaling, but no cross-service joins or transactions), independent deploys, "smart endpoints, dumb pipes"	Team independence, independent scaling and failure	**Distributed systems tax**: network failures/latency, no cross-service transactions (sagas), tracing, per-service ops
Serverless / FaaS	Functions run per event, scaled from zero, billed per invocation (Lambda, Cloud Functions, Workers)	No servers, idle costs nothing	**Cold starts** (~100 ms to seconds), 15-minute Lambda limit, lock-in, costly at steady load

Conway's law: systems mirror the organisation's communication structure. Avoid a **distributed monolith** (services deployed in lockstep, shared DB, long sync chains) and splitting by technical layer. Advice: monolith first; migrate with the **strangler fig pattern** (proxy in front, move features one by one). Prime Video (2023) cut one tool's cost > 90% by moving from Step Functions + Lambda to a single process — pick per workload.

→ Full explanation

Synchronous vs asynchronous communication

Synchronous: A calls B (REST, gRPC) and blocks until a reply or timeout. **Asynchronous**: A publishes to a queue/stream and returns; B processes later; results come by callback, webhook or status polling (HTTP **202 Accepted** + job ID). Sync chains multiply availability (five 99.9% services ≈ 99.5%, ~44 h/year down instead of ~9) and cascade slowness; async absorbs failures but gives eventual results, needs idempotent consumers and correlation IDs. Rule: synchronous with a timeout only if the user needs the result now (card authorisation); everything else (email, analytics, indexing) as events.

→ Full explanation

Service discovery, sidecars and service mesh

Service discovery answers "which addresses run service X now?". A **service registry** (Consul, etcd, ZooKeeper, Eureka, Kubernetes) records instances that **register**, send heartbeats and are removed when they stop. **Client-side discovery**: the caller queries the registry and load-balances (Eureka + Ribbon); **server-side discovery**: the caller hits a fixed router/LB that looks up the registry (AWS target groups, Kubernetes Services — the usual default); **DNS-based discovery** publishes instances as DNS names (`payments.default.svc.cluster.local`; caching can go stale). A **sidecar** is a helper process beside each instance handling networking, TLS, retries, logging; a **service mesh** is a fleet of sidecar proxies (the **data plane**, usually Envoy) configured by a **control plane** (Istio, Linkerd), giving mTLS, uniform retries/timeouts, traffic splitting and per-call telemetry without code changes — at the cost of an extra hop each side, CPU/memory per pod and operational complexity (overkill for ten services). Don't configure retries in both app and mesh.

→ Full explanation

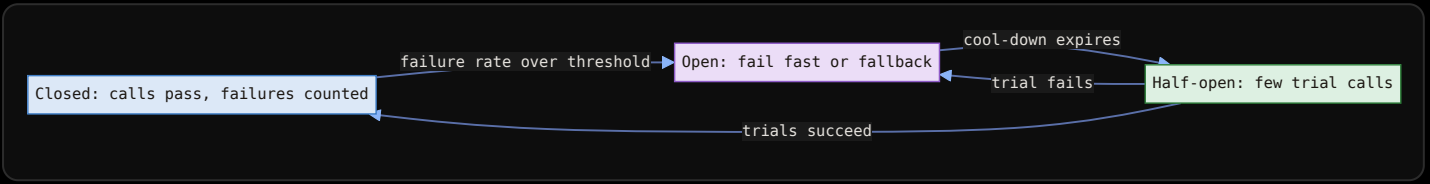
Timeouts, retries, exponential backoff and jitter

A **timeout** caps how long a caller waits; a **retry** repeats a request that hit a **transient fault**; **exponential backoff** doubles the wait each attempt up to a cap; **jitter** randomises it. Without timeouts, waiting callers exhaust threads and fail too — a **cascading failure**; careless retries multiply load on a struggling service — a **retry storm**. Set timeouts from measured latency (p99 300 ms → ~0.5–1 s) and propagate a **deadline** across the request (gRPC does). Retry only transient errors (timeouts, resets, 502/503/504, 429 after `Retry-After`), only idempotent operations (or with an idempotency key), 2–3 attempts, at one layer only (four layers × 3 attempts = 81 calls). **Full jitter**: wait random(0, min(cap, base × 2^n)). A **retry budget** caps retries at, say, 10% extra load. **Hedged requests** cut tail latency for a few percent extra load. AWS SDKs back off with jitter by default.

→ Full explanation

Circuit breakers, bulkheads and fallbacks

A **circuit breaker** wraps calls to a dependency: **Closed** (normal, counting failures in a rolling window) → **Open** when failures exceed a threshold (e.g. 50% of ≥ 20 calls; calls fail instantly or go to a fallback) → **Half-open** after a cool-down (e.g. 30 s; a few trial calls close it on success or re-open it). A **bulkhead** isolates resources (separate thread/connection pools per dependency, separate queues or clusters, **cell-based architecture** limiting **blast radius**) so one failure can't consume everything. A **fallback** returns cached, default or partial data. Costs: tuning thresholds, reserved idle capacity, stale answers (fine for recommendations, not balances). Netflix Hystrix (maintenance mode) → Resilience4j; meshes offer breakers and outlier ejection.



→ Full explanation

Rate limiting and throttling

Rate limiting caps requests per client per window and rejects the excess with **429 Too Many Requests** + `Retry-After` and `X-RateLimit-*` headers; **throttling** strictly means delaying/queueing instead. Purposes: protection, fairness, cost control, slowing brute force, business tiers. Limits are per key (user, API key, IP, user + endpoint).

Algorithm	How	Bursts	Memory	Accuracy
Token bucket	Bucket of B tokens refilled at R/s, one per request	Up to B	Tiny	Good (most used)
Leaky bucket	Queue drained at a fixed rate	Smoothed	Queue	Good
Fixed window	Counter per calendar window	2× at edges	Tiny	Rough
Sliding window log	Timestamp per request	No	Per request	Exact
Sliding window counter	current + previous × overlap fraction	Little	Two counters	Very good

Enforce at the edge/CDN/WAF (by IP), the API gateway (per key), inside services, and outbound to partners. **Distributed rate limiting** uses a shared store (Redis atomic Lua script) or local shares synced periodically; if the store fails, choose **fail open** (usual for user traffic) or **fail closed**. GitHub: 5,000 requests/hour authenticated, 60 unauthenticated; NGINX `limit_req` is a leaky bucket. Don't limit by IP alone or answer 500/503.

→ Full explanation

Load shedding and graceful degradation

Load shedding deliberately rejects some requests when overloaded so the rest succeed; **graceful degradation** turns off non-essential features (recommendations, reviews) to keep the core (search, checkout). Overload otherwise leads to **congestion collapse**: queues grow, requests time out after work is done, clients retry, **goodput** (useful completed work) falls to near zero. Steps: detect overload (CPU, in-flight limit, queue wait > ~100 ms); reject cheaply with 503 + `Retry-After`; shed by priority (bots and background first, checkout last); drop requests whose callers already timed out; keep **kill switches**. Backpressure is the cooperative cousin. Rejecting must be cheaper than serving.

→ Full explanation

Autoscaling

Autoscaling adds capacity (**scaling out**) and removes it (**scaling in**) automatically — elasticity in practice. Pick a metric (CPU, RPS per instance, queue depth, p95 latency) and a target (e.g. 60% CPU leaves headroom); the controller (Kubernetes HPA checks every 15 s) computes desired = current × metric/target (10 at 90% with a 60% target → 15); new instances boot (seconds for containers, ~1–5 min for VMs); scale in after a **cool-down** (5–10 min) with draining; set **min/max** bounds. Types: **reactive** (late), **scheduled** (known peaks), **predictive** (learned patterns; AWS, Netflix Scryer), horizontal vs vertical (VPA), **cluster autoscaling** (adds nodes). Watch for the DB connection limit, too-high targets, slow startup and unbounded bills. Scale queue workers on lag; KEDA scales on event sources.

→ Full explanation

Virtual machines, containers and Docker

A **virtual machine (VM)** is an emulated computer with its own OS kernel, created by a **hypervisor** (KVM, Xen, ESXi, AWS Nitro). A **container** is an isolated process sharing the host kernel, built from Linux **namespaces** (what it sees) and **cgroups** (what it may use). A **container image** is a read-only package of app + dependencies built in cached **layers**; **Docker** (2013) popularised the `Dockerfile`, runtime and **registries** (Docker Hub, ECR). Containers start in ms–seconds and weigh MB (dense, portable) but isolate less than VMs (separate kernel, tens of seconds, GB); usually containers run inside cloud VMs, and micro-VMs like Firecracker (Lambda, Fargate) boot in ~125 ms. Keep data out of the container filesystem (volumes/external storage), keep images small, don't run as root or bake in secrets.

→ Full explanation

Kubernetes basics

Kubernetes (K8s) is a **container orchestrator**: you declare desired state and it keeps reality matching (open-sourced by Google 2014 from Borg, now CNCF; managed as EKS, GKE, AKS). Vocabulary: **cluster** (machines + control plane); **node** (a machine running the **kubelet** agent); **control plane** (**API server**, **etcd** state store, **scheduler**, **controllers**); **pod** (smallest unit: one or more containers sharing an IP and volumes, replaced not repaired); **Deployment** (replicas + rolling updates for stateless apps); **Service** (stable name and virtual IP load-balancing over matching pods — built-in discovery); **Ingress/Gateway API** (external HTTP routing); **ConfigMap/Secret**; **Namespace**; **StatefulSet** (stable names and storage per pod); **liveness** (restart) and **readiness** (send traffic) **probes**. The **reconciliation loop** compares desired and actual state ("3, want 5" → create 2). It is complex; ECS, Cloud Run or a **PaaS** may suit small teams. Set resource requests/limits and readiness probes.

→ Full explanation

Infrastructure as code

Infrastructure as code (IaC) describes servers, networks, LBs, databases and permissions in version-controlled files applied by a tool, giving reproducible environments, code review and fast rebuilds. **Declarative** tools describe the end state (Terraform, CloudFormation, Kubernetes YAML, Pulumi): write, **plan** (diff), review, **apply**, recorded in a **state file**; **imperative** scripts list steps; **configuration management** (Ansible, Chef, Puppet) configures existing servers; **immutable infrastructure** replaces servers rather than patching them.

Trade-offs and common mistakes

Learning curve; state files must be stored and locked; manual console edits cause **drift** that the next apply may undo.

→ Full explanation

CI/CD

Continuous integration (CI): merge small changes at least daily, each triggering automated build and tests (minutes). **Continuous delivery (CD)**: every passing change is packaged and releasable at a button; **continuous deployment**: it goes to production automatically. A **pipeline**: push → build, lint, unit tests, security scans (< ~10 min) → one versioned artifact promoted unchanged through environments → staging and integration tests → production with a safe strategy and automatic rollback. Small frequent releases are safer; the **DORA metrics** (deployment frequency, lead time, change failure rate, time to restore) show high performers deploy more often with fewer failures. Avoid flaky tests, rebuilding per environment, long-lived branches, and non-backward-compatible schema changes — use **expand and contract**.

→ Full explanation

Deployment strategies and rollback

A **deployment strategy** replaces a running version without downtime while limiting **blast radius** (most outages are caused by changes).

Strategy	How	Downtime	Rollback	Blast radius
Recreate (big bang)	Stop all, start new	Yes	Slow	Everyone
Rolling deployment	Replace a few at a time behind health checks (Kubernetes default)	No	Medium	Grows per batch
Blue-green deployment	Switch all traffic from blue to a tested green environment	No	Instant	Everyone at switch; 2× capacity
Canary release (deployment)	1% → 5% → 25% … with automated metric comparison	No	Fast	Tiny at first
Feature flags	Ship dark, enable at runtime per audience; doubles as kill switch	No	Seconds	Chosen users

A **shadow (dark) launch** copies real traffic to the new version and discards responses (beware side effects). Flags separate **deploy** from **release**. **Rollback** to the last good version first, investigate later; when data or schema changed incompatibly, **roll forward**. Lessons: Knight Capital (2012) lost ~$440 million in ~45 minutes after a partial deploy reactivated dormant code; CrowdStrike (July 2024) pushed an update everywhere at once, crashing ~8.5 million Windows machines.

→ Full explanation

Observability: logs, metrics and traces

Monitoring watches known numbers; **observability** is the ability to explain unexpected behaviour from emitted data, via three pillars: **logs** (timestamped event records, ideally **structured** JSON), **metrics** (cheap numeric time series: **counters**, **gauges**, **histograms**), and **traces** (one request's path as a tree of timed **spans** sharing a **trace ID**). Instrument with **OpenTelemetry**, propagate a **correlation/trace ID** (W3C `traceparent`), ship to log stores (OpenSearch, Loki, Splunk), metrics DBs (Prometheus, Datadog, CloudWatch) and trace backends (Jaeger, Zipkin, Tempo), visualise in Grafana/Kibana, alert on metrics. Google's **Dapper** paper (2010) introduced large-scale tracing (inspiring Zipkin and Jaeger). **Sampling** keeps e.g. 1% of normal and all error/slow traces (**tail-based sampling**). Avoid high-**cardinality** metric labels (user IDs). Logs are rich but expensive, metrics cheap but aggregated, traces precise but sampled; don't log secrets.

Types: what to measure

Four golden signals (Google SRE): latency (successes and failures separately), traffic, errors, saturation. the **RED method** (Tom Wilkie) for services: rate, errors, duration. the **USE method** (Brendan Gregg) for resources: utilisation, saturation, errors. RED shows what users feel; USE shows why.

→ Full explanation

Dashboards, alerting, on-call and SLO-based alerts

A **dashboard** shows key metrics at a glance; an **alert** fires when a metric rule breaks; **on-call** engineers respond; a **page** is urgent, a **ticket** waits for working hours; a **runbook** says what to do; a **postmortem** analyses an incident **blamelessly**. Page only on user-facing symptoms that need human judgement, with runbook and dashboard links, to avoid **alert fatigue**. **SLO-based alerting** pages on **burn rate** — how fast the error budget is spent (1 = exactly the budget over 30 days). Google's SRE workbook scheme: 1 h window at 14.4× (2% of budget) → page; 6 h at 6× (5%) → page; 3 days at 1× (10%) → ticket; pair each long window with a short one. In incidents an **incident commander** coordinates, and **mitigation** (rollback, failover, shed load) comes before root cause.

→ Full explanation

Disaster recovery

Disaster recovery (DR) restores service and data after large failures (site loss, region outage, ransomware, deleted database); **business continuity** is the wider plan. **RPO** (recovery point objective) = how much data, in time, you may lose; **RTO** (recovery time objective) = how long you may be down. Lower values cost more. Backups follow **3-2-1**: full, incremental, snapshots, continuous WAL archiving (point-in-time recovery), **immutable / air-gapped** copies; rehearse restores. A cloud **region** contains several **availability zones** (separate data centres, ~1–2 ms apart): **multi-AZ** (synchronous replication practical; the baseline) vs **multi-region** (survives regional outages and serves users nearer home, but 50–150 ms+ apart, so usually async with RPO > 0, and costlier).

Types: DR strategies

Strategy	DR site runs	RTO	RPO
Backup and restore	Only backups	Hours–a day	Hours
Pilot light	Replicated data, servers off	Tens of minutes	Minutes
Warm standby	Scaled-down full copy	Minutes	Seconds–minutes
Active-active	Full copy serving traffic	Near zero	Near zero–seconds

Active-passive leaves idle capacity and a rarely exercised failover; active-active needs headroom and multi-writer conflict handling. **Chaos engineering** injects controlled failures (Netflix **Chaos Monkey**, 2012) to prove recovery works; **game days** rehearse it. Keep backups in another account/region and ensure DR doesn't depend on the failed region (Facebook's 2021 BGP/DNS outage locked engineers out of internal tools).

→ Full explanation

Authentication and authorization

Authentication (AuthN) proves who you are (something you know, have, or are; two or more = **MFA**); failure → **401 Unauthorized** (really "unauthenticated"). **Authorization (AuthZ)** decides what you may do; failure → **403 Forbidden**. Models: **ACL** (per-resource list of who may do what; fine-grained, hard at scale), **RBAC** (permissions via roles; simple, auditable, most common), **ABAC** (rules over user, resource and context attributes; flexible, harder to reason about), **ReBAC** (permissions follow graph relationships; Google Zanzibar). Always check authorization on the server for every request.

→ Full explanation

Sessions, cookies, tokens and JWT

Session-based authentication: after login the server stores a **session** (Redis/DB) and gives the browser a random **session ID** in a **cookie** — instantly revocable, but a store lookup per request. **Token-based authentication**: the client sends a token, usually `Authorization: Bearer <token>`; **opaque tokens** are looked up, **self-contained tokens** carry signed claims. A **JWT** ("jot") is `header.payload.signature`, Base64URL-encoded, signed with HS256 (shared secret, HMAC) or RS256 (RSA key pair), verified locally by any service via signature and `exp` — ideal for microservices, but **signed, not encrypted** (no secrets inside) and hard to revoke. Use short-lived **access tokens** (5–15 min) plus a carefully stored **refresh token** checked against a DB. Cookie flags: `HttpOnly` (no JS access, anti-XSS), `Secure` (HTTPS only), `SameSite=Lax/Strict` (anti-CSRF). Reject `alg: none`; prefer HttpOnly cookies over `localStorage` in browsers.

→ Full explanation

OAuth 2.0, OpenID Connect, SSO and API keys

OAuth 2.0 is **delegated authorization**: an app gets a scoped token to your data elsewhere without your password. Roles: **resource owner** (user), **client** (app), **authorization server** (issues tokens after consent), **resource server** (API checking token and **scopes**). **Authorization code flow with PKCE** (recommended): redirect to the authorization server with client ID, scopes, redirect URL and a hashed one-time secret → user logs in and consents → short-lived single-use **authorization code** returned → client exchanges code + PKCE secret (and client secret for server apps) server-to-server → **access token** (+ refresh token) → API calls with Bearer token. Other flows: **client credentials** (machine-to-machine), **device code** (TVs); implicit and password flows are discouraged. **OpenID Connect (OIDC)** adds authentication via a signed **ID token** (JWT) — "Sign in with Google". **SSO** reuses one identity-provider login across apps (Okta, Entra ID; OIDC or XML-based **SAML**). An **API key** identifies a calling program (header such as `X-API-Key`): simple for metering and rate limits, but a bearer secret without built-in expiry or user identity — never commit keys to Git. Don't treat OAuth access tokens as identity proof.

→ Full explanation

Encryption, hashing, passwords and secrets

Encryption is reversible with a key (**decrypt** to recover it): **symmetric encryption** (one key, AES; fast, bulk data) or **asymmetric** (public key encrypts/verifies, private key decrypts/signs; RSA, elliptic-curve; key exchange and signatures). **In transit** = TLS (mTLS between services); **at rest** = disks, DBs, backups, typically AES-256 with keys from a **KMS** (key management service). **Hashing** makes an irreversible fixed-size **fingerprint** (**hash/digest**) (SHA-256 = 256 bits) for matching, integrity and dedup. Passwords: never plain or encrypted; store a per-user random **salt** (defeats rainbow tables) with a deliberately slow hash — **bcrypt** (cost factor), **scrypt** or **Argon2id** (memory-hard; 2015 Password Hashing Competition winner's variant, OWASP's first choice) — tuned to tens–hundreds of ms; fast hashes allow billions of GPU guesses per second. **Envelope encryption**: data keys encrypt data and are themselves encrypted by a master key that never leaves the KMS. **Secrets management**: keep credentials out of code, images and logs, in Vault / AWS Secrets Manager / Google Secret Manager / KMS-backed Kubernetes Secrets, scoped per service and **rotated** automatically. Base64 is encoding, not encryption; never roll your own crypto.

→ Full explanation

Network security: firewalls, WAF and DDoS

A **firewall** filters by IP, port and protocol (L3/L4; cloud **security groups** and network ACLs). A **web application firewall (WAF)** inspects HTTP (L7) for SQL injection, XSS, bad bots and floods (AWS WAF, Cloudflare, ModSecurity with the OWASP Core Rule Set). **Network segmentation** keeps databases in **private subnets**, reachable only via LBs or a **bastion host**. A **DDoS** attack floods a service from many machines (often a **botnet**): **volumetric** (UDP floods, **amplification** via open DNS/memcached), **protocol** (**SYN flood**), **application layer** (expensive HTTP requests). Mitigate by absorbing at scale (anycast scrubbing: Cloudflare, AWS Shield, Cloud Armor, Akamai), edge filtering and SYN cookies, rate limiting and challenges, caching with hidden origin IPs, and capped autoscaling. Records: ~398 million HTTP requests/s (HTTP/2 "Rapid Reset", disclosed by Google October 2023); Mirai (2016) took down Dyn.

→ Full explanation

Common web vulnerabilities: CORS, CSRF, XSS and SQL injection

Same-origin policy stops a page from one **origin** (scheme + domain + port) reading another's responses; **CORS** lets a server opt in (`Access-Control-Allow-Origin`, `OPTIONS` **preflight**) — a browser relaxation, not server protection; never `*` with credentials. **CSRF** tricks a logged-in browser into sending a forged request with its cookies; fix with `SameSite` cookies, anti-CSRF tokens and no state changes on GET. **XSS** runs attacker script inside your site; fix with output escaping (React does by default), a **Content Security Policy (CSP)** and `HttpOnly` cookies. **SQL injection** splices input into SQL (`' OR '1'='1`); fix

with **parameterised queries**. The **OWASP Top 10** lists the most critical web risks; broken access control tops recent editions.

→ Full explanation

Least privilege and zero trust

Least privilege: each user, service and process gets only the permissions it needs, only for as long as needed. **Defence in depth**: independent layers (firewall, WAF, AuthN, AuthZ, encryption, monitoring). **Zero trust**: trust no request because it is "inside" (unlike the castle-and-moat **perimeter model** with a **VPN**) — authenticate user and device, authorise, encrypt (mTLS) and log every request, limiting **lateral movement**. Practice: narrow per-service **IAM** roles, no shared admin accounts, expiring just-in-time access, private subnets, device checks, audit logs. Google **BeyondCorp** (from 2014) based access on user and device identity rather than network location.

→ Full explanation

Cost awareness

Cost drivers: compute, storage (per GB-month, cheaper cold tiers), managed databases and **data transfer** — ingress usually free, **egress** to the internet ~$0.05–0.09/GB on AWS, cross-AZ ~$0.01/GB each way (why CDNs matter for video). Pricing: **on-demand**, **reserved instances / savings plans** (1–3-year commitments), **spot/preemptible** (up to ~90% off, reclaimable; batch jobs). Levers: autoscale and stop idle environments, right-size, cache, lifecycle old data to cold tiers, sample logs and traces, compare serverless vs always-on at real traffic. Every nine, region and lower RPO costs money; engineering time too. **FinOps** makes teams see and own their spend. In interviews name your design's dominant cost (video: egress; logging: storage) and one reduction.

→ Full explanation

Part 5 glossary

Every Part 5 term is defined in bold above. Extra one-liners:

Term	Meaning
Strangler fig pattern	Migrate by routing features one at a time from the old system to new services
Transient fault	A temporary failure likely to succeed on retry
Goodput	Useful completed work, excluding wasted or timed-out work
Lateral movement	An attacker moving from one compromised system to others

→ Full explanation

Part 6 — Putting it together

Parts 1–5 examined each piece alone; this part reassembles them: one request's full journey, a worked URL-shortener design, a decision cheat sheet, the 60 most common beginner interview questions with short answers, and an A–Z index of terms.

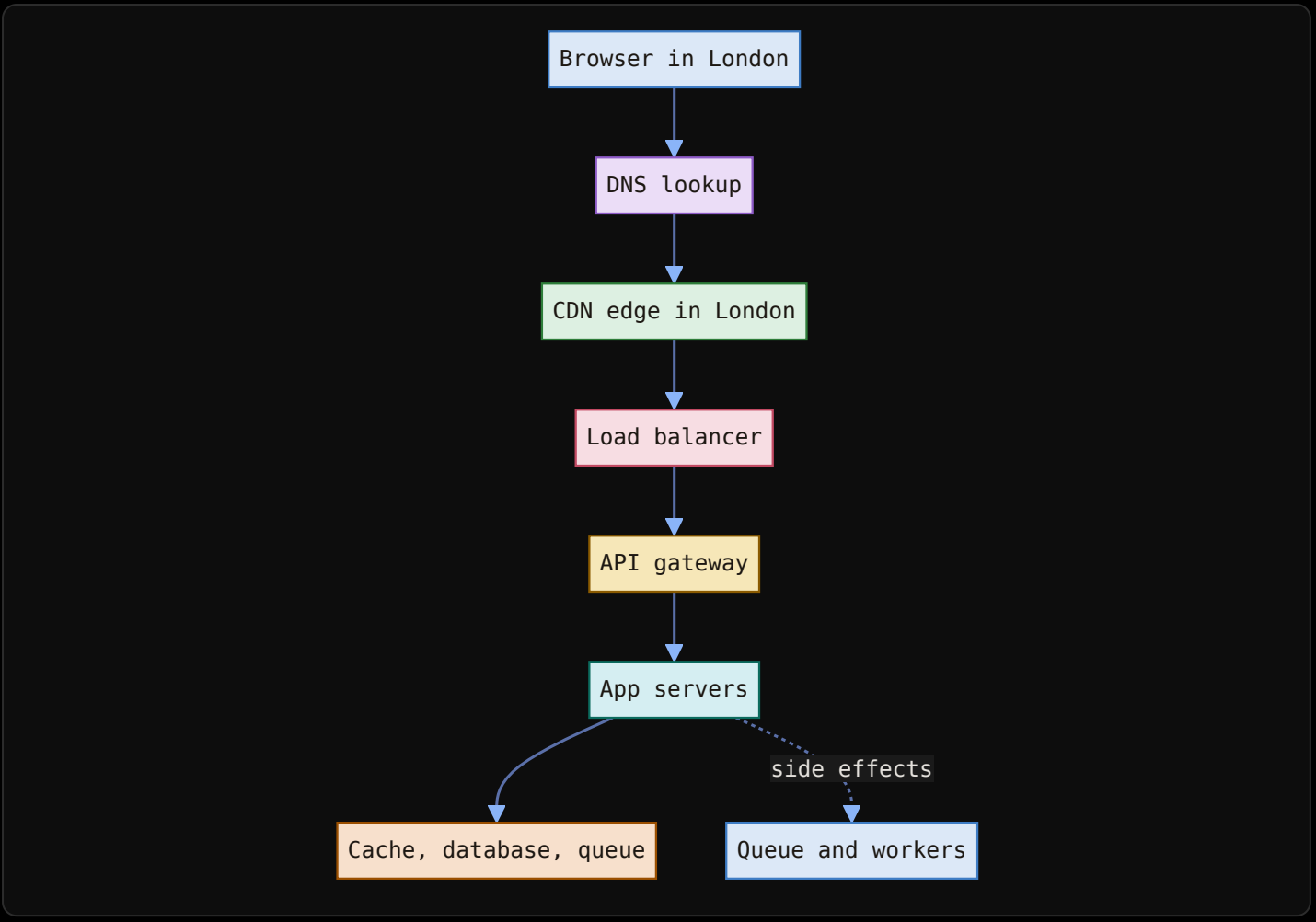
> **Part 6 TL;DR:** One click goes browser cache → DNS → TCP/TLS → CDN → load balancer → API gateway → stateless app server → cache → database and back, with side effects pushed through a queue; every hop costs time and can fail, so cache it, duplicate it or take it off the critical path. Every "reach for X" carries a "watch out for Y" — say both.

→ Full explanation

A request's journey

A **request's journey** is the full path one user action takes through a system — the classic "what happens when you type a URL and press Enter?" warm-up, showing where time goes and where things break.

The map



1. **Browser** splits the URL into **scheme** (`https`), **host** and **path**, and checks its **browser cache**.
2. **DNS** (0–50 ms): recursive resolver, root, TLD, authoritative; a CDN CNAME returns a nearby edge (GeoDNS/anycast).
3. **TCP** handshake (1 RTT; 5–10 ms to a London edge vs ~70 ms to New York).
4. **TLS** 1.3 handshake (1 RTT; **0-RTT resumption** for repeat visits; HTTP/3 merges 3 and 4).
5. **HTTP request** (`GET /feed`, cookie carrying the session — HTTP is stateless).
6. **CDN edge** serves static hits; personal `/feed` is forwarded to the origin (DDoS absorption, WAF here).
7. **Load balancer** (L7, TLS termination, health checks, algorithm; itself redundant).
8. **API gateway** authenticates (session/JWT), rate-limits (429), routes.
9. **Stateless app server** in containers on Kubernetes, autoscaled, with service discovery and timeouts.
10. **Cache** (cache-aside, ~0.5 ms; 90–99% hits).
11. On a miss, **database**: read replica, index, shard key, connection pool (2–20 ms).
12. **Fill the cache** with a TTL — readers see eventual consistency.
13. **Response** returns through gateway, LB and edge (compressed); the browser renders and fetches images from the CDN.
14. **Writes** (`POST` like) commit to the primary and publish an event; idempotent **workers** handle notifications and counters (at-least-once).
15. **Observability**: logs, metrics, trace ID; SLO alerts page on-call.

Where the time goes

Step	Typical	Shrink with
DNS	0–50 ms	Caching, long TTL
TCP + TLS	10–20 ms to nearby edge	CDN, HTTP/3, keep-alive
Edge → origin	5–80 ms	Origin in user's region
Gateway + app	5–20 ms	Fewer hops, parallel calls
Cache / DB	0.5 ms / 5–20 ms	Hit ratio, indexes
Rendering	50–200 ms	Smaller pages

On a first visit, network round trips often cost more than server work; repeat visits reuse DNS, connections and cached files.

What can go wrong at each hop

Hop	Failure	Protection
DNS	Provider outage	Two providers, sensible TTLs
CDN	Edge outage, stale content	Multi-CDN, invalidation
Load balancer	Node dies	Redundant pair or managed LB
App server	Crash, bad deploy	Health checks, canaries, rollback
Cache	Node dies, stampede	Replicas, request coalescing
Database	Primary dies	Replica failover, backups
Queue/worker	Crash mid-job	At-least-once, retries, DLQ
Any call	Slow dependency	Timeouts, circuit breaker, fallback

→ Full explanation

Worked example: designing a URL shortener

A **URL shortener** maps a short code (`sho.rt/aZ3kQ9b`) to a long URL and **redirects** (Bitly, TinyURL) — a classic 45-minute problem.

Step 1 — Requirements

Functional: create a short link; redirect; optional **custom alias**, expiry, click counts. Non-functional: 99.99% redirect availability, p99 < 50 ms, codes not guessable in sequence, links never lost, read-heavy (assume 100 reads per write). Out of scope: accounts, dashboards, previews.

Step 2 — Estimates (back of the envelope)

100 million links/month ÷ 2.6 million s ≈ 40 writes/s; × 100 = 4,000 reads/s, ~12,000 at 3× peak; 6 billion links in 5 years × ~500 B ≈ 3 TB; read bandwidth ~2 MB/s. **Base62** (a–z, A–Z, 0–9): $62^6 \approx 56.8$ billion, $62^7 \approx 3.5$ trillion → 7 characters (500× headroom). Cache: ~350 million redirects/day over ~50 million distinct links; the hottest 20% (10 million × 500 B) ≈ 5 GB fits one cache node (the **80/20 rule**).

Step 3 — API

`POST /api/v1/links` (`long_url`, optional `custom_alias`, `expires_at`; `Idempotency-Key` header) → `201 Created` with the code; `GET /{code}` → redirect with `Location`. Versioned path, rate-limited creation (429), `404` for unknown and `410 Gone` for expired codes. **301 Moved Permanently** lets browsers cache the redirect

(less load, but lost click counts and no expiry); **302 Found** re-checks every time — choose 302 if analytics or expiry matter.

Step 4 — Data model

One table: `code` (primary key), `long_url`, `owner_id` (nullable), `created_at`, `expires_at`. Every read is one primary-key lookup, so PostgreSQL or a key-value store (DynamoDB, Cassandra) works. Don't increment a click counter in this row on each redirect.

Step 5 — Generating the short code

Approach	Pros	Cons
Hash the URL (7 base62 chars of MD5/SHA-256)	Same URL → same code	**Hash collisions** must be checked and retried
Random code	Simple, unguessable	Collision checks
Counter + base62	No collisions	Bottleneck, guessable
Key generation service (KGS)	Fast, no collisions	Another service

Recommended: **counter ranges** handed to each server by a coordination store (DB row, ZooKeeper/etcd), counted in memory (crashed ranges are simply skipped), **scrambled** with a reversible mixing function, then base62-encoded. Snowflake IDs need no coordination but give 11 base62 characters. Custom aliases are inserted directly; a duplicate key → **409 Conflict**.

Step 6 — High-level design

Client → CDN/DNS → load balancer → stateless link service → Redis, database (primary + replicas), click-event queue → analytics workers. **Redirect path**: Redis (cache-aside) → on miss a replica, cached with a ~24 h TTL → check expiry → publish click event asynchronously → 302. **Create path**: gateway auth and rate limit → validate URL (malware blocklist) → next number, scramble, encode → insert into primary → 201.

Step 7 — Deep dives

Caching: 90%+ hits turn 12,000 reads/s into ~1,200 DB reads; links rarely change; **request coalescing** prevents stampedes; the CDN can cache redirects without analytics. **Scaling**: shard by hash of `code` (every query knows it) with consistent hashing. **Availability**: two+ copies per tier across AZs; redirects keep working from replicas and cache while a primary fails over (graceful degradation). **Analytics**: clicks to Kafka, batched per minute into a column store (ClickHouse); de-duplicate by event ID. **Abuse**: rate limits, malware checks, disabling links.

Trade-offs we made

302 over 301 (more load, accurate analytics and expiry); counter ranges over hashing (no collision checks, small coordination store); eventual consistency for click counts; long cache TTLs (links effectively immutable). Don't skip estimates, ignore hash collisions, write on the redirect path, or open with 50 shards for 40 writes/s.

→ Full explanation

Decision cheat sheet

How to use it

Read the left column as "the interviewer just told me…", name the tool, then say why and what it costs — the "watch out for" earns the marks.

The table

If you need…	Reach for…	Watch out for…
Faster repeated reads	Cache (Redis), cache-aside	Staleness, invalidation, stampedes
Fast static files worldwide	CDN	Invalidation delay, cost per GB
Low latency far away	CDN, edge, multi-region	Cross-region consistency
More stateless capacity	Horizontal scaling behind an LB	Move session state out
Quick bump, small system	Vertical scaling	Ceiling, still one machine
Spread traffic	Load balancer (L7 HTTP, L4 TCP)	LB redundancy
Daily traffic swings	Autoscaling	Start-up time, right metric
One entry point for many services	API gateway	Bottleneck, "god" layer
Strong consistency, transactions	Relational DB	Scaling writes past one primary
Flexible nested records	Document store	Weaker joins/transactions
Key lookups at huge scale	Key-value store	Few query patterns
High write rate, time-ordered	Wide-column / LSM store	Query-driven tables, eventual consistency
Metrics over time	Time-series DB	Retention, downsampling
Friends of friends	Graph DB	Hard to shard
Full-text search	Search engine	It's a copy — keep in sync
Similarity on embeddings	Vector DB / pgvector	Approximate, memory hungry
Large files	Object storage	Not for small frequent updates
More DB read capacity	Read replicas	Lag, read-your-writes
More write capacity / data	Sharding with a good key	Hot spots, cross-shard queries, resharding
Change nodes without moving most data	Consistent hashing + vnodes	Some movement, balancing
Fast column queries	B-tree index	Slower writes, storage
Slow work off the request	Queue + workers	Duplicates, idempotency, DLQ
Many services react to one event	Pub/sub or stream (Kafka, SNS)	Ordering, schema changes
Replay, high-throughput logs	Kafka	Partitions cap parallelism
Safe payment retries	Idempotency key	Key storage and expiry
Absorb spikes	Queue buffer, backpressure	Growth hides overload
Stop abusive clients	Token-bucket rate limiting, 429	Fair limits, distributed counters
Failing dependency	Timeouts + circuit breaker + fallback	Thresholds, test the open state
Transient failures	Backoff + jitter retries	Retry storms; idempotent calls only
Contain failures	Bulkheads	Idle reserved capacity
Survive a data-centre loss	Multi-AZ, then multi-region	Cost, lag, split brain
Cross-service consistency	Saga with compensations	Interim states

If you need…	Reach for…	Watch out for…
Atomic commit across two DBs	2PC	Blocking coordinator, slow
One worker at a time	Lock with fencing tokens, leader election	Lease expiry mid-work
Agree on a value/leader	Raft via etcd/ZooKeeper	Majority needed, write latency
Uncoordinated unique IDs	UUIDv7 / Snowflake	UUIDv4 index locality, clocks
"Seen before?" cheaply	Bloom filter	False positives
Distinct counts	HyperLogLog	~1% error
Nearby places	Geohash / quadtree	Cell borders
Server → browser updates	SSE	One direction
Two-way real time	WebSockets	Stateful connections
Notify another company	Webhooks	Retries, signatures, duplicates
Fast internal calls	gRPC + Protobuf	Browser needs a proxy
Clients choose fields	GraphQL	Caching, query cost
Overnight analysis	Batch (Spark), warehouse	Hours-old results
Results in seconds	Stream processing (Flink)	State, correctness
Production visibility	Metrics, logs, traces, SLO alerts	Alert fatigue, log cost
Safe releases	Canary, blue-green, feature flags	Schema changes
One login, many apps	SSO with OIDC	Critical identity provider
Act for a user elsewhere	OAuth 2.0	Token storage, scopes
Stateless auth	JWT	Revocation
Store passwords	bcrypt/scrypt/Argon2	Never encrypt or fast-hash
Protect secrets	Secrets manager	Rotation, audit

→ Full explanation

Top 60 interview questions

One-line model answers; the section of the same name above has the detail.

Foundations and performance

1. **System design?** Choosing parts and how they fit to meet features, scale, speed and reliability — a trade-off exercise.
2. **Latency vs throughput?** Time per request vs requests per second; a cargo ship has huge throughput and high latency.
3. **Why p99, not average?** The mean hides slow requests; p99 shows the unlucky 1%, which fan-out makes common.
4. **Little's law?** L = λ × W: 1,000 req/s × 0.2 s = 200 in flight, sizing threads and pools.
5. **Estimate QPS?** DAU × actions ÷ ~10^5 s, × 2–3 for peak.
6. **Four nines?** 99.99%: ~52.6 min/year, ~4.4 min/month down.
7. **SLA vs SLO vs SLI?** Measurement, internal target, customer contract (looser, with penalties).
8. **Error budget?** 1 − SLO; spend on releases, stop and fix reliability when used up.
9. **Vertical vs horizontal?** Bigger machine (simple, capped, SPOF) vs more machines (needs LB, statelessness).

10. **Stateless service?** No per-user memory between requests, so any instance serves any request.
11. **SPOF?** A part whose loss stops everything; remove with redundancy, health checks, automatic failover.
12. **Reliability/availability/durability?** Correct over time / answering now / saved data survives (S3: eleven nines durability).
13. **Interview approach?** Requirements, estimates, API, data model, high-level design, deep dives, trade-offs.

Networking and communication

14. **How does DNS work?** Resolver → root → TLD → authoritative, each answer cached for its TTL.
15. **TCP vs UDP?** Reliable ordered connection with handshake vs connectionless datagrams for real-time.
16. **HTTP/1.1 → 2 → 3?** One request per connection → multiplexed streams over TCP → streams over QUIC without TCP HOL blocking.
17. **What HTTPS adds?** TLS encryption, integrity and server authentication via certificates.
18. **Idempotent methods?** GET, HEAD, PUT, DELETE, OPTIONS — safe to retry; POST needs an idempotency key.
19. **REST vs gRPC vs GraphQL?** Public CRUD and caching / internal typed streaming / clients picking fields.
20. **Real-time to browsers?** Polling, long polling, SSE (one-way), WebSockets (two-way).
21. **Load balancer?** Spreads traffic over healthy servers for scale and availability.
22. **L4 vs L7?** Connections by IP/port vs HTTP-aware per-request routing.
23. **LB algorithms?** Round robin, weighted, least connections, least response time, IP/consistent hash, random, P2C.
24. **Sticky sessions?** Same user → same server; avoided because they unbalance load and lose sessions.
25. **CDN?** Edge caches near users cut latency and origin load.
26. **Forward vs reverse proxy?** Acts for clients vs acts for servers.
27. **API gateway?** Front door doing auth, rate limits, routing, transformation.
28. **Offset or cursor pagination?** Offset for small admin lists; cursor for large or changing feeds.

Data storage

29. **SQL or NoSQL?** By access pattern and consistency; SQL by default.
30. **ACID?** Atomicity, consistency, isolation, durability.
31. **Index cost?** Fast lookups for slower writes, space and memory.
32. **Replication vs sharding?** Copies for reads and availability vs splitting for writes and size.
33. **Sync vs async replication?** No loss but slower writes vs fast but may lose recent writes.
34. **Replication lag?** Followers behind the leader; users may not see their own writes.
35. **Shard key?** High cardinality, even load, in the main query, not monotonic for ranges.
36. **Consistent hashing?** Ring placement so membership changes move ~1/N keys.
37. **Normalize or denormalize?** Normalize first; denormalize measured hot reads.
38. **Object storage?** For large files, backups, data lakes; keep keys in the DB.
39. **OLTP vs OLAP?** Small transactional queries on row stores vs big scans on column stores.
40. **N+1 problem?** One query per list item; fix with eager loading or batching.

Caching, async and consistency

41. **Cache-aside?** Read cache, on miss load DB and fill; on write update DB and delete key.
42. **Write-through vs write-back?** Synchronous DB write (fresh, slower) vs deferred (fast, can lose data).
43. **LRU?** Evict the least recently used entry.
44. **Cache stampede?** Many misses on one expired hot key; prevent with single flight, stale-while-revalidate, jitter.
45. **Why a queue?** Fast responses, decoupling, spike absorption, retries.
46. **Queue vs pub/sub?** One consumer per message vs every subscriber gets a copy.
47. **Delivery guarantees?** Maybe lost / maybe duplicated / effectively once via dedup.

48. **Idempotency?** Repeating has no extra effect; implement with idempotency keys stored transactionally.
49. **Dead-letter queue?** Where repeatedly failing messages go for inspection and redrive.
50. **Strong vs eventual?** Reads see the latest write vs converge eventually.
51. **CAP?** During a partition choose consistency or availability.
52. **Quorum?** R + W > N replicas so reads overlap writes.
53. **Why Raft?** Majority agreement on leader and log despite failures, preventing split brain.
54. **2PC or saga?** Saga across services; 2PC blocks and couples availability.

Architecture, reliability and security

55. **Monolith or microservices?** Modular monolith first; split for team or scaling needs.
56. **Circuit breaker?** Stops calling a failing dependency (closed, open, half-open).
57. **Jitter?** Desynchronises retries so they don't arrive in waves.
58. **Token bucket?** B tokens refilled at R/s, one per request; allows bursts.
59. **AuthN vs AuthZ?** Who you are (401) vs what you may do (403).
60. **RPO vs RTO?** Data you may lose vs time you may be down.

→ Full explanation

Master A–Z glossary

Every term below is defined in bold in Parts 1–6 above (search the page for it); use this list as a self-test index.

What it is

.

A

A / AAAA record; ABAC; ACID; Active-active; Active-passive; Active / passive health check; Anycast; API; API gateway; API key; At-least-once / at-most-once delivery; Authentication; Authoritative name server; Authorization; Autoscaling; Availability; Availability zone (AZ).

B

Back-of-the-envelope estimate; Backpressure; Bandwidth; BASE; Base62; Batch processing; bcrypt; Block storage; Bloom filter; Blue-green deployment; B-tree; Bulkhead; Burn rate.

C

Cache; Cache-aside; Cache hit / miss; Cache invalidation; Cache stampede; Canary deployment; CAP theorem; Causal consistency; CDN; Certificate (TLS); Chaos engineering; CI/CD; Circuit breaker; CNAME record; Cold start; Column-oriented storage; Compensating action; Concurrency / parallelism; Connection draining; Connection pool; Consensus; Consistency (ACID); Consistency (distributed); Consistent hashing; Consumer; Consumer group; Container; Cookie; CORS; Count-min sketch; Covering index; CQRS; CSRF.

D

Data lake; Data warehouse; DAU; DDoS; Dead-letter queue (DLQ); Denormalization; Dirty / non-repeatable / phantom read; Disaster recovery (DR); Distributed cache; Distributed lock; Distributed system; DNS; Document database; Durability.

E

Elasticity; Encryption in transit / at rest; Error budget; Event-driven architecture; Event sourcing; Event streaming; Eventual consistency; Exactly-once; Exponential backoff.

F

FaaS; Failover; Fallback; Fault / failure; Fault tolerance; Feature flag; Federation; Fencing token; File storage; Firewall; Foreign key; Forward proxy; fsync; Functional requirement.

G

GeoDNS; Geohash; Global server load balancing (GSLB); Golden signals; Gossip protocol; Graceful degradation; Graph database; GraphQL; gRPC.

H

Hashing; Head-of-line blocking; Health check; Heartbeat; Horizontal scaling; Horizontal / vertical partitioning; Hot key / hot spot; HTTP; HTTP/2 / HTTP/3; HTTPS; HyperLogLog.

I

Idempotency; Idempotency key; Index; Infrastructure as code (IaC); In-process cache; Inverted index; IOPS; IP address; IP hash; Isolation level.

J

Jitter; JWT.

K

Kafka; Keep-alive; Key-value store; Kubernetes.

L

L4 / L7; Lamport clock; Latency; Leader election; Leader-follower replication; Leaderless replication; Leaky bucket; Least connections; Least privilege; Linearizability; Little's law; Load balancer; Load shedding; Long polling; Lost update; LRU / LFU; LSM-tree.

M

MapReduce; Merkle tree; Message queue; Microservices; Modular monolith; Monolith; Monotonic reads; MTBF; MTTR; Multi-leader replication; Multi-region.

N

N+1 query; NAT; Network partition; Nines; Node; Non-functional requirement; Normalization; NoSQL; NS record; NTP.

O

OAuth 2.0; Object storage; Observability; Offset (messaging); OLAP; OLTP; On-call; OpenID Connect (OIDC); Origin; Origin shield; ORM; OSI model.

P

p50 / p95 / p99 / p999; PACELC; Pagination; Partition; Paxos; Peak factor; Percentile; PKCE; Point of presence (PoP); Port; Power of two choices; Primary key; Private / public IP; Protobuf; Pub/sub; Push / pull CDN.

Q

QPS; Quadtree; QUIC; Quorum.

R

Raft; Rate limiting; RBAC; Reactive / scheduled / predictive scaling; Read replica; Read-through / write-through / write-around; Read-your-writes; Recursive resolver; Redis; RED method; Redundancy; Region; Relational database; Reliability; Replication; Replication lag; Resharding; Response time; REST; Reverse proxy; Rollback; Rolling deployment; Root server; Round robin; Round trip time (RTT); RPC; RPO; RTO.

S

Saga; Salt; Saturation; Scalability; Secrets management; Serverless; Server-Sent Events (SSE); Service discovery; Service mesh; Session; Sharding; Shard key; Sidecar; SLA; SLI; Sliding window; SLO; Snowflake ID; SOA; Split brain; SQL injection; SSO; Stateless / stateful service; Sticky session; Stream processing; Strong consistency; Synchronous / asynchronous; Synchronous replication.

T

Tail latency; TCP; Three-way handshake; Throttling; Throughput; Thundering herd; Timeout; Time-series database; TLD; TLS; TLS termination; Token bucket; Topic; Trace; Transaction; TTL; Two-phase commit (2PC).

U

UDP; USE method; UUID.

V

Vector clock; Vector database; Vertical scaling; Virtual machine (VM); Virtual node.

W

WAF; WAL; Webhook; WebSocket; Wide-column store; Worker; Write-back / write-behind; Write skew.

X, Y, Z

XSS; Zero trust; ZooKeeper.

→ Full explanation

Part 6 glossary

Term	Meaning
Short code	The few characters (e.g. `aZ3kQ9b`) identifying a short link
Counter range	Block of numbers handed to one server for ID generation
Hash collision	Two different inputs producing the same hash
Request coalescing	One request refills a cache entry while others wait for it
80/20 rule	Rough rule that 20% of items get 80% of traffic

→ Full explanation

Chapter 1 — Reliable, Scalable, and Maintainable Applications

Book: Chapter 1, pp. 3–26

Overview

Part I in brief (Foundations of Data Systems). Chapters 1–4 hold for every data system, on one machine or a cluster: Ch 1 sets the vocabulary (reliability, scalability, maintainability); Ch 2 compares data models and query languages; Ch 3 opens up storage engines (the part of a database that writes data to disk and reads it back), whose fit to the workload (mix of reads, writes and queries) can dominate performance; Ch 4 covers encoding (serialization: turning in-memory data into bytes) and schema evolution (a *schema* is the declared structure of the data). Part II then turns to distributed data.

What question does Chapter 1 answer? "What exactly do we mean when we say a data system is *good*?" Three goals, each given a measurable meaning:

- **Reliability**: keep working correctly even when things break.
- **Scalability**: have sensible options when load grows.
- **Maintainability**: let many people keep operating and changing the system productively for years.

Why it matters. Most modern apps are *data-intensive* (the pain is data volume, complexity or rate of change, not CPU) and are assembled from standard parts:

Need	Building block
Keep data so it can be found later	Database
Remember the result of a costly computation for fast reads	Cache
Search by keyword or filter	Search index
Hand a message to another process to handle asynchronously	Stream processing
Periodically crunch a big pile of accumulated data	Batch processing

There are many databases, caching styles and indexing approaches, each suited to different needs; picking and combining them well is the real skill.

Key terms

Term	Plain-English meaning
Data-intensive / compute-intensive	Main difficulty is data (amount, complexity, change speed) / CPU cycles.
Data system	Umbrella for databases, queues, caches, search indexes, etc., which increasingly overlap.
Reliability	Keeps doing the right thing at the needed performance despite faults.
Fault vs failure	One component deviating from its spec vs the whole system no longer serving users.
Fault-tolerant / resilient	Able to anticipate and cope with certain kinds of faults.
Chaos Monkey	Netflix tool that deliberately kills processes to exercise fault handling.
MTTF (mean time to failure)	Average time before a component fails; hard disks about 10–50 years.
RAID	Several disks arranged so one can fail without data loss.
Rolling upgrade	Upgrading one node at a time so the system never goes down.
Systematic error	A fault correlated across many nodes, e.g. a shared software bug.
Cascading failure	A small fault in one part triggers faults in others.
Telemetry	Detailed monitoring data (metrics, error rates) about a running system.
Scalability	Ability to cope with increased load: "what are our options if load grows like X?"
Load parameters	A few numbers describing current load (requests/sec, read/write ratio, active users, cache hit rate…).
Fan-out	How many further requests (or writes) one incoming request triggers.
Throughput	Records processed per second, or job time for a dataset of some size.
Response time / service time / latency	What the client sees (send to reply) / time actually processing / time waiting to be handled.
Percentile (p50, p95, p99, p999)	Value below which that percentage of measurements fall; p50 = median.
Tail latency	High percentiles of response time (slow outliers).
SLO / SLA	Service level objective / agreement: targets or contracts for performance and availability.
Head-of-line blocking	Slow requests at the front of a queue delay fast ones behind them.
Tail latency amplification	One user request needing many backend calls makes it likelier that at least one is slow.
Scaling up (vertical) / out (horizontal, shared-nothing)	Bigger machine / many smaller machines not sharing memory or disk.
Elastic system	Automatically adds resources when load rises.
Stateless / stateful service	Keeps no durable data / owns data that must be preserved.
Maintainability: operability, simplicity, evolvability	Easy for ops to run / for newcomers to understand / to adapt to new requirements (also extensibility, modifiability, plasticity).
Big ball of mud	A project drowning in complexity.
Accidental complexity	Complexity from the implementation, not from the problem users need solved.

Term	Plain-English meaning
Abstraction	A clean, reusable interface hiding implementation detail.
Functional vs nonfunctional requirements	What the system must do vs general properties (security, reliability, scalability…).

Thinking About Data Systems

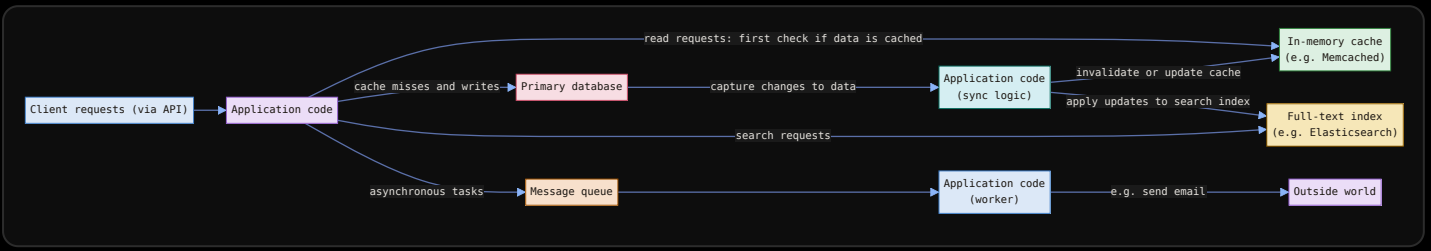
What it is. The book lumps databases, queues and caches together as **data systems**, even though they are accessed, perform and are built very differently. Two reasons:

1. **The categories are blurring.** Redis is an in-memory key-value datastore also used as a message queue; Apache Kafka is a message queue with database-like durability guarantees (written data survives crashes).
2. **One tool is rarely enough.** Work is split into tasks, each handled by a specific tool, and *your application code* stitches them together.

Example. An app with its own caching layer (e.g. Memcached) and a full-text search server (Elasticsearch or Solr: finds documents by any words they contain) beside the main database. Usually the application must keep cache and index in step with the database.

Key term — cache, hit, miss, invalidation: a **cache** is a fast copy of data whose authoritative home is slower (here, the database). A read found in the cache is a **hit**; one that must go to the database is a **miss**. When the underlying data changes, the stale copy must be **invalidated** (thrown away) or updated, or readers see old values; getting this right is famously tricky. See System Design Fundamentals.

Figure 1-1 (redrawn): one possible architecture combining several data components behind an application.



Clients only see the service's API. Behind it you have built a **new, special-purpose data system** from general-purpose parts, which may promise things like "the cache is correctly invalidated or updated on every write." You are now a **data system designer**, not just an app developer.

Designer's questions: How do we keep data correct and complete when parts break? How do we give steady performance when parts are degraded? How do we handle growing load? What should the API look like? Situational factors (team skills, legacy dependencies, deadlines, risk appetite, regulation) also matter, but the book focuses on three concerns that matter for nearly all software:

Concern	Short definition
Reliability	Keeps working correctly (right function, acceptable performance) despite hardware faults, software faults, even human mistakes.
Scalability	As data volume, traffic, or complexity grows, there are reasonable ways to cope.
Maintainability	Over time many people (engineering and operations) can work on it productively, both keeping it running and adapting it.

Reliability

What it is. Users expect software to do what they expected, tolerate their mistakes or odd usage, perform well enough at the expected load and data volume, and block unauthorised access and abuse. Reliability roughly means **continuing to work correctly when things go wrong**.

Faults vs failures.

	Fault	Failure
Scope	One component	The whole system
Meaning	A part deviates from its spec	Users no longer get the required service
Goal	Can never be driven to zero probability	Prevent faults from turning into failures

Systems that cope with faults are **fault-tolerant** or **resilient**. No system survives every fault (the book's joke: surviving a black hole swallowing Earth would need hosting in space), so we always mean tolerating *specific kinds* of faults, building reliable systems out of unreliable parts.

Deliberately causing faults. Randomly killing processes (the **Netflix Chaos Monkey**) constantly exercises recovery machinery; many serious bugs come from poor error handling, so this raises confidence that real faults will be handled.

Prevention vs cure. Tolerating faults usually beats preventing them, but some faults have no cure. **Security** is the main case: stolen data cannot be un-stolen. The book deals mostly with curable faults.

Hardware Faults

What they are. Disks crash, RAM goes bad, power blacks out, someone unplugs the wrong cable; in a large datacenter this happens constantly.

The numbers. Hard disks have an MTTF of roughly **10 to 50 years**, so a cluster of **10,000 disks** should expect about **one disk failure per day**.

First response: hardware redundancy. **RAID** disks, **dual power supplies**, **hot-swappable CPUs** (replaceable while running), datacenter **batteries and diesel generators**. When one part dies, the redundant one takes over. This cannot stop every failure, but it is well understood and often keeps one machine up for years.

Why that stopped being enough.

- Redundancy made losing a whole machine rare, and restoring a backup onto a new machine was usually quick enough; few apps truly needed multiple machines for high availability.
- Now more apps use **many machines**, which raises the total rate of hardware faults proportionally.
- On cloud platforms such as **AWS**, virtual machine instances can vanish without warning, because the platforms favour **flexibility and elasticity** over single-machine reliability.

The shift. Systems increasingly use **software fault tolerance** to survive losing a *whole machine*, instead of or alongside hardware redundancy. Bonus: a single-server system needs planned downtime to reboot (e.g. for OS security patches), whereas a node-loss-tolerant system can be patched **node by node** with no overall downtime: a **rolling upgrade** (Chapter 4).

Software Errors

How they differ. Hardware faults are usually **random and independent** (one disk dying does not make another die, apart from weak correlations like rack temperature). **Systematic errors** are **correlated across nodes**, harder to predict, and cause far more system failures. Examples:

- A bug that crashes *every* app server instance on a particular bad input. Real case: the **leap second on June 30, 2012** made many applications hang simultaneously because of a **Linux kernel** bug.
- A runaway process exhausting a shared resource (memory, CPU, network bandwidth, disk).
- A dependency (service or library) that slows down, stops responding, or returns corrupted data.
- **Cascading failures**: a minor fault in one part triggers faults in others, and so on.

Why they hide. Such bugs lie dormant until an unusual circumstance exposes an assumption about the environment that was *usually* true and one day is not.

What helps (no silver bullet): think carefully about assumptions and interactions; test thoroughly; isolate processes; let processes crash and restart; measure, monitor and analyse production behaviour; and if the system promises a guarantee, have it **check itself continuously** and alert on discrepancies (e.g. a message queue checking messages in = messages out).

Human Errors

The problem. Humans design, build and operate systems and are unreliable. One study of large internet services found **operator configuration errors were the leading cause of outages**, while hardware faults (servers or network) played a role in only **10–25%**.

How the best systems cope (combining approaches):

Approach	What it means in practice
Design to minimise error opportunities	Good abstractions, APIs and admin interfaces make the right thing easy and the wrong thing hard; too restrictive and people work around them.
Decouple mistake-prone places from failure-causing places	Full-featured **non-production sandbox** environments with real data, where people can experiment without affecting users.
Test at every level	Unit, whole-system integration, and manual tests; automated tests are great for rare corner cases.
Make recovery quick and easy	Fast rollback of configuration changes, **gradual rollouts** of new code so a bug hits few users, tools to **recompute data** if an old computation was wrong.
Detailed, clear monitoring	Performance metrics and error rates (**telemetry**, as in rocketry): early warnings, checking assumptions, diagnosis.
Good management and training	Important but outside the book's scope.

How Important Is Reliability?

Not only for nuclear plants and air traffic control: business-software bugs cost productivity (and wrong reported figures bring **legal risk**); **ecommerce** outages cost revenue and reputation; even a "noncritical" photo app holding a parent's only copies of their children's photos carries responsibility if its database is corrupted.

Sometimes we knowingly trade reliability for lower **development cost** (a prototype for an unproven market) or **operational cost** (razor-thin margins). Do it consciously, knowing where you cut corners.

Scalability

What it is. A system working today may not tomorrow, commonly because of **increased load**: concurrent users grow from 10,000 to 100,000, or 1 million to 10 million, or data volume grows. **Scalability** is how well a system handles growing load.

> **Common confusion:** scalability is *not* a yes/no label; "X scales" is meaningless. Ask instead: "If the system grows *in this particular way*, what are our options?" and "How do we add computing resources to absorb the extra load?"

Describing Load

What it is. First describe today's load with a few **load parameters**, chosen to fit the architecture: requests/sec to a web server, database read/write ratio, simultaneously active users in a chat room, cache hit rate, etc. Sometimes the average matters; sometimes a few extreme cases are the bottleneck.

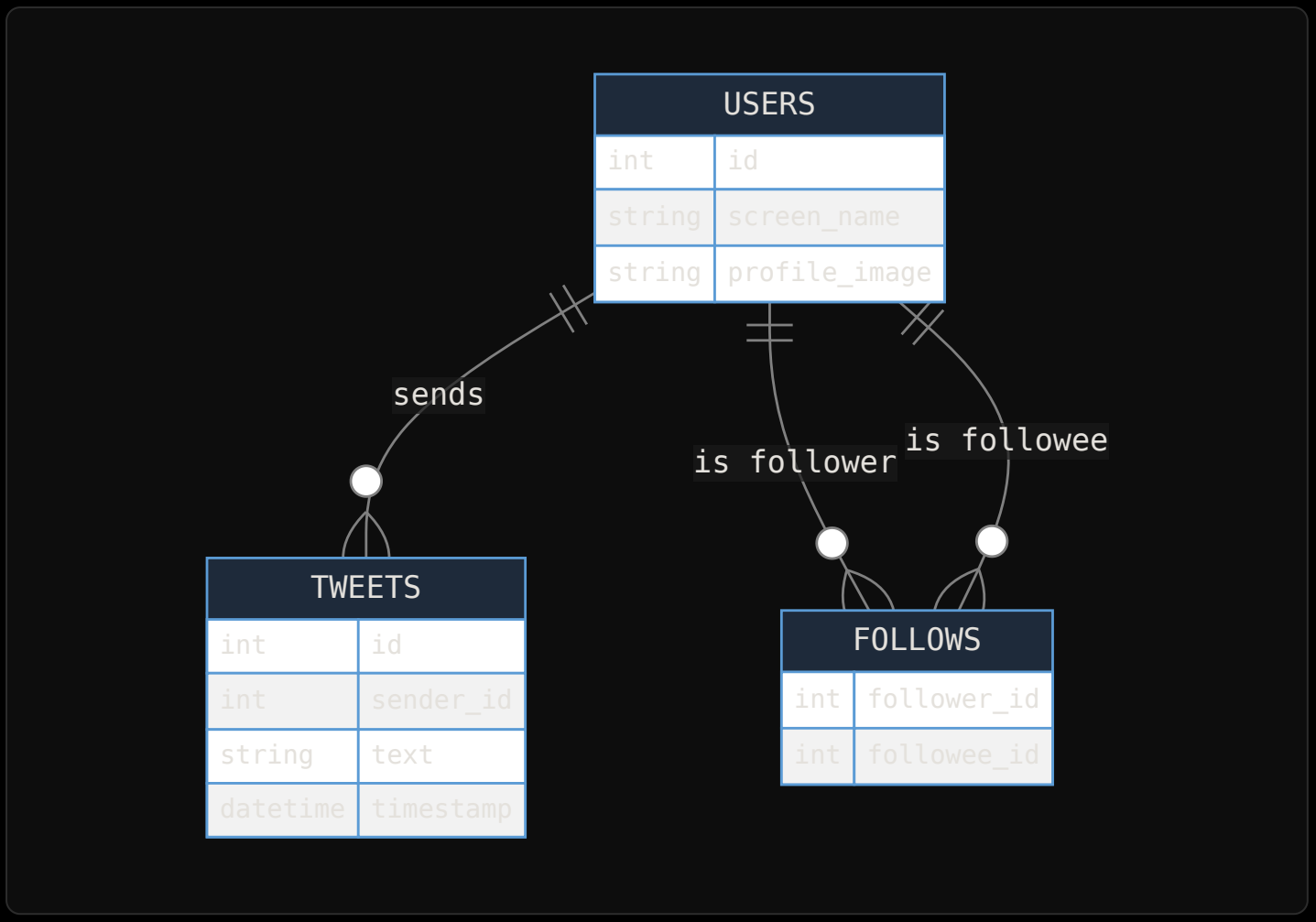
Worked example: Twitter (data from November 2012).

Operation	What it does	Load
Post tweet	Publish a message to your followers	**4.6k** requests/sec average, over **12k** at peak
Home timeline	View tweets from people you follow	**300k** requests/sec

12,000 writes/sec alone would be fairly easy. The challenge is **fan-out**: each user follows many people and is followed by many. (The term comes from electronics, where it counts how many logic-gate inputs one gate's output drives; in transaction processing it means how many requests to other services one incoming request causes.)

Approach 1 — compute on read. Posting inserts the tweet into one global collection. Reading a home timeline: look up everyone the user follows, fetch all their tweets, merge sorted by time.

Figure 1-2 (redrawn): a simple relational schema for a Twitter-style home timeline.



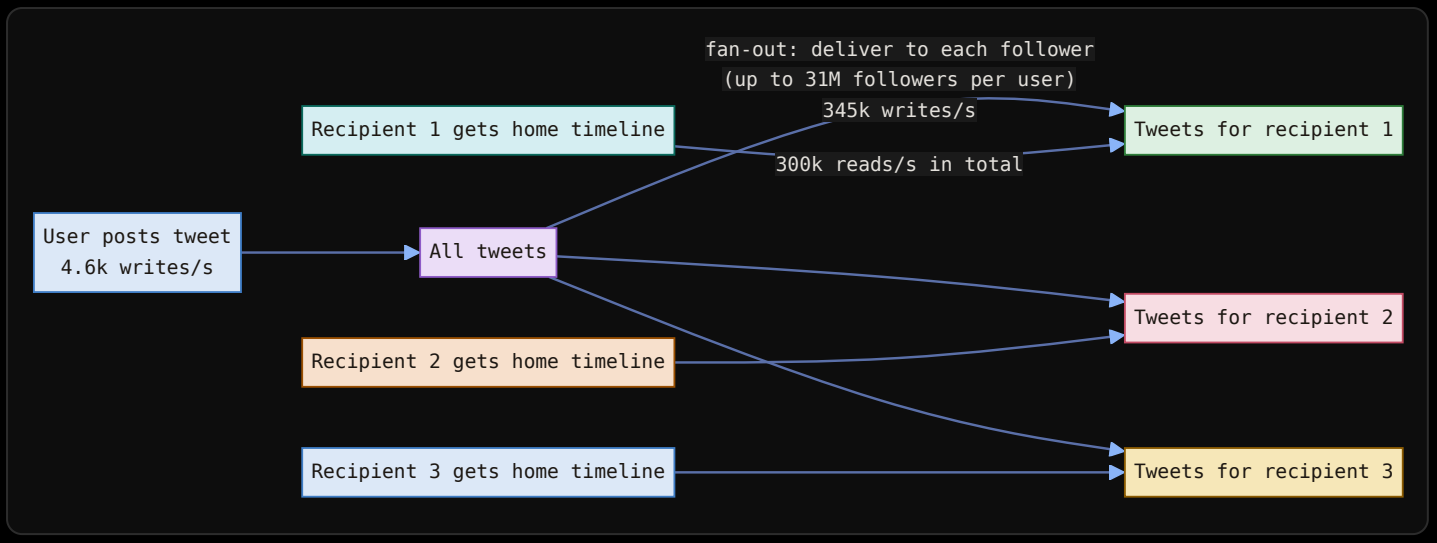
In SQL this is one join-heavy query (a *join* combines rows from tables sharing a value). The same shape for a recipe site where you subscribe to chefs:

```sql
SELECT recipes.*, chefs.display_name      -- the recipe plus who wrote it
FROM recipes
  JOIN chefs         ON recipes.author_id = chefs.id          -- attach author details
  JOIN subscriptions ON subscriptions.chef_id = chefs.id      -- keep chefs someone subscribes to
WHERE subscriptions.reader_id = :me                           -- ...and that someone is me
ORDER BY recipes.published_at DESC;                           -- newest first
```

All the work happens at **read time**.

Approach 2 — precompute on write. Keep a per-user home timeline cache, like a mailbox. On post: look up the poster's followers and insert the tweet into each follower's cache. Reading is now cheap because the result was built in advance.

Figure 1-3 (redrawn): Twitter's pipeline for delivering tweets to followers, with November 2012 load parameters.



Which is better? Twitter started with approach 1 but could not keep up with timeline reads, so it switched to approach 2. Tweets are published almost **two orders of magnitude** (~100x) less often than timelines are read, so shifting work to the write path pays off.

The downside of approach 2. Posting becomes expensive. An average tweet reaches about **75 followers**, turning **4.6k tweets/sec into 345k cache writes/sec**. But some users have **over 30 million followers**, so one tweet may trigger 30M+ writes, and Twitter aims to deliver tweets within **five seconds**.

Lesson. For Twitter, the **distribution of followers per user** (perhaps weighted by how often each tweets) is a key load parameter because it drives fan-out. Reason about your own app's load the same way.

The final twist: a hybrid. Most users' tweets are fanned out on write, but a few **celebrities** with huge follower counts are exempt: their tweets are fetched at read time and merged in, like approach 1. This gives consistently good performance. (Revisited in Chapter 12.)

	Approach 1: fan-out on read	Approach 2: fan-out on write	Hybrid
Cost of posting	Cheap (one insert)	Expensive (one write per follower)	Cheap for celebrities, per-follower for others
Cost of reading	Expensive (join + merge)	Cheap (read one cache)	Cheap cache read + merge in celebrity tweets
Pain point	Read rate is ~100x write rate	Users with 30M+ followers	More moving parts

Describing Performance

With load described, ask what happens as it grows: (1) **fixed resources** — raise a load parameter, keep CPU/memory/bandwidth the same: how does performance change? (2) **fixed performance** — how much must resources grow to keep performance unchanged? Both need performance numbers.

Throughput vs response time. Batch systems (e.g. Hadoop) care about **throughput**: records/sec, or total job time on a dataset of given size. **Online** systems (a client is waiting) care about **response time**: from sending a request to receiving the reply.

> **Aside: batch running time.** Ideally dataset size ÷ throughput; in practice longer because of **skew** (data unevenly spread across workers) and waiting for the slowest task.

> **Latency vs response time.** Often used as synonyms, but **response time** is what the client sees: **service time** (actual processing) plus network and queueing delays. **Latency** is only the time a

request spends *waiting* to be handled (latent).

Response time is a distribution, not a number. Identical requests still vary because of random extra delays: a context switch to a background process, a lost network packet that TCP must retransmit, a garbage-collection pause, a page fault forcing a disk read, even mechanical vibration in the rack. Some requests are also genuinely more expensive (more data).

Figure 1-4 (described): response times of 100 sample requests as bars: mostly short, a few tall outliers, with mean, median, p95 and p99 marked. A single average hides the spread; high percentiles reveal the outliers.

Why the mean is misleading. The arithmetic mean does not tell you how many users actually waited that long, so it poorly describes the *typical* experience.

Use percentiles instead. Sort response times fastest to slowest. The **median** (**p50**) is the halfway point: if p50 = 200 ms, half of requests finish faster, half slower — the *typical* wait. Higher percentiles (**p95, p99, p999**) show the outliers: if p95 = 1.5 s, 5 in 100 requests take 1.5 s or more.

Common confusion: the median is about a *single request*. A user making several requests (over a session, or several resources per page) has well over a 50% chance that *at least one* is slower than the median.

Why tail latencies matter.

- **Amazon** sets internal response-time requirements at the **99.9th percentile** (1 in 1,000 requests) because the slowest requests often come from customers with the most data — those who bought the most, i.e. the **most valuable customers**.
- Amazon observed **100 ms** more response time cuts sales by **1%**; others report a **1-second** slowdown lowers a customer satisfaction metric by **16%**.
- But optimising the **99.99th percentile** was judged too costly at Amazon: very high percentiles are dominated by uncontrollable random events, with diminishing returns.

SLOs and SLAs. Percentiles appear in **service level objectives** (internal targets) and **service level agreements** (contracts). Example SLA: "up" means median response time below **200 ms** and p99 below **1 s** (slower counts as down), met at least **99.9%** of the time; clients can claim refunds if missed.

Queueing delays and head-of-line blocking. A server handles only a few things at once (e.g. limited by CPU cores); requests arriving while busy wait in a **queue**, and a few slow ones hold up everything behind them (**head-of-line blocking**). Later requests may be fast to process yet look slow to the client. Therefore **measure response time on the client side**.

Load-testing pitfall. The load-generating client must keep sending requests **regardless of response time**. If it waits for each reply first, queues stay artificially short and results are skewed.

Percentiles in practice (sidebar).

- **Tail latency amplification:** a request calling several backends (even in parallel) waits for the *slowest* call, so a small fraction of slow backend calls makes a larger fraction of user requests slow.
- **Dashboards:** e.g. keep a rolling 10-minute window of response times and every minute compute and plot the median and other percentiles.
- The naive way sorts all values in the window every minute; cheaper probabilistic algorithms estimate percentiles closely with little CPU and memory: **forward decay**, **t-digest**, **HdrHistogram**.
- **Never average percentiles** (to lower time resolution or merge machines) — mathematically meaningless. **Add up the histograms** (counts per value range, e.g. 0–10 ms, 10–20 ms) instead.

Figure 1-5 (redrawn): one end-user request fans out to seven backends (92, 76, 103, 143, 86, 487, 133 ms); the single 487 ms call sets the time for the whole request.

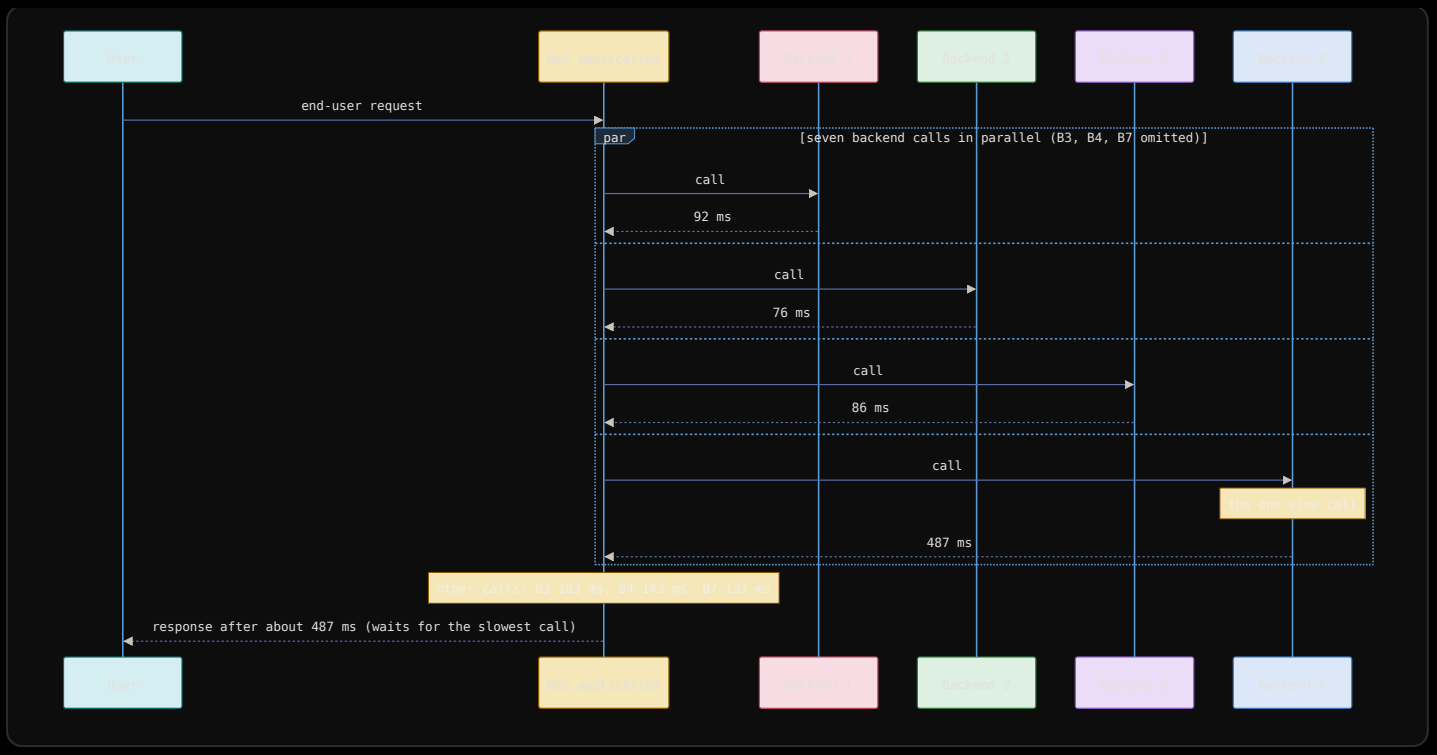


Illustration (our numbers, not the book's): if each call has a 1% chance of being slow and a page makes 100 independent calls, P(at least one slow) = 1 − 0.99^100 ≈ 63%.

Approaches for Coping with Load

Core idea. An architecture suited to one load level is unlikely to cope with **10x** that load; a fast-growing service will probably rethink its architecture at **every order of magnitude** of growth, or more often.

	Scaling up (vertical)	Scaling out (horizontal, shared-nothing)
How	Move to a more powerful machine	Spread load over many smaller machines
Pros	Simpler; everything on one machine	Can go beyond any single machine's limits
Cons	High-end machines get very expensive	More complexity, especially for stateful data

Good architectures mix both pragmatically; a few fairly powerful machines can be simpler and cheaper than many small VMs.

Elastic vs manual scaling. Elastic systems add resources automatically — useful for highly unpredictable load. **Manually scaled** systems rely on a human analysing capacity and adding machines — simpler, with fewer operational surprises (see "Rebalancing Partitions", Chapter 6).

Stateless vs stateful. Distributing **stateless** services is fairly easy; taking a **stateful** data system from one node to many adds a lot of complexity. Hence the conventional wisdom until recently: **keep your database on one node (scale up)** until cost or high-availability needs force distribution. As distributed tools improve, distributed data systems may become the default even for modest workloads.

No magic scaling sauce. Large-scale architectures are highly application-specific. The problem may be read volume, write volume, data volume, data complexity, response-time needs, access patterns (which data is read or written, how often, in what shape), or a mix. Example: **100,000 requests/sec of 1 kB each** needs a very different design from **3 requests/minute of 2 GB each**, despite the same data throughput.

Assumptions matter. A scalable architecture assumes which operations are common and which rare (the load parameters); wrong assumptions make scaling effort wasted or harmful. In an **early-stage startup or unproven product**, fast feature iteration usually matters more than scaling for imagined load. Still, scalable designs are built from **general-purpose building blocks in recurring patterns**, which the book teaches.

Maintainability

Why it matters. Most software cost comes **after** initial development: fixing bugs, keeping systems running, investigating failures, moving platforms, adapting to new use cases, repaying technical debt, adding features. Engineers dislike **legacy systems** (old code by people who left, outdated platforms, systems stretched beyond their design); each is painful in its own way, but we can design so maintenance hurts less and avoid creating legacy software ourselves.

Principle	Goal
Operability	Make it easy for operations to keep the system running smoothly.
Simplicity	Remove as much complexity as possible so newcomers can understand the system (internals, not a simple UI).
Evolvability	Make it easy to change the system later for unanticipated needs (extensibility, modifiability, plasticity).

As with reliability and scalability, there is no quick recipe; keep these principles in mind while designing.

Operability: Making Life Easy for Operations

A good operations team can often compensate for weak software, while excellent software runs unreliably under poor operations. Some work can be automated, but humans must set up and check the automation.

What a good operations team does: monitors health and quickly restores service; tracks down causes of failures or degraded performance; keeps platforms current, including security patches; understands how systems affect each other to catch risky changes; anticipates problems (e.g. **capacity planning**); establishes good deployment and configuration-management practices and tools; performs complex maintenance like platform migrations; keeps security intact as configuration changes; defines processes that make operations predictable; preserves organisational knowledge as people come and go.

What data systems can do to make routine tasks easy:

- Give **visibility** into runtime behaviour and internals via good monitoring.
- Support **automation** and integration with standard tools.
- **Avoid depending on individual machines** (any one can be taken down for maintenance).
- Provide good documentation and an easy **operational model** ("doing X leads to Y").
- Offer **good defaults** that admins can override.
- **Self-heal** where sensible, with manual admin control when needed.
- Behave **predictably**, with few surprises.

Simplicity: Managing Complexity

The problem. Large projects tend to become complex; complexity slows everyone and raises maintenance cost. A project stuck in it is a **big ball of mud**.

Symptoms: exploding state space, tightly coupled modules, tangled dependencies, inconsistent naming, performance hacks, special cases working around problems elsewhere.

Why it hurts. Budgets and schedules overrun, and changes are riskier because hidden assumptions, unintended consequences and surprising interactions slip by. So **simplicity should be a key goal**.

Accidental vs essential complexity. Simpler need not mean fewer features; it can mean removing **accidental complexity**, which Moseley and Marks define as complexity not inherent in the problem as users see it but arising only from the implementation.

Abstraction is the best tool. A good abstraction hides detail behind a clean façade and is reusable, so improvements to it benefit every user. Examples: **high-level languages** hide machine code, CPU registers and syscalls; **SQL** hides on-disk and in-memory data structures, concurrent requests from other clients, and inconsistencies left by crashes.

The catch. Good abstractions are hard to find. Distributed systems have many good algorithms but it is much less clear how to package them into abstractions; the book keeps looking for ones that turn parts

of large systems into well-defined, reusable components.

Evolvability: Making Change Easy

Requirements change constantly: new facts and use cases, shifting business priorities, feature requests, platform replacements, new laws, growth forcing architectural changes.

- **Agile** working patterns provide an organisational framework for change, plus technical practices like **test-driven development (TDD)** and **refactoring** (restructuring code without changing behaviour).
- Those discussions are mostly small-scale (a few source files). This book seeks agility at the level of a **whole data system**, e.g. how would you "refactor" Twitter's timeline from approach 1 to approach 2?
- Ease of change depends heavily on **simplicity and abstractions**.

The book names this system-level agility **evolvability**.

Summary

- Applications have **functional requirements** (store, retrieve, search, process data) and **nonfunctional** ones (security, reliability, compliance, scalability, compatibility, maintainability). This chapter covered three nonfunctional ones.
- **Reliability** = working correctly despite faults from **hardware** (random, uncorrelated), **software** (systematic, hard to handle) and **humans** (inevitable mistakes). Fault tolerance can hide some faults from users.
- **Scalability** = strategies to keep performance good as load grows. Describe **load** quantitatively (Twitter fan-out) and **performance** quantitatively (response-time percentiles).
- **Maintainability** = making life better for engineers and operators: good abstractions reduce complexity and ease change; good operability means visibility and effective management.
- No easy fix exists for any of the three, but patterns recur; the coming chapters study them, and Part III returns to multi-component systems like Figure 1-1.

Interview angle

1. "What is the difference between a fault and a failure, and how do you build reliable systems?" *30-second answer:* A fault is one component misbehaving; a failure is the whole system no longer serving users. Faults cannot be eliminated, so design fault tolerance so they do not become failures: redundancy (RAID, dual power, multiple machines), software tolerance of whole-node loss, process isolation and restarts, monitoring, quick rollback. Injecting faults (Chaos Monkey) keeps recovery paths tested.

- *Follow-up: Why are software bugs worse than hardware faults?* Hardware faults are mostly independent; a bug is systematic and hits every node seeing the same input at once (e.g. the 2012 leap-second Linux bug).
- *Follow-up: Biggest cause of outages?* In the study cited, operator configuration errors; hardware played a role in only 10–25%.

2. "Design a Twitter home timeline. Fan-out on write or on read?" *30-second answer:* Reads (300k/s) outnumber posts (4.6k/s avg) ~100x, so precompute timelines at write time into per-user caches. Celebrities with tens of millions of followers make write fan-out explode, so go hybrid: fan out for normal users, fetch and merge celebrity tweets at read time.

- *Follow-up: Key load parameter?* Distribution of followers per user (weighted by tweet frequency).
- *Follow-up: Cache writes per second?* ~75 followers on average, so ~345k writes/s.

3. "Why use percentiles instead of the average for latency?" *30-second answer:* The mean hides how many users suffer; p50 is the typical experience, p99/p999 the tail. Tails matter because the slowest requests often belong to the most valuable users and fan-out amplifies tail latency.

- *Follow-up: Can I average p99s from 10 servers?* No; merge histograms (or use t-digest/HdrHistogram).

- *Follow-up: Where to measure?* Client side, since queueing and head-of-line blocking add delay the server does not see.
- *Follow-up: Latency vs response time?* Response time = service time + network + queueing delays, as seen by the client; latency is only the waiting time.

4. "Scale up or scale out?" *30-second answer:* Scaling up is simpler but high-end hardware gets very expensive; scaling out (shared-nothing) goes further but adds complexity, especially for stateful data. Stateless services scale out easily; classic advice is to keep the database on one node until cost or availability forces distribution. Real systems mix both and redesign at each order of magnitude.

- *Follow-up: Elastic or manual?* Elastic for unpredictable load; manual is simpler with fewer surprises.

5. "What makes a system maintainable?" *30-second answer:* Operability (monitoring, automation, no single-machine dependencies, predictability), simplicity (remove accidental complexity with good abstractions), evolvability (easy to adapt).

- *Follow-up: Accidental complexity?* Complexity from the implementation, not from the problem users need solved.

Self-check

1. **What makes an application "data-intensive"?** Its main challenge is the data (volume, complexity, rate of change) rather than CPU.
2. **Give two reasons the book groups databases, queues and caches under "data systems".** Tools overlap (Redis as a queue, Kafka with database-like durability), and apps combine several tools with application code into a new composite data system.
3. **A cluster has 10,000 disks with MTTF of 10–50 years. Roughly how many fail per day?** About one per day.
4. **Why did the industry shift from hardware redundancy toward software fault tolerance?** More machines mean more total hardware faults, cloud VMs can vanish without warning, and tolerating node loss enables rolling upgrades without downtime.
5. **Your p50 is 100 ms and p99 is 2 s. A page makes 20 backend calls in parallel. Is the typical page load closer to 100 ms or 2 s? Why?** Neither describes it well. The page waits for the slowest of 20 calls, so it is almost never as fast as the per-call median (P(all 20 beat p50) = 0.5^20, about one in a million). With a 1% chance per call of being p99-slow, about 18% of pages (1 − 0.99^20 ≈ 0.18) wait 2 s or more, versus 1% of single calls: tail latency amplification. (Our illustrative numbers, assuming independent calls.)
6. **Why did Twitter switch from approach 1 to approach 2, and why then move to a hybrid?** Reads vastly outnumbered posts, so precomputing on write made reads cheap; but 30M+-follower users made write fan-out huge, so celebrity tweets are fetched at read time.
7. **A load-test client waits for each response before sending the next request. What goes wrong?** Queues stay artificially short, so measured response times look better than reality.
8. **Two systems each handle the same bytes per second: one with 100,000 × 1 kB requests/sec, one with 3 × 2 GB requests/minute. Should they share an architecture?** No. Their load parameters differ completely; there is no generic scalable design.
9. **Name three things a data system can do to improve operability.** Any three of: monitoring/visibility, automation support, no single-machine dependency, clear documentation and operational model, overridable defaults, self-healing with manual override, predictable behaviour.
10. **When is it acceptable to sacrifice reliability?** When deliberately trading it for lower development cost (prototype for an unproven market) or operational cost (thin margins), knowing you are cutting corners.

Chapter 2 — Data Models and Query Languages

Book: Chapter 2, pp. 27–68

Overview

What question does this chapter answer? "In what *shape* should I store my data, and how do I ask questions of it?" It compares the three big general-purpose **data models** (relational, document, graph) and their **query languages** (languages for asking a database for data): SQL, relational algebra, MapReduce, the MongoDB aggregation pipeline, Cypher, SPARQL and Datalog, with CSS/XSL (web styling/XML transformation languages) as a side analogy.

Why it matters. The data model is arguably the most important decision in software: it shapes both the code and how you *think* about the problem. Every model makes some operations easy and fast and others awkward, slow or impossible; a model that fits your data's relationships saves years of pain.

Layers of data models. Each layer hides the one below behind a cleaner model (an **abstraction**):

1. **Application developer:** models the real world (people, goods, actions, money flows, sensors…) as objects/data structures plus APIs that manipulate them.
2. **General-purpose data model:** those structures expressed as JSON or XML documents (text formats for nested data: `{"name": "Asha"}` / `<name>Asha</name>`), relational tables, or a graph. *This chapter.*
3. **Database engineers:** represent JSON/XML/tables/graphs as bytes in memory, on disk or on the network, in ways that allow querying and processing. *Chapter 3.*
4. **Hardware engineers:** represent bytes as electrical currents, light pulses, magnetic fields.

Bigger systems have more layers, but each hides the complexity beneath, letting different groups (e.g. database-vendor engineers and app developers) work together. Mastering even one model is a lot of work, so choose carefully.

Key terms

Term	Plain-English meaning
Relational model	Data as relations (tables), each an unordered collection of tuples (rows). Edgar Codd, 1970. RDBMS = relational database management system.
Transaction / batch processing (here)	Day-to-day operations (sales, banking, airline reservations, stock-keeping) / periodic bulk jobs (invoicing, payroll, reporting).
Network / hierarchical model	Pre-relational models of the 1970s (CODASYL, IMS).
NoSQL	Loose label for nonrelational databases; later "Not Only SQL".
Polyglot persistence	Using relational and various nonrelational stores side by side.
Impedance mismatch	Awkward translation between application objects and tables/rows/columns.
ORM (object-relational mapping)	Frameworks (ActiveRecord, Hibernate) that reduce that translation boilerplate.
One-to-many / many-to-one / many-to-many	A user has many jobs (a tree) / many people live in one region / people and the companies they worked at.
Normalization / denormalization	Store each human-meaningful fact once, referenced by ID / deliberately duplicate (e.g. to avoid joins) and keep copies in sync.
Join	Combining records from different tables/collections that reference each other.
Foreign key / document reference	An ID pointing at another record (relational / document).
Locality	Storing related data together so it is read in one go.
Access path	In the network model, the chain of links followed from a root record to reach a record.
Query optimizer	Decides automatically how to execute a query (order, indexes, join methods).
Shredding	Splitting a document-like structure into several tables.
Schema-on-write / schema-on-read	Explicit schema enforced on write (like static typing) / implicit structure interpreted by the reader (like dynamic typing). "Schemaless" is a misleading name for schema-on-read.
BSON	MongoDB's binary encoding of JSON-like documents.
Imperative / declarative language	Says *which operations in which order* / *what result you want*.
Relational algebra	The mathematical operators behind SQL, e.g. σ (selection).
MapReduce	Bulk processing model across many machines using map and reduce functions.
Pure function	Uses only its inputs, makes no extra database queries, has no side effects.
Aggregation pipeline	MongoDB's declarative, JSON-syntax query language (added in 2.2).
Graph, vertex, edge	Vertices (nodes/entities) connected by edges (relationships/arcs). Tail / head vertex = where an edge starts / ends.
Property graph	Vertices and edges have IDs and key-value properties; edges have labels.
Cypher	Neo4j's declarative property-graph query language.
Recursive common table expression	SQL:1999 `WITH RECURSIVE` for variable-length traversals.

Term	Plain-English meaning
Triple-store, RDF, Turtle/N3	Facts as (subject, predicate, object); RDF is the semantic web's triple model; Turtle (a subset of Notation3) is a readable text format for triples.
Semantic web	Idea that websites publish machine-readable data combinable into a "web of data".
SPARQL	Query language for RDF triple-stores ("sparkle").
Datalog / rule	1980s logic query language, a subset of Prolog / a definition of a derived predicate from facts and other rules.

Relational Model Versus Document Model

What it is. The **relational model** behind SQL (**Edgar Codd, 1970**): data sits in **relations** (SQL: *tables*), each a collection of **tuples** (SQL: *rows*) with **no inherent order**.

A short history.

- At first a theoretical proposal many doubted could be implemented efficiently; by the **mid-1980s** RDBMSes and SQL were the default for regularly structured data, a dominance lasting **25–30 years**.
- Roots in **business data processing** on 1960s–70s **mainframes**: **transaction processing** and **batch processing**. Other databases of that era made developers think about internal representation; the relational model **hid that detail behind a cleaner interface**.
- Challengers were hyped and faded: **network** and **hierarchical models** (1970s–early '80s), **object databases** (late '80s–early '90s), **XML databases** (early 2000s, niche).
- Relational databases generalised far beyond business data: much of today's web (publishing, discussion, social networking, ecommerce, games, SaaS productivity apps) runs on them.

The Birth of NoSQL

NoSQL, the **2010s** challenger, names no particular technology: it began as a **Twitter hashtag for a 2009 meetup** on open source, distributed, nonrelational databases, and was later reinterpreted as **"Not Only SQL"**.

Why people adopted it:

1. **More scalability** than relational databases easily provide (huge datasets, very high write throughput).
2. Preference for **free and open source** software over commercial databases.
3. **Specialised query operations** the relational model supports poorly.
4. Frustration with **restrictive relational schemas**; desire for a more dynamic, expressive model.

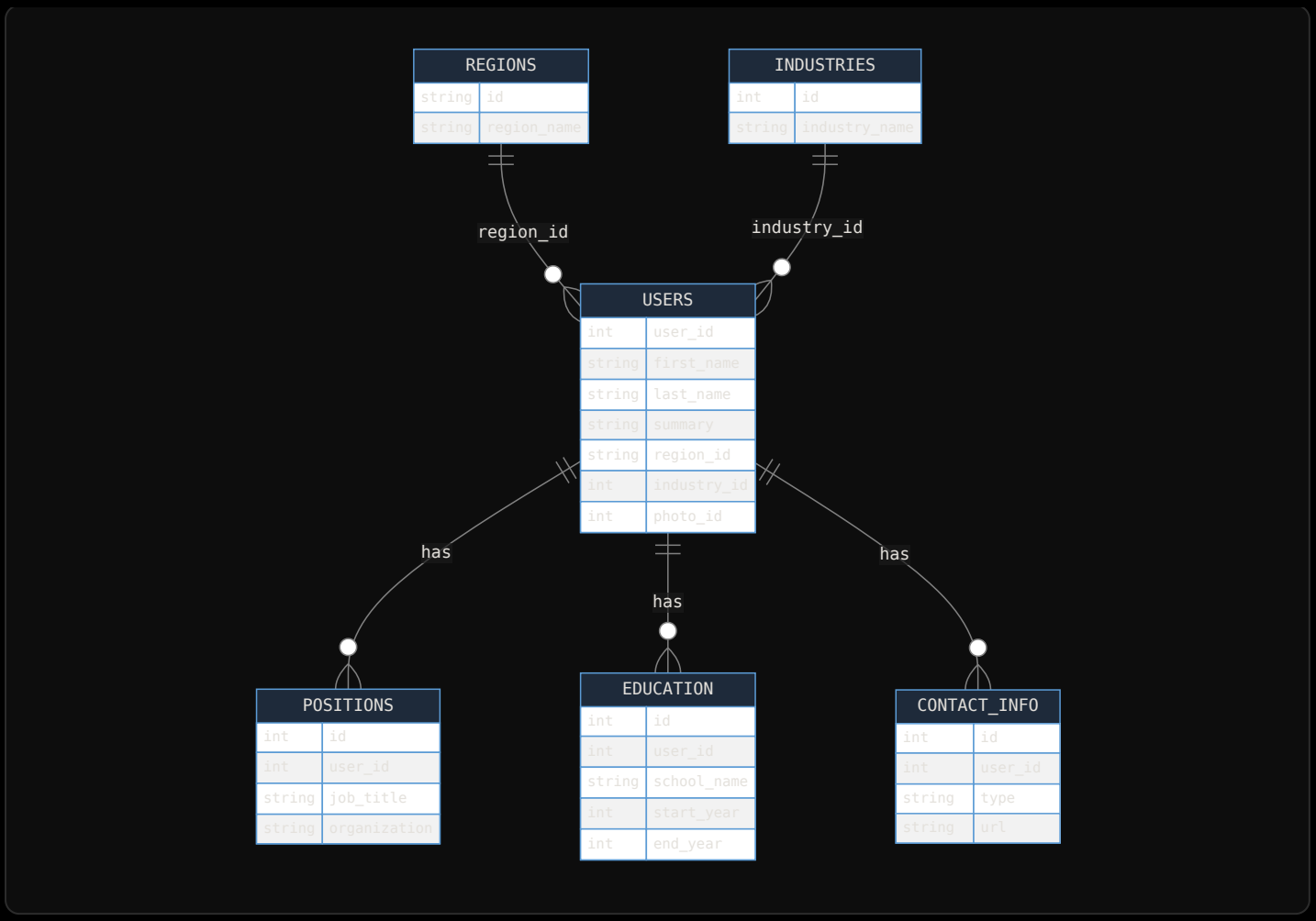
Relational databases will likely keep being used **alongside** many nonrelational stores: **polyglot persistence**.

The Object-Relational Mismatch

The problem. Most application code is object-oriented (Java, Python, C#); data in relational tables needs an awkward translation layer between objects and tables/rows/columns: the **impedance mismatch** (an electronics term: power transfers best when one circuit's output impedance matches the next one's input impedance). **ORMs** like **ActiveRecord** and **Hibernate** reduce the boilerplate but cannot fully hide the differences.

The résumé example. The book models a **LinkedIn profile** (Bill Gates's) relationally: the profile has a unique `user_id`; once-per-user fields (`first_name`, `last_name`) are columns in `users`; but positions, education entries and contact info vary in number — **one-to-many** relationships.

Figure 2-1 (redrawn): a LinkedIn-style profile represented with a relational schema.



(The book's figure has `photo_id`; its JSON version, Example 2-1, has a `photo_url` string.)

Three ways to store one-to-many data in SQL:

Option	How	Notes
1. Separate tables (classic normalized)	`positions`, `education`, `contact_info` each with a foreign key to `users`	Common approach before SQL:1999.
2. Structured / XML / JSON datatypes	Multi-valued data inside one row, with querying and indexing inside it	Oracle, IBM DB2, MS SQL Server, PostgreSQL support this to varying extents; several (IBM DB2, MySQL, PostgreSQL) also have a JSON datatype.
3. Encode as JSON/XML text	Text column interpreted by the application	You usually **cannot query** inside it with the database.

The document alternative. A résumé is mostly a **self-contained document**, so JSON (much simpler than XML) fits well. Document databases: **MongoDB**, **RethinkDB**, **CouchDB**, **Espresso**. Our own fictional profile:

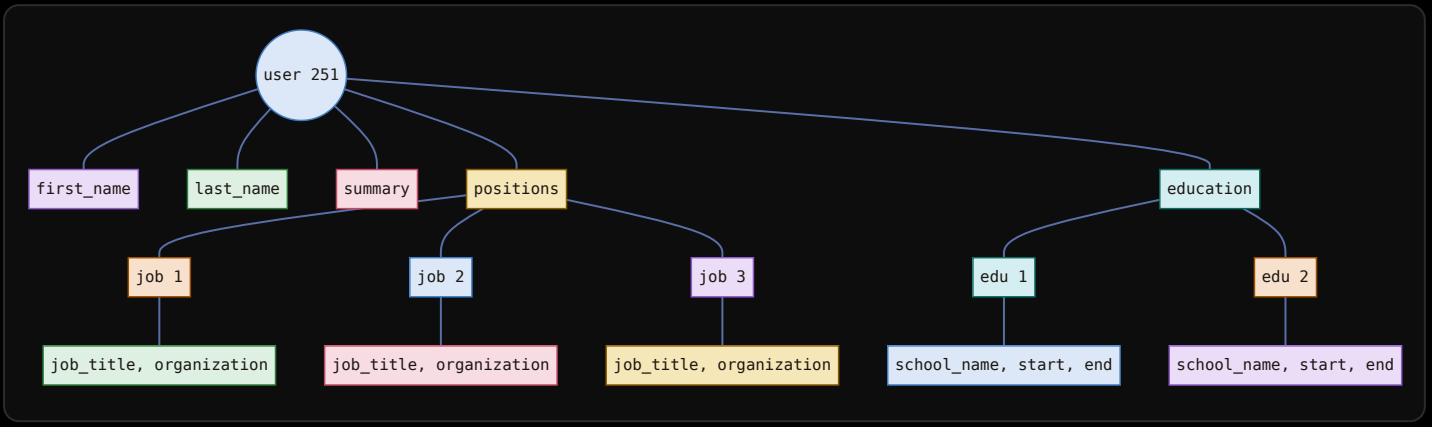
```
{
  "user_id": 7042,
  "first_name": "Asha",
  "last_name": "Rao",
  "summary": "Marine ecologist and science writer.",
  "region_id": "in:ka",
  "industry_id": 57,
  "positions": [
    {"job_title": "Research Lead", "organization": "Coral Watch Trust"},
    {"job_title": "Field Scientist", "organization": "Ocean Survey Institute"}
  ],
  "education": [
    {"school_name": "University of Mysore", "start": 2006, "end": 2010},
    {"school_name": "Bishop Cotton School", "start": null, "end": null}
  ],
  "contact_info": {
    "blog": "https://example.org/asha",
    "mastodon": "@asha@example.social"
  }
}
```

`positions` and `education` are nested **arrays** replacing whole tables; `contact_info` is a nested **object**; `region_id` / `industry_id` are **IDs**, not text (next section).

Pros of the JSON version:

- Some developers find it **reduces the impedance mismatch** (JSON's own encoding problems: Chapter 4).
- The **lack of a schema** is often called an advantage (see "Schema flexibility" below).
- **Better locality:** relationally, a profile needs several queries (one per table) or a messy multi-way join; in JSON one query fetches everything.
- The one-to-many relationships form a **tree**, which JSON makes **explicit**.

Figure 2-2 (redrawn): the book's user 251 as a tree. Scalar fields hang directly off the user; positions and education are one-to-many branches. (The book's tree omits contact_info.)



Many-to-One and Many-to-Many Relationships

Why IDs instead of text? The book stores `region_id` / `industry_id` rather than `"Greater Seattle Area"` / `"Philanthropy"`. Standardised lists (drop-downs, autocomplete) give: **consistent** spelling; **no ambiguity** between same-named cities; **easy updates** (a renamed city changes in one place); **localisation** (lists can be translated into the viewer's language); **better search** (the region list can encode that Seattle is in Washington, so a search for philanthropists in Washington finds this profile).

ID vs text is really about duplication. With an ID, the human-meaningful information is stored **once** and referenced by an ID meaningful only inside the database, so the ID **never needs to change**. With text, that information is **copied into every record**, and any change must update every copy: **write overhead** and risk of **inconsistency**. Removing such duplication is the core idea of **normalization**. (Relational theory's several normal forms differ little in practice; rule of thumb: if values that could live in one place are duplicated, the schema is not normalized. DBAs and developers love arguing normalize vs denormalize; the book defers the debate to Part III's treatment of caching and derived data.)

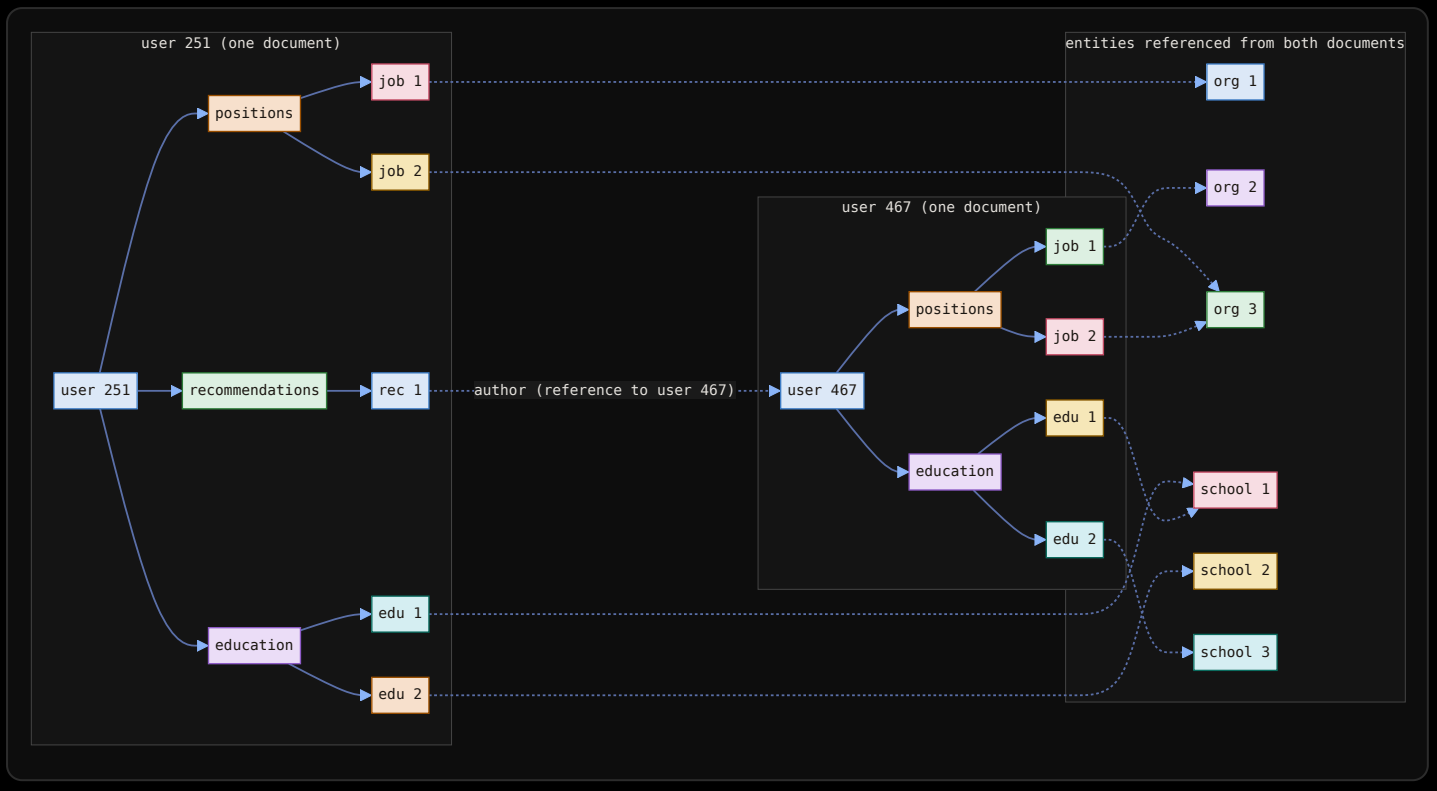
The catch for documents. Normalizing needs **many-to-one** relationships, which fit documents poorly:

- Relational: referring to rows by ID is normal because **joins are easy**.
- Document: one-to-many trees need no joins, and **join support is often weak** — at the time of writing **RethinkDB supports joins, MongoDB does not, CouchDB only in predeclared views**.
- Without database joins you **emulate them in application code** with multiple queries (fine for small, rarely changing lists like regions, which can be cached in memory — but the join work has moved into your code).

Data grows more connected over time. Two résumé extensions add links even if v1 fitted a join-free model:

- **Organizations and schools as entities** with their own pages (logo, news feed), referenced from résumés instead of plain strings. *(Figure 2-3, described: a LinkedIn screenshot where a company name links to the company's page with logo.)*
- **Recommendations** written by one user for another, showing the **author's name and photo**; if the author changes their photo, every recommendation must show the new one, so it must **reference the author's profile**.

Figure 2-4 (redrawn): two résumés (user 251 and user 467) extended with many-to-many links. Each dashed box could be one document; arrows leaving a box are references needing joins.



Are Document Databases Repeating History?

Document databases reopened a debate about many-to-many relationships far older than NoSQL.

IMS and the hierarchical model. The leading 1970s business database was IBM's **Information Management System (IMS)**, built to track stock for the **Apollo space program**, commercially released in **1968**, and still used on **OS/390** on IBM mainframes. Its **hierarchical model** stored all data as a **tree of records nested inside records**, much like JSON. Like document databases it handled **one-to-many** well, made **many-to-many** hard, and had **no joins**, forcing a choice between **duplicating (denormalizing)** data and **manually resolving references** — the same problems document databases face today.

Two solutions were proposed: the **relational model** (became SQL, conquered the industry) and the **network model** (early fans, then obscurity). Their "great debate" ran through much of the 1970s.

The network model

- Standardised by the **Conference on Data Systems Languages (CODASYL)**, hence the **CODASYL model**.
- It **generalised** the hierarchy: a record could have **multiple parents** (e.g. one "Greater Seattle Area" record linked from every user there), enabling many-to-one and many-to-many links.

- Links were like **pointers** (stored on disk), not **foreign keys**.
- Reaching a record meant starting at a **root record** and following chains of links: the **access path**. A query moved a **cursor** through the database, iterating over record lists and following links (like walking a linked list). With many-to-many relationships several paths could reach one record and the programmer had to track them all; CODASYL members likened it to **navigating an n-dimensional data space**.

Trade-off. Hand-chosen access paths used scarce 1970s hardware efficiently (e.g. **tape drives**, where seeks are extremely slow), but query and update code became **complicated and inflexible**. Without a path to the data you needed, you were stuck; changing access paths meant rewriting lots of query code, so the data model was hard to change.

The relational model

What it did differently: put all data **in the open** — a relation is just a collection of tuples, with no nested structures or access paths. You can read any rows **matching an arbitrary condition**, fetch a row by its **key** (the columns uniquely identifying it), and **insert into any table** without caring about links in or out.

> **Foreign key constraints aside.** Constraints can restrict modifications but are not required by the model. Even with them, joins on foreign keys happen **at query time**; CODASYL effectively did the join **at insert time**.

The query optimizer decides the execution order of a query's parts and which indexes to use. These choices *are* the access path, but made by the optimizer, not the developer.

> **Key term — index:** an extra structure kept alongside a table that finds rows matching a value without scanning every row, like a book's index. Indexes speed reads but cost extra work on every write. Built in Chapter 3.

- To query in a new way, **declare a new index**; queries use it automatically without changes, making new features much easier.
- Optimizers took years of R&D, but the insight is **build it once and every application benefits**. Hand-coding one query's access path is easier than a general optimizer, yet over time the **general-purpose approach wins**.

Comparison to document databases

- Document databases **returned to the hierarchical model in one respect**: nested one-to-many records (positions, education, contact_info) live **inside the parent**.
- For **many-to-one and many-to-many**, they are **basically like relational**: a unique ID (**foreign key** / **document reference**) resolved **at read time** via a join or follow-up queries.
- So far they have **not** followed CODASYL's path.

Relational Versus Document Databases Today

Fault tolerance (Chapter 5) and concurrency (Chapter 7) also differ; here we compare only **data models**.

	Document model	Relational model
Main arguments for	Schema flexibility; locality performance; closer to app data structures for some apps	Better joins; better many-to-one / many-to-many support

Which data model leads to simpler application code?

- **Document-like data** (a tree of one-to-many relationships, usually loaded whole) → **document model**; relational **shredding** gives clumsy schemas and needless complexity.
- **Document limitation:** no direct reference to a nested item; you say "the second item in user 251's positions list" (like a hierarchical access path). Fine if nesting is **shallow**.

- **Weak joins** may not matter: an **analytics app** recording events with timestamps may never need many-to-many.
- **Many-to-many needed** → documents get less attractive: **denormalizing** means the app must keep copies consistent; **emulating joins** with several requests moves complexity into the app and is usually **slower** than in-database joins.
- **Bottom line:** it depends on relationships between data items. For **highly interconnected** data: documents awkward, relational acceptable, **graph models most natural**.

Schema flexibility in the document model

Most document databases, and the JSON support in relational databases, do **not enforce a schema**; XML support usually offers **optional** validation. Any keys and values may be added, and readers get **no guarantee** which fields exist.

> **Common confusion: "schemaless" is misleading.** Reading code assumes some structure, so there is an **implicit schema** the database does not enforce.

	Schema-on-read	Schema-on-write
Structure	Implicit; interpreted when read	Explicit; database checks all writes conform
Typical of	Document databases	Traditional relational databases
Programming analogy	**Dynamic (runtime) type checking**	**Static (compile-time) type checking**

As with typing, there is **no universally right answer**.

Where it shows: changing the data format. The book splits a full-name field into first and last name. Our variant: splitting a `location` string like `"Pune, India"` into a `city` field.

Document database (schema-on-read): write new documents with the new field and handle old ones when reading:

```
if (member && member.location && !member.city) {
    // Records created before the 2021 redesign lack a "city" field
    member.city = member.location.split(",")[0].trim();
}
```

Relational database (schema-on-write): run a **migration** (one-time scripted change to schema and data):

```
ALTER TABLE members ADD COLUMN city text;                          -- new column, NULL by default
UPDATE members SET city = split_part(location, ',', 1);            -- PostgreSQL
UPDATE members SET city = substring_index(location, ',', 1);       -- MySQL
```

Are schema changes slow? Their bad reputation is **not entirely deserved**:

- Most relational databases run `ALTER TABLE` in **a few milliseconds**.
- **MySQL is the notable exception**: `ALTER TABLE` **copies the entire table**, so a big table can mean minutes or hours of downtime (tools exist to work around this).
- `UPDATE` on a large table is **slow on any database** (every row rewritten). Alternative: leave the new column `NULL` and fill it at read time, as a document database would.

When schema-on-read is better: heterogeneous items — **many object types**, impractical to give each a table; or structure dictated by **external systems** you do not control that may change anytime. When all records share a structure, a schema usefully **documents and enforces** it. (Schema evolution: Chapter 4.)

Data locality for queries

A document is usually stored as **a single contiguous string** — JSON, XML, or a binary variant such as **MongoDB's BSON**. If the app needs the **whole document** (e.g. to render a page), this **storage locality** wins over a multi-table layout needing several index lookups and disk seeks.

Limitations: the benefit exists only when reading **large portions** together, since the database usually loads the **whole document** even for a small piece; **updates** usually rewrite the whole document, and only

changes that **keep the encoded size** are easy in place. Hence the advice: keep documents **small-ish** and avoid writes that **grow** them. This significantly narrows where document databases are useful.

Locality is not unique to documents:

System	Locality feature
Google's **Spanner**	Relational; schema can declare a table's rows **interleaved (nested)** in a parent table
Oracle	**Multi-table index cluster tables**
Bigtable model (**Cassandra**, **HBase**)	**Column families** control locality

(More in Chapter 3.)

Convergence of document and relational databases

- Since the **mid-2000s**, most relational databases **except MySQL** support **XML**, including local modification and indexing/querying inside documents.
- **JSON support**: **PostgreSQL since 9.3**, **MySQL since 5.7**, **IBM DB2 since 10.5**; others likely to follow given JSON's popularity in web APIs.
- From the document side: **RethinkDB** supports relational-style joins; some **MongoDB drivers** resolve references automatically — a **client-side join**, likely slower than in-database joins (extra network round-trips, less optimisation).

> **Codd's nonsimple domains.** Codd's 1970 paper already allowed a row value to be a nested relation rather than a primitive, permitting arbitrarily nested trees — much like the XML/JSON support SQL gained 30+ years later.

Takeaway. The models are **converging** and complement each other; a **hybrid** handling document-like data plus relational queries is a good direction.

Query Languages for Data

Imperative vs declarative. SQL is declarative; **IMS and CODASYL** were queried with **imperative code**.

Our own example: return only the citrus fruits. Imperative JavaScript:

```
function getCitrus() {
    var result = [];
    for (var i = 0; i < fruits.length; i++) {        // walk the list in order
        if (fruits[i].group === "Citrus") {         // test each item
            result.push(fruits[i]);                 // collect matches
        }
    }
    return result;
}
```

In **relational algebra**, `citrus = σ group = "Citrus" (fruits)`, where **σ (sigma)** is the **selection operator**. SQL follows relational algebra closely:

```
SELECT * FROM fruits WHERE "group" = 'Citrus';   -- "group" is quoted because it is a SQL keyword
```

	Imperative	Declarative
You specify	Which operations, in which order (loop, test, update variables)	Only the result pattern: conditions and transformations (sort, group, aggregate)
Who picks the algorithm	You	The **query optimizer** (indexes, join methods, order)

Why declarative wins:

1. **Concise** and easier to work with.
2. **Hides engine internals**, so performance can improve **without changing queries**. If the database reshuffles records to reclaim disk space, imperative code relying on iteration order might break, and the database cannot tell; SQL promises **no order**, so reordering is safe. Being **more limited** gives more room for optimisation.
3. **Parallel execution.** CPUs now gain **cores**, not clock speed. Imperative code fixes an instruction order and is hard to parallelise; declarative queries specify only the result, so the database may run them **in parallel**.

Declarative Queries on the Web

The advantage extends beyond databases. The book uses a web page about ocean animals (user on "Sharks"); our version is a planets site with "Mars" selected:

```
<ul>
    <li class="selected">          <!-- the current page's menu item -->
        <p>Mars</p>                <!-- title of the selected page -->
        <ul>
            <li>Olympus Mons</li>
            <li>Valles Marineris</li>
        </ul>
    </li>
    <li>
        <p>Venus</p>
        <ul>
            <li>Maxwell Montes</li>
        </ul>
    </li>
</ul>
```

Goal: give the selected page's title a blue background.

Declarative CSS: the selector states the pattern; `<p>Mars</p>` matches, `<p>Venus</p>` does not.

```
li.selected > p {           /* any <p> whose direct parent is <li class="selected"> */
    background-color: blue;
}
```

Declarative XSL, using the **XPath** expression `li[@class='selected']/p` (equivalent to the CSS selector):

```
<xsl:template match="li[@class='selected']/p">   <!-- same pattern in XPath -->
    <fo:block background-color="blue">
        <xsl:apply-templates/>                   <!-- keep processing children -->
    </fo:block>
</xsl:template>
```

Imperative JavaScript DOM API (the DOM is the browser's in-memory tree of page elements):

```
var items = document.getElementsByTagName("li");
for (var i = 0; i < items.length; i++) {
    if (items[i].className === "selected") {                    // find selected <li>
        var kids = items[i].childNodes;
        for (var j = 0; j < kids.length; j++) {
            var k = kids[j];
            if (k.nodeType === Node.ELEMENT_NODE && k.tagName === "P") {
                k.setAttribute("style", "background-color: blue"); // colour it
            }
        }
    }
}
```

Problems with the imperative version (besides length):

- If the `selected` class is **removed** (user clicks another page), the blue stays — even if the code reruns — until reload. With CSS the browser **notices automatically** and removes it.

- Using a newer, faster API (`getElementsByClassName`, `document.evaluate()`) means **rewriting the code**; browser vendors can speed up CSS and XPath without breaking compatibility.

Conclusion. Declarative CSS beats imperative styling in browsers, as declarative SQL beat imperative query APIs in databases. (IMS and CODASYL offered only imperative APIs, typically used from **COBOL** (a 1959 business language), stepping through records one by one.)

MapReduce Querying

What it is. **MapReduce**, popularised by **Google**, is a model for bulk processing of large datasets across many machines. **MongoDB** and **CouchDB** offer a **restricted version** for **read-only queries over many documents**. (General MapReduce: Chapter 10.)

Where it sits: between declarative and imperative. Query logic is **code snippets** the framework calls repeatedly, based on functional programming's **map** (*collect*: apply a function to every item) and **reduce** (*fold/inject*: combine many items into one).

The book's example: a marine biologist's report of **sharks seen per month**. Our parallel: a birdwatcher counting **herons per month**.

PostgreSQL (declarative): filter to herons, group by month, sum the counts. `date_trunc('month', timestamp)` returns the start of the calendar month containing the timestamp (rounds down).

```
SELECT date_trunc('month', seen_at) AS sighting_month,   -- round timestamp down to month start
       sum(bird_count)              AS total_birds       -- add up birds per month
FROM sightings
WHERE family = 'Herons'                                  -- only herons
GROUP BY sighting_month;                                 -- one row per month
```

MongoDB MapReduce:

```
db.sightings.mapReduce(
    function map() {                                     // called once per matching document
        var y = this.seenAt.getFullYear();
        var m = this.seenAt.getMonth() + 1;              // JS months start at 0
        emit(y + "-" + m, this.birdCount);               // key = "year-month", value = count
    },
    function reduce(key, values) {                       // called once per distinct key
        return Array.sum(values);                        // total birds that month
    },
    {
        query: { family: "Herons" },                     // declarative filter (MongoDB extension)
        out: "monthlyHeronReport"                        // collection to write results into
    }
);
```

How it runs:

1. The `query` filter selects documents declaratively (a MongoDB extension to MapReduce).
2. `map` is called **once per matching document** (`this` = the document) and **emits** a key (year-month string such as `"2013-12"` or `"2014-1"`) and a value (the count).
3. Emitted pairs are **grouped by key**; `reduce` is called **once per key** and sums the counts.
4. Output goes to the named collection (book: `monthlySharkReport`).

Worked example from the book: shark sightings on **25 Dec 1995** (3 animals, *Carcharodon carcharias*) and **12 Dec 1995** (4 animals, *Carcharias taurus*). `map` emits `("1995-12", 3)` and `("1995-12", 4)`; then `reduce("1995-12", [3, 4])` returns **7**.

Restriction: pure functions. `map` and `reduce` may use only their inputs, may **not** run extra database queries, and must have **no side effects**, so the database can run them **anywhere, in any order, and rerun them after failure**. They can still parse strings, call libraries, compute.

Clearing up confusions:

- MapReduce is a fairly **low-level** model for cluster execution. SQL **can** be implemented as MapReduce pipelines (Chapter 10), but many distributed SQL implementations do **not** use it.
- Nothing restricts SQL to one machine; MapReduce has **no monopoly** on distributed querying.

- JavaScript inside queries is not unique to MapReduce: some **SQL databases accept JavaScript extensions**.

Usability problem and fix. Writing **two coordinated functions** is often harder than one query, and declarative languages give the optimizer more room. So **MongoDB 2.2** added the declarative **aggregation pipeline**:

```
db.sightings.aggregate([
    { $match: { family: "Herons" } },                    // filter, like WHERE
    { $group: {                                          // group, like GROUP BY
        _id: {
            year:  { $year:  "$seenAt" },
            month: { $month: "$seenAt" }
        },
        totalBirds: { $sum: "$birdCount" }               // aggregate, like sum()
    } }
]);
```

Its expressiveness is similar to a **subset of SQL** with **JSON-based syntax** (arguably a matter of taste). Moral: a NoSQL system may end up **accidentally reinventing SQL** in disguise.

Graph-Like Data Models

When to use graphs. Many-to-many relationships are the key differentiator: mostly one-to-many or none → **documents**; simple many-to-many → **relational**; **very common, complex** connections → **graph**.

What a graph is: vertices (*nodes*, *entities*) and **edges** (*relationships*, *arcs*).

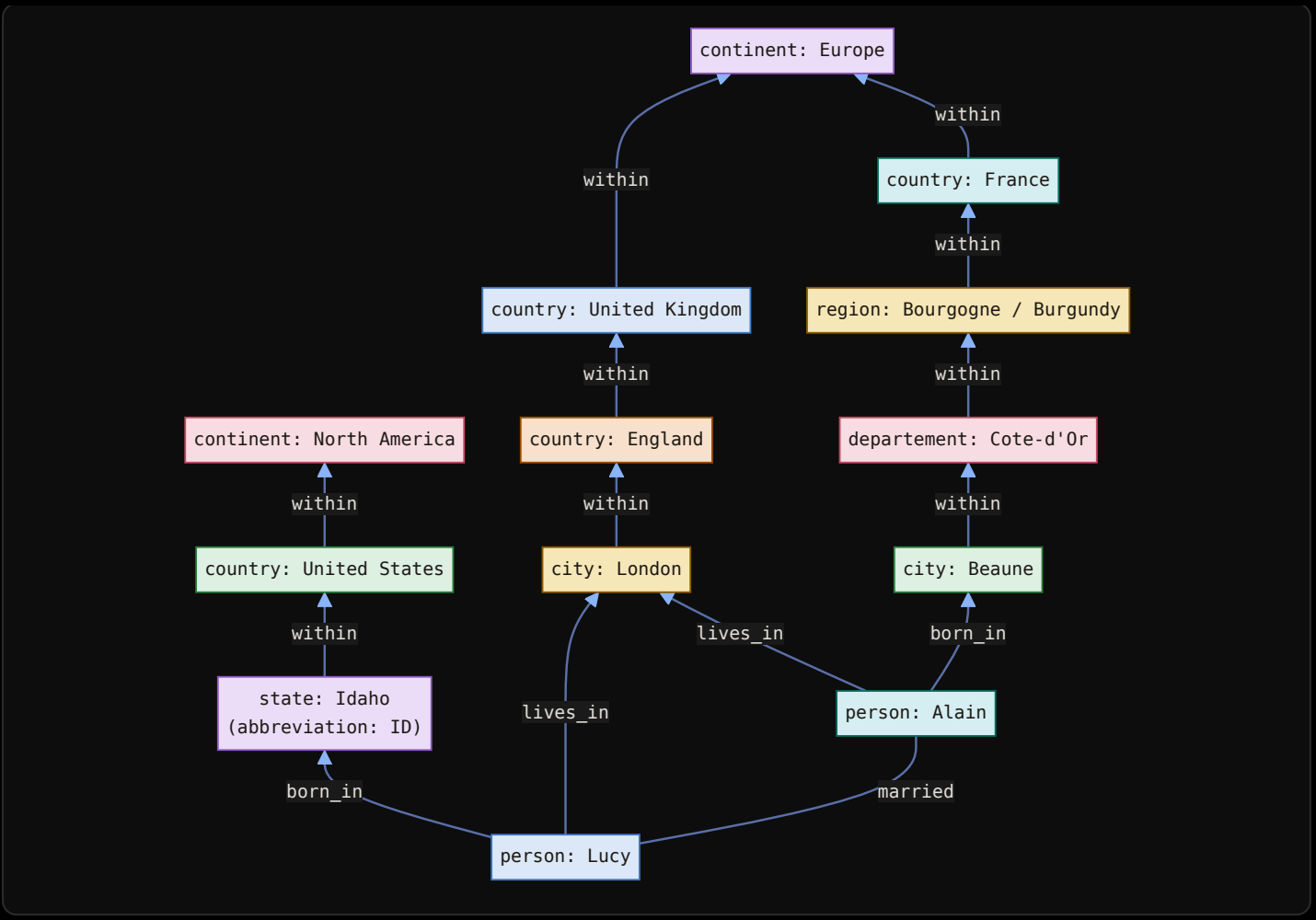
Graph	Vertices	Edges
Social graphs	People	Who knows whom
The web graph	Web pages	HTML links
Road or rail networks	Junctions	Roads or railway lines

Well-known algorithms: **car navigation** (shortest paths); **PageRank** (page popularity on the web graph, for search ranking).

Heterogeneous graphs. Vertices need not be the same kind; one graph can store completely different object types. **Facebook**'s single graph has vertices for **people, locations, events, checkins, comments**, and edges for **friendship, checkin location, who commented on which post, who attended which event**, etc.

Running example (Figure 2-5): Lucy, originally from Idaho, and **Alain, originally from Beaune, France, married and living in London** (could be a social network or genealogy database).

Figure 2-5 (redrawn): graph-structured data. France's hierarchy (city, département, région, country) differs from the US one (state, country).



	Models	Implementations
Property graph	vertices + edges with properties	**Neo4j, Titan, InfiniteGraph**
Triple-store	(subject, predicate, object)	**Datomic, AllegroGraph**, others

Declarative graph languages covered: **Cypher, SPARQL, Datalog**. Others exist: **imperative** languages such as **Gremlin**, and graph processing frameworks like **Pregel** (Chapter 10).

Property Graphs

Each vertex has	Each edge has
A unique identifier	A unique identifier
A set of outgoing edges	The **tail vertex** (start)
A set of incoming edges	The **head vertex** (end)
Properties (key-value pairs)	A **label** for the relationship
	Properties (key-value pairs)

As relational tables: a vertex table and an edge table (the book's Example 2-2 uses PostgreSQL's `json` type for properties). Our own sketch:

```
CREATE TABLE nodes (
    node_id  integer PRIMARY KEY,                       -- unique vertex ID
    attrs    json                                       -- key-value properties
);

CREATE TABLE links (
    link_id  integer PRIMARY KEY,                       -- unique edge ID
    from_node integer REFERENCES nodes (node_id),       -- tail vertex
    to_node   integer REFERENCES nodes (node_id),       -- head vertex
    kind     text,                                      -- the edge label
    attrs    json                                       -- edge properties
);

CREATE INDEX links_from ON links (from_node);           -- fast outgoing-edge lookups
CREATE INDEX links_to   ON links (to_node);             -- fast incoming-edge lookups
```

Important properties:

1. **Any vertex can connect to any other**; no schema restricts associations.
2. From any vertex you can efficiently find **incoming and outgoing** edges, so you can **traverse** forwards and backwards (hence indexes on both columns).
3. **Different labels** let one graph hold many kinds of information cleanly.

Why the flexibility matters (Figure 2-5): easy to represent **different regional structures** (French *départements* and *régions* vs US *counties* and *states*), **historical quirks** like a **country within a country** (England within the UK), and **varying granularity** (Lucy's residence is a **city**, her birthplace only a **state**).

Evolvability. E.g. **food allergies**: a vertex per allergen, edges from people to their allergens, and links from allergens to foods containing them; a query can then find **foods safe for each person**.

The Cypher Query Language

Cypher is a **declarative** property-graph language created for **Neo4j** (named after a *Matrix* character, unrelated to ciphers).

Inserting data. The book's Example 2-3 inserts the left part of Figure 2-5. Each vertex gets a **symbolic name**; `(Idaho) -[:WITHIN]-> (USA)` creates a WITHIN edge with Idaho as **tail** and USA as **head**. Our own data:

```
CREATE
  (Asia:Location    {name:'Asia',      type:'continent'}),   // vertex with label + properties
  (India:Location   {name:'India',     type:'country'}),
  (Kerala:Location  {name:'Kerala',    type:'state'}),
  (Priya:Person     {name:'Priya'}),
  (Kerala) -[:WITHIN]-> (India) -[:WITHIN]-> (Asia),        // chain of WITHIN edges
  (Priya)  -[:BORN_IN]-> (Kerala)                           // Priya's birthplace
```

Querying. The book finds **everyone who emigrated from the United States to Europe** (a `BORN_IN` edge to a place within the US **and** a `LIVES_IN` edge to a place within Europe). Ours: people **born in India now living in Canada**:

```
MATCH
  (p) -[:BORN_IN]->  () -[:WITHIN*0..]-> (:Location {name:'India'}),
  (p) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (:Location {name:'Canada'})
RETURN p.name
```

- `(p) -[:BORN_IN]-> ()` matches a BORN_IN edge, binding its tail to `p`.
- `-[:WITHIN*0..]->` follows WITHIN edges **zero or more times** (like regex `*`) until a Location named India.
- The second line requires the **same** `p` to have a LIVES_IN path to Canada; return `p.name`.

Execution is the optimizer's job. It might scan **forward** (all people, check birthplace and residence) or go **backward** (use a `name` index to find the two Location vertices, follow **incoming** WITHIN edges to collect all places inside each, then find people via incoming BORN_IN/LIVES_IN edges). You do not choose.

Graph Queries in SQL

Possible, with difficulty. Relational queries usually **know in advance which joins** are needed; graph queries traverse a **variable number of edges**, so the **number of joins is not fixed**. A LIVES_IN edge might point to a street, city, district, region, state…, and the place you want may be several WITHIN levels up.

The SQL tool: since **SQL:1999**, **recursive common table expressions** (`WITH RECURSIVE`: a named sub-query that refers to itself, repeating until no new rows appear), supported by **PostgreSQL, Oracle, IBM DB2, SQL Server**. It works but is **very clumsy**. Our India→Canada query over `nodes` / `links`:

```
WITH RECURSIVE
  in_india(node_id) AS (                                -- all places inside India
      SELECT node_id FROM nodes WHERE attrs->>'name' = 'India'
    UNION
      SELECT links.from_node FROM links
        JOIN in_india ON links.to_node = in_india.node_id
        WHERE links.kind = 'within'
  ),
  in_canada(node_id) AS (                               -- all places inside Canada
      SELECT node_id FROM nodes WHERE attrs->>'name' = 'Canada'
    UNION
      SELECT links.from_node FROM links
        JOIN in_canada ON links.to_node = in_canada.node_id
        WHERE links.kind = 'within'
  ),
  born_in_india(node_id) AS (                           -- people born somewhere in India
    SELECT links.from_node FROM links
      JOIN in_india ON links.to_node = in_india.node_id
      WHERE links.kind = 'born_in'
  ),
  lives_in_canada(node_id) AS (                         -- people living somewhere in Canada
    SELECT links.from_node FROM links
      JOIN in_canada ON links.to_node = in_canada.node_id
      WHERE links.kind = 'lives_in'
  )
SELECT nodes.attrs->>'name'
FROM nodes
JOIN born_in_india   ON nodes.node_id = born_in_india.node_id     -- intersect the
JOIN lives_in_canada ON nodes.node_id = lives_in_canada.node_id;  -- two sets
```

Steps: seed `in_india` with the India vertex and repeatedly add tails of **incoming** `within` edges until nothing new; same for Canada; collect people with incoming `born_in` edges from `in_india` and `lives_in` edges from `in_canada`; **intersect** the two sets.

Lesson. The book notes the same query is **4 lines** in one language and **29 lines** in another: different data models suit different use cases.

Triple-Stores and SPARQL

What it is. The **triple-store** model is **mostly equivalent** to the property graph in different words, but worth knowing for its tools and languages.

Triples: (subject, predicate, object), e.g. (*Jim, likes, bananas*). The **subject** is like a vertex. The **object** is either:

1. **A primitive value** (string, number): predicate + object act as a **property** of the subject, e.g. *(lucy, age, 33)* ≈ vertex `lucy` with `{"age":33}`.
2. **Another vertex**: the predicate is an **edge** (subject = tail, object = head), e.g. *(lucy, marriedTo, alain)*.

Turtle format. The book writes Figure 2-5 as triples in **Turtle** (a subset of **Notation3 (N3)**). In full form, one triple per line: `_:priya :bornIn _:kerala.` (an edge: object is a vertex) or `_:india :name "India".` (a property: object is a literal). Vertices are written `_:someName`; the name means nothing outside the file, it only marks which triples refer to the same vertex, and `a` means "is of type". **Semicolons** say several things about one subject; our data in that compact form:

```
@prefix : <urn:example:>.
_:priya  a :Person;   :name "Priya";  :bornIn _:kerala.
_:kerala a :Location; :name "Kerala"; :type "state";     :within _:india.
_:india  a :Location; :name "India";  :type "country";   :within _:asia.
_:asia   a :Location; :name "Asia";   :type "continent".
```

The semantic web

- Triple-stores are **completely independent** of the semantic web (**Datomic** claims no link to it), but many associate the two.
- **The idea:** websites publish **machine-readable data** alongside human content; the **Resource Description Framework (RDF)** would let data from different sites combine into a **"web of data"** — an internet-wide "database of everything".
- **Reality: overhyped in the early 2000s**, not realised in practice, and weighed down by acronyms, over-complex standards and hubris, making many cynical.
- **But** it produced useful work: **triples are a good internal data model** even with no intent to publish RDF.

> **Datomic aside.** Datomic actually stores **5-tuples**: the two extra fields are **metadata for versioning**.

The RDF data model

Turtle is a **human-readable** RDF format; RDF can also be written in much more verbose **XML**. Tools like **Apache Jena** convert between formats automatically. Part of our data in RDF/XML:

```
<rdf:RDF xmlns="urn:example:" xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <Location rdf:nodeID="kerala">
    <name>Kerala</name>
    <type>state</type>
    <within>
      <Location rdf:nodeID="india">
        <name>India</name>
        <type>country</type>
      </Location>
    </within>
  </Location>
  <Person rdf:nodeID="priya">
    <name>Priya</name>
    <bornIn rdf:nodeID="kerala"/>
  </Person>
</rdf:RDF>
```

RDF quirks from internet-wide data exchange: subjects, predicates and objects are often **URIs** (globally unique names like web addresses), e.g. `<http://my-company.com/namespace#within>` instead of `WITHIN`, so your data can merge with others' without clashes (theirs might be `<http://other.org/foo#within>`). The URL **need not resolve**; it is just a namespace. The book uses non-resolvable `urn:example:within`, declared **once** at the top.

The SPARQL query language

SPARQL (**SPARQL Protocol and RDF Query Language**, said "sparkle") queries RDF triple-stores. It **predates Cypher**, whose pattern matching was **borrowed from SPARQL**. The emigration query is **even more concise** than in Cypher; ours:

```
PREFIX : <urn:example:>

SELECT ?who WHERE {
  ?p :name ?who.                          # bind the person's name
  ?p :bornIn  / :within* / :name "India".  # born somewhere inside India
  ?p :livesIn / :within* / :name "Canada". # living somewhere inside Canada
}
```

Variables start with `?`; `/` chains predicates into a path; `:within*` follows `within` zero or more times. Correspondence: Cypher `(p) -[:BORN_IN]-> () -[:WITHIN*0..]-> (place)` ≈ SPARQL `?p :bornIn / :within* ?place.`, and Cypher `(ind {name:'India'})` ≈ SPARQL `?ind :name "India".` Because RDF uses predicates for **both properties and edges**, one syntax matches both. SPARQL is useful internally even if the semantic web never happens.

> **Graph Databases Compared to the Network Model (sidebar).** Are graph databases CODASYL again? **No**:
>
> | CODASYL (network model) | Graph databases |
> |---|---|
> | A schema dictated which record type could nest within which | Any vertex can link to any other, so apps adapt more easily |
> | The only way to reach a record was an access path | Refer to any vertex directly by **unique ID**, or find vertices by value via an **index** |
> | Children were an **ordered set** maintained by the database (affecting storage), and inserts had to care about positions | Vertices and edges are **unordered**; sort at query time |
> | All queries imperative, hard to write, easily broken by schema changes | Imperative traversal possible, but most offer **declarative** languages (Cypher, SPARQL) |

The Foundation: Datalog

What it is. **Datalog** is much **older** than SPARQL or Cypher (studied extensively in the **1980s**), less known among engineers but the **foundation** later languages build on. Used by **Datomic**, and **Cascalog** implements it for big datasets on **Hadoop**. (Both use Clojure S-expression syntax; the book uses equivalent **Prolog** syntax.)

Data model: like triples, generalised — **predicate(subject, object)**. Our facts:

```
name(asia, 'Asia').
type(asia, continent).
name(india, 'India').
type(india, country).
within(india, asia).
name(kerala, 'Kerala').
type(kerala, state).
within(kerala, india).
name(priya, 'Priya').
born_in(priya, kerala).
```

Queries are built from rules (Datalog is a **subset of Prolog**). Our emigration query (assume Priya also lives in Toronto, within Canada):

```
inside(Place, Name) :- name(Place, Name).                  /* Rule 1 */
inside(Place, Name) :- within(Place, Via),                 /* Rule 2 */
                       inside(Via, Name).

moved(Name, From, To) :- name(Person, Name),               /* Rule 3 */
                         born_in(Person, BirthPlace),
                         inside(BirthPlace, From),
                         lives_in(Person, HomePlace),
                         inside(HomePlace, To).

?- moved(Who, 'India', 'Canada').
/* Who = 'Priya'. */
```

(The book's rules are `within_recursive` and `migrated`, querying `'United States'` to `'Europe'` and answering `Who = 'Lucy'`.)

How rules work:

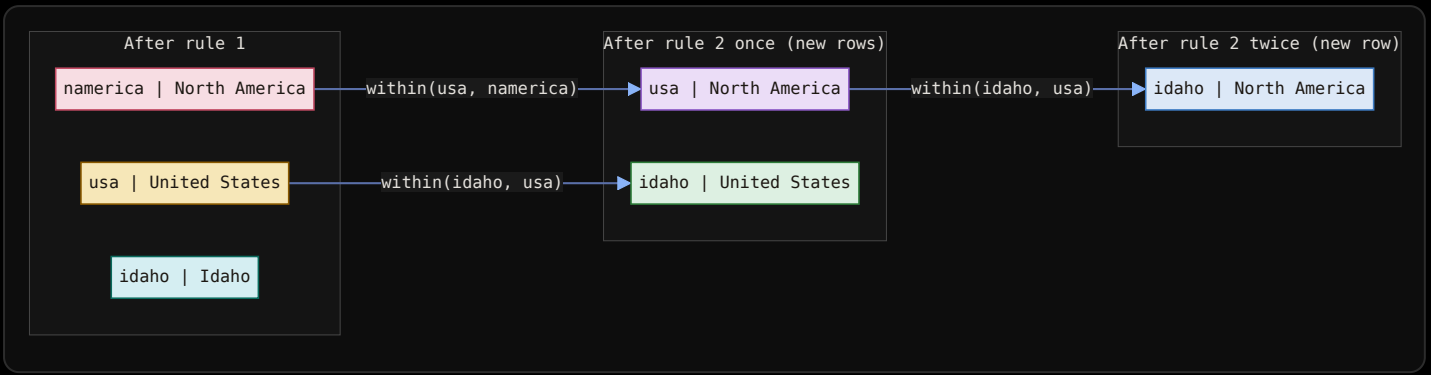
- Datalog proceeds **a small step at a time**: rules define **new predicates** (`inside`, `moved`) that are **derived**, not stored.
- Rules can call other rules or themselves **recursively**, building complex queries piece by piece.
- **Uppercase words are variables**; `name(Place, Name)` matches `name(asia, 'Asia')` with `Place = asia`, `Name = 'Asia'`.
- A rule **applies** when **all** predicates right of `:-` match; then the left side (variables substituted) is as if **added to the database**.

One possible evaluation (our data):

1. `name(asia, 'Asia')` exists → **rule 1** generates `inside(asia, 'Asia')`.
2. `within(india, asia)` + step 1 → **rule 2** generates `inside(india, 'Asia')`.
3. `within(kerala, india)` + step 2 → **rule 2** generates `inside(kerala, 'Asia')`.

Repeating rules 1 and 2 lists every location inside Asia (or any named place).

Figure 2-6 (redrawn): the within_recursive table (Location, Name) as the book's rules are applied. Rule 1 seeds each place with its own name; each pass of rule 2 copies a name one level down. After two passes, (idaho, North America) appears.



Rule 3 then finds people born in some place `BornIn` and living in some place `LivingIn`; the book's query sets `BornIn = 'United States'`, `LivingIn = 'Europe'` and leaves the person as variable `Who`, which gives the same answer as Cypher and SPARQL.

Trade-off. Datalog needs a different way of thinking, but **rules combine and are reused across queries**: less handy for simple one-off queries, better for **complex data**.

Summary

- **History:** data began as **one big tree** (**hierarchical model**), which handled **many-to-many** badly, so the **relational model** was invented. Applications that did not fit relational led **NoSQL** in two directions: **document databases** (self-contained documents, **rare** relationships between them) and **graph databases** (**anything may relate to everything**).
- All three models are widely used, each good in its domain. One can **emulate** another (graphs in relational tables), but **awkwardly** — hence different systems for different purposes.
- Document and graph databases usually **do not enforce a schema**, easing change; the app still assumes structure, so the question is **explicit (on write)** vs **implicit (on read)**.
- Each model has its query language: **SQL, MapReduce, MongoDB aggregation pipeline, Cypher, SPARQL, Datalog**; **CSS and XSL/XPath** show parallels.
- **Models not covered: genome data** needs **sequence-similarity search** (matching a very long DNA string against a database of similar ones), served by specialised software like **GenBank**; **particle physics** (the **Large Hadron Collider (LHC)** handles **hundreds of petabytes**, needing custom solutions to control hardware cost); **full-text search** (information retrieval; search indexes in Chapter 3 and Part III).
- Next chapter: trade-offs in **implementing** these models.

Interview angle

1. "Relational or document database — how do you choose?" *30-second answer:* Look at the relationships. A self-contained tree of one-to-many data usually loaded whole (a profile, an order with line items) suits documents: locality, schema flexibility, no shredding. Many-to-one or many-to-many relationships favour relational joins; emulating joins in app code is slower and more complex. Highly interconnected data → consider a graph database. The line is blurring (JSON in PostgreSQL/MySQL, joins in RethinkDB).

- *Follow-up: Is a document DB "schemaless"?* No, it is schema-on-read: code assumes structure the database does not enforce. Schema-on-write ≈ static typing, schema-on-read ≈ dynamic typing.
- *Follow-up: When is document locality a disadvantage?* When you need a small part of a large document (whole thing is loaded) or when updates grow it (usually rewritten). Keep documents small.

2. "What is normalization and why store IDs instead of text?" *30-second answer:* Store each human-meaningful fact once and reference it by ID. IDs never change, so renames are one update with no inconsistent copies, plus consistent spelling, disambiguation, localisation and better search. The cost is many-to-one relationships needing joins, which document databases handle poorly.

- *Follow-up: When denormalize?* To avoid joins for read performance, accepting extra write work and inconsistency risk.

3. "Why is SQL declarative and why does that matter?" *30-second answer:* You describe the result; the optimizer chooses indexes, join methods and order. Queries are shorter, the database can improve or reorganise storage without breaking queries (no promised order), and parallel execution is easier as CPUs gain cores rather than speed.

- *Follow-up: Where does MapReduce fit?* In between: map and reduce code snippets (pure functions) run by the framework. MongoDB added the declarative aggregation pipeline in 2.2 because coordinating two functions is harder and limits optimisation.

4. "When would you use a graph database, and how does it differ from the old network model?" *30-second answer:* When many-to-many relationships are common and complex and queries traverse a variable number of hops (social networks, web graph, road networks, heterogeneous data like Facebook's graph). Unlike CODASYL: no schema restricting links, direct access by ID or index, unordered children, declarative languages like Cypher and SPARQL.

- *Follow-up: Can I do this in SQL?* Yes, with recursive CTEs (`WITH RECURSIVE`, SQL:1999), but clumsily: 29 lines of SQL vs 4 of Cypher in the book's example.
- *Follow-up: Property graph vs triple-store?* Mostly equivalent; triples express properties (object is a value) and edges (object is a vertex) as (subject, predicate, object).

5. "Why did the relational model beat the network model?" *30-second answer:* CODASYL made programmers navigate pointer access paths by hand, so code was complex and hard to change. Relational exposed plain tables and let an optimizer choose access paths, so new indexes and queries needed no code rewrites. Build the optimizer once and every app benefits.

Self-check

1. **What are the four layers of data models the chapter describes, and which one does this chapter focus on?** Application objects/data structures; general-purpose model (JSON/XML, tables, graph); bytes in memory/disk/network; electrical/optical/magnetic hardware. The chapter focuses on the second.
2. **Name the four driving forces behind NoSQL.** Greater scalability (large datasets, high write throughput); preference for free/open source; specialised queries poorly supported by relational; frustration with restrictive schemas.
3. **Why does a document database struggle once résumés gain organisation pages and recommendations?** These add many-to-one and many-to-many links; with weak join support you must denormalize or emulate joins in application code.
4. **How is IMS similar to modern document databases?** Its hierarchical model stored nested record trees like JSON, handled one-to-many well, made many-to-many hard, and lacked joins, forcing duplication

or manual reference resolution.

5. **What was an "access path" in CODASYL, and what replaced it in relational databases?** A programmer-chosen chain of pointer links from a root record; replaced by the query optimizer choosing execution paths automatically.
6. **Scenario: you need to split a full-name field into a new `first_name` column across 500 million rows in MySQL with no downtime. What does the chapter warn about, and what workaround does it suggest?** MySQL copies the entire table on `ALTER TABLE` (possible hours of downtime; external tools exist), and a mass `UPDATE` is slow on any database. Add the column with a NULL default and fill values at read time, schema-on-read style.
7. **Why can a database reorder records safely under SQL but not under imperative query code?** SQL promises no order; imperative code might depend on iteration order, and the database cannot tell.
8. **In the book's MapReduce example, what do `map` and `reduce` output for the two December 1995 shark sightings?** Two emits for key `"1995-12"` with values 3 and 4; reduce receives `[3, 4]` and returns 7.
9. **Why must map and reduce be pure functions?** So the database can run them anywhere, in any order, and rerun them after failures.
10. **Give three ways graph databases differ from the CODASYL network model.** Any three of: no schema restricting connections; direct access by ID or index; unordered vertices/edges; declarative query languages.

Chapter 3 — Storage and Retrieval

Book: Chapter 3, pp. 69–110

Overview

A database's core jobs: store data safely, and give it back when asked. Chapter 2 took the application developer's view (data models, query languages); this chapter looks *inside*: how does a **storage engine** (the part that lays bytes out on disk or in memory and finds them again) work?

Why care if you will never write one? You still have to *pick* and *tune* one, which needs a rough model of what it does. Engines for **transactional** workloads (many small user reads/writes) behave very differently from engines for **analytics** (a few huge scans).

Three parts:

1. **Indexes for OLTP engines** (online transaction processing, defined below): *log-structured* engines (append-only files, e.g. LSM-trees) and *page-oriented* engines (update-in-place, e.g. B-trees), plus other index kinds and in-memory databases.
2. **OLTP vs analytics**: why companies build separate *data warehouses*, and star/snowflake schemas.
3. **Column-oriented storage**: storing by column, compression, sorting, writing, precomputed aggregates.

Key terms

Term	Plain-English meaning
Storage engine	Component that stores data on disk/memory and retrieves it.
Log	An append-only sequence of records (not necessarily human-readable).
Index	Extra structure derived from the main data to find records quickly. Speeds reads, slows writes.
Key-value store	Maps a key to a value, like a hash map.
Hash index	In-memory hash map from each key to the **byte offset** (position in the file) of its value.
Segment	One chunk of the log; a file closed once it reaches a certain size.
Compaction / merging	Rewriting a segment keeping only the latest value per key / combining segments into one new one.
Tombstone	Special record meaning "this key was deleted".
Checksum	Small value computed from data to detect corruption (e.g. half-written records).
SSTable (Sorted String Table)	Segment file with key-value pairs sorted by key, each key once.
Memtable	In-memory balanced tree (e.g. red-black) of recent writes, sorted by key.
LSM-tree (Log-Structured Merge-Tree)	Memtable plus a cascade of SSTables merged in the background.
Bloom filter	Memory-cheap structure that can say "this key is definitely not here".
Size-tiered / leveled compaction	Two strategies for choosing when and which SSTables to merge.
B-tree	Most common index: a balanced tree of fixed-size **pages** (usually 4 KB) updated in place. **Leaf pages** hold the keys; **branching factor** = child references per page (typically several hundred).
Write-ahead log (WAL) / redo log	Append-only file recording each B-tree change before pages are modified, for crash recovery.
Latch	Lightweight lock protecting a data structure from concurrent threads.
Copy-on-write	Write a modified copy of a page elsewhere and update parents, instead of overwriting.
B+ tree	B-tree variant whose interior pages store abbreviated keys.
Write amplification	One logical write causing multiple physical disk writes over time.
Fragmentation	Unused gaps inside files/pages.
Primary key / secondary index	Uniquely identifies a row/document/vertex / index on a non-unique field.
Heap file	Rows stored in no particular order; indexes point into it.
Forwarding pointer	Note at a record's old heap location pointing to its new one.
Clustered / covering index	Stores the whole row in the index / stores some columns (included columns) so some queries need only the index.
Concatenated / multi-dimensional index	Several columns joined into one key in fixed order / filters on several columns at once (e.g. lat *and* lon).

Term	Plain-English meaning
R-tree / space-filling curve	Specialised spatial index / maps multi-D points to one number for a normal B-tree.
Edit distance / Levenshtein automaton	Number of single-letter insertions, deletions, substitutions between words / state machine finding words within a given edit distance.
In-memory database	Serves all reads from RAM (may log to disk for durability).
Anti-caching	Evicting least-recently-used records from an in-memory DB to disk, reloading on access.
NVM (non-volatile memory)	Memory that keeps contents without power.
Transaction (here)	Group of reads and writes as one logical unit (need not be ACID).
OLTP / OLAP	Online transaction / analytic processing: many small key-based user reads and writes / few queries scanning huge numbers of rows for aggregates.
Business intelligence	Reports and analyses that help management decide.
Data warehouse / ETL	Separate read-only-copy database for analysts / Extract–Transform–Load process feeding it.
Star schema (dimensional modeling) / snowflake schema	Central **fact table** (one row per event) surrounded by **dimension tables** (who/what/where/when/how/why) / dimensions split into sub-dimensions.
Row- / column-oriented storage	All values of a row adjacent / all values of a column together, each column in its own file.
Bitmap / run-length encoding	Per distinct value, one bit per row / store "value, count" runs.
Vectorized processing / SIMD	Tight loops over compressed column chunks in CPU cache / CPU instructions processing many values at once.
Sort key	Column(s) by which a column store orders rows.
Materialized / virtual view	Stored copy of query results kept up to date / saved query expanded on each use.
Data cube / OLAP cube	Grid of precomputed aggregates grouped by several dimensions.

Data Structures That Power Your Database

The world's simplest database. The book builds a key-value store from two **Bash** functions (`db_set`, `db_get`) storing JSON documents about cities. Our equivalent:

```
#!/bin/bash
put () {                        # usage: put <key> <value>
  echo "$1,$2" >> store.log     # append "key,value" as a new line at the end of the file
}
fetch () {                      # usage: fetch <key>
  grep "^$1," store.log |       # find every line whose key matches
    sed -e "s/^$1,//" |         # strip the "key," prefix, leaving only the value
    tail -n 1                   # keep only the LAST match = the newest value
}
```

`put` never edits old lines, so all versions of a key stay; `fetch` takes the **last** match because the newest write wins (`put 7 '{"city":"Oslo"}'` then `put 7 '{"city":"Bergen"}'` → `fetch 7` returns Bergen).

- **Writes are fast:** appending is about the cheapest disk operation, and many real databases use exactly this internally: a **log**.
- **Reads are slow:** `fetch` scans the whole file, **O(n)** — double the records, double the lookup time.

> **"Log" in this book** means any *append-only sequence of records*, possibly binary, not debug text output.

Real databases also handle concurrency, stopping the log growing forever, and crashes leaving half-written records — but the principle stays.

Indexes. Extra metadata acting as a signpost to find keys without scanning:

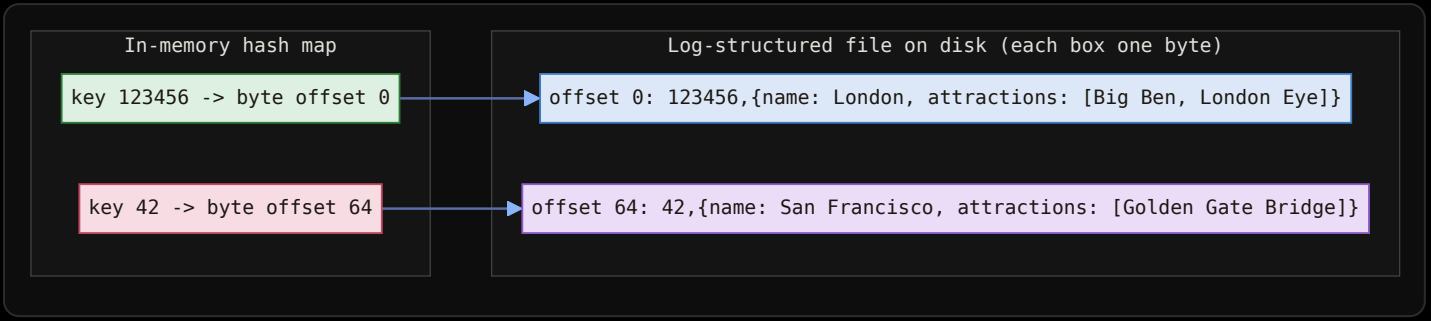
1. An index is **derived** from the primary data; adding or dropping one changes only query speed.
2. Every index must be updated on every write, so **indexes slow down writes** (nothing beats a plain append).
3. The core **trade-off**: indexes speed reads but cost writes, so databases do not index everything; the developer or DBA chooses based on actual query patterns.

Hash Indexes

What it is. An **in-memory hash map** (roughly constant-time lookup) mapping each key to the **byte offset** of its latest value in the data file.

How it works: **write** — append `key,value`, then set `hashmap[key] = offset` (new keys and updates alike); **read** — look up the offset, seek there, read the value.

Figure 3-1 (redrawn): an append-only CSV-like log of key-value records, with an in-memory hash map pointing at each key's byte offset



Real system: essentially **Bitcask**, the default storage engine of **Riak**. High-performance reads and writes **as long as all keys fit in RAM**; values may exceed memory, since a read costs one disk seek (or none if that part of the file is in the OS filesystem cache).

Good fit: few distinct keys, many updates each. Book example: key = cat-video URL, value = play count incremented on every play.

Problem: the log grows forever. Split it into **segments** (close a file at a certain size, start a new one); run **compaction** on closed segments (drop older duplicates, keep the latest value per key); and since compaction shrinks segments, **merge** several while compacting.

Figure 3-2 (redrawn): compacting one segment of a cat-video play-count log keeps only the newest value per key

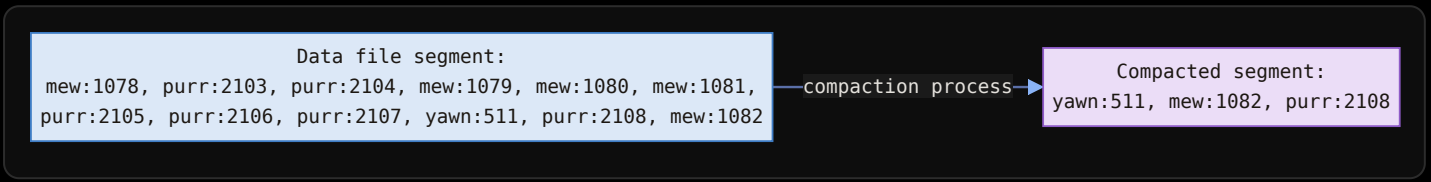
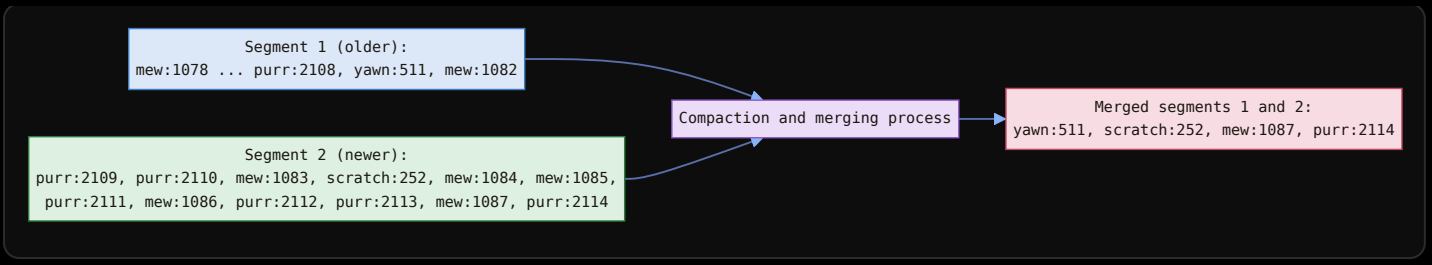


Figure 3-3 (redrawn): compaction and merging of two segments done together, producing one new segment



Merging details: segments are **never modified**; the result goes to a **new file**, built by a **background thread** while reads and writes continue on old segments; then reads switch over and old files are deleted. **Each segment has its own hash map**; reads check newest to oldest, and merging keeps the count small.

Practical issues in a real implementation:

Issue	How it's handled
File format	Binary (length in bytes, then raw bytes) instead of CSV: faster, no escaping.
Deleting records	Append a **tombstone**; merging then drops all earlier values for that key.
Crash recovery	In-memory maps vanish on restart; rebuilding by reading whole segments is slow, so Bitcask stores a **snapshot of each segment's hash map** on disk to load quickly.
Partially written records	Bitcask uses **checksums** to detect and ignore corrupt tails after a crash.
Concurrency control	Writes are strictly sequential, so typically **one writer thread**; immutable segments allow **many concurrent readers**.

Why append-only instead of overwriting in place?

- Appends and merges are **sequential writes**, much faster than random writes on HDDs and, to a lesser degree, SSDs.
- **Concurrency and crash recovery are simpler** with immutable files: no value left half old, half new by a crash mid-overwrite.
- **Merging avoids fragmentation** over time.

Limitations:

- **The hash table must fit in memory**; on-disk hash maps perform poorly (random I/O, costly growth, fiddly collision handling).
- **Range queries are inefficient**: scanning `kitty00000`–`kitty99999` requires looking up each key individually.

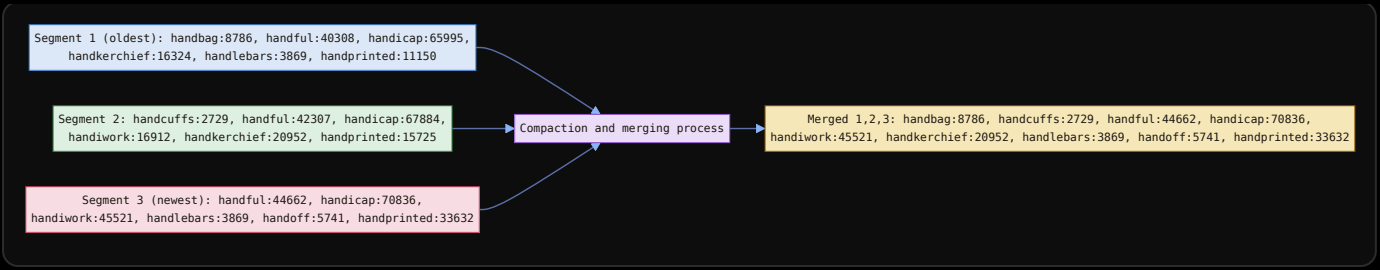
SSTables and LSM-Trees

The change. Require each segment to be **sorted by key**, with each key **once** per merged segment: a **Sorted String Table (SSTable)**.

Three advantages over hash-indexed log segments:

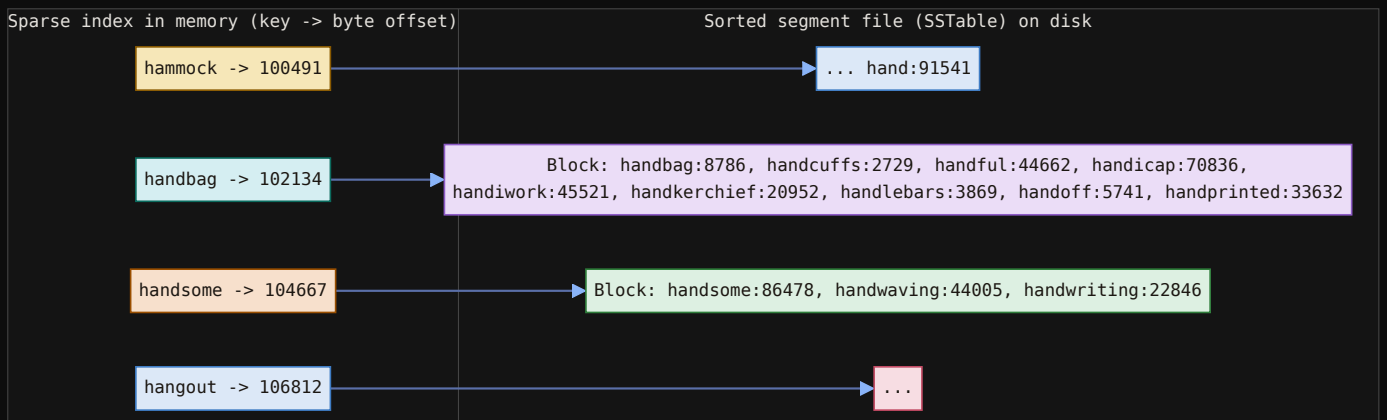
1. **Merging is simple and efficient even for files bigger than memory**, like mergesort's merge step: read the inputs side by side, repeatedly emit the lowest current key; the output is sorted. For a key in several inputs, keep the **newest segment's** value (each segment covers a time period, assuming adjacent segments are merged).

   Figure 3-4 (redrawn): merging three SSTable segments mergesort-style; for a key found in several segments, the value from the newest segment wins



2. **Not every key needs to be in memory.** A **sparse** index suffices: to find `handiwork` knowing only the offsets of `handbag` and `handsome`, jump to `handbag` and scan forward. One entry per **few kilobytes** is enough because scanning a few KB is fast. (With fixed-size records a binary search would need no index at all, but records vary in length.)
3. **Blocks can be compressed.** Group neighbouring records into a block and compress it before writing; each sparse-index entry points to a block start. Saves disk space *and* I/O bandwidth.

Figure 3-5 (redrawn): an SSTable with a sparse in-memory index; each entry points to the start of a compressible block of the sorted file



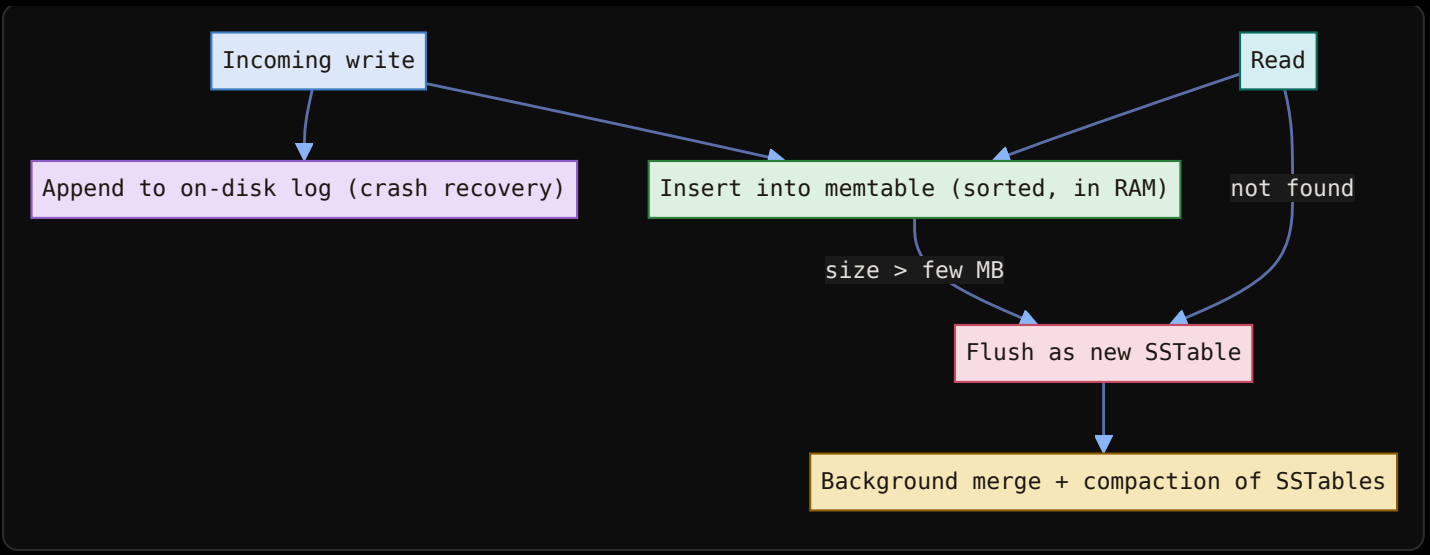
Constructing and maintaining SSTables

Writes arrive in random key order. A sorted structure **on disk** is possible (B-tree), but **in memory** it is easy with a self-balancing tree such as a **red-black tree** or **AVL tree**: insert in any order, read back sorted.

How the engine works:

1. **Write:** insert into the in-memory balanced tree, the **memtable**.
2. **Flush:** when it exceeds a threshold (typically a **few megabytes**), write it out as a new SSTable (already sorted), which becomes the newest segment; new writes go to a fresh memtable meanwhile.
3. **Read:** memtable, then newest segment, then older ones.
4. **Background:** merge and compact segments, dropping overwritten and deleted values.

The crash problem. A crash loses the memtable. Fix: also append every write to a separate **unsorted log** on disk, used only to rebuild the memtable after a crash and discarded once that memtable is flushed.



Making an LSM-tree out of SSTables

- This is essentially **LevelDB** and **RocksDB**, key-value engine libraries embedded into other applications; LevelDB can replace Bitcask in **Riak**.
- Similar engines power **Cassandra** and **HBase**, inspired by Google's **Bigtable** paper, which coined *SSTable* and *memtable*.
- First described by **Patrick O'Neil et al.** as the **Log-Structured Merge-Tree (LSM-Tree)**, building on log-structured filesystems; engines that merge and compact sorted files are **LSM storage engines**.
- **Lucene** (behind **Elasticsearch** and **Solr**) uses a similar approach for its **term dictionary**: a full-text index maps each word (**term**) to the IDs of documents containing it (the **postings list**), kept in SSTable-like sorted files merged in the background.

Performance optimizations

- **Missing-key lookups are slow** (memtable plus every segment back to the oldest, possibly disk reads each). **Bloom filters** approximate set contents and say for certain a key is *absent*, saving those reads.
- **Compaction strategies:**

Strategy	How it works	Used by
Size-tiered	Newer, smaller SSTables are successively merged into older, larger ones.	HBase; Cassandra (supports both)
Leveled	Key range split into smaller SSTables; older data moves into separate "levels"; more incremental, less disk space.	LevelDB (hence the name), RocksDB; Cassandra

Why LSM-trees work well: a simple, powerful idea (a cascade of SSTables tidied by background merging) that works with data far larger than memory, supports **range queries** (data is sorted), and gives **very high write throughput** (sequential disk writes).

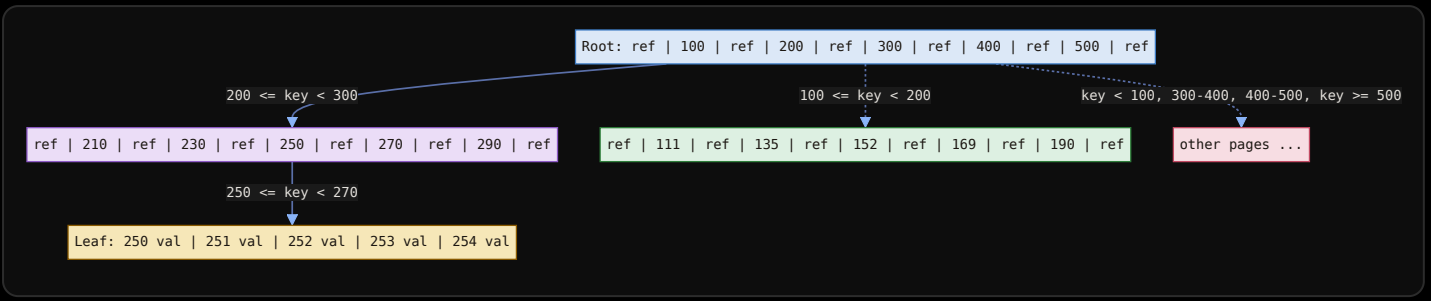
B-Trees

What it is. The **most widely used** index: introduced in **1970**, called "ubiquitous" less than 10 years later, still standard in almost all relational and many non-relational databases. Like SSTables, keys are sorted (efficient lookups and range queries), but the design philosophy differs:

	Log-structured (LSM)	B-tree
Unit of storage	Variable-size segments (several MB+)	Fixed-size **pages** (traditionally **4 KB**, sometimes bigger)
Write style	Always write a whole segment sequentially	Read/write one page at a time, overwriting in place
Hardware match	—	Close to how disks are organised (fixed-size blocks)

How it works: each page has an address, so pages reference each other like on-disk pointers, forming a **tree**. Lookups start at the **root**. A page holds keys interleaved with child references; each child covers a **continuous key range** bounded by the keys around its reference. Follow the matching reference down to a **leaf page**, which holds the keys with values inline or references to them. Book example: for key **251**, follow the root's reference between **200** and **300**, and keep descending.

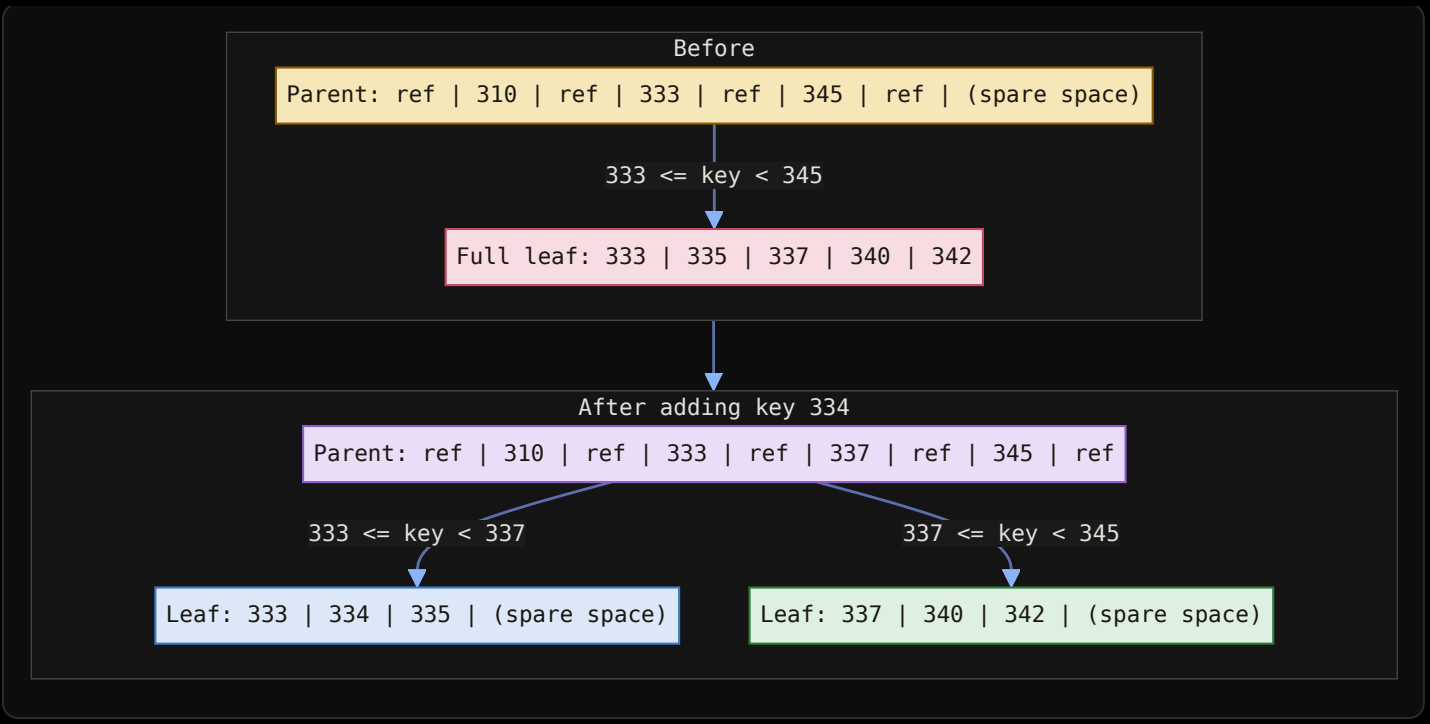
Figure 3-6 (redrawn): looking up user_id 251 by descending from the root through range boundaries to a leaf page



Branching factor: six in the figure, **several hundred** in practice (depends on the space references and boundaries take).

Updating and inserting: to **update**, find the leaf, change the value, write the page back (references stay valid). To **insert**, add the key to the page covering its range; if **full**, **split** it into two half-full pages and update the **parent**.

Figure 3-7 (redrawn): adding key 334 to a full leaf splits it into two pages with spare space; the parent gains the new boundary key 337



- The tree stays **balanced**: depth **O(log n)**; most databases fit in **three or four levels**.
- **Numbers to remember: 4 KB pages**, **branching factor 500**, **four levels** → up to **256 TB**.
- Insertion is intuitive; deletion while keeping balance is more involved.

Making B-trees reliable

- The basic B-tree write **overwrites a page in place**, assuming its location never changes — the opposite of LSM-trees, which only append and delete whole files. On an HDD that means moving the head, waiting for the platter to rotate, and rewriting the sector; an SSD must erase and rewrite a fairly large block of its chip.
- **Multi-page operations are dangerous**: a split writes two children *and* the parent; a crash halfway can corrupt the index (e.g. an **orphan page** with no parent).
- **Fix — the write-ahead log (WAL, a.k.a. redo log):** every change is appended to it **before** touching tree pages; after a crash it is replayed to restore a consistent tree.
- **Concurrency is harder:** threads could see the tree mid-change, so B-trees use **latches**. Log-structured engines merge in the background and swap segments atomically, without disturbing queries.

B-tree optimizations

- **Copy-on-write (append-only B-trees)** instead of overwrite + WAL (e.g. **LMDB**): write the modified page elsewhere and create new parent versions pointing to it. Also helps concurrency control (snapshot isolation, Chapter 7).
- **Abbreviated keys:** interior pages store only enough of a key to separate ranges → higher branching factor, fewer levels (a **B+ tree**, though the optimisation is so common it is rarely singled out).
- **Sequential leaf layout:** pages can be anywhere, so range scans may seek per page; many implementations try to lay leaves out in key order, hard to maintain as the tree grows. LSM-trees do this naturally by rewriting large chunks when merging.
- **Sibling pointers:** each leaf links to its neighbours, so ordered scans need not go back up to parents.
- **Fractal trees:** B-tree variants borrowing log-structured ideas to reduce seeks (nothing to do with fractals).

Comparing B-Trees and LSM-Trees

Rule of thumb: LSM-trees are usually **faster for writes**; B-trees are thought to **read faster** (LSM reads may check several structures and SSTables at different compaction stages). Benchmarks are often inconclusive and workload-dependent — test *your* workload.

Advantages of LSM-trees

- **Write amplification.** A B-tree writes data **at least twice** (WAL, then the page, plus more on splits), writes a **whole page** even for a few changed bytes, and some engines write a page **twice** to avoid a torn (partially written) page on power loss. LSM-trees also rewrite data repeatedly through compaction. One database write causing multiple disk writes over its lifetime is **write amplification**; it matters especially on **SSDs**, which wear out after a limited number of overwrites per block.
- **Higher write throughput.** When disk bandwidth is the bottleneck, every extra byte written costs throughput. LSM-trees often sustain more writes because they *sometimes* have lower write amplification (configuration- and workload-dependent) and write compact SSTables **sequentially** rather than overwriting scattered pages — most important on magnetic disks.
- **Better compression, smaller files.** B-trees waste space through **fragmentation** (after splits, or rows not fitting a page); LSM-trees periodically rewrite SSTables and have lower overhead, **especially with leveled compaction**.
- **On SSDs**, firmware often internally uses a log-structured algorithm to turn random writes sequential, so the engine's pattern matters less — but lower write amplification and less fragmentation still let more requests fit in the I/O bandwidth.

Downsides of LSM-trees

- **Compaction can interfere with live traffic:** requests may wait behind expensive compaction. Average throughput and response time barely suffer, but **high-percentile (tail) latencies** can be quite high. **B-trees are more predictable.**
- **Compaction can fall behind at high write rates.** Disk bandwidth is shared between initial writes (logging, memtable flushes) and compaction, which needs more as data grows. If it cannot keep up, **unmerged segments pile up** until the disk fills, and reads slow as they check more files. SSTable engines usually **don't throttle** incoming writes, so **monitor explicitly**.
- In a B-tree **each key lives in exactly one place**; an LSM engine may hold copies in several segments. So B-trees suit **strong transactional semantics**: many relational databases implement isolation with **locks on key ranges** attached directly to the B-tree (Chapter 7).

Aspect	B-tree	LSM-tree
Typical strength	Reads, predictable latency	Writes, throughput
Write amplification	WAL + full page writes (sometimes twice)	Repeated compaction (sometimes lower overall, depending on configuration and workload)
Disk pattern	Random page overwrites	Sequential segment writes
Space usage	Fragmentation leaves empty space	More compact, better compression
Background work	No background compaction	Compaction may hurt tail latency / fall behind
Key location	Exactly one place (good for range locks)	Possibly many segments
Maturity	Very mature, ingrained in databases	Increasingly popular in new datastores

Conclusion: B-trees are deeply embedded and consistently good, so they won't disappear; log-structured indexes are increasingly popular in new systems. No quick rule — test empirically.

Other Indexing Structures

So far: **key-value indexes**, like a **primary key** index (uniquely identifying a row, document, or vertex; other records refer to it by that key).

Secondary indexes are very common (`CREATE INDEX`), often essential for efficient joins:

```sql
-- A secondary index on a non-unique column
CREATE INDEX idx_orders_customer ON orders (customer_id);
-- Now this query can use the index instead of scanning every order:
SELECT * FROM orders WHERE customer_id = 812;
```

The difference from a primary index: **keys are not unique**. Fix by making each value a **list of matching row IDs** (like a postings list), or making keys unique by **appending the row ID**. Either way, secondary indexes can be **B-trees or log-structured**.

Storing values within the index

An index entry's value is either **the row itself** or a **reference** to it.

- **Heap file:** rows stored in no particular order (append-only, or reusing deleted space). With several secondary indexes, data lives in **one place** and each index points to it. Updating a value **without changing the key** can overwrite in place if not larger; if it **grows** and moves, update **all indexes** or leave a **forwarding pointer**.
- **Clustered index:** when the extra hop to the heap hurts reads, store the **whole row in the index**. **MySQL InnoDB**: the primary key is always clustered, and secondary indexes refer to the **primary key**, not a heap location. **SQL Server**: **one** clustered index per table.
- **Covering index / index with included columns:** store **some** columns in the index so certain queries are answered from it alone (the index **covers** the query):

```sql
-- SQL Server-style covering index
CREATE INDEX idx_emp_dept ON employees (dept_id) INCLUDE (salary);
SELECT salary FROM employees WHERE dept_id = 4;  -- answered from the index alone
```

Trade-off: clustered and covering indexes duplicate data → faster reads but more storage, write overhead, and effort to keep transactional guarantees (no visible inconsistency from the duplication).

Option	What the index stores	Read cost	Write/storage cost
Nonclustered (heap)	Pointer to row	Extra hop	Lowest
Covering	Some columns	No hop for covered queries	Medium
Clustered	Entire row	No hop	Highest

Multi-column indexes

- **Concatenated index** (most common): several columns joined into one key in fixed order. Like a **phone book** sorted by (last name, first name): finds a last name or last+first, but is **useless for first name alone**.
- **Multi-dimensional indexes:** important for **geospatial** data. A restaurant map needs everything inside the visible rectangle — a **2D range query**:

```sql
SELECT * FROM cafes
WHERE lat BETWEEN 48.850 AND 48.862      -- north-south band
  AND lon BETWEEN 2.330  AND 2.355;      -- east-west band
```

A normal B-tree or LSM index gives all rows in the latitude band (any longitude) *or* the longitude band, not both efficiently. **Solutions:** map 2D points to one number with a **space-filling curve** and use a B-tree; or, more commonly, a spatial index like an **R-tree** — **PostGIS** builds them on PostgreSQL's **Generalized Search Tree (GiST)**.

Not just maps: a 3D index on **(red, green, blue)** for colour search in e-commerce; **(date, temperature)** for weather readings in **2013** between **25 and 30 ℃** without scanning all of 2013 (or vice versa). **HyperDex** uses this technique.

Full-text search and fuzzy indexes

The indexes above need **exact** keys or ranges; **fuzzy** querying finds **similar** keys (e.g. misspellings).

- Full-text engines expand to **synonyms**, ignore **grammatical variations**, find words **near each other**, and more.
- **Lucene** finds words within a given **edit distance** (1 = one letter added, removed or replaced).
- Lucene's term dictionary is SSTable-like with a small in-memory index: where **LevelDB** uses a sparse key set, **Lucene** uses a **finite state automaton** over the keys' characters (like a **trie**), which can be turned into a **Levenshtein automaton** to search efficiently within an edit distance.
- Other fuzzy techniques lean toward document classification and machine learning (see information-retrieval texts).

Keeping everything in memory

We tolerate awkward disk layouts because disks are **durable** and **cheaper per GB** than RAM. As RAM gets cheaper, many datasets fit in memory (possibly across machines): **in-memory databases**.

- **Cache-only** stores like **Memcached** accept losing data on restart.
- **Durable** ones use battery-backed RAM, a **change log** on disk, **periodic snapshots**, or **replication** (Chapter 5), reloading from disk or a replica on restart. Disk is only an append-only durability log — **all reads come from RAM** — and disk files also ease backups and external analysis.

Named systems: VoltDB, MemSQL, Oracle TimesTen (in-memory relational; vendors claim big speedups from dropping on-disk structure overhead); **RAMCloud** (open-source durable in-memory key-value store, log-structured in RAM and on disk); **Redis, Couchbase** (weak durability: write to disk **asynchronously**, so a crash can lose recent writes).

> **Common confusion:** in-memory databases are *not* fast mainly because they skip disk reads — a disk-based engine with enough RAM may never read disk either, thanks to the OS cache. The real win is avoiding **encoding in-memory structures into a disk-friendly format**.

Richer data models: structures hard to build on disk, e.g. **Redis** priority queues and sets, with a comparatively simple implementation.

Anti-caching: research suggests in-memory systems can exceed RAM by evicting **least recently used** (LRU) records to disk and reloading on access — like OS virtual memory/swap, but per **individual record** rather than whole pages. Indexes must still fit in memory (like Bitcask).

Future: non-volatile memory (NVM) may require further engine design changes; an active research area.

Transaction Processing or Analytics?

Early database writes usually matched a **commercial transaction** (a sale, supplier order, salary payment). The term **transaction** stuck, now meaning a group of reads and writes forming one logical unit.

> **Note:** here a transaction need not be ACID (Chapter 7). *Transaction processing* just means **low-latency** reads and writes, versus periodic **batch processing** jobs (Chapter 10).

OLTP. Blog comments, game actions, contacts — all look like business transactions: find a few records by key via an index; insert or update based on user input. Interactive, hence **online transaction processing (OLTP)**.

OLAP. Analytics is different: scan a **huge number of records**, read **a few columns** each, compute **aggregates** (count, sum, average) rather than return raw rows. E.g. total revenue per store in January? Banana sales during the latest promotion vs normal? Which baby-food brand is most often bought with brand X diapers? Such queries feed **business intelligence** reports: **online analytic processing (OLAP)** (the "online" is vague; probably means interactive exploratory queries).

Table 3-1 (paraphrased): OLTP vs OLAP characteristics

Property	OLTP	OLAP
Main read pattern	Each query fetches a few records by key	Aggregates computed across very many records
Main write pattern	Low-latency, random-access writes driven by user input	Event stream or bulk import (ETL)
Primarily used by	Customers/end users, through a web app	Internal analysts supporting decisions
What data represents	Latest state (current point in time)	History of events over time
Dataset size	GB up to TB	TB up to PB

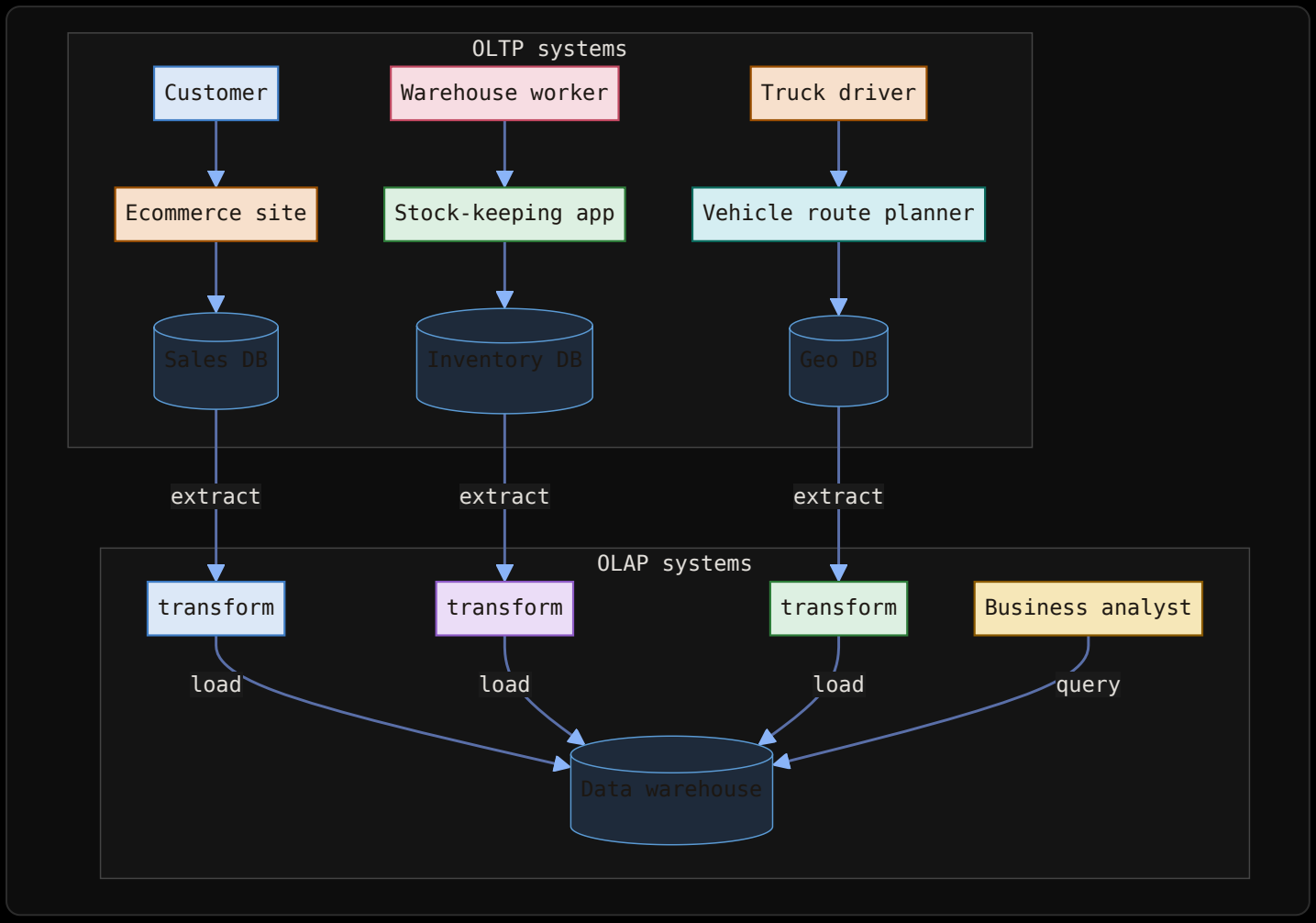
SQL handles both, and at first the same databases served both; from the **late 1980s and early 1990s** companies moved analytics to a **separate** database: the **data warehouse**.

Data Warehousing

A large company may run **dozens** of OLTP systems (website, point-of-sale, inventory, route planning, supplier management, HR…), each complex, with its own team. They must be **highly available** and **low-latency**, so DBAs resist analysts running expensive **ad hoc** scans on them.

A **data warehouse** is a separate database holding a **read-only copy** of data from all OLTP systems, which analysts can query freely. Data arrives via **Extract–Transform–Load (ETL)**: **extract** from OLTP databases (periodic dump or continuous update stream), **transform** into an analysis-friendly, cleaned schema, **load** into the warehouse.

Figure 3-8 (redrawn): simplified ETL. Each OLTP system is extracted and transformed separately, then loaded into one data warehouse that business analysts query



- Common in **large** enterprises, rare in **small** ones (few OLTP systems, little data — a normal SQL database or spreadsheet suffices).
- **Big advantage:** optimised for analytic access; the OLTP indexes above are **poor at analytic queries**.

The divergence between OLTP databases and data warehouses

- Warehouses are usually **relational** (SQL suits analytics); graphical tools generate SQL, visualise results, and support **drill-down** (from a total to its details) and **slicing and dicing** (cutting by different attributes).
- Both speak SQL, but **internals differ greatly**; vendors increasingly specialise in one. **Microsoft SQL Server** and **SAP HANA** support both, increasingly as two engines behind one SQL interface.
- **Commercial warehouses:** Teradata, Vertica, SAP HANA, ParAccel (expensive licences); **Amazon RedShift** is hosted ParAccel.
- **Open-source SQL-on-Hadoop** (younger): Spark SQL, Apache Hive, Apache Drill, Apache Tajo, Facebook Presto, Cloudera Impala; some build on Google's **Dremel**.

Stars and Snowflakes: Schemas for Analytics

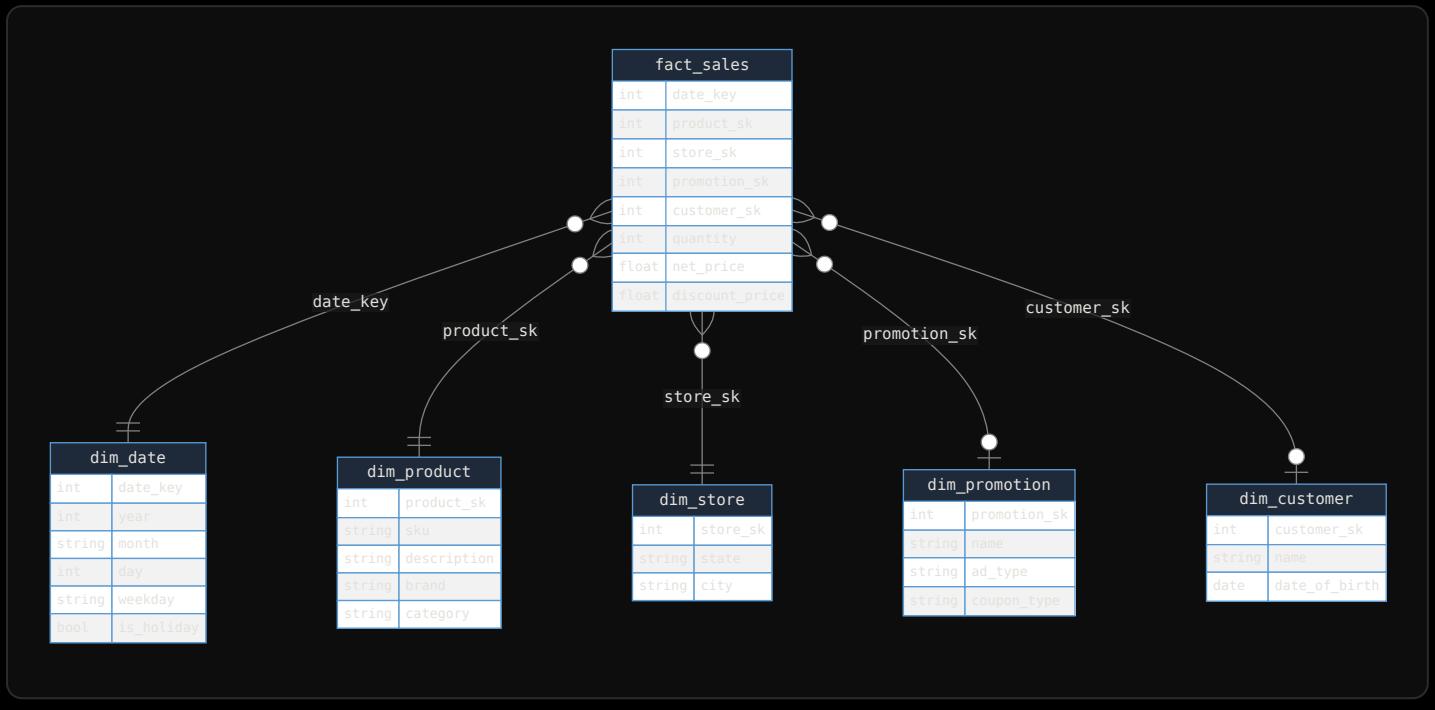
Analytics uses far fewer data models than OLTP; most warehouses use a **star schema** (**dimensional modeling**).

Fact table (e.g. `fact_sales` for a grocery chain): each row is an **event** at a particular time (here, a customer buying a product; in web analytics, a page view or click).

- Recording individual events maximises flexibility, but fact tables become **enormous**: **Apple, Walmart, eBay** may hold **tens of petabytes**, mostly facts.
- Some columns are **attributes** (sale price, supplier cost → profit margin); others are **foreign keys** to **dimension tables** describing the event's **who, what, where, when, how and why**.

Dimension tables: e.g. `dim_product`, one row per product type (SKU, description, brand, category, fat content, package size…); each fact row references the product sold (several products in one purchase = several fact rows). Even **date and time** get a dimension so extra info like **public holidays** can be attached and compared.

Figure 3-9 (redrawn): star schema for a grocery retailer. fact_sales sits in the centre; its foreign keys point at five dimension tables (sample rows in the book: product 31 = Fish food by Aquatech, store 3 = Palo Alto CA, promotion 19 = Aquarium deal, customer 191 = Bob, date 140102 = Thu 2 Jan 2014)



- **Why "star":** the fact table in the middle with dimensions around it like rays.
- **Snowflake schema:** dimensions split into sub-dimensions (e.g. separate `brand` and `category` tables referenced by `dim_product`): more **normalised**, but stars are usually preferred as **simpler for**

analysts.

- **Tables are wide:** fact tables often exceed **100 columns**, sometimes several hundred; dimensions too (e.g. `dim_store` with services offered, in-store bakery, floor area, opening date, last remodel, distance to highway).

Column-Oriented Storage

Fact tables can have **trillions of rows** and **petabytes**; dimension tables are much smaller (**millions** of rows), so focus on **facts**.

Key observation: a typical warehouse query touches only **4 or 5** of 100+ columns (`SELECT *` is rare). The book's Example 3-1 asks whether people prefer fresh fruit or candy by weekday across all of 2013, touching only `date_key`, `product_sk` and `quantity`. A similar query of ours:

```sql
SELECT d.month, s.region, SUM(f.net_price) AS revenue   -- only a few columns needed
FROM fact_sales f
  JOIN dim_date  d ON f.date_key = d.date_key            -- attach date details
  JOIN dim_store s ON f.store_sk = s.store_sk            -- attach store details
WHERE d.year = 2015 AND s.region IN ('North', 'West')    -- filter
GROUP BY d.month, s.region;                              -- one total per month per region
```

It reads just `date_key`, `store_sk` and `net_price` from `fact_sales`, but across a huge number of rows.

Why row storage struggles: OLTP databases are **row-oriented** (a row's values adjacent; document databases similarly). Even with indexes on `date_key` or `product_sk`, a row store loads whole 100+-attribute rows from disk, parses and filters them — slow.

The column-oriented idea: store each **column** together in its **own file**; queries read and parse only the columns they need. (Works for non-relational data too: **Parquet** is a columnar format supporting a document model, based on Google's **Dremel**.)

Figure 3-10 (redrawn): the book's eight fact_sales rows stored as one file per column; position k in every file belongs to row k



Crucial rule: every column file lists rows in the **same order**; row 23 = the 23rd entry of each file.

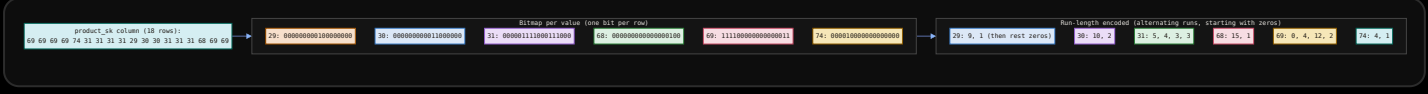
Column Compression

Columns are often repetitive, so **compression** cuts I/O further.

Bitmap encoding (especially effective in warehouses):

1. Columns usually have few **distinct values** relative to rows (**billions** of sales, **100,000** products).
2. For **n distinct values**, make **n bitmaps**, **one bit per row** (1 if the row has that value).
3. Very small n (e.g. ~**200** countries): store bitmaps as-is.
4. Larger n: bitmaps are mostly zeros (**sparse**), so add **run-length encoding** (lengths of alternating runs of 0s and 1s).

Figure 3-11 (redrawn): an 18-row product_sk column turned into one bitmap per distinct value, then run-length encoded



Bitmaps make common filters cheap: `WHERE product_sk IN (30, 68, 69)` → bitwise **OR** of three bitmaps; `product_sk = 31 AND store_sk = 3` → bitwise **AND** of two bitmaps (works because bit k is the same row in every column). Many other compression schemes exist.

> **Column-oriented storage vs column families (common confusion):** **Cassandra** and **HBase** have *column families* from **Bigtable**, but within a family they store all of a row's columns together with a row

> key and use no column compression. The Bigtable model is mostly **row-oriented**.

Memory bandwidth and vectorized processing

Besides disk-to-memory bandwidth, analytic engines care about moving data from **RAM into the CPU cache**, avoiding **branch mispredictions** and **bubbles** in the CPU pipeline, and using **SIMD** (single-instruction-multi-data) instructions. Column layouts help: the engine takes a chunk of compressed column data that fits in the **L1 cache** and loops over it tightly with **no function calls** — far faster than per-record code full of calls and branches. Compression fits **more rows** in L1, and operators like bitwise AND/OR can run **directly on compressed chunks**. This is **vectorized processing**.

Sort Order in Column Storage

Rows can stay in **insertion order** (insert = append to each column file), or be given a **sort order**, like SSTables, usable as an index.

- **Columns cannot be sorted independently** (you would lose which values form a row); sort **whole rows**.
- The DBA picks **sort keys** from common queries: if queries target date ranges, make `date_key` the **first** key so only that range is scanned. A **second** key orders ties, e.g. `date_key` then `product_sk` groups same-day sales of a product.

Sorting boosts compression: a first sort column with few distinct values forms **long runs**, so run-length encoding can shrink it to a **few kilobytes** even for **billions** of rows. The first key benefits most; later columns look nearly random. Still a net win.

Several different sort orders

From **C-Store**, adopted by **Vertica**: data is **replicated** to several machines anyway for fault tolerance, so store each replica **sorted differently** and let each query use the best-fitting one.

	Multiple sort orders (column store)	**Multiple secondary indexes (row store)**
Where the full data lives	Each sorted copy contains the column values themselves	Each row in one place (heap file or clustered index)
What the extra structures hold	No pointers — just columns of values	Pointers to matching rows

Writing to Column-Oriented Storage

Columnar storage, compression and sorting speed **reads** (the main warehouse load) but make **writes harder**: **update-in-place is impossible** with compressed columns, and inserting **midway** through a sorted table would likely mean **rewriting every column file** (rows are identified by position).

Solution: LSM-trees again. Writes go to a sorted **in-memory store** (row- or column-oriented, it doesn't matter); when enough accumulate, they are **merged** with on-disk column files into **new files in bulk** (essentially what **Vertica** does). **Queries** combine on-disk data with recent in-memory writes, and the **query optimizer hides** this, so inserts, updates and deletes appear immediately.

Aggregation: Data Cubes and Materialized Views

Not every warehouse is columnar, but column stores can be **much faster for ad hoc analytics** and are gaining popularity.

Materialized aggregates. Queries often use `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`; if many reuse the same aggregates, **cache** them rather than recompute from raw data.

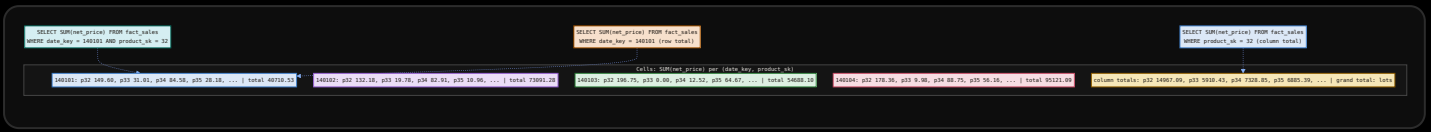
	Virtual view	Materialized view
What it is	A named query shortcut	The query's results really stored **on disk**
On read	SQL engine expands it into the underlying query on the fly	Read precomputed results directly
On data change	Nothing to do	Must be **updated** (denormalised copy); database can do it automatically
Cost	Slow reads for heavy queries	More expensive **writes**

Because of write cost, materialized views are rare in **OLTP** but more sensible in **read-heavy warehouses** (case-dependent).

```
-- PostgreSQL-style example
CREATE MATERIALIZED VIEW daily_store_revenue AS
  SELECT date_key, store_sk, SUM(net_price) AS revenue
  FROM fact_sales
  GROUP BY date_key, store_sk;          -- precompute totals per day per store
REFRESH MATERIALIZED VIEW daily_store_revenue;  -- recompute after new sales arrive
```

Data cube / OLAP cube: a materialized view that is a **grid of aggregates by several dimensions**. With two dimensions, **date** and **product**, each cell holds e.g. `SUM(net_price)` for that date–product pair; summing a row or column gives totals with one dimension removed (per date, or per product).

Figure 3-12 (redrawn): two dimensions of a data cube (date_key by product_sk). Each cell is SUM(net_price) for that pair; summing along a row gives the per-date total, along a column the per-product total



- More dimensions (Figure 3-9 has **five**: date, product, store, promotion, customer) make a **hypercube**, same idea.
- **Advantage:** some queries become near-instant (e.g. yesterday's sales per store = read totals along one dimension).
- **Disadvantage: less flexible** than raw data: e.g. the share of sales from items priced above **$100** is impossible if price isn't a dimension.
- So most warehouses **retain raw data** and use cubes only as a **speed-up for particular queries**.

Summary

- **Two categories** of storage engines: **OLTP** (user-facing, huge request volume, each query touches few records found by key via an index; **disk seek time** is often the bottleneck) and **analytics/OLAP** (business analysts, fewer queries each scanning millions of records; the limit is **disk bandwidth**, answered by **column-oriented storage**).
- **Two OLTP schools: log-structured** (only append and delete obsolete files; Bitcask, SSTables, LSM-trees, LevelDB, RocksDB, HBase, Cassandra, Lucene) and **update-in-place** (fixed-size overwritable pages; **B-trees**, in all major relational and many non-relational databases). Log-structured engines turn **random writes into sequential writes**, raising write throughput on HDDs and SSDs.
- **Hash indexes** (Bitcask): keys must fit in RAM, no range queries; segments + compaction + merging, tombstones, checksums.
- **SSTables/LSM-trees:** memtable → sorted SSTables → background merging (size-tiered or leveled); sparse index, compressed blocks, Bloom filters, crash log.
- **B-trees:** 4 KB pages, in-place updates, splits keep balance (4 levels × 500 branching × 4 KB = 256 TB); WAL and latches; copy-on-write (LMDB), abbreviated keys (B+ tree), sibling pointers, fractal trees.

- **LSM vs B-tree:** LSM usually better for writes and space; B-tree for reads, predictable latency, key-range locking. Benchmark your workload.
- **Other indexes:** secondary; heap vs clustered (InnoDB, SQL Server) vs covering; concatenated vs multi-dimensional (R-trees, PostGIS, HyperDex); fuzzy (Lucene automata). **In-memory DBs** win by avoiding encoding overhead, not disk reads.
- **Warehouses** (via **ETL**, **star**/snowflake schemas): indexes matter less than **very compact encoding** — columnar storage, bitmap/RLE compression, vectorised processing, sort orders (C-Store/Vertica), LSM-style writes, and **materialized views/data cubes** trading flexibility for speed.
- **Takeaway:** knowing engine internals helps you choose tools, predict the effect of tuning parameters, and read your database's documentation.

Interview angle

1. B-tree vs LSM-tree — which would you pick? *30-second answer:* B-trees update fixed-size pages in place with a WAL; each key lives in one place, reads are fast and latency predictable — the relational default. LSM-trees buffer writes in a memtable, flush sorted SSTables sequentially, and compact in the background — higher write throughput and better compression, but reads may check several files and compaction can hurt tail latency. Write-heavy (logs, metrics, time-series) → LSM; read-heavy/transactional → B-tree. Benchmark.

- *Follow-up: What is write amplification?* One logical write causing several physical writes over time (WAL + page writes in B-trees; repeated compaction in LSM). Limits throughput and wears out SSDs.
- *Follow-up: How does an LSM avoid checking every SSTable for a missing key?* Per-SSTable Bloom filters say "definitely not here".
- *Follow-up: What if compaction can't keep up?* Unmerged segments pile up, reads slow, disk can fill; engines usually don't throttle writes, so monitor.

2. How does an LSM-tree handle a write and survive a crash? *30-second answer:* The write is appended to an on-disk log and inserted into the sorted memtable; at a few MB the memtable is flushed as an immutable SSTable. After a crash the log rebuilds the memtable; after a flush it is discarded.

- *Follow-up: Deletes?* A tombstone; compaction drops older values.
- *Follow-up: Size-tiered vs leveled?* Size-tiered merges smaller/newer into bigger/older tables (HBase); leveled splits key ranges into small tables across levels, more incremental and less disk space (LevelDB, RocksDB). Cassandra supports both.

3. How does a B-tree stay balanced and crash-safe? *30-second answer:* An overflowing page splits into two half-full pages with a new boundary in the parent, keeping depth O(log n), usually 3–4 levels. Changes go to a write-ahead log first so a crash mid-split can be repaired; latches protect concurrent access.

- *Follow-up: Alternative to a WAL?* Copy-on-write pages (LMDB).

4. OLTP vs OLAP, and why a separate data warehouse? *30-second answer:* OLTP = many small key-based user reads/writes, latest state, GB–TB. OLAP = few large aggregate scans for analysts, event history, TB–PB. Analyst queries on OLTP databases hurt customer latency and OLTP indexes are poor for scans, so ETL copies data into a warehouse optimised for analytics (often columnar, star schema).

- *Follow-up: Star schema?* Central fact table of events with foreign keys to dimension tables; snowflake further normalises dimensions.

5. Why are column stores fast for analytics? *30-second answer:* Queries touch 4–5 of 100+ columns, so reading separate column files avoids most I/O. Columns compress very well (bitmaps, RLE), especially the sort key, and compressed chunks fit CPU cache for vectorised processing.

- *Follow-up: Writes?* Buffer in memory (LSM-style) and periodically merge into new column files; queries combine both (Vertica).
- *Follow-up: Are Cassandra/HBase column stores?* No — column families are row-oriented within each family, with no column compression.

6. Clustered vs covering vs heap-file indexes? *30-second answer:* Heap-file indexes point to rows stored elsewhere (no duplication, extra hop). Clustered indexes hold the full row (InnoDB primary key; one per

table in SQL Server). Covering indexes hold some columns so certain queries need only the index. More duplication = faster reads, more storage and write cost.

Self-check

1. **Why is appending to a log fast, and why is reading from an unindexed log slow?** Appending is a sequential write at the end of the file; reading without an index scans the whole file, O(n).
2. **What is the main limitation of Bitcask-style hash indexes, and what workload suits them?** All keys must fit in RAM and range queries are inefficient. Suits many updates to a modest number of keys (e.g. play counts per video URL).
3. **Why can an SSTable use a sparse index while a hash-indexed log cannot?** Keys are sorted, so neighbouring keys' offsets bound the range to scan; in an unsorted log a key could be anywhere.
4. **Trade-off: your service ingests millions of sensor readings per second and rarely updates old data. B-tree or LSM-tree? Why?** LSM-tree: sequential writes, higher write throughput, better compression. Watch compaction and tail latency.
5. **What happens in a B-tree when you insert a key into a full page, and why is that risky without a WAL?** The page splits and the parent is updated — several page writes. A crash in between could leave an orphan page; the WAL restores consistency.
6. **How big can a 4-level B-tree with 4 KB pages and branching factor 500 get?** Up to 256 TB.
7. **Scenario: a map app needs "all shops inside this rectangle". Why won't a concatenated (lat, lon) B-tree index work well, and what should you use?** It narrows by latitude but must then scan all longitudes in that band. Use a spatial index such as an R-tree (e.g. PostGIS on GiST) or a space-filling curve.
8. **What is the real reason in-memory databases are faster?** They avoid encoding in-memory structures into disk-friendly form — not just avoiding disk reads, which the OS cache already avoids for disk-based engines with enough RAM.
9. **In a column store, why can't each column be sorted independently? What benefit does sorting give?** Rows are reconstructed by position; independent sorting breaks alignment. Sorting whole rows speeds range filters on the sort key and creates long runs that compress extremely well.
10. **What's the downside of relying on a data cube instead of raw data?** Inflexibility: only questions along chosen dimensions (e.g. no share of sales for items over $100 if price isn't a dimension). Keep raw data; use cubes as a speed-up.

Chapter 4 — Encoding and Evolution

Book: Chapter 4, pp. 111–144

Overview

Applications change constantly, and nearly every change also changes the **data**: a new field, a new record type, old data presented differently. Chapter 1 called coping with this **evolvability**.

Chapter 2's data models handle change differently:

- **Relational databases** assume **one schema** at any moment; **schema migrations** (`ALTER TABLE`) change it, but only one version is in force at a time.
- **Schema-on-read ("schemaless") databases** enforce no schema, so old- and new-format records coexist.

Code that reads and writes the data must change too, but in a big system it **cannot change everywhere at once**:

- **Server-side:** a **rolling upgrade** (**staged rollout**): deploy to a few nodes, check, continue node by node. No downtime, so releases can be frequent.
- **Client-side:** users may not install updates for a long time.

So **old and new code, and old and new data formats, coexist**, needing compatibility **in both directions**:

Kind	Meaning	Difficulty
Backward compatibility	*Newer* code can read data written by *older* code.	Usually easy — the new code's author knows the old format and can handle it explicitly.
Forward compatibility	*Older* code can read data written by *newer* code.	Harder — old code must *ignore* additions it has never heard of.

Two questions:

1. How do **encoding formats** (JSON, XML, Protocol Buffers, Thrift, Avro) **handle schema changes**?
2. How are they used when **data flows** between processes — through databases, services (REST and RPC), and asynchronous messages (brokers and actors)?

Key terms

Term	Plain-English meaning
Evolvability	How easily a system can be changed over time.
Schema migration / schema-on-read	Changing a relational schema (`ALTER TABLE`) / no schema enforced on write; readers interpret structure.
Rolling upgrade / staged rollout	Deploying a new version gradually, a few nodes per step.
Backward / forward compatibility	New code reads old data / old code reads new data.
Encoding (serialization, marshalling)	In-memory data → self-contained byte sequence.
Decoding (parsing, deserialization, unmarshalling)	Byte sequence → in-memory data.
Language-specific format	Encoding built into one language (Java Serializable, Ruby Marshal, Python pickle).
Textual format	Human-readable encodings: JSON, XML, CSV.
Binary string	Raw bytes with no character encoding.
Base64	Writes binary as text; about 33% larger.
Binary encoding / MessagePack	Compact non-human-readable byte format / one binary encoding of JSON.
Schema / IDL	Formal description of data structure and types / the language it is written in (Thrift IDL, Avro IDL).
Code generation	Tool turning a schema into classes in your language.
Field tag	Number identifying a field in Thrift/Protocol Buffers instead of its name.
Variable-length integer (varint)	Uses fewer bytes for small numbers.
Schema evolution	Changing a schema over time while keeping compatibility.
Writer's / reader's schema	Schema used to encode the data / schema the decoding app expects (Avro).
Union type	Avro type allowing one of several types, e.g. `null` or `long`.
Object container file	Avro file storing the writer's schema once, then many records; **self-describing**.
ASN.1 / DER	1984 schema language and its binary encoding, still used for X.509 certificates.
Dataflow	How encoded data travels from one process to another.
Data outlives code	Old data keeps its original encoding long after the code that wrote it is gone.
Migrating data / archival storage	Rewriting stored data into a new schema / snapshots or dumps for backups or warehouses.
Client / server / service	Clients send requests to servers; the API a server exposes is a service.
Ajax	Browser JavaScript making HTTP requests (XMLHttpRequest) for data.
SOA / microservices	Application built from many smaller services calling each other.
Web service / middleware	Service accessed over HTTP / software supporting service-to-service calls inside one organisation.
REST / RESTful	Design philosophy built on HTTP principles / an API following it.

Term	Plain-English meaning
SOAP / WSDL	XML-based network API protocol with WS-* standards / XML language describing a SOAP API, enabling code generation.
OpenAPI (Swagger)	Format for describing RESTful APIs and generating documentation.
RPC (remote procedure call) / location transparency	Making a network request look like a local function call / hiding whether something is local or remote.
Idempotence	Doing an action twice has the same effect as once — needed for safe retries.
Futures / promises	Objects representing a result that will arrive later (or fail).
Service discovery	Finding the IP address and port a service runs on.
API versioning	How a client indicates which API version it wants.
Message broker (message queue, message-oriented middleware)	Intermediary temporarily storing messages; destinations are named **queues** or **topics**; **producers** send, **consumers (subscribers)** receive.
Actor model / distributed actor framework	Concurrency model of actors with private state exchanging async messages / the same across nodes (Akka, Orleans, Erlang OTP).

Formats for Encoding Data

Programs hold data in **two representations**:

1. **In memory:** objects, structs, lists, arrays, hash tables, trees, tuned for CPU access and usually full of **pointers**.
2. **On disk / on the network:** a **self-contained sequence of bytes** (e.g. JSON). Pointers mean nothing to another process, so it looks very different. (Exceptions: some memory-mapped files, or operating directly on compressed data as in Chapter 3's column compression.)

Translation: **encoding** (**serialization**, **marshalling**): in-memory → bytes; **decoding** (**parsing**, **deserialization**, **unmarshalling**): bytes → in-memory.

> **Terminology clash:** "serialization" also means something different for transactions (Chapter 7), so the book says **encoding**. Encoding is unrelated to **encryption** (not covered).

Language-Specific Formats

What they are. Built-in object-to-bytes support: **Java** `java.io.Serializable`, **Ruby** `Marshal`, **Python** `pickle`; third-party ones like **Kryo** for Java. Convenient: save/restore objects with almost no code.

Why they're a bad idea:

1. **Tied to one language**: reading from another language is very hard, locking you in and hindering integration with other organisations.
2. **Security risk**: to rebuild objects the decoder must **instantiate arbitrary classes**; if an attacker can feed bytes to decode, this often enables **remote code execution**.
3. **Versioning is an afterthought**: forward/backward compatibility often ignored.
4. **Efficiency is an afterthought**: CPU cost and size may be poor; **Java's built-in serialization** is infamous for slowness and bloat.

Verdict: only for **very short-lived** purposes.

```
import pickle
blob = pickle.dumps({"sku": "A-17", "qty": 3})  # encode a dict into Python-only bytes
item = pickle.loads(blob)                       # decode; never do this with untrusted input!
```

JSON, XML, and Binary Variants

JSON and **XML** are the standard cross-language choices — widely known, supported, and disliked. XML is called verbose and needlessly complicated; JSON is popular because browsers support it natively (a subset of JavaScript) and it is simpler. **CSV** is another language-independent, less powerful format. All are **textual**, somewhat human-readable, with **subtle problems**:

1. **Number ambiguity. XML and CSV** cannot distinguish numbers from digit strings (without an external schema). **JSON** distinguishes strings from numbers but not **integers from floating-point** and specifies **no precision**. Integers above **2^53** cannot be represented exactly in an **IEEE 754 double**, so languages parsing JSON numbers as floats (**JavaScript**) get them wrong. **Twitter** uses **64-bit** tweet IDs, so its API returns each ID **twice**: as a JSON number and as a **decimal string**.
2. **No binary strings.** JSON and XML handle Unicode text but not raw bytes; the hacky workaround is **Base64** (marked as such in the schema), growing data by **33%**.
3. **Optional, complicated schemas.** XML and JSON schema languages are powerful and hence complex; XML schemas are fairly common, many JSON tools skip schemas. Without one, apps must **hardcode** how to interpret numbers and binary strings.
4. **CSV has no schema**: the app defines what each row and column means and handles changes manually. CSV is also **vague** about commas or newlines inside values; its escaping rules are specified but not every parser implements them correctly.

Still good enough for many uses, especially **data interchange formats** between organisations: agreement matters more than prettiness or efficiency, and getting organisations to agree on *anything* is the hard part.

Binary encoding

Within one organisation you can choose something more compact or faster to parse than a lowest-common-denominator format; negligible for small data, significant at **terabyte** scale.

JSON is less verbose than XML, but both are much larger than binary formats, hence many **binary encodings**: for JSON **MessagePack, BSON, BJSON, UBJSON, BISON, Smile**; for XML **Fast Infoset, WBXML**. All niche. Some add datatypes (int vs float, binary strings) but keep the JSON/XML data model; and **with no schema, they must include every field name** in the bytes.

The running example (Example 4-1): a person record: `userName` = "Martin", `favoriteNumber` = 1337, `interests` = ["daydreaming", "hacking"]. Our similar record:

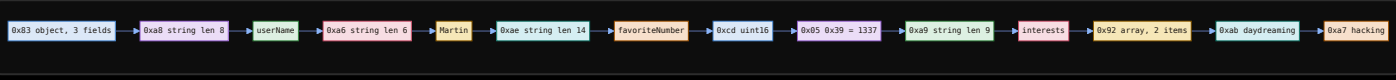
```
{
  "bookTitle": "Dune",
  "pageCount": 412,
  "genres": ["sci-fi", "classic"]
}
```

MessagePack walkthrough (book's Example 4-1):

1. First byte **0x83**: high nibble 0x80 = "object", low nibble 0x03 = **three fields**. (Objects with more than 15 fields get a different type marker and a 2- or 4-byte count.)
2. **0xa8**: high nibble 0xa0 = "string", low nibble = length **eight bytes**.
3. Eight bytes of ASCII `userName` (length known, so no terminator or escaping).
4. **0xa6** then the six-letter `Martin`; and so on.

Result: 66 bytes vs **81 bytes** for whitespace-free JSON — only slightly smaller, as for all binary JSON variants; unclear whether worth losing readability. The schema-based formats below reach **32 bytes**.

Figure 4-1 (redrawn): Example 4-1 encoded with MessagePack — every field name travels inside the bytes



Total: 66 bytes.

Thrift and Protocol Buffers

What they are. Apache Thrift (from **Facebook**) and **Protocol Buffers / protobuf** (from **Google**): binary encoding libraries on the same principle, open-sourced **2007–08**. Both **require a schema** written in an **interface definition language (IDL)**. Thrift-style, our book record — each field has a **tag number**, `required` / `optional`, a type and a name:

```
struct Book {
  1: required string       bookTitle,   // tag 1, must be present
  2: optional i64          pageCount,   // tag 2, 64-bit integer, may be absent
  3: optional list<string> genres       // tag 3, list of strings
}
```

Protocol Buffers:

```
message Book {
  required string book_title = 1;   // field tag 1
  optional int64  page_count = 2;   // field tag 2
  repeated string genres     = 3;   // "repeated" = may appear many times (a list)
}
```

Each has a **code generation tool** producing classes in many languages, which your app calls to encode/decode.

Thrift has two binary formats: BinaryProtocol and **CompactProtocol** (plus **DenseProtocol**, C++ only so not cross-language, and two JSON-based encodings — "What fun!").

Thrift BinaryProtocol — 59 bytes. Like MessagePack, each field has a **type annotation** and, where needed, a **length**; strings are UTF-8. **Big difference: no field names** — each field carries a **field tag** (1, 2, 3), a compact **alias** for the field.

Figure 4-2 (redrawn): Example 4-1 in Thrift BinaryProtocol — type byte, 2-byte tag, then length/value



Total: 59 bytes.

Thrift CompactProtocol — 34 bytes. Packs **field type and tag into one byte**, and uses **variable-length integers**: 1337 takes **two bytes** instead of eight. Each byte's top bit says whether more follow, so **–64 to 63** fits in one byte, **–8192 to 8191** in two, and so on.

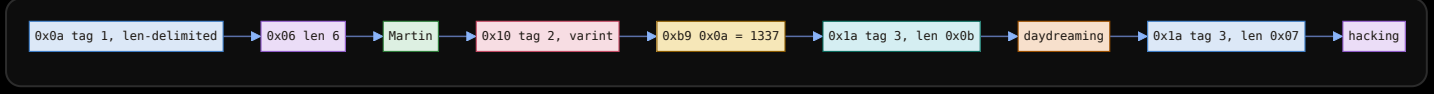
Figure 4-3 (redrawn): Example 4-1 in Thrift CompactProtocol — type+tag share a byte, integers are varints



Total: 34 bytes.

Protocol Buffers — 33 bytes. One binary format, bit packing very similar to CompactProtocol.

Figure 4-4 (redrawn): Example 4-1 in Protocol Buffers — a repeated field simply appears once per element



Total: 33 bytes.

Byte-size scoreboard for Example 4-1:

Encoding	Bytes
Textual JSON (whitespace removed)	81
MessagePack	66
Thrift BinaryProtocol	59
Thrift CompactProtocol	34
Protocol Buffers	33
Avro	32

> **Common confusion — required vs optional:** the marker does not change the encoding. `required` only enables a **runtime check** that errors if the field is missing — useful for catching bugs.

Field tags and schema evolution

A record is its encoded fields concatenated, each with a **tag number** and **datatype** annotation; unset fields are **omitted**. The tags give the bytes meaning:

- **Renaming a field is fine** (names are never encoded).
- **Changing a field's tag is not** (all existing data becomes invalid).

Adding a field: give it a **new tag**. **Forward compatibility:** old code **ignores** unknown tags, using the datatype annotation to know **how many bytes to skip**. **Backward compatibility:** tags keep their meaning, so new code reads old data — **but** the new field **cannot be** `required` (old data lacks it, failing the check). Every field added after initial deployment must be **optional or have a default**.

Removing a field: only **optional** fields (never required ones), and the tag number may **never be reused** (old data with it still exists).

Datatypes and schema evolution

Changing a field's type may be possible (check the docs) but risks **lost precision or truncation**. E.g. 32-bit → 64-bit integer: new code reading old data is fine (missing bits zero-filled); old code reading new data puts the value in a **32-bit** variable and **truncates** it if too large.

Protocol Buffers `repeated`**:** protobuf has **no list type**, only a `repeated` marker (beside `required` and `optional`): the **same tag appearing multiple times** (Figure 4-4). So an **optional (single)** field can later become **repeated**: new code reading old data sees a list of **zero or one** item; old code reading new data sees only the **last** item.

Thrift has a real **list** type parameterised by element type (e.g. `list<string>`): no single→multi evolution, but it supports **nested lists**.

Avro

What it is. **Apache Avro**, begun in **2009** as a **Hadoop** subproject because Thrift did not fit Hadoop's use cases well. Two schema languages: **Avro IDL** (human-edited) and a **JSON-based** one (machine-friendly). Our example in Avro IDL:

```
record Book {
  string              bookTitle;            // plain string, no tag number
  union { null, long } pageCount = null;    // may be null or a long; default null
  array<string>       genres;               // list of strings
}
```

JSON form (`["null", "long"]` is a union; `"default": null` fills the field when the writer lacked it):

```
{
  "type": "record",
  "name": "Book",
  "fields": [
    {"name": "bookTitle", "type": "string"},
    {"name": "pageCount", "type": ["null", "long"], "default": null},
    {"name": "genres",    "type": {"type": "array", "items": "string"}}
  ]
}
```

No tag numbers, and Example 4-1 takes **32 bytes**, the most compact. The bytes contain **nothing identifying fields or types** — just concatenated values: a string is a length prefix plus UTF-8 bytes, but nothing says it is a string. Integers use variable-length encoding (like CompactProtocol).

Figure 4-5 (redrawn): Example 4-1 in Avro — only values, in schema order, with no tags or type markers (lengths and counts are zigzag varints, so 0x0c means 6)



Total: 32 bytes.

Consequence: parsing walks fields **in schema order**, using the schema for types, so data decodes correctly **only with exactly the writer's schema**. So how does Avro evolve?

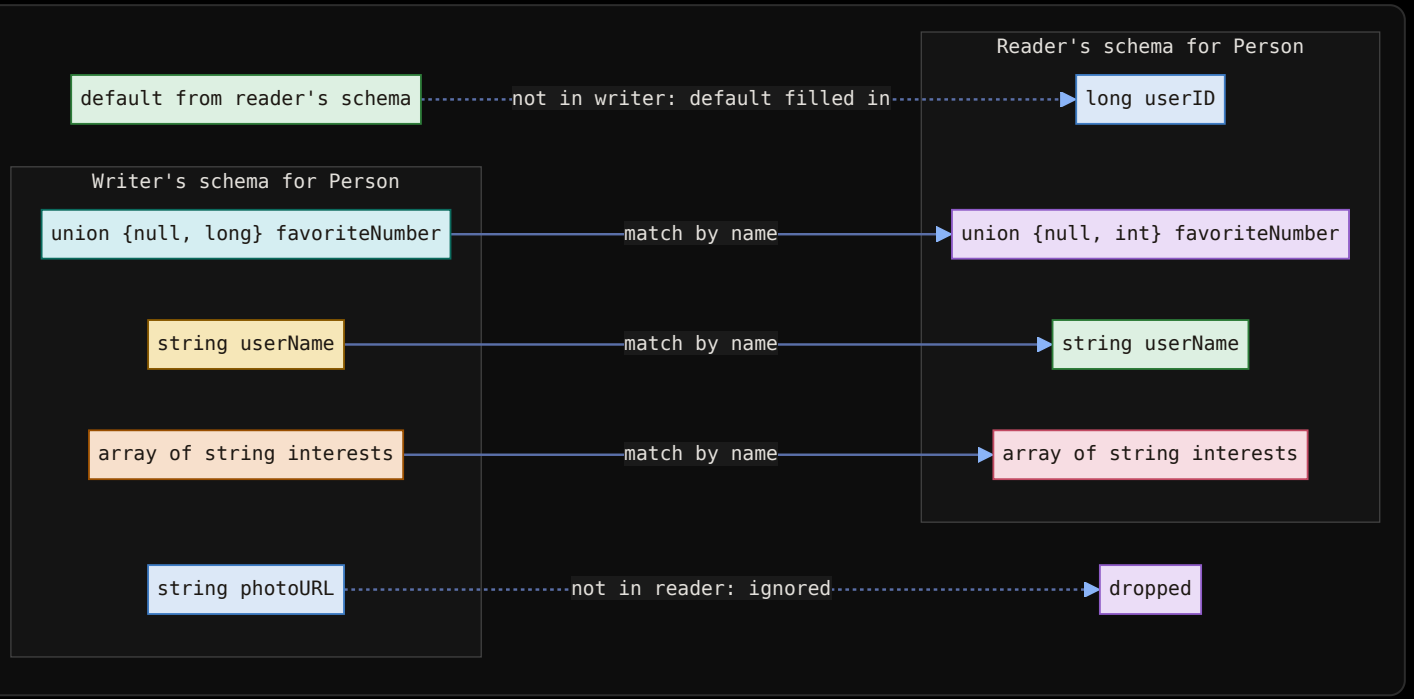
The writer's schema and the reader's schema

- **Writer's schema:** the version the encoding app knows (perhaps compiled in).
- **Reader's schema:** the version the decoding app's code expects (perhaps generated at build time).

Key idea: they **need not be identical, only compatible**. The Avro library compares them **side by side** and translates writer → reader (**schema resolution**, defined in the Avro spec):

1. **Field order may differ** — fields match **by name**.
2. Field only in the **writer's** schema → **ignored**.
3. Field only in the **reader's** schema → filled with the reader's **default**.

Figure 4-6 (redrawn): an Avro reader resolves the writer's and reader's Person schemas by field name. Order differs between the two; photoURL exists only in the writer's schema, userID only in the reader's



Schema evolution rules

In Avro, **forward compatibility** = new schema as **writer**, old as **reader**; **backward compatibility** = new as **reader**, old as **writer**.

Rule: add or remove a field **only if it has a default** (the book's `favoriteNumber` defaults to `null`).

- **Add with default:** new readers fill the default for old records.
- **Add without default:** new readers cannot read old data → **breaks backward compatibility**.
- **Remove without default:** old readers cannot read new data → **breaks forward compatibility**.

Null via unions. Nothing is nullable by default in Avro; use a **union**, e.g. `union { null, long, string } field;`. **`null` can be a default only if it is a union branch** (strictly, the default must match the **first branch** — an Avro-specific limitation). Verbose, but explicit nullability prevents bugs. Hence Avro has **no `optional`/`required` markers**; it uses **unions and defaults**.

Other changes: changing a type is allowed if Avro can convert; **renaming** works via **aliases** in the reader's schema (matching old writer names), so it is **backward but not forward compatible**; **adding a union branch** is likewise **backward but not forward compatible**.

But what is the writer's schema?

Attaching the full schema to every record would cost more than binary encoding saves. It depends on context:

Context	How the reader gets the writer's schema
Large file with lots of records (common with Hadoop; Chapter 10)	One schema for millions of records, written **once at the start of the file**: Avro **object container files**.
Database with individually written records	Records may use different schemas. Prefix each with a **version number** and keep a **list of schema versions in the database**; the reader fetches the matching writer's schema. **Espresso** works this way.
Sending records over a network connection	Two processes on a **bidirectional** connection **negotiate the schema version at setup** and use it throughout (**Avro RPC**).

A **database of schema versions** is useful anyway as documentation and to **check compatibility** of changes. Versions can be an **incrementing integer** or a **hash of the schema**.

Dynamically generated schemas

Having **no tag numbers** makes Avro friendlier to **dynamically generated schemas**. **Example: dumping a relational database to a binary file:** generate an Avro schema with **one record schema per table** and **one field per column** (**column name = field name**), encode the contents into an **object container file**; when the database schema changes (a column added or removed), just **regenerate** the schema and export. Fields match **by name**, so new writers' schemas still resolve against old readers'.

With Thrift or protobuf, tags would likely be assigned **by hand**, requiring an administrator to update the column → tag mapping on every schema change; automation must be careful **never to reuse tags**. Dynamic generation **wasn't a design goal** for them; it **was** for Avro.

Code generation and dynamically typed languages

- **Thrift and protobuf rely on code generation**, which suits **statically typed** languages (**Java, C++, C#**): efficient in-memory structures, type checking, IDE autocompletion.
- In **dynamically typed** languages (**JavaScript, Ruby, Python**) there is no compile-time checker to satisfy, and a compile step is otherwise avoided, so code generation adds little; with **dynamically generated schemas** (e.g. from a database table) it just gets in the way.
- **Avro** offers **optional** code generation and works without it. An **object container file** embeds the writer's schema, so the Avro library can open and browse it like JSON — it is **self-describing**.
- Handy with dynamically typed data-processing languages like **Apache Pig**: open Avro files, analyse, write derived Avro datasets without thinking about schemas.

The Merits of Schemas

Protobuf, Thrift and Avro schema languages are **far simpler** than **XML Schema** or **JSON Schema** (which offer elaborate validation such as regex-matched strings or integer ranges), so they are easy to implement and have libraries in many languages.

Not a new idea:

- **ASN.1**, standardised in **1984**, defined many network protocols; its binary encoding **DER** still encodes **SSL certificates (X.509)**. It supports evolution with **tag numbers** like protobuf/Thrift, but is very complex and badly documented — unwise for new applications.
- Many data systems use **proprietary binary encodings**, e.g. each relational database's own **network protocol**, decoded by a vendor **driver** (e.g. via **ODBC** or **JDBC** APIs).

Benefits of schema-based binary encodings:

1. **Much more compact** than binary JSON (no field names).
2. The schema is **documentation guaranteed up to date**, since decoding needs it.
3. A **database of schemas** allows checking forward/backward compatibility **before deploying**.
4. **Generated code** enables compile-time type checking in static languages.

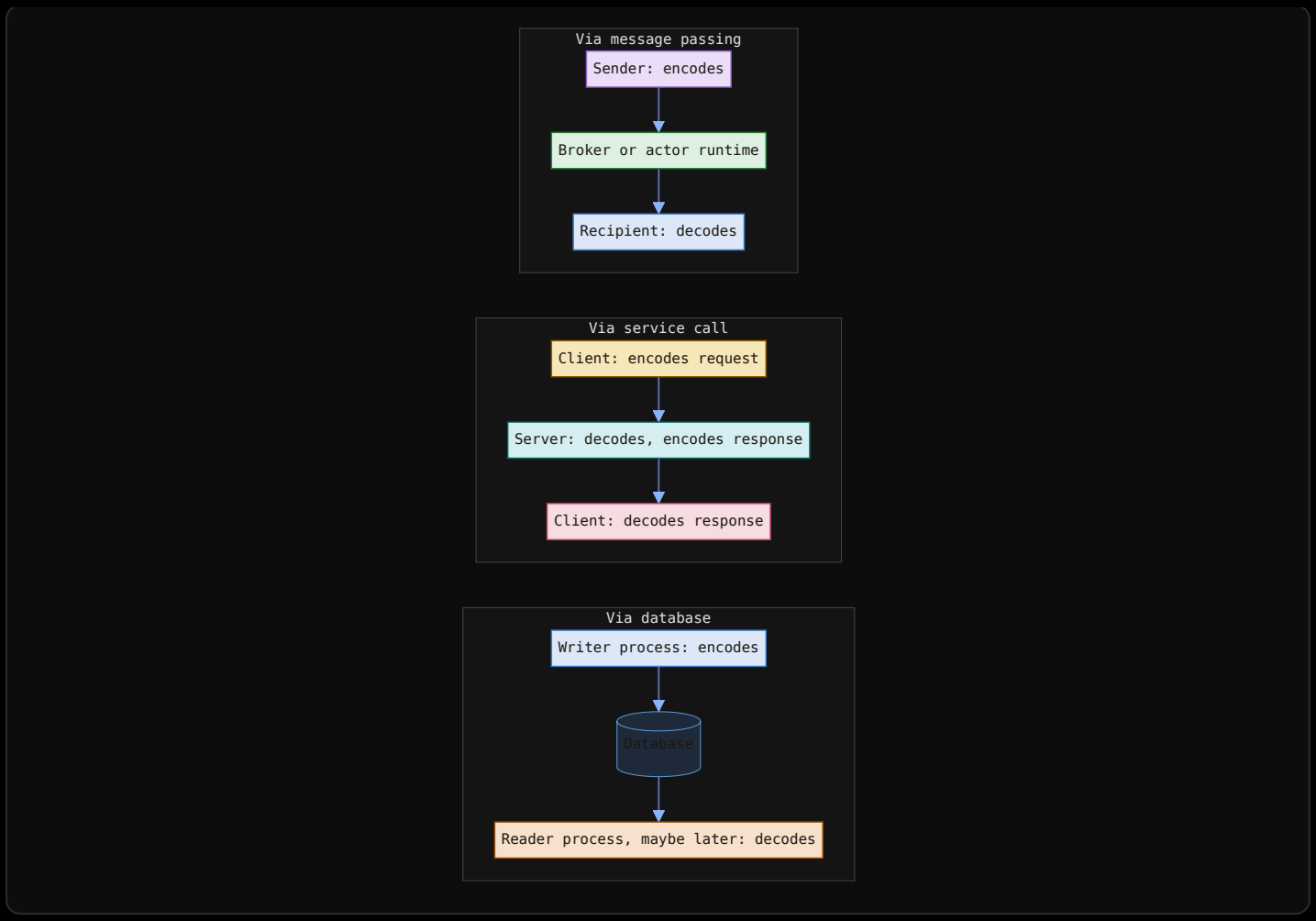
Bottom line: schema evolution gives flexibility comparable to schemaless JSON databases **plus** stronger data guarantees and better tooling.

Format family	Examples	Size	Human-readable	Schema	Compatibility story
Language-specific	Java Serializable, Ruby Marshal, Python pickle, Kryo	Often bloated	No	Implicit in classes	Often none; insecure
Textual	JSON, XML, CSV	Large	Yes	Optional (XML/JSON), none (CSV)	Depends on usage; number/binary ambiguity
Binary JSON/XML	MessagePack, BSON, WBXML…	Slightly smaller	No	None	Like JSON; field names still stored
Schema-driven binary	Thrift, Protocol Buffers	Very compact (tags)	No	Required, with tags	Clear rules via tag numbers
Schema-driven binary	Avro	Most compact (no tags)	No	Writer's + reader's schema	Clear rules via name matching + defaults

---

Modes of Dataflow

Data sent to a process you don't share memory with (over the network or via a file) must be encoded. Compatibility is a relationship between **the encoding process** and **the decoding process**, and it lets you **upgrade parts of a system independently**. Who encodes and decodes? Three common flows: **databases**, **service calls** (REST and RPC), **asynchronous message passing**.



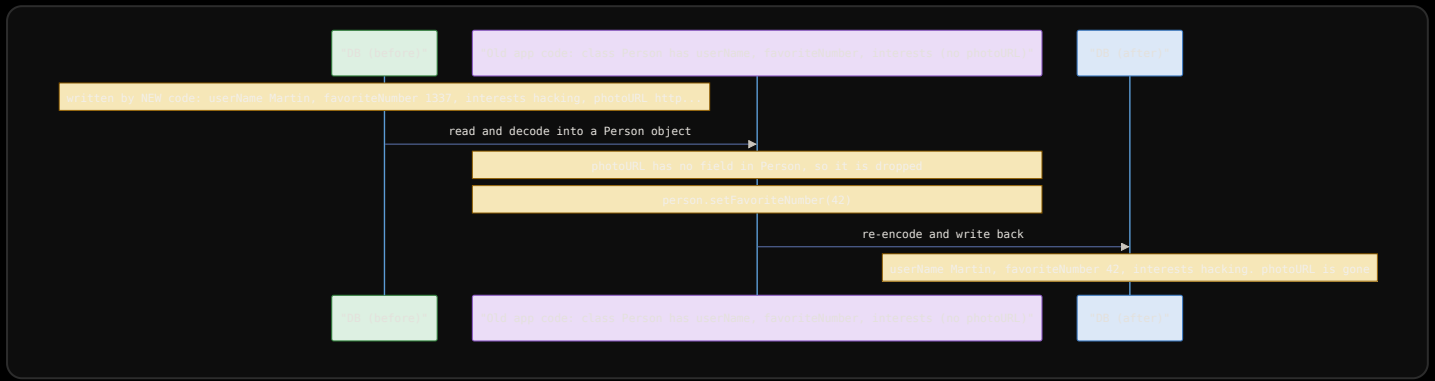
Dataflow Through Databases

The **writer** encodes, the **reader** decodes.

- **Single process:** the reader may be a **later version** of the same process — like **leaving a note for your future self** — so **backward compatibility** is obviously needed.
- **Many processes:** several apps or service instances access the database; during a **rolling upgrade** some run new code, some old. A value **written by new code may be read by old code**, so **forward compatibility** is needed too.

Preserving unknown fields. New code writes a new field; then **old code** reads the record, **modifies** something, and writes it back. Usually you want the new field **left untouched**. The formats support this, but if the app decodes into **model objects** and re-encodes, the unknown field can be **lost** — not hard to fix, just be aware.

Figure 4-7 (redrawn): an older app version reads a record written by a newer version, updates one field through its model object, and writes it back. The newer field (`photoURL`) is silently lost.



Different values written at different times

- Any value can be updated at any time, so one database holds values written **five milliseconds ago** and **five years ago**.
- A new app version replaces the old within **minutes**, but the **five-year-old data keeps its original encoding** unless rewritten: **"data outlives code."**
- **Migrating** data to a new schema is **expensive** on large datasets, so most databases avoid it. **Most relational databases** allow simple changes like **adding a column with a null default without rewriting** data (missing columns read as null) — except **MySQL**, which often rewrites the whole table (Chapter 2).
- **LinkedIn's Espresso** stores data in **Avro** to use its evolution rules.
- **Net effect:** the database **appears** to have one schema while storage holds many historical versions.

Archival storage

Snapshots for **backups** or loading a **data warehouse** (Chapter 3) are typically encoded with the **latest schema** (you are copying everything anyway). Written once and **immutable**, so **Avro object container files** fit well, or a **column-oriented** format like **Parquet** for analytics. More in Chapter 10.

Dataflow Through Services: REST and RPC

Clients and servers. Servers expose an API over the network (a **service**); **clients** send requests. (Background on HTTP, proxies, load balancers, idempotency: System Design Fundamentals.)

- **The web:** browsers issue **GET** requests for HTML, CSS, JavaScript and images and **POST** requests to send data, using **standard** protocols and formats (HTTP, URLs, SSL/TLS, HTML), so any browser can in principle access any site.
- **Other clients:** native mobile/desktop apps, and browser JavaScript using **XMLHttpRequest** (**Ajax**), get data (e.g. **JSON**) rather than HTML; HTTP is the transport but the API is **application-specific** and must be agreed by both sides.
- **A server can be a client** of another service (e.g. of its database). Decomposing an application by functionality into services calling each other is **service-oriented architecture (SOA)**, refined and rebranded as **microservices**.

Services vs databases. Databases accept **arbitrary queries**; services expose an **application-specific API** whose inputs and outputs are fixed by business logic, giving **encapsulation**: fine-grained control over what clients may do.

Why compatibility matters: SOA/microservices aim for services that are **independently deployable and evolvable**, each owned by **one team** releasing **without coordinating** with others. So **old and new clients and servers run simultaneously**, and encodings must be compatible across API versions.

Web services

A service using **HTTP** is a **web service** (a misnomer, since use extends beyond the web):

1. **Client app on a user's device** calling over the **public internet**.
2. **Service to service within one organisation**, often in the **same datacenter** (supporting software: **middleware**).
3. **Service to service across organisations**: **public APIs** such as **credit card processing**, or **OAuth** for shared access to user data.

Two approaches, REST and SOAP, with nearly opposite philosophies and heated debate (even within camps, e.g. over **HATEOAS**: responses carrying links to the client's next possible actions).

REST (Representational State Transfer): not a protocol but a **design philosophy** built on HTTP: **simple data formats**, **URLs identifying resources**, and **HTTP features** for cache control, authentication and content-type negotiation. Increasingly popular, especially for **cross-organisational** integration, and associated with **microservices**. An API following it is **RESTful**.

SOAP: an **XML-based protocol**, usually over HTTP but aiming to be **independent of it** and avoiding most HTTP features. Comes with a sprawling **web service framework, WS-***. APIs are described in **WSDL (Web Services Description Language)**, enabling **code generation** so clients use local classes and method calls (useful in static languages, less in dynamic). WSDL is not human-readable and messages are hard to

build by hand, so users depend on **tools, code generation and IDEs**; unsupported languages struggle. Despite standards, **interoperability between vendors' implementations** is often problematic. SOAP persists in **large enterprises**; most smaller companies **moved away**. (**SOAP is not required for SOA**: one technology vs a broad architectural approach.)

RESTful APIs favour simpler approaches with less code generation; **OpenAPI** (**Swagger**) can describe them and produce documentation.

	REST	SOAP
Nature	Design philosophy on top of HTTP	XML-based protocol, HTTP-independent
Data format	Simple (commonly JSON)	XML
Uses HTTP features	Yes (URLs, caching, auth, content negotiation)	Mostly avoids them
Description	OpenAPI / Swagger (optional)	WSDL
Tooling dependence	Low	High (code generation, IDEs)
Where popular	Public/cross-org APIs, microservices	Large enterprises

The problems with remote procedure calls (RPCs)

Web services are the latest of many hyped but flawed network-API technologies: Java's **Remote Method Invocation (RMI)** and **Enterprise JavaBeans (EJB)** (**Java-only**); **DCOM (Distributed Component Object Model)** (**Microsoft-only**); **CORBA (Common Object Request Broker Architecture)** (**excessively complex**, **no backward or forward compatibility**).

All are based on the **remote procedure call (RPC)** (since the **1970s**), which makes a remote call **look like a local function call: location transparency**. This is fundamentally flawed:

Local function call	Network request
Predictable: succeeds or fails based only on parameters you control.	**Unpredictable**: request/response may be lost, remote machine slow or down. Failures are common, so you must expect them (e.g. **retry**).
Returns a result, throws an exception, or never returns.	Can also **time out** with no result — you **don't know** whether it got through (Chapter 8).
Calling twice is your choice.	**Retrying** may repeat an action whose response was lost, unless the protocol has **deduplication (idempotence)** (Chapter 11).
Roughly constant time.	**Much slower** and **wildly variable**: under a millisecond normally, **many seconds** under congestion or overload.
Can pass **pointers/references** efficiently.	Every parameter must be **encoded into bytes** — problematic for large objects.
Same language on both sides.	Possibly **different languages**, so datatypes must be translated, sometimes uglily (e.g. JavaScript and numbers above **2^53**).

Conclusion: disguising a remote service as a local object is pointless. Part of REST's appeal is that it **doesn't hide** being a network protocol (though RPC libraries are built on REST too).

Current directions for RPC

RPC frameworks exist for every encoding here: **Thrift** and **Avro** include RPC; **gRPC** uses **Protocol Buffers**; **Finagle** uses **Thrift**; **Rest.li** uses **JSON over HTTP**.

The new generation is more honest that remote ≠ local: **Finagle** and **Rest.li** use **futures (promises)** for asynchronous, possibly failing calls, which also simplify calling **several services in parallel**; **gRPC** supports **streams** of many requests and responses over time in one call; some frameworks provide **service discovery** (finding a service's IP and port; see "Request Routing", Chapter 6).

RPC vs REST trade-off: custom binary RPC can **outperform** JSON-over-REST, but **RESTful APIs** are easy to **experiment with and debug** (a browser or **curl**, no code generation), supported in **all mainstream languages**, and have a **huge ecosystem** (servers, caches, proxies, load balancers, firewalls, monitoring,

testing tools). So **REST dominates public APIs**, while **RPC frameworks** focus on requests **between services of one organisation**, usually in the **same datacenter**.

```
# Poking a RESTful API directly from the command line, no generated client needed
curl -H "Accept: application/json" https://api.example.com/v2/books/42
```

Data encoding and evolution for RPC

Services allow a **simplifying assumption**: **servers are updated first, clients second**. So you need **backward compatibility on requests** (new server reads old requests) and **forward compatibility on responses** (old client reads new responses).

Compatibility is **inherited from the encoding**: **Thrift, gRPC (Protocol Buffers), Avro RPC** follow their format's rules; **SOAP** uses **XML schemas**, evolvable with subtle pitfalls; **RESTful APIs** typically use **JSON without a formal schema** for responses and JSON or **URI-encoded / form-encoded** parameters (e.g. `?page=2`) for requests — **new optional request parameters** and **extra response fields** are generally safe.

Harder across organisations: providers cannot force clients to upgrade, so compatibility may be needed **for a very long time, perhaps forever**; breaking changes lead to **multiple API versions side by side**.

API versioning has no agreed approach: for REST, the **version in the URL** (like `/v2/` above) or in the **HTTP** `Accept` **header**; for services using **API keys**, the server can **store each client's chosen version**, changeable via a **separate admin interface**.

Message-Passing Dataflow

Asynchronous message-passing sits **between** RPC and databases: **like RPC**, a client's **message** reaches another process with **low latency**; **like a database**, it goes via an intermediary — a **message broker** (**message queue**, **message-oriented middleware**) that **stores it temporarily**.

Advantages of a broker over direct RPC:

1. **Buffers** when the recipient is down or overloaded → more reliable.
2. **Redelivers** to crashed processes → no message loss.
3. Sender needn't know the recipient's **IP address and port** — useful in the cloud where **virtual machines come and go**.
4. One message can reach **several recipients**.
5. **Logically decouples** sender and recipient.

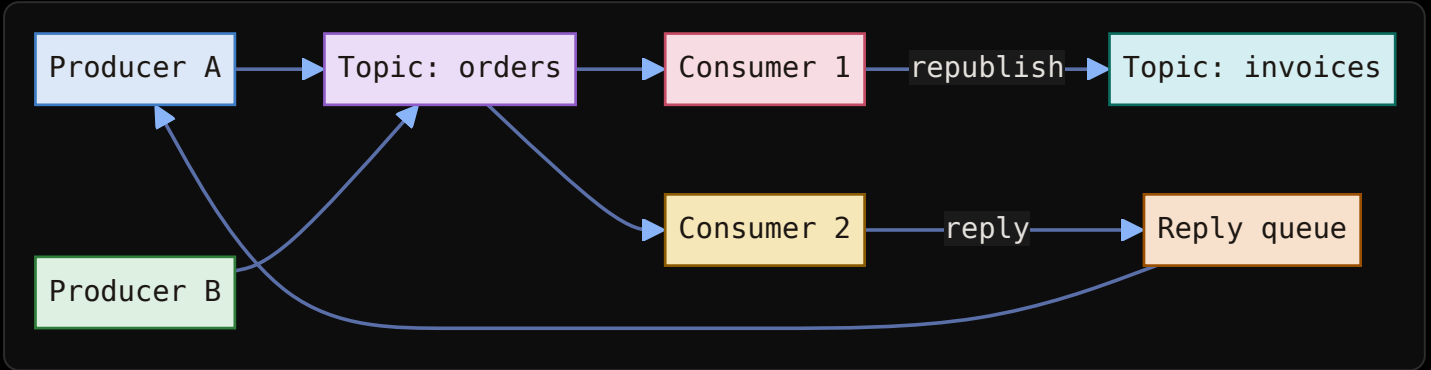
Difference from RPC: communication is usually **one-way**; replies, if any, go on a **separate channel**. It is **asynchronous**: the sender does not wait for delivery.

Message brokers

Formerly dominated by commercial **TIBCO**, **IBM WebSphere** and **webMethods**; now open-source **RabbitMQ**, **ActiveMQ**, **HornetQ**, **NATS** and **Apache Kafka** (compared in Chapter 11).

General pattern: a producer sends a message to a named **queue** or **topic**; the broker delivers it to one or more **consumers** or **subscribers**; a topic can have **many producers and many consumers**.

Message broker dataflow (illustrative, not a numbered book figure)



- Topics are **one-way**, but a consumer can **publish to another topic** (chaining, Chapter 11) or a **reply queue** for the original sender (RPC-like request/response).

- Brokers usually **don't enforce a data model** (a message is **bytes plus metadata**), so any encoding works; a **backward- and forward-compatible** one lets publishers and consumers change and deploy **in any order**.
- A consumer that **re-publishes** must **preserve unknown fields**, as with databases (Figure 4-7).

Distributed actor frameworks

The actor model is a concurrency model **within one process**. Instead of raw threads (with race conditions, locking, deadlock), logic lives in **actors**: each represents **one client or entity**, may hold **local state** not shared with others, and communicates via **asynchronous messages**. **Delivery is not guaranteed** (messages can be lost in some error cases). Each actor processes **one message at a time**, so it needs no thread handling, and the framework schedules actors independently.

Distributed actor frameworks scale this across **multiple nodes**: the same messaging whether actors share a node or not; across nodes the framework **encodes, sends and decodes** the message transparently.

Location transparency fits actors better than RPC, because actors already assume **messages may be lost**, even locally; latency is higher over the network, but local and remote are less mismatched.

Such a framework effectively **combines a message broker with the actor model**. **Rolling upgrades** still need forward and backward compatibility, since messages flow between new- and old-version nodes.

Framework	Default encoding	Rolling upgrades
Akka	**Java's built-in serialization** — no forward/backward compatibility	Possible if you swap in e.g. **Protocol Buffers**
Orleans	A **custom** encoding without rolling-upgrade support	Set up a **new cluster**, move traffic, shut the old one down; custom serialization plug-ins possible, as with Akka.
Erlang OTP	Record schemas are surprisingly hard to change, despite many high-availability features	Possible but must be **planned carefully**; the experimental JSON-like **maps** datatype (**Erlang R17, 2014**) may ease this.

Summary

Encoding choices affect not only **efficiency** but **application architecture** and **deployment options**.

- **Rolling upgrades** spread a new version gradually across nodes: releases **without downtime** (encouraging frequent small releases) and **less risk** (bad releases caught and rolled back early) — both aiding **evolvability**. Since nodes run different versions, all flowing data needs **backward** (new reads old) and **forward** (old reads new) compatibility.
- **Language-specific formats** (Java Serializable, Ruby Marshal, Python pickle, Kryo): one language, insecure, often no compatibility, inefficient.
- **Textual formats (JSON, XML, CSV):** widespread; compatibility depends on use; optional schemas help or hinder; vague datatypes — beware **numbers** (2^53; Twitter's string IDs) and **binary strings** (Base64 +33%). Binary JSON (MessagePack: 66 vs 81 bytes) saves little.
- **Binary schema-driven formats** — compact, efficient, **clearly defined** compatibility, documentation and code generation; downside: data must be **decoded before humans can read it**:
  - **Thrift/Protocol Buffers** (BinaryProtocol 59, CompactProtocol 34, protobuf 33 bytes): **field tags**; rename freely; never change or reuse tags; new fields optional/defaulted; only optional fields removable.
  - **Avro** (32 bytes): no tags; **writer's vs reader's schema** resolved by name; add/remove only defaulted fields; nulls via unions; writer's schema via file header, per-record version, or connection negotiation; great for **dynamically generated schemas**.
- **Schemas** give compactness, up-to-date docs, compatibility checks, code generation (ASN.1 did it in 1984).
- **Dataflow: databases** (writer encodes, reader decodes; data outlives code; preserve unknown fields; dumps with latest schema); **RPC and REST** (client encodes request, server decodes and encodes

response; REST vs SOAP; RPC's location transparency is flawed; servers upgrade first; versioning via URL, Accept header, or per-API-key); **asynchronous messages** via brokers or actors (sender encodes, recipient decodes; buffering, redelivery, decoupling, fan-out; Akka/Orleans/Erlang OTP still need compatible encodings).
- With a little care, both compatibility directions — and thus rolling upgrades — are entirely realistic.

Looking ahead

That closes Part I (data on a single machine). Part II, *Distributed Data*, opens with why data is spread across machines (scalability, fault tolerance, latency) and shared-memory, shared-disk and shared-nothing architectures — summarised at the start of the Chapter 5 notes, which then cover replication.

Interview angle

1. What are backward and forward compatibility, and why do you need both? *30-second answer:* Backward = new code reads old data; forward = old code reads new data. Rolling upgrades and un-updated clients mean versions coexist and read each other's data. Backward is easy (you know the old format); forward requires old code to ignore unknown additions.

- *Follow-up: Which does a database need?* Both: future versions read old data, and during rolling upgrades old instances read rows from new ones. Also preserve unknown fields on read-modify-write.
- *Follow-up: Which do services need?* Assuming servers upgrade first: backward-compatible requests, forward-compatible responses.

2. How do Protocol Buffers / Thrift handle schema evolution? *30-second answer:* Each field has a numeric tag; data stores tag + type + value, never names. Old code skips unknown tags (type gives the length) → forward compatible. Tags keep their meaning → backward compatible, if new fields are optional or defaulted. Rename freely, never change or reuse tags, remove only optional fields.

- *Follow-up: int32 → int64?* New code reads old fine; old code may truncate new values.
- *Follow-up: optional → repeated in protobuf?* Allowed: new code sees 0/1-element lists, old code sees the last element.

3. How is Avro different? *30-second answer:* No tags or type markers — values in schema order (32 bytes). Decoding needs the writer's schema, resolved against the reader's by field name: unknown fields ignored, missing ones defaulted. Only defaulted fields may be added/removed. The writer's schema comes from a file header, a per-record version with a schema registry, or connection negotiation. Great for auto-generated schemas, e.g. from database tables.

- *Follow-up: Null?* Via unions like `union { null, long }`; null can be the default only as a branch (strictly the first).

4. Why is RPC's "make a remote call look local" idea flawed? *30-second answer:* Network calls can be lost, time out with unknown outcome, be duplicated on retry (need idempotence), have variable latency, require encoding all parameters, and cross language type systems. Modern frameworks use futures, streams and service discovery; REST is explicit about being a network protocol.

- *Follow-up: REST or RPC?* REST for public APIs (curl-debuggable, universal tooling); binary RPC (gRPC, Thrift) for internal calls where performance matters.

5. When would you use a message broker instead of direct RPC? *30-second answer:* For buffering against slow/offline consumers, redelivery after crashes, not needing recipient addresses, fan-out, and decoupling. Trade-off: one-way async communication; replies need a reply queue.

- *Follow-up: Encoding pitfall?* Republishing consumers must preserve fields they don't understand.

6. Why avoid Java Serializable / pickle for stored data? *30-second answer:* Language lock-in, remote-code-execution risk on untrusted bytes, weak versioning, poor efficiency (Java serialization is notoriously bloated). Only for very short-lived data.

Self-check

1. **Define backward and forward compatibility. Which is usually harder, and why?** Backward: new code reads old data. Forward: old code reads new data. Forward is harder because old code must ignore unknown additions.
2. **Why can't JavaScript reliably read Twitter tweet IDs sent as JSON numbers, and how did Twitter work around it?** IDs are 64-bit, but JavaScript uses IEEE 754 doubles, exact only up to 2^53. Twitter sends each ID as a number and as a decimal string.
3. **Rank these by encoded size for Example 4-1: Protobuf, JSON, Avro, MessagePack, Thrift Compact, Thrift Binary.** Avro 32 < Protobuf 33 < Thrift Compact 34 < Thrift Binary 59 < MessagePack 66 < JSON 81.
4. **In Protocol Buffers, can you rename a field? Change its tag? Add a required field?** Rename: yes. Change tag: no (existing data invalid). Add required: no (new code fails on old data).
5. **Avro data has no field tags or types. How can a reader possibly decode it after the schema changes?** It has the writer's schema (file header, version lookup, or negotiation) and its own reader's schema; the library matches fields by name, ignores extras and fills defaults.
6. **Scenario: you add a field without a default to an Avro schema. What breaks?** Backward compatibility: new readers can't read old records (no value to fill in).
7. **Scenario: an old service version reads a user record, changes the email, and writes it back — and a newly added field disappears. Why, and how do you prevent it?** The model object had no slot for the new field, so re-encoding dropped it. Preserve unknown fields and avoid lossy model translation.
8. **Why is it reasonable to need only backward-compatible requests and forward-compatible responses for RPC?** Servers are upgraded before clients: new servers must accept old requests, old clients must accept new responses.
9. **Give three advantages of a message broker over direct RPC and one disadvantage.** Buffering, redelivery, no need for recipient addresses, fan-out, decoupling. Disadvantage: usually one-way/asynchronous.
10. **Why does location transparency work better for actors than for RPC?** Actors already assume messages can be lost even locally, so the network doesn't break their assumptions — unlike RPC, which pretends remote calls are as reliable as local ones.

Chapter 5 — Replication

Book: Chapter 5, pp. 151–198 (plus the Part II introduction that precedes it)

Overview

First, the Part II introduction: why spread data over many machines?

Part I covered data on one computer; Part II asks what changes when **several machines** store and serve it. Three motivations:

Reason	What it buys you
Scalability	When data volume or read/write load outgrows one machine, share the work.
Fault tolerance / high availability	If a machine (or several, the network, or a whole datacenter) dies, a redundant one takes over.
Latency	Servers on several continents let each user talk to a nearby datacenter instead of sending packets across the planet.

Scaling to Higher Load

Architecture	How it works	Strengths	Weaknesses
Shared-memory (vertical scaling, "scaling up")	One bigger machine: many CPUs, RAM chips and disks under one OS, joined by a fast interconnect so any CPU reaches any memory or disk.	Simple: still "one machine".	Cost grows faster than linearly, and bottlenecks mean twice the hardware does not give twice the throughput. Some fault tolerance via hot-swappable parts (disks, memory, even CPUs), but in a single location.
Shared-disk	Several machines with their own CPUs and RAM sharing one disk array over a fast network (NAS = Network Attached Storage, SAN = Storage Area Network).	Used for some data-warehouse workloads.	Contention and locking overhead limit scaling.
Shared-nothing (horizontal scaling, "scaling out")	Each machine or VM running the database is a **node**, using only its own CPUs, RAM and disks; nodes coordinate in software over an ordinary network.	No special hardware, so best price/performance. Data can span regions, cutting latency and surviving loss of a whole datacenter; with cloud VMs even small companies can go multi-region.	More application complexity; can restrict usable data models.

> **NUMA aside.** Even in one shared-memory machine, some memory banks are nearer to certain CPUs (nonuniform memory access). Using it well means splitting work so each CPU mostly touches nearby memory — partitioning even on "one machine".

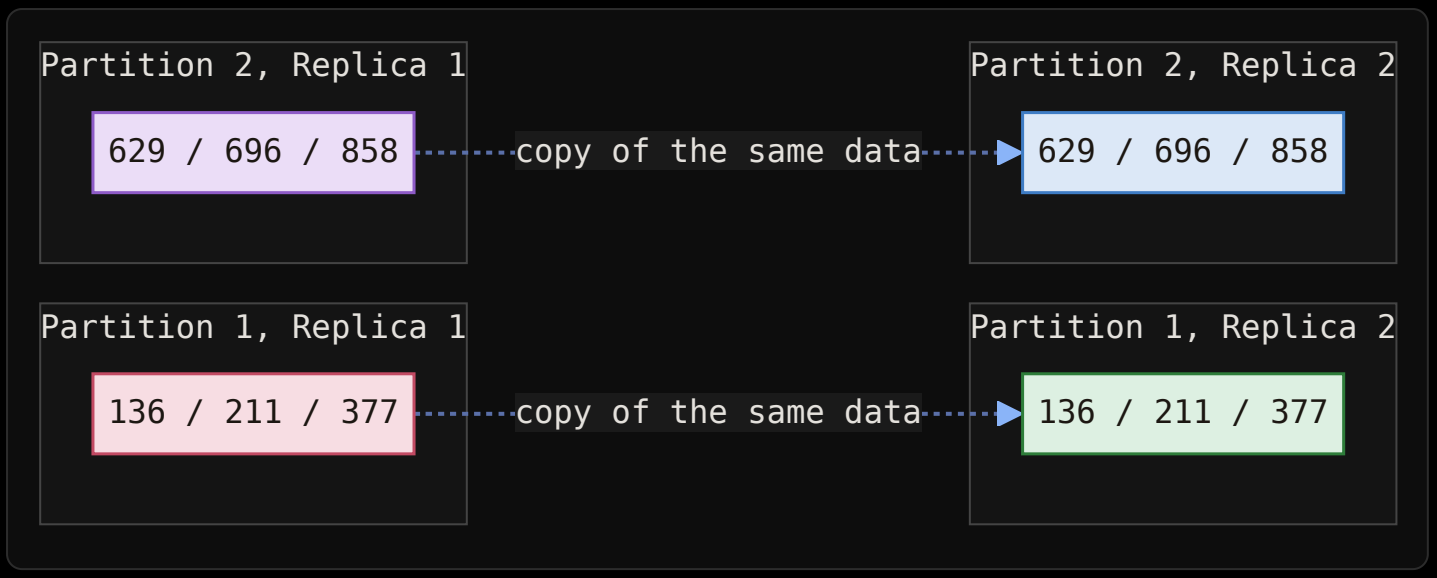
Part II focuses on **shared-nothing** — not because it is always best, but because it demands the most care from developers: the database cannot hide all the constraints of distribution. A simple single-threaded program can sometimes beat a cluster of 100+ CPU cores, so distribution is not free performance.

Replication Versus Partitioning

- **Replication**: the *same* data copied onto several nodes, possibly in different places — redundancy (others serve data when some are down) and sometimes better performance. Chapter 5.
- **Partitioning** (**sharding**): a large database split into **partitions** placed on different nodes. Chapter 6.

Independent mechanisms, usually combined.

Figure II-1 (redrawn): a database split into two partitions, each partition kept as two replicas holding the same records (the book shows records keyed 136, 211, 377 in partition 1 and 629, 696, 858 in partition 2).



Then Chapter 7 covers transactions, Chapters 8–9 the fundamental limits of distributed systems, and Part III combining several data stores into larger systems.

What this chapter answers

Replication keeps copies of the same data on several networked machines. *When data keeps changing, how do we keep the copies in step, and what goes wrong when they are not?*

Reasons to replicate: keep data near users (**latency**); keep working when parts fail (**availability**); let more machines serve reads (**read throughput**). (Primer on latency, availability, replication and sharding: System Design Fundamentals.)

Assume each machine holds the whole dataset (Chapter 6 lifts this). If data never changed, replication would be trivial; the difficulty is **changes**. Three algorithms cover almost every distributed database: **single-leader**, **multi-leader**, **leaderless**. Also covered: synchronous vs asynchronous replication, failed replicas, and **eventual consistency**. The theory dates to the 1970s (network limits haven't changed); what is new is mainstream use, which causes much misunderstanding.

Key terms

Term	Plain-English meaning
Replica	A node holding its own full copy of the data.
Leader (master, primary)	The replica that accepts writes and sends them to the others.
Follower (read replica, slave, secondary, hot standby)	Copies changes from the leader; serves only reads.
Leader-based replication (active/passive, master–slave)	All writes go to one leader; followers copy them.
Replication log / change stream	Ordered list of changes the leader sends followers.
Synchronous / asynchronous / semi-synchronous	Leader waits for a follower's confirmation before reporting success / doesn't wait / one follower synchronous, rest async.
Chain replication	Synchronous-replication variant (e.g. Microsoft Azure Storage).
Snapshot	Consistent copy of the whole database at one moment.
Log sequence number / binlog coordinates	PostgreSQL's / MySQL's name for an exact replication-log position.
Caught up / catch-up recovery	Follower has applied all changes to the leader's current position / restarted follower requesting missed changes.
Failover	Promoting a follower to leader when the leader dies.
Controller node	Previously elected node that can appoint a new leader.
Split brain	Two nodes both believing they are leader.
Fencing / STONITH	Forcibly shutting down a node when two leaders are detected ("Shoot The Other Node In The Head").
Statement-based replication	Sending SQL statements themselves to followers.
Nondeterministic function	E.g. `NOW()` or `RAND()`: may differ each call.
Write-ahead log (WAL) shipping	Sending the storage engine's low-level append-only log.
Logical (row-based) log	Row-level change log independent of storage format.
Change data capture	Streaming a database's changes to outside systems (Chapter 11).
Trigger / trigger-based replication	Custom code run automatically on data changes / using it to record changes for an external replicator.
Read-scaling architecture	Many followers sharing read traffic.
Replication lag	Delay between a write on the leader and its appearance on a follower.
Eventual consistency	If writes stop, replicas *eventually* agree; no time bound.
Read-after-write (read-your-writes) / cross-device	A user always sees their own updates / also across their devices.
Monotonic reads	A user never sees data go "back in time".
Consistent prefix reads	Writes are seen in the order they happened.
Multi-leader (master–master, active/active)	Several nodes accept writes; each leader follows the others.
Write conflict	Two leaders concurrently accept different changes to the same data.
Conflict avoidance	Routing all writes for a record through one leader.

Term	Plain-English meaning
Convergent conflict resolution	All replicas reach the same final value.
Last write wins (LWW)	Keep the write with the highest timestamp, discard the rest.
CRDT	Conflict-free replicated datatype; merges concurrent edits automatically.
Mergeable persistent data structure	Tracks history like Git; three-way merges.
Operational transformation	Conflict resolution algorithm of Google Docs and Etherpad.
Replication topology	Paths writes travel between leaders (circular, star, all-to-all).
Leaderless (Dynamo-style) replication	Any replica accepts writes; client or **coordinator node** (which imposes no order) sends to many replicas.
Read repair / anti-entropy	Client writes a newer value back to a stale replica seen during a read / background process fixing differences.
Quorum (n, w, r)	n replicas; writes need w acks, reads query r; w + r > n gives overlap.
Strict / sloppy quorum	Uses only a value's designated n nodes / during failures may use nodes outside them.
Hinted handoff	Returning temporarily held writes to their home nodes.
Concurrent operations / happens-before / causally dependent	Neither knows of the other / B knows about, depends on, or builds on A / B depends on A, so A applies first.
Siblings	Riak's name for concurrent values kept side by side.
Tombstone	Marker for a deleted item so deletion survives merges.
Version vector / dotted version vector / causal context	Per-replica version counters distinguishing overwrites from concurrent writes / variant in Riak 2.0 / Riak's string encoding of it.

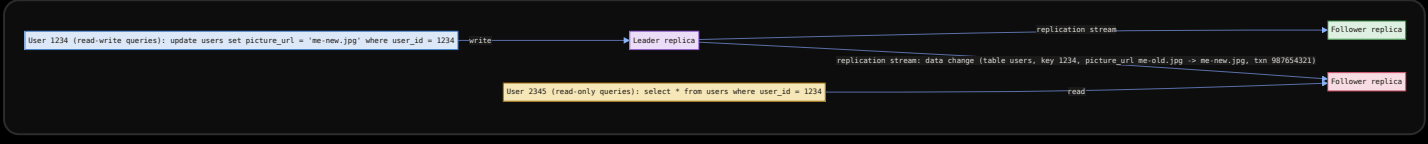
Leaders and Followers

Every write must reach every replica. The most common approach is **leader-based replication**:

1. One replica is the **leader**; all writes go to it and it writes them to local storage first.
2. **Followers** receive each change via the **replication log** and apply changes **in the leader's order**.
3. Reads may go to leader or followers; only the leader accepts writes.

> **"Hot" vs "warm" standby.** Definitions vary; in PostgreSQL a hot standby serves reads, a warm standby applies changes but answers no queries. The book treats the difference as unimportant.

Figure 5-1 (redrawn): leader-based replication. User 1234 changes their profile picture by writing to the leader; the leader streams the row change to both followers; user 2345 reads user 1234's profile from a follower.

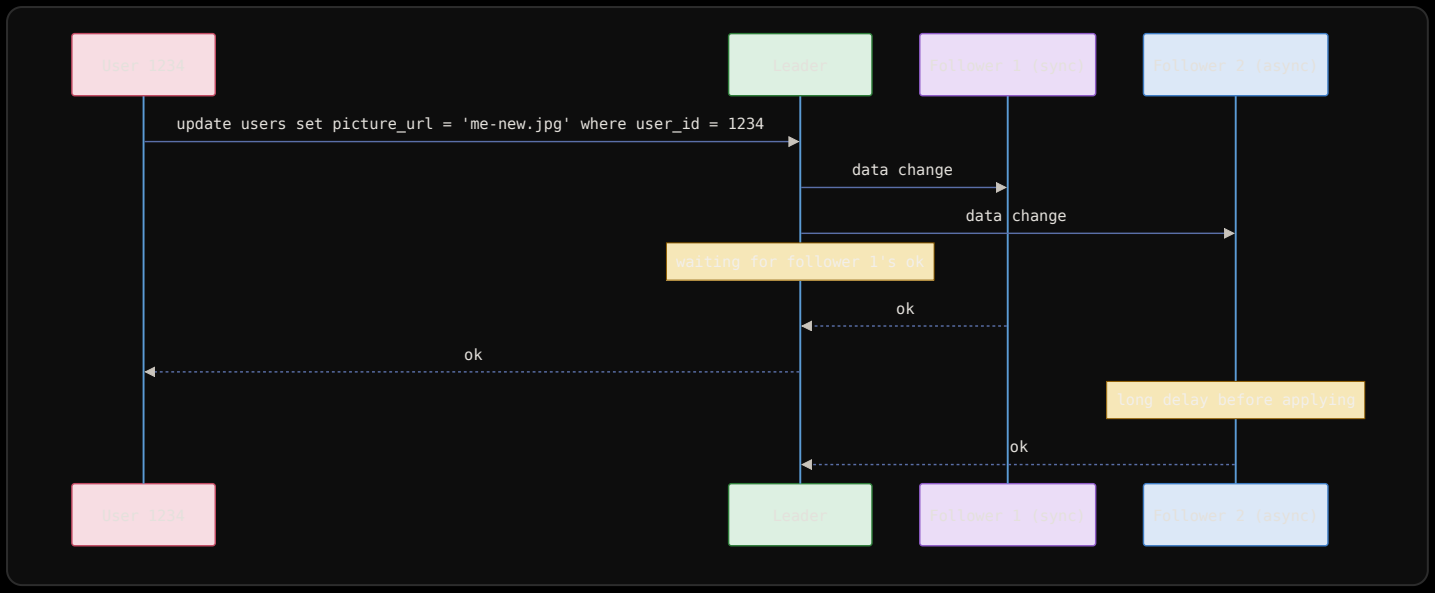


Who uses it: PostgreSQL (since 9.0), MySQL, Oracle Data Guard, SQL Server AlwaysOn Availability Groups; MongoDB, RethinkDB, Espresso; message brokers Kafka and RabbitMQ highly available queues; DRBD (replicated block device) and some network filesystems.

Synchronous Versus Asynchronous Replication

What it is. Does the leader wait for a follower's confirmation before telling the client "done"? Yes → **synchronous** follower; no → **asynchronous**. Relational databases often make this configurable; other systems hard-code it.

Figure 5-2 (redrawn): follower 1 is synchronous (leader waits for its ok before answering the user), follower 2 is asynchronous (leader answers without waiting; follower 2 applies the change and acknowledges much later).



Followers usually apply changes within a second, but **there is no upper bound**: minutes behind when recovering from a crash, near maximum capacity, or under network problems.

	Synchronous follower	**Asynchronous follower**
Up-to-date copy guaranteed?	Yes: if the leader dies, the data is safe on this follower	No: unreplicated writes can be lost
Effect if the follower is down/slow	Leader must **block all writes** until it returns	Leader carries on regardless

Making *all* followers synchronous is impractical (one stuck node halts everything). The compromise is **semi-synchronous**: one synchronous follower, the rest async; if it becomes slow or unavailable, an async follower is made synchronous. Data is then current on at least two nodes.

Often replication is **fully asynchronous**: a write confirmed to the client can still be lost if the leader fails permanently before forwarding it, so durability is **not guaranteed** — but the leader keeps accepting writes even if all followers lag. This trade-off is widely accepted, especially with many or geographically distributed followers.

> **Research on replication. Chain replication**, a synchronous variant, has been deployed successfully in a few systems, such as Microsoft Azure Storage. Replication consistency is closely tied to **consensus** (Chapter 9).

Setting Up New Followers

Copying data files doesn't work (writes keep arriving, so the copy mixes different moments), and locking the database kills availability. The no-downtime procedure:

1. Take a **consistent snapshot** of the leader, ideally without locking (backups need this anyway; sometimes a third-party tool such as *innobackupex* for MySQL, part of Percona XtraBackup).
2. **Copy** it to the new follower.
3. The follower **requests all changes since the snapshot**, which must be tied to an exact log position: PostgreSQL's **log sequence number**, MySQL's **binlog coordinates**.

4. After processing that backlog it has **caught up** and follows the live stream.

Some databases automate this; others need an obscure multi-step admin procedure.

Handling Node Outages

Nodes go down from faults and planned maintenance (e.g. rebooting for a kernel security patch). Restarting individual nodes without downtime is a big operational win.

Follower failure: Catch-up recovery

Each follower logs received changes on local disk. After a crash, restart or network break, it finds the last transaction it processed, requests all later changes from the leader, applies them, and resumes the stream.

Leader failure: Failover

Failover: promote a follower, point client writes at it, and have other followers consume from it — manually or automatically. Automatic failover:

1. **Decide the leader has failed.** No foolproof detector, so use a **timeout**: nodes exchange messages, and one silent for, say, **30 seconds** is presumed dead (not applicable to planned shutdowns).
2. **Choose a new leader** by **election** (majority of remaining replicas) or appointment by a previously elected **controller node**. Best candidate: the replica with the most up-to-date data, minimising loss. Agreement is a **consensus** problem (Chapter 9).
3. **Reconfigure** clients to send writes to the new leader ("Request Routing", Chapter 6). If the old leader returns believing it is still leader, force it to become a follower.

What can go wrong:

- **Lost writes.** With async replication, the new leader may lack the old leader's last writes, and may have accepted conflicting writes meanwhile. Usually the old leader's unreplicated writes are **discarded**, breaking clients' durability expectations.
- **Danger with outside systems.** At **GitHub**, an out-of-date MySQL follower was promoted; its autoincrement primary-key counter lagged, so it **reused primary keys** the old leader had assigned. Those keys were also used in **Redis**, so the stores disagreed and some private data was shown to the wrong users.
- **Split brain.** Two nodes believe they are leader (Chapter 8); if both accept writes without conflict resolution, data can be lost or corrupted. Some systems shut one down (**fencing**, **STONITH**), but a badly designed mechanism can shut down *both* (the book cites a GitHub outage postmortem).
- **Choosing the timeout.** Too long: slow recovery. Too short: needless failovers from load spikes or network glitches, which usually make an already struggling system **worse**.

No easy fixes, so some teams fail over **manually** even when software could automate it. Node failures, unreliable networks, and trade-offs among consistency, durability, availability and latency recur in Chapters 8 and 9.

Implementation of Replication Logs

Four methods are used in practice.

Statement-based replication

How it works. The leader logs every write statement (`INSERT`, `UPDATE`, `DELETE`); followers parse and execute each as if from a client. Illustrative statement log (our example):

```sql
-- statement 1: fine to replay anywhere
INSERT INTO orders (customer, item) VALUES ('ana', 'lamp');
-- statement 2: NOW() gives a different time on each replica when replayed
UPDATE orders SET shipped_at = NOW() WHERE customer = 'ana';
-- statement 3: result depends on current data and on execution order
DELETE FROM orders WHERE shipped_at < '2024-01-01';
```

Where it breaks:

- Nondeterministic functions (`NOW()`, `RAND()`) give different values per replica.

- Statements using an autoincrement column or depending on existing data (`UPDATE ... WHERE <condition>`) must run in **exactly** the same order everywhere — limiting with concurrent transactions.
- Side effects (triggers, stored procedures, user-defined functions) may differ unless fully deterministic.

Workarounds exist (e.g. the leader substitutes a fixed value for `NOW()`), but edge cases abound, so other methods are preferred.

Who uses it: MySQL before 5.1. Still used for compactness, but MySQL now switches to row-based replication whenever a statement is nondeterministic. **VoltDB** uses it safely by requiring deterministic transactions.

Write-ahead log (WAL) shipping

Storage engines append every write to a log (Chapter 3): in **log-structured** engines (SSTables/LSM-trees) the log *is* the storage, compacted and garbage-collected in the background; in **B-trees**, the **write-ahead log** records each change before the tree is touched, for crash recovery. Either way, the leader can send this byte log to followers, which replay it to build identical structures. **Used by PostgreSQL and Oracle**.

Downside: the WAL is very low level (which bytes changed in which disk blocks), so replication is **tightly coupled to the storage engine**. If the on-disk format changes between versions, leader and followers usually can't run different versions — ruling out **zero-downtime upgrades** (upgrade followers, then fail over to one), so upgrades need downtime.

Logical (row-based) log replication

Use a **separate log format for replication**, decoupled from engine internals: a **logical log**. For relational databases, a sequence of row-level records:

- **Insert:** new values of all columns.
- **Delete:** enough to identify the row — normally the primary key, else all old column values.
- **Update:** row identifier plus new values (all columns, or at least the changed ones).

A multi-row transaction produces several records plus a commit record. **MySQL's binlog** in row-based mode works like this. Illustrative records (our data):

```
{"op": "insert", "table": "books", "row": {"id": 7, "title": "Dune", "stock": 3}}
{"op": "update", "table": "books", "key": {"id": 7}, "changed": {"stock": 2}}
{"op": "delete", "table": "books", "key": {"id": 4}}
{"op": "commit", "txid": 912}
```

Benefits: easier **backward compatibility**, so leader and followers can run different versions or even storage engines; easier for **external applications** to parse (data warehouses, custom indexes and caches) — **change data capture** (Chapter 11).

Trigger-based replication

For more flexibility — replicating a subset of data, between different databases, or with custom **conflict resolution** — move replication to the application layer:

- Tools like **Oracle GoldenGate** read the database log and expose changes to an application.
- Or use **triggers** and **stored procedures**: a trigger logs each change into a separate table, which an external process reads, applies application logic to, and replicates. **Databus for Oracle** and **Bucardo for Postgres** work this way.

Minimal illustration (PostgreSQL-style, our names); an outside process would poll `change_queue` and ship the rows:

```
CREATE TABLE change_queue (tbl text, row_id int, changed_at timestamptz);

CREATE FUNCTION log_change() RETURNS trigger AS $$
BEGIN
  INSERT INTO change_queue VALUES (TG_TABLE_NAME, NEW.id, now());
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER books_changes AFTER INSERT OR UPDATE ON books
  FOR EACH ROW EXECUTE FUNCTION log_change();
```

Trade-off: more overhead and more bug-prone than built-in replication, but flexible.

Method	What is sent	Pros	Cons	Examples
Statement-based	SQL statements	Compact	Nondeterminism, ordering, side effects	MySQL < 5.1, VoltDB
WAL shipping	Low-level disk-block changes	Exact copy, no extra log	Tied to storage format; version upgrades need downtime	PostgreSQL, Oracle
Logical (row-based)	Row-level changes	Version-independent; good for CDC	Book names no specific drawback; it is simply a second log format separate from storage	MySQL binlog (row mode)
Trigger-based	Whatever app code records	Very flexible	Overhead, bugs	Databus, Bucardo

Problems with Replication Lag

Replication also serves **scalability** and **latency**. With one leader, reads can go anywhere, so read-heavy workloads (common on the web) can add many followers: a **read-scaling architecture**. This only works with **asynchronous** replication — synchronous replication to all followers would let any node failure or glitch halt writes, more likely with more nodes.

But async followers can return **outdated data**: leader and follower may answer the same query differently at the same moment. Stop writing and wait, and followers catch up: **eventual consistency** (coined by Douglas Terry et al., popularised by Werner Vogels, a NoSQL rallying cry — though async-replicated relational databases behave the same).

"Eventually" is deliberately vague: lag has no bound. Normally a fraction of a second, it can grow to seconds or minutes near capacity or with network trouble, and then the inconsistencies become real application bugs. Three examples follow.

Reading Your Own Writes

Problem. A user submits data (a record, a comment) then views it; the write went to the leader but the read hits a follower that hasn't received it, so the data seems lost.

Figure 5-3 (redrawn): user 1234 posts a comment via the leader, then immediately reads from follower 2, which has not received the insert yet, and gets no results.



Guarantee: read-after-write consistency (**read-your-writes**): after reloading, users always see their *own* updates (no promise about others').

Techniques:

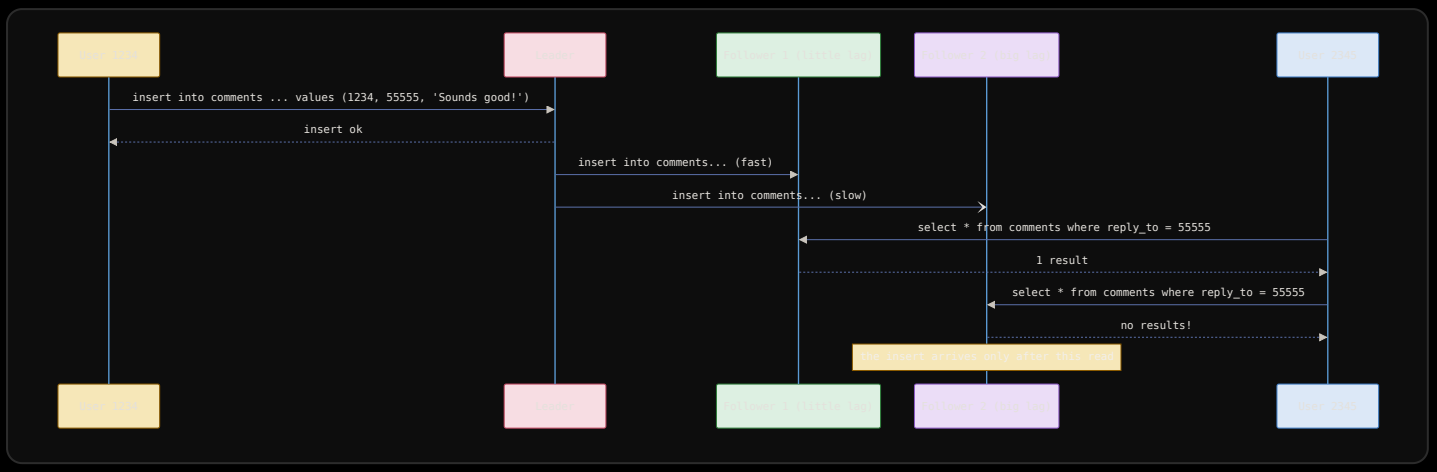
1. **Read possibly-modified data from the leader**, the rest from followers. Needs a way to know what might have changed without querying, e.g. users edit only their own profile, so read your own profile from the leader and others' from followers.
2. If most things are user-editable, that defeats read scaling; use other criteria, e.g. read from the leader for **one minute after the user's last update**, or avoid followers more than one minute behind.
3. **Client remembers its latest write's timestamp**; the serving replica must reflect at least that, else use another replica or wait. The timestamp can be **logical** (e.g. log sequence number) or the **system clock** (then clock sync matters; Chapter 8).
4. With **multiple datacenters**, requests needing the leader must be routed to its datacenter.

Cross-device read-after-write (enter on laptop, view on phone) adds issues: last-write metadata must be **centralised** (one device doesn't know what another wrote); and devices may reach **different datacenters** (home broadband vs mobile network), so leader-read approaches may first require routing all of a user's devices to one datacenter.

Monotonic Reads

Problem: time appears to go backwards when successive reads hit a fresh then a stale follower (likely if each refresh hits a random server).

Figure 5-4 (redrawn): user 2345 first sees user 1234's new comment on a fresh follower, then refreshes, hits a stale follower and the comment is gone.



Seeing nothing twice would go unnoticed; seeing the comment appear then vanish is the confusion.

Guarantee: monotonic reads — weaker than strong consistency, stronger than eventual: data may be old, but one user's successive reads never move to an *older* state.

How: each user always reads from the **same replica** (different users may use different ones), e.g. chosen by **hashing the user ID** rather than randomly; if that replica fails, reroute.

Consistent Prefix Reads

Problem: causality violated. The book's dialogue: Mr. Poons asks how far into the future Mrs. Cake can see; she answers about ten seconds. If an observer reads the answer through a fast replica and the question through a slow one, the answer appears **before** the question, as if Mrs. Cake were psychic.

Figure 5-5 (redrawn): Mr. Poons's question goes to partition 1, whose follower lags a lot; Mrs. Cake's answer goes to partition 2, whose follower lags little. Each speaker hears the other via the leaders, but the observer, reading from the followers, receives the answer first.



Guarantee: consistent prefix reads — writes made in some order are read in that order.

Mostly a problem in **partitioned (sharded)** databases (Chapter 6): a single global write order would always give consistent prefixes, but partitions operate independently with **no global order**, so a reader may see some parts old and some new.

Fixes: write causally related data to the **same partition** (not always efficient), or use algorithms that **track causal dependencies** (see "The happens-before relationship" below).

Solutions for Replication Lag

- Ask: *what if lag grew to minutes or hours?* If users would suffer, design for a stronger guarantee such as read-after-write. **Pretending async replication is synchronous invites future trouble.**
- Applications can provide stronger guarantees themselves (e.g. leader reads), but it is complex and error-prone.
- Better if the database "does the right thing" — the purpose of **transactions**.
- Many distributed systems abandoned transactions, claiming they cost too much performance and availability and that eventual consistency is inevitable at scale. The book calls this partly true but **too simplistic**, revisiting it in Chapters 7, 9 and Part III.

Anomaly	What the user sees	Guarantee	Typical fix
Own write missing	"My post disappeared"	Read-after-write	Read own data from leader; track last-write timestamp
Time goes backwards	Item appears, then vanishes	Monotonic reads	Pin each user to one replica (hash of user ID)
Effect before cause	Answer before question	Consistent prefix reads	Causally related writes in same partition; causal tracking

Multi-Leader Replication

Single-leader's weakness: **all writes go through one leader**; if you can't reach it (e.g. a network break), you can't write. (In a partitioned database each partition has its own single leader, possibly on different nodes, but still exactly one per partition.)

Extension: **several nodes accept writes**, each forwarding changes to all others: **multi-leader** (master–master, active/active) replication, where each leader is also a follower of the others.

Use Cases for Multi-Leader Replication

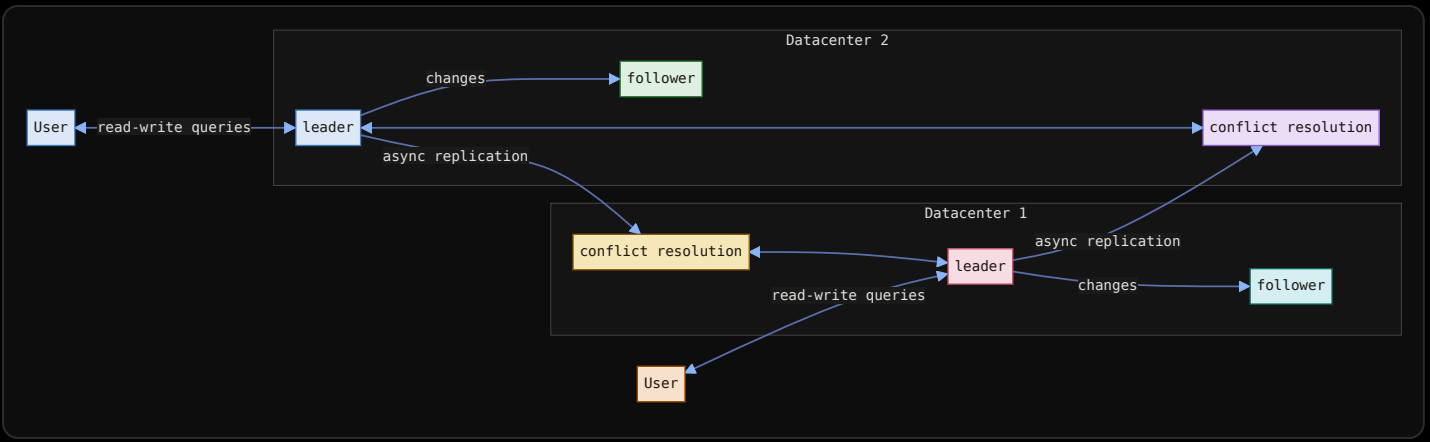
Rarely worth the complexity within one datacenter, but useful in these cases.

Multi-datacenter operation

With replicas in several datacenters, single-leader forces every write to the leader's datacenter. Multi-leader gives **each datacenter its own leader**: leader–follower replication inside each, leader-to-leader replication between them.

Figure 5-6 (redrawn): each datacenter has its own leader, which serves the local users' read-write queries and sends changes to a local follower; the two leaders replicate to each other, each passing

incoming changes through a conflict-resolution step.



Aspect	Single-leader across datacenters	Multi-leader
Performance	Every write crosses the internet to the leader's datacenter, adding latency and undermining multiple sites	Writes handled locally, replicated asynchronously; inter-datacenter delay hidden from users
Datacenter outage	Failover promotes a follower in another datacenter	Each datacenter runs independently; replication catches up when the failed one returns
Network problems	Very sensitive to the less reliable inter-datacenter (public internet) link, since writes cross it synchronously	Async replication tolerates temporary interruptions; writes continue

Some databases support it natively; often it is added via external tools: **Tungsten Replicator** (MySQL), **BDR** (PostgreSQL), **GoldenGate** (Oracle).

Big downside: the same data can be changed concurrently in two datacenters, causing **write conflicts**. And because it is often retrofitted, it interacts badly with **autoincrementing keys, triggers, and integrity constraints**. Often regarded as dangerous territory to avoid if possible.

Clients with offline operation

Offline-capable clients, e.g. a **calendar** on phone, laptop and other devices: you must read and add meetings anytime, syncing when back online. **Each device's local database acts as a leader**, with async multi-leader replication (sync) between devices and lag of **hours or days**. It is the multi-datacenter case taken to the extreme: each device a "datacenter" on a very unreliable link. Calendar sync's history of bugs shows how hard this is. **CouchDB** was designed for this mode.

Collaborative editing

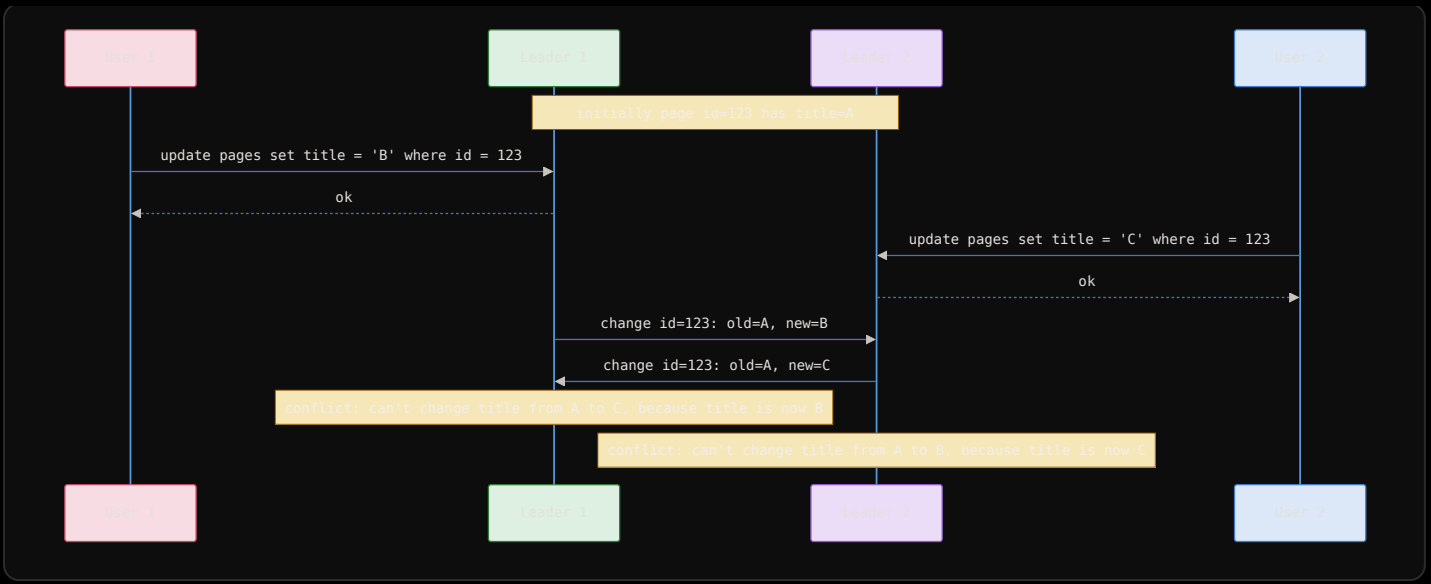
Real-time collaborative editors (**Etherpad**, **Google Docs**) apply each user's edits instantly to their local replica and replicate asynchronously to the server and other editors, like the offline case.

- **Guaranteeing no conflicts** means locking the document so one user edits at a time — equivalent to **single-leader replication with transactions on the leader**.
- Faster collaboration uses tiny units of change (e.g. **one keystroke**) and no locking, bringing all multi-leader challenges including conflict resolution.

Handling Write Conflicts

Conflicts are multi-leader replication's central problem.

Figure 5-7 (redrawn): page id=123 starts with title A. User 1 changes it to B on leader 1 while user 2 changes it to C on leader 2; both get "ok", and each leader only finds the conflict when the other's change arrives.



This cannot happen with a single leader.

Synchronous versus asynchronous conflict detection

- **Single leader:** the second writer blocks until the first completes, or is aborted and retries.
- **Multi-leader:** both writes succeed locally; the conflict is found **later, asynchronously**, when asking the user may be impossible.
- Synchronous detection (waiting for all replicas before confirming) would forfeit multi-leader's main benefit of independent writes; then just use single-leader.

Conflict avoidance

Simplest: route all writes for a given record through **the same leader**, so conflicts can't occur — frequently recommended, since many implementations handle conflicts poorly. E.g. give each user a **"home" datacenter** (perhaps the nearest) whose leader handles their reads and writes: single-leader from that user's view.

Breaks when the designated leader must change: a datacenter fails, or a user moves nearer another datacenter. Concurrent writes on different leaders become possible again.

Converging toward a consistent state

A single leader applies writes sequentially, so the last write determines the value. With multiple leaders there is **no defined order**: in Figure 5-7 leader 1 sees B then C, leader 2 sees C then B. Applying in arrival order would leave leader 1 with C and leader 2 with B — permanent inconsistency. Resolution must be **convergent**: all replicas reach the same final value once changes have replicated.

Ways to converge:

1. **Unique ID per write** (timestamp, long random number, UUID (128-bit random identifier), or hash of key and value); highest ID wins. With timestamps this is **last write wins (LWW)**: popular but **dangerously prone to data loss**.
2. **Unique ID per replica**; writes from higher-numbered replicas win. Also loses data.
3. **Merge the values**, e.g. sort and concatenate (title "B/C").
4. **Record the conflict** in a data structure preserving all information, and let application code resolve it later (perhaps by asking the user).

Custom conflict resolution logic

Resolution is application-specific, so most multi-leader tools accept custom code run:

- **On write:** called when replicated changes conflict. **Bucardo** allows a Perl snippet. Runs in a background process; must be fast and cannot prompt a user.
- **On read:** all conflicting versions are stored and returned on the next read; the application prompts the user or resolves automatically and writes back. **CouchDB** works this way.

Resolution usually applies **per row or document, not per transaction**: a transaction's several writes (Chapter 7) are each resolved separately.

> **Automatic Conflict Resolution.** Custom handlers get complicated and buggy; **Amazon**'s shopping cart once preserved *additions* but not *removals*, so deleted items reappeared. Research directions:
>
> - **CRDTs (conflict-free replicated datatypes)**: sets, maps, ordered lists, counters, etc. that resolve concurrent edits sensibly by themselves; some are in **Riak 2.0**.
> - **Mergeable persistent data structures**: track history like **Git** and use a **three-way merge** (CRDTs use two-way merges).
> - **Operational transformation**: behind **Etherpad** and **Google Docs**, designed for concurrent edits to ordered sequences such as a document's characters.
>
> Database implementations are young but promising.

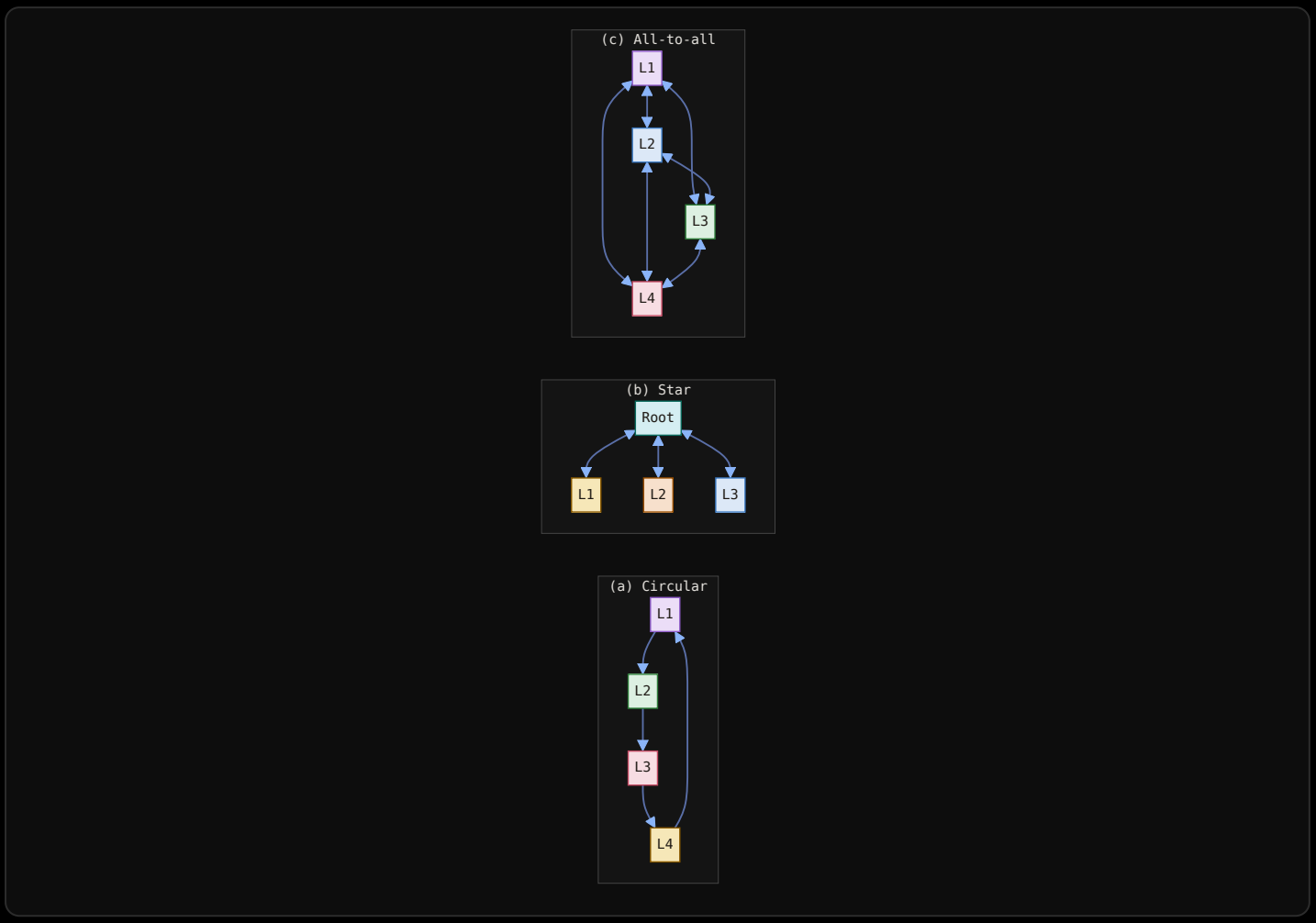
What is a conflict?

Some are obvious (two concurrent writes giving the same field different values). Others are subtle: a **meeting-room booking system** must prevent overlapping bookings, yet two bookings made on **two different leaders** can conflict even if each checked availability. No quick answer; Chapter 7 gives more examples, Chapter 12 scalable detection and resolution.

Multi-Leader Replication Topologies

A **replication topology** is the set of paths writes take between nodes. Two leaders have one sensible option (each sends to the other); more leaders allow several.

Figure 5-8 (redrawn): three example topologies with four leaders each: (a) circular, (b) star (one root in the middle), (c) all-to-all.



- **All-to-all:** every leader sends to every other; most general.

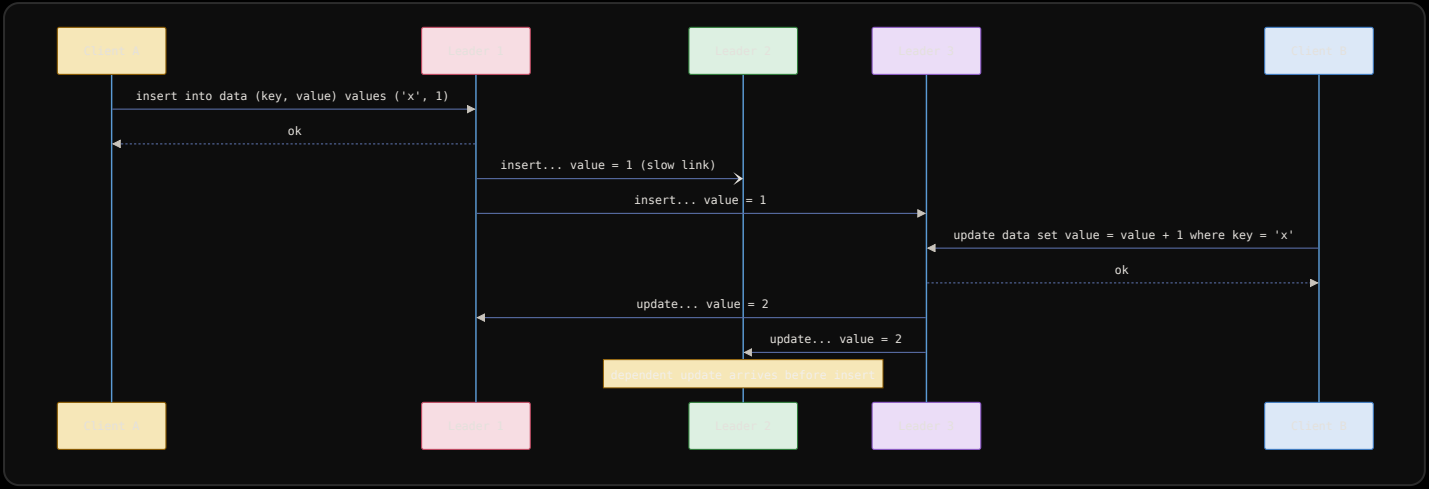
- **Circular:** each node receives from one neighbour and forwards those plus its own writes to the next. **MySQL supports only this by default.**
- **Star:** a root forwards writes to all others; generalises to a **tree**. (Unrelated to the warehouse *star schema*.)

Avoiding infinite loops. In circular and star layouts writes pass through several nodes; each write is tagged with the IDs of nodes it has passed through, and a node ignores changes tagged with its own ID.

Fault tolerance. In circular and star layouts **one failed node can block replication** until fixed (rerouting is usually manual). Denser topologies like all-to-all avoid this single point of failure.

All-to-all's problem: messages overtaking each other. Some links are faster (e.g. due to congestion elsewhere), so replication messages arrive out of order.

Figure 5-9 (redrawn): client A inserts row x on leader 1, client B increments it on leader 3, but the insert travels slowly to leader 2, so leader 2 receives the dependent update before the insert.



A **causality** problem like consistent prefix reads: the update depends on the insert, which must be applied first. **Timestamps are not enough** (clocks aren't reliably in sync; Chapter 8); the right tool is **version vectors** (below). Many systems handle this poorly: at the time of writing, **PostgreSQL BDR** gives no causal ordering, and **Tungsten Replicator for MySQL** doesn't even try to detect conflicts.

Advice: with multi-leader replication, know these issues, read the docs, and test that you really get the guarantees you assume.

Leaderless Replication

Single- and multi-leader designs let a leader fix the write order. **Leaderless** systems let **any replica accept client writes directly**.

Some early replicated systems were leaderless; the idea faded during relational dominance and returned with **Amazon**'s in-house **Dynamo**. **Riak, Cassandra, and Voldemort** are open-source leaderless stores inspired by it: **Dynamo-style**. (**DynamoDB**, AWS's hosted product, is a different design using single-leader replication.)

Clients send writes to several replicas themselves, or via a **coordinator node**, which — unlike a leader — does **not impose an order** on writes, with major consequences.

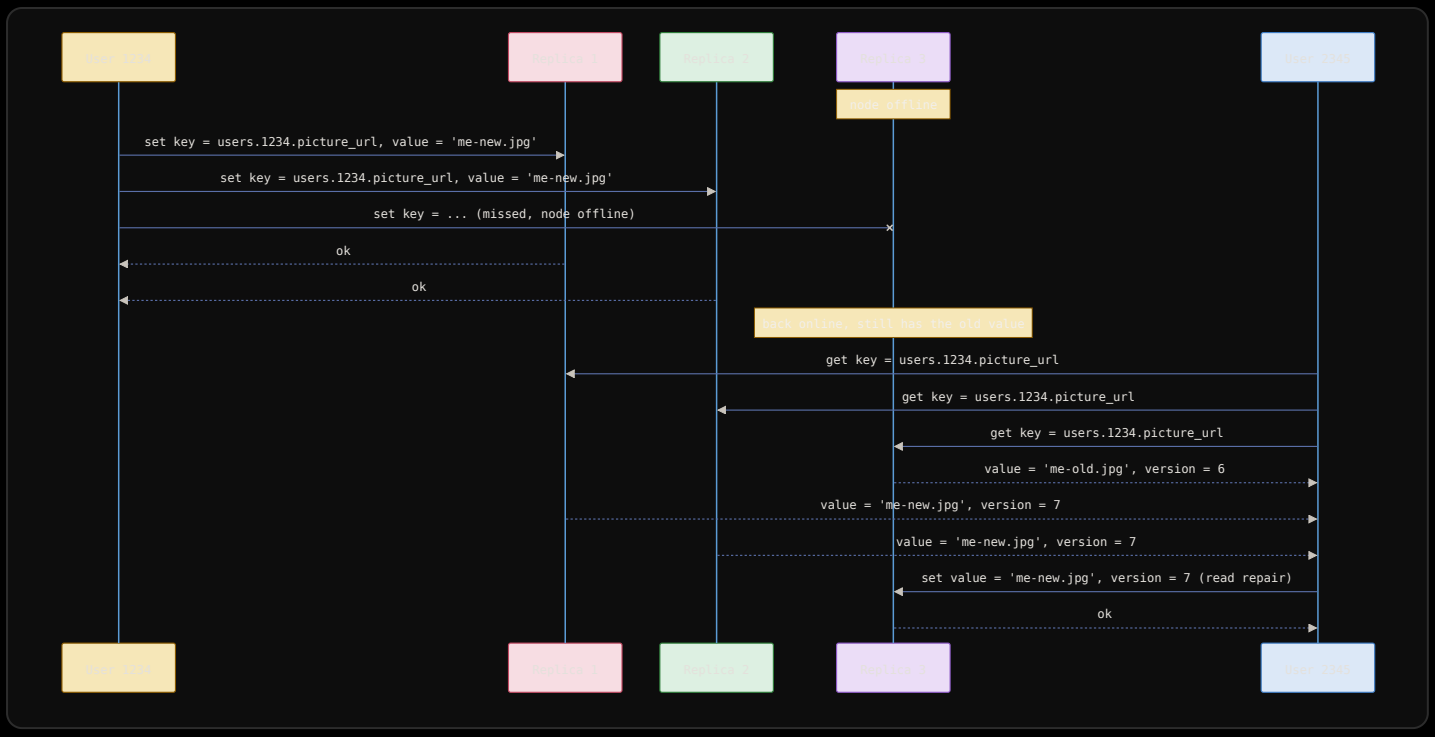
Writing to the Database When a Node Is Down

With three replicas and one down, a leader-based system might need failover. Leaderless systems have **no failover**: the client sends the write to **all three** in parallel; the two available replicas accept it; two of three acknowledgements suffice, so the write **succeeds** and the missed replica is ignored.

The returning node lacks missed writes and would serve **stale** values, so **reads also go to several nodes in parallel**; replies may differ, and **version numbers** identify the newest.

Figure 5-10 (redrawn): user 1234 writes a new picture URL to all three replicas while replica 3 is offline; two oks are enough. Later user 2345 reads from all three, gets version 7 from replicas 1 and 2

but the old version 6 from replica 3, and repairs replica 3.



Read repair and anti-entropy

Mechanism	How it works	Notes
Read repair	During a parallel read, the client spots stale replies (e.g. version 6 vs 7) and writes the newer value back to the stale replica.	Works well for frequently read values.
Anti-entropy process	Background process compares replicas and copies missing data.	Unlike a replication log, copies in **no particular order**, possibly with long delay.

Not all systems have both: **Voldemort currently has no anti-entropy process**, so rarely read values may be missing on some replicas — **less durable**, since read repair only fixes what is read.

Quorums for reading and writing

If every successful write is on at least 2 of 3 replicas, at most one is stale, so reading 2 guarantees at least one fresh reply.

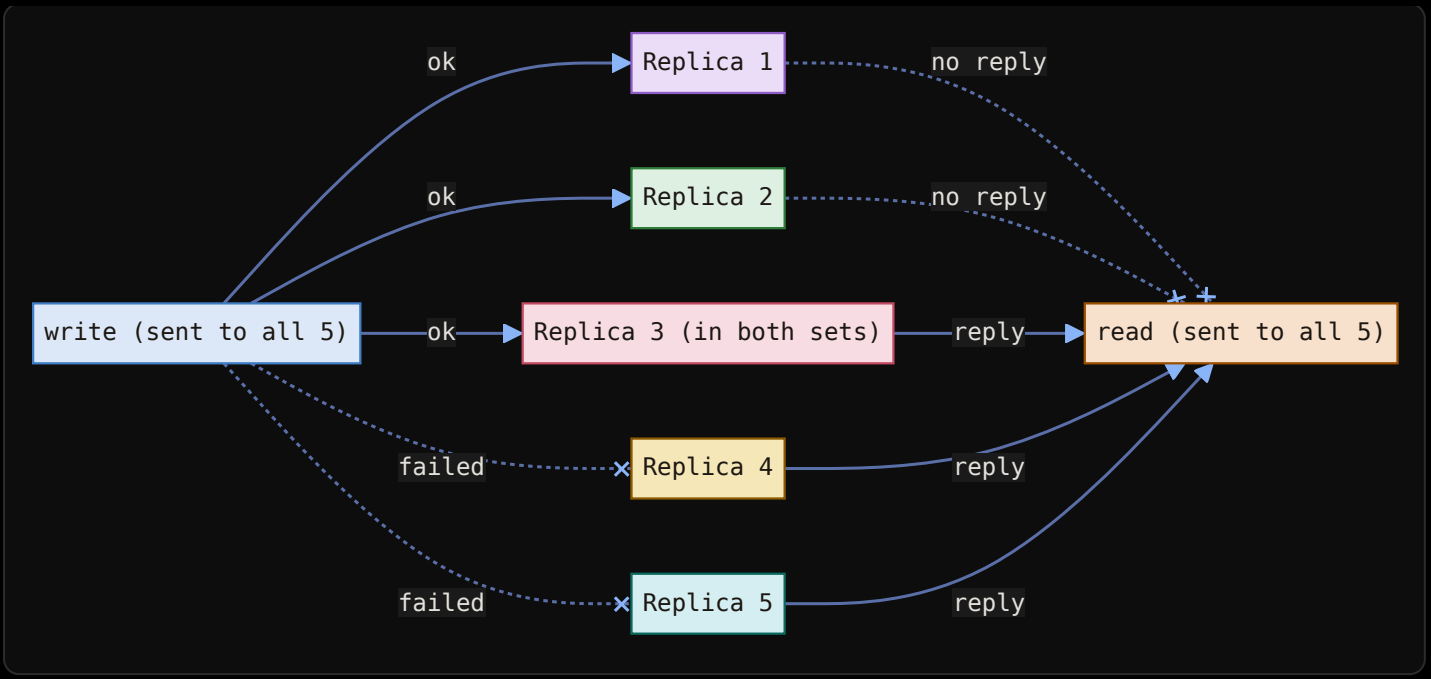
General rule: **n** replicas; writes confirmed by **w** nodes; reads query at least **r** nodes. If **w + r > n**, the read and write sets overlap, so some node read is up to date: **quorum reads and writes** (w and r are minimum votes; above, n = 3, w = 2, r = 2). This is a **strict quorum**, as opposed to sloppy quorums below.

Tuning (all configurable in Dynamo-style databases):

- Common: **n odd (typically 3 or 5)**, **w = r = (n + 1) / 2 rounded up**.
- Few writes, many reads: **w = n, r = 1** — fast reads, but one failed node makes **all writes fail**.
- **n is per value, not per cluster**: a cluster may have more than n nodes, each value stored on n of them — how data is partitioned across a large cluster (Chapter 6).

Fault tolerance: **w < n** lets writes proceed with a node down; **r < n** likewise for reads. **n = 3, w = 2, r = 2** tolerates **one** unavailable node; **n = 5, w = 3, r = 3** tolerates **two**. Requests go to **all n** replicas in parallel; w and r only set how many replies to **wait for**.

Figure 5-11 (redrawn): with n = 5, w = 3, r = 3, the write is sent to all five replicas but only replicas 1-3 succeed; the read is sent to all five but only replicas 3-5 answer. Replica 3 is in both sets, so the read sees the latest write.



Fewer than w or r successful replies → the operation **returns an error**. Why a node failed (crash, power loss, full disk, network break) doesn't matter.

Limitations of Quorum Consistency

Why w + r > n usually works: the written and read sets share at least one node with the latest value.

- w and r are often a **majority** (> n/2), satisfying w + r > n while tolerating up to n/2 failures; but quorums need only **overlap**, not be majorities, giving flexibility in algorithm design.
- **w + r ≤ n** is allowed: requests still go to n nodes but fewer successes are needed — stale reads **more likely**, but **lower latency and higher availability** (work continues until reachable replicas fall below w or r).

Stale reads even with w + r > n:

1. **Sloppy quorum**: writes and reads may hit different nodes, with no guaranteed overlap.
2. **Concurrent writes**: order unclear; the only safe option is merging. LWW can lose writes to **clock skew**.
3. **Write concurrent with a read**: the write may be on only some replicas; the read may return old or new.
4. **Partial write failure**: succeeded on fewer than w replicas (others e.g. had full disks); successful replicas **don't roll back**, so a "failed" write may or may not appear in later reads.
5. **Restore from a stale replica**: a node with the new value fails and is restored from one with the old value, dropping the count below w.
6. **Unlucky timing** even when all is well ("Linearizability and quorums", Chapter 9).

So quorums are not the simple guarantee they seem. Dynamo-style databases target workloads that **tolerate eventual consistency**; treat w and r as adjusting the *probability* of stale reads. The lag guarantees above (read-your-writes, monotonic reads, consistent prefix) are usually **absent**; stronger guarantees need transactions or consensus (Chapters 7, 9).

Monitoring staleness

Even if staleness is tolerable, you must monitor replication health and alert on large lag.

- **Leader-based:** easy: writes apply in the same order everywhere, so **lag = leader's log position minus follower's**, usually exposed as a metric.
- **Leaderless:** no fixed write order, so harder; with only read repair (no anti-entropy), nothing **limits how old** a rarely read value can be.

Research exists on measuring staleness and predicting stale-read fractions for given n, w, r, but it is not common practice. The book argues staleness should be a standard metric: you need to **quantify**

"eventual".

Sloppy Quorums and Hinted Handoff

Quorums tolerate individual node failures **without failover** and slow nodes (only w or r replies needed), suiting **high availability, low latency, and occasional stale reads**.

But a network break can cut a client off from many nodes — alive, but effectively dead to that client — leaving fewer than w or r reachable. In a large cluster (far more than n nodes) the client can probably reach *some* nodes, just not the value's n "home" nodes. Options: fail requests that can't reach w or r, **or** accept writes on reachable nodes outside the home n.

The latter is a **sloppy quorum**: w and r successes are still required but may include non-home nodes (like sleeping on a neighbour's couch when locked out). When the network recovers, those writes are sent to the right nodes: **hinted handoff** (the neighbour sends you home when you find your keys).

Trade-off: better **write availability** (any w reachable nodes), but w + r > n no longer **guarantees reading the latest value**, which may sit outside the n. Until handoff completes, it is only a **durability assurance** (data stored on w nodes somewhere), not a traditional quorum.

Optional in all common Dynamo implementations: **enabled by default in Riak; disabled by default in Cassandra and Voldemort.**

Multi-datacenter operation

Leaderless replication also suits multiple datacenters, tolerating concurrent conflicting writes, network breaks and latency spikes.

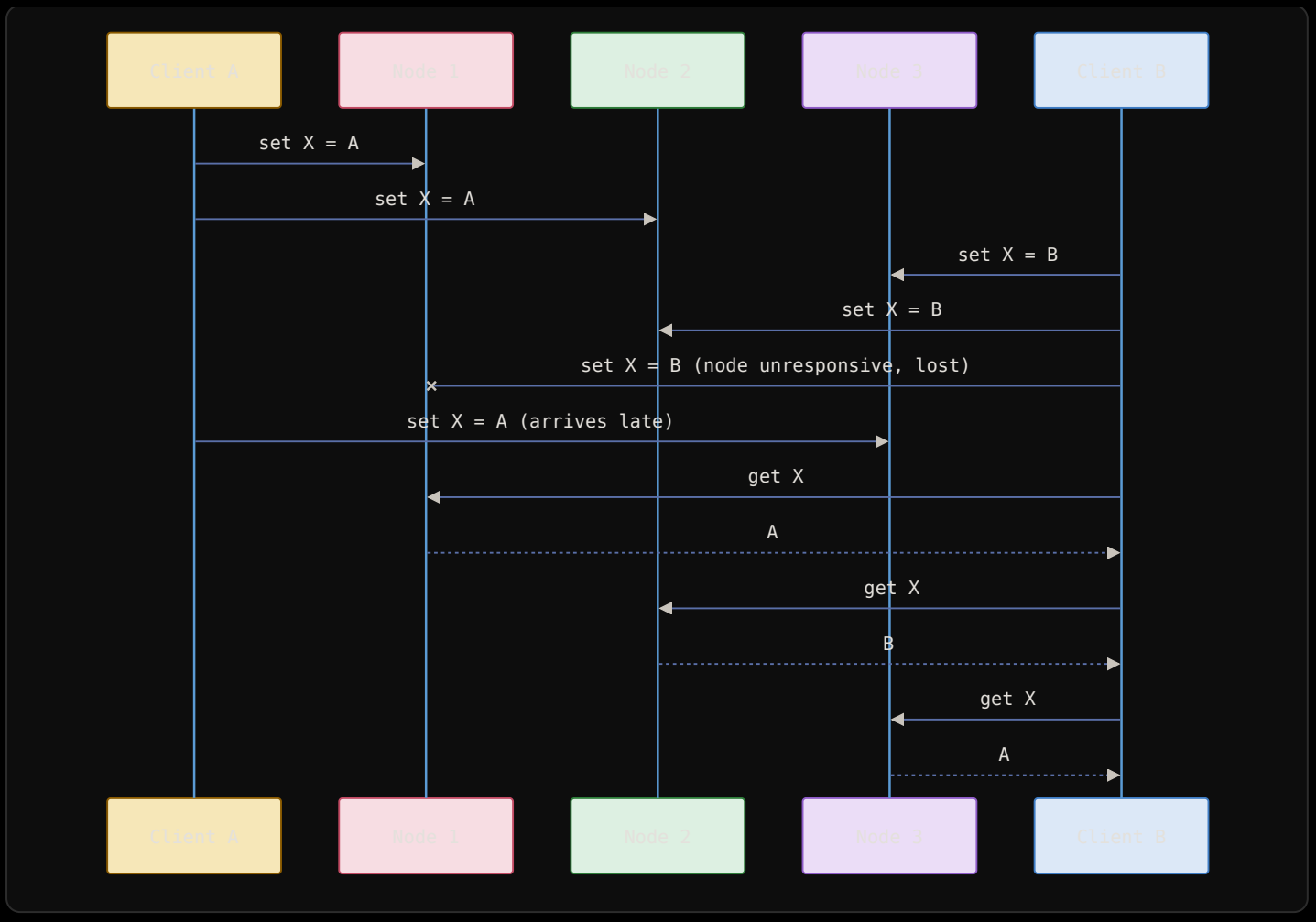
System	How it handles multiple datacenters
Cassandra, Voldemort	Normal leaderless model: n counts nodes in **all** datacenters, configurable per datacenter. Writes go to all replicas, but the client usually waits for a quorum **in its local datacenter**, unaffected by cross-datacenter delays; remote writes are often asynchronous (configurable).
Riak	Client-node traffic stays **within one datacenter**; n counts replicas there. Cross-datacenter replication between clusters is asynchronous in the background, like multi-leader.

Detecting Concurrent Writes

Dynamo-style databases allow concurrent writes to one key, so **conflicts occur even with strict quorums** — similar to multi-leader conflicts, and **read repair** and **hinted handoff** can create them too.

Root cause: variable network delays and partial failures deliver events to nodes in different orders. Clients A and B writing key X on three nodes: node 1 gets A's write but never B's (transient outage); node 2 gets A then B; node 3 gets B then A.

Figure 5-12 (redrawn): clients A and B concurrently set key X; node 1 is unresponsive to B's write, and nodes 2 and 3 receive the two writes in opposite orders. If each node just overwrites on arrival, a later get X returns A, B and A from nodes 1, 2 and 3: they disagree forever.



Replicas must **converge** for eventual consistency. Most implementations handle this poorly, so **developers must understand their database's conflict handling** to avoid data loss.

Last write wins (discarding concurrent writes)

Idea: each replica keeps only the "most recent" value, overwriting "older" ones; if we can always tell which write is newer and every write reaches every replica, copies converge.

Why "recent" is quoted: in Figure 5-12 neither client knew of the other, so neither came "first": the writes are **concurrent** and their order **undefined**. LWW **imposes** an arbitrary order, e.g. via timestamps (keep the largest).

- **Cassandra**'s only resolution method; optional in **Riak**.
- **Cost:** convergence at the expense of **durability**: of several concurrent writes, all reported successful (each reached w replicas), only one survives; the rest are **silently discarded**. LWW can even drop **non-concurrent** writes ("Timestamps for ordering events", Chapter 8).
- Acceptable for e.g. **caching**; avoid where data loss is unacceptable.
- **Only safe use:** write each key **once**, treating it as immutable. Cassandra recommends a **UUID as the key** for every write.

The "happens-before" relationship and concurrency

- **Figure 5-9**: **not concurrent** — B's increment builds on A's insert, so B is **causally dependent** on A.
- **Figure 5-12**: **concurrent** — neither client knew of the other.

Definition: A **happens before** B if B knows about, depends on, or builds on A. A and B are **concurrent** if **neither** happens before the other. So there are three cases: A before B, B before A, or concurrent. If ordered, the later overwrites; if concurrent, there is a conflict to resolve.

> **Concurrency, Time, and Relativity.** "Concurrent" doesn't mean "at the same instant" (exact simultaneity is hard to judge anyway; Chapter 8). It means each was **unaware** of the other,

> regardless of physical time. The analogy is special relativity: events closer in time than light could travel between them cannot affect each other. In computers, operations can be concurrent even when light would have had ample time, e.g. if a slow or broken network kept one from learning of the other.

Capturing the happens-before relationship

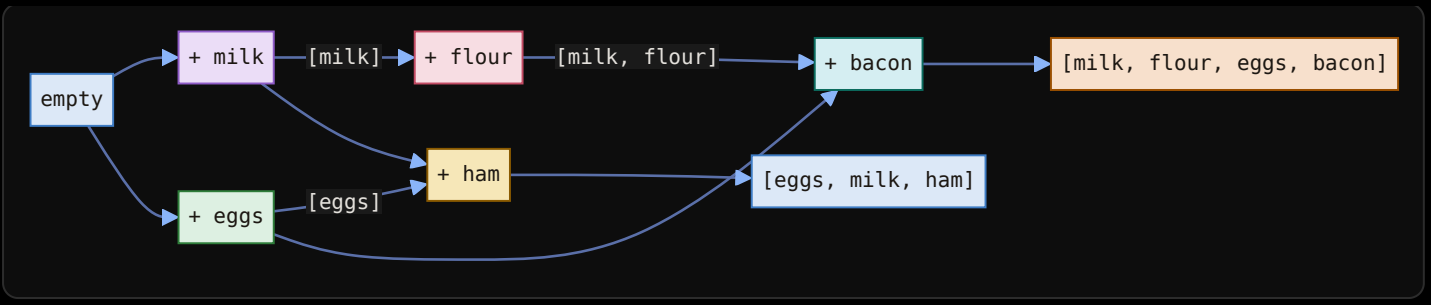
Start with a **single replica**. Two clients add items to one shopping cart (or picture two air-traffic controllers adding aircraft to a sector). Five writes:

1. **Client 1 adds milk.** First write; stored as **version 1**, value and version returned.
2. **Client 2 adds eggs**, unaware of milk. Stored as **version 2**; `[milk]` and `[eggs]` kept as **two separate values**, both returned.
3. **Client 1 adds flour**, unaware of eggs, sending `[milk, flour]` with **version 1**: supersedes `[milk]` but concurrent with `[eggs]`. Stored as **version 3**; `[milk]` dropped, `[eggs]` kept.
4. **Client 2 adds ham**: merges its last-read `[milk]` and `[eggs]`, sends `[eggs, milk, ham]` with **version 2**. Replaces `[eggs]`; `[milk, flour]` (concurrent) stays. Server holds `[milk, flour]` (v3) and `[eggs, milk, ham]` (**v4**).
5. **Client 1 adds bacon**: merges its last-read `[milk, flour]` and `[eggs]` (version 3), sends `[milk, flour, eggs, bacon]` with **version 3**. Overwrites `[milk, flour]`; `[eggs, milk, ham]` is concurrent and stays.

Figure 5-13 (redrawn): two clients editing one cart; the server uses version numbers to decide what to overwrite and what to keep side by side.



Figure 5-14 (redrawn): the causal-dependency graph of those writes. An arrow means the later operation knew about (built on) the earlier one; edge labels show the cart value passed along, and the two arrows leaving on the right are the final siblings.



Clients are never fully current, yet old versions are overwritten and **no write is lost**. The server only compares version numbers, so values can be any data structure.

The algorithm:

1. The server keeps a **version number per key**, incremented on every write and stored with the value.
2. A **read** returns **all values not yet overwritten** plus the latest version. Every write **must be preceded by a read**.
3. A **write** includes the **version from the prior read** and must **merge** all values from that read. (Write responses can look like reads, allowing chained writes as above.)
4. On a write with version v, the server **overwrites all values with version ≤ v** (merged in) and **keeps higher versions** (concurrent).

A write **without a version** is concurrent with everything: it overwrites nothing and appears as another value on later reads.

Merging concurrently written values

No data is silently lost, but clients must **merge** the concurrent values, which **Riak calls siblings**. This is the multi-leader conflict problem again; LWW loses data, so application logic may be needed.

- **Cart:** take the **union**: `[milk, flour, eggs, bacon]` and `[eggs, milk, ham]` → `[milk, flour, eggs, bacon, ham]`, deduplicated (milk and eggs appear in both despite each being added once).
- **Removals break plain union:** an item removed in one sibling reappears (the Amazon cart problem). Fix: on removal leave a marker with a version number — a **tombstone** (as in hash-index log compaction, Chapter 3).

Manual merging is complex and error-prone; **Riak's datatype support** uses **CRDTs**, which merge siblings sensibly, **including preserving deletions**.

Tiny union-with-tombstones illustration (Python, our example); the naive rule lets a removal always beat a concurrent re-add, whereas real CRDTs use version information:

```
a = {"pens": "added", "tape": "added"}
b = {"pens": "removed", "glue": "added"}   # "removed" acts as a tombstone
merged = {}
for k in a.keys() | b.keys():
    states = {a.get(k), b.get(k)} - {None}
    merged[k] = "removed" if "removed" in states else "added"
visible = sorted(k for k, s in merged.items() if s == "added")   # ['glue', 'tape']
```

Version vectors

With **multiple replicas and no leader**, one version per key is not enough: keep a **version number per replica per key**. Each replica increments its own counter on writes it processes and tracks counters seen from the others, which tells it what to overwrite and what to keep as siblings.

The collection is a **version vector**; the most interesting variant is the **dotted version vector** in **Riak 2.0**, working much like the cart example.

- Sent to clients on reads and **returned with the next write** (Riak encodes it as a string, the **causal context**).
- Distinguishes **overwrites from concurrent writes**; the application may still merge siblings.
- Makes it safe to **read from one replica and write to another**: siblings may appear, but nothing is lost if merged correctly.

> **Version vectors vs vector clocks.** Version vectors are sometimes called vector clocks, but they differ subtly; for comparing replica state, use version vectors.

Summary

Replication serves four purposes:

Purpose	Meaning
High availability	Keep running when a machine, several machines, or a whole datacenter fails
Disconnected operation	Keep working during a network interruption
Latency	Put data near users so interaction is faster
Scalability	Serve more reads than one machine could, by reading from replicas

The simple goal is tricky: you must reason about concurrency and failures (unavailable nodes, network interruptions, even silent corruption from bugs).

Approach	How writes flow	Pros	Cons
Single-leader	All writes to one leader, which streams changes to followers; reads anywhere (followers may be stale)	Easy to understand; no conflict resolution	Every write depends on reaching the one leader; failover is fraught
Multi-leader	Each write goes to one of several leaders; leaders stream changes to each other and to followers	More robust to faulty nodes, network problems, latency spikes	Conflicts; hard to reason about; weak consistency
Leaderless	Each write to several nodes; reads from several nodes in parallel to detect and fix stale data	Same robustness benefits; no failover	Very weak consistency guarantees; conflicts

Sync vs async strongly affects fault behaviour: async is fast when healthy, but consider growing lag and failures — promoting an async follower can **lose recently committed data**.

Consistency models for lag: read-after-write (users see their own submissions), **monotonic reads** (no going back to an earlier state), **consistent prefix reads** (causally sensible order).

Concurrency: multi-leader and leaderless allow concurrent writes and hence conflicts; version numbers / version vectors decide whether one operation happened before another or they were concurrent, and conflicts are resolved by merging.

Next: **partitioning**.

Interview angle

1. Single-leader vs multi-leader vs leaderless? 30-second answer: Single-leader routes every write through one node streaming an ordered log to followers — simple, no conflicts, but writes depend on that leader and failover is risky. Multi-leader lets several nodes (one per datacenter, or each offline device) accept writes and replicate asynchronously — better write latency and outage tolerance, but conflicts must be resolved. Leaderless (Dynamo-style: Riak, Cassandra, Voldemort) sends reads and writes to several replicas with quorums — no failover, very available, weak consistency.

- *Follow-up: why not multi-leader in one datacenter?* The complexity (conflicts, trouble with autoincrement keys, triggers, constraints) rarely pays off on a fast, reliable local network.
- *Follow-up: is DynamoDB leaderless?* No — AWS DynamoDB uses single-leader replication; Dynamo was Amazon's internal system.

2. Synchronous vs asynchronous replication? 30-second answer: Sync: the leader waits for a follower's ack, so that follower is current, but if it stalls all writes block. Async: no waiting — fast and

available, but confirmed writes can be lost if the leader dies. In practice semi-synchronous (one sync follower) or fully async.

- *Follow-up: why not all followers sync?* Any node outage would halt writes, likelier with more nodes.

3. What can go wrong in failover? 30-second answer: With async replication the new leader may lack the old leader's last writes, usually discarded (durability violation). Discarded writes can clash with outside systems (GitHub reused MySQL autoincrement keys also used in Redis, leaking private data). Split brain can leave two leaders accepting writes. The timeout trades slow recovery against needless failovers that worsen overload.

- *Follow-up: split brain handling?* Fencing / STONITH shuts one down; badly designed, it can shut down both.

4. Explain quorums (n, w, r). 30-second answer: A value lives on n replicas; writes need w acks, reads ask r replicas. If w + r > n the sets overlap, so a read sees the latest write. Typical: n = 3 or 5, w = r = majority; n = 3, w = r = 2 tolerates one failure; n = 5, w = r = 3 tolerates two. Read repair and anti-entropy fix stale replicas.

- *Follow-up: does w + r > n guarantee fresh reads?* No: sloppy quorums, LWW on concurrent writes (clock skew), writes concurrent with reads, partially failed writes not rolled back, restores from stale replicas, timing edge cases. It is a probability knob.
- *Follow-up: sloppy quorum?* Writes go to reachable non-home nodes when home nodes are unreachable, later moved home by hinted handoff. Better write availability, no overlap guarantee. On by default in Riak, off in Cassandra and Voldemort.

5. Replication-lag anomalies and fixes. 30-second answer: Read-your-writes: read own editable data from the leader, or track last-write timestamp/LSN. Monotonic reads: pin each user to one replica via hash of user ID. Consistent prefix (mostly partitioned stores): keep causally related writes in one partition or track causality.

- *Follow-up: multiple devices?* Centralise last-write metadata; devices may hit different datacenters, so route all of a user's devices to one.

6. How do you detect concurrent writes, and why not just LWW? 30-second answer: Writes are concurrent if neither knew of the other. Servers track a version per key (per replica per key in a version vector); clients return the version they read; the server overwrites values at or below it and keeps higher ones as siblings to merge. LWW converges but silently drops all but one concurrent write — safe only for write-once keys (e.g. UUID keys in Cassandra).

- *Follow-up: deletions when merging?* Tombstones, or CRDTs (Riak) that merge automatically and preserve deletions.

Self-check

1. **Name the four ways a leader's changes can be shipped to followers and one weakness of each.** Statement-based (nondeterminism, ordering, side effects); WAL shipping (coupled to storage format; upgrades may need downtime); logical/row-based (book names no specific drawback; it is the version-tolerant option decoupled from the engine); trigger-based (overhead, bug-prone).
2. **Why is copying the leader's data files not enough to set up a new follower, and what is done instead?** Ongoing writes make a file copy inconsistent. Take a consistent snapshot tied to a log position (LSN / binlog coordinates), copy it, then apply all later changes until caught up.
3. **Scenario: users complain that after posting a comment and reloading, it sometimes disappears. Which guarantee is missing and give two fixes.** Read-after-write. Read the user's own content from the leader (or for a minute after their last write), or have the client remember its last write timestamp/LSN and read only from replicas that have reached it.
4. **With n = 5, what w and r would you choose to tolerate two failed nodes while keeping the quorum condition?** w = 3, r = 3 (3 + 3 > 5).
5. **Trade-off: what do you gain and lose by setting w = n and r = 1?** Very fast reads, but any single node failure makes all writes fail.

6. **Why can timestamps not correctly order the insert and update arriving at leader 2 in Figure 5-9? What does work?** Clocks aren't reliably synchronised, so timestamps may misorder causally related events. Version vectors capture the dependency.
7. **What is the difference between read repair and anti-entropy, and which system lacks anti-entropy?** Read repair fixes stale replicas noticed during reads (good for hot data); anti-entropy is a background process copying missing data in no particular order. Voldemort lacks anti-entropy, so rarely read values can be less durable.
8. **In the version-number algorithm, what does the server do when it receives a write carrying version 3 while it holds values at versions 3 and 4?** Overwrites values with version ≤ 3 and keeps the version 4 value as a concurrent sibling.
9. **Why does conflict avoidance stop working sometimes?** It relies on a fixed home leader per record; a datacenter failure or a user moving changes the leader, allowing concurrent writes on different leaders.
10. **Scenario: an offline-first notes app syncs across phone and laptop. Which replication style is it, and what must you plan for?** Multi-leader (each device a leader). Plan for very long lag (hours or days) and conflict resolution, e.g. CRDTs or on-read resolution as in CouchDB.

Chapter 6 — Partitioning

Book: Chapter 6, pp. 199–220

Overview

Chapter 5 kept full copies of the data on several nodes. That is not enough when the dataset is **too big for one machine** or the **query rate too high**: the data must be split into **partitions** (**sharding**) spread over many nodes. (Primer: System Design Fundamentals.)

This chapter answers: *which record lives on which node, how do secondary indexes keep working, how is data moved when nodes join or leave, and how does a request find the right node?*

> **Terminology zoo.** A partition is a **shard** in MongoDB, Elasticsearch and SolrCloud, a **region** in HBase, a **tablet** in Bigtable, a **vnode** in Cassandra and Riak, a **vBucket** in Couchbase. The book uses **partition**.

> **Not a network partition.** Here partitioning means deliberately splitting a database; it is unrelated to *network partitions* (netsplits, a network fault; Chapter 8).

Key ideas:

- Usually each record belongs to **exactly one** partition, which acts like a small database of its own (some operations span partitions).
- The goal is **scalability**: in a shared-nothing cluster, partitions on different nodes spread data over many disks and queries over many processors.
- Single-partition queries are answered by one node, so adding nodes raises throughput; large complex queries can in principle be parallelised across nodes, but that is much harder.
- History: **Teradata** and **Tandem NonStop SQL** pioneered partitioned databases in the 1980s; NoSQL stores and Hadoop-based data warehouses rediscovered the idea. Transactional vs analytic workloads change tuning, not the fundamentals.

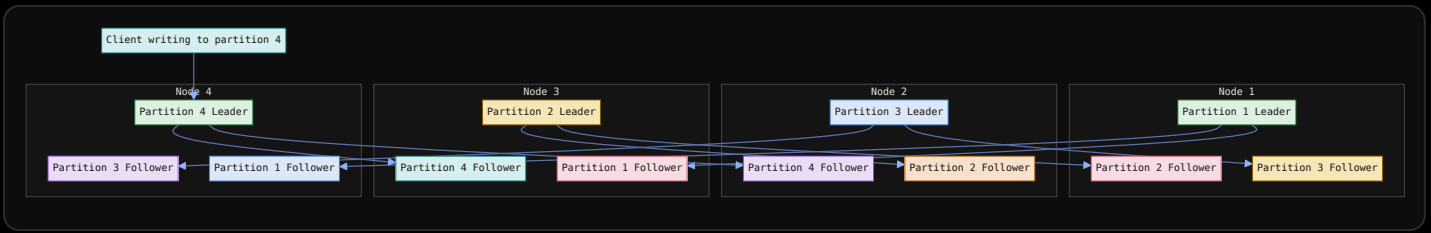
Key terms

Term	Plain-English meaning
Partition (shard, region, tablet, vnode, vBucket) / sharding	A subset of the data; each record belongs to exactly one / partitioning.
Skew / hot spot	Some partitions get more data or queries than others / a partition with disproportionately high load.
Key-range partitioning	Each partition owns a continuous key range, like encyclopedia volumes.
Partition boundaries	Key (or hash) values where one partition ends and the next begins.
Range scan / range query	Fetch all keys between a lower and upper bound.
Concatenated index	Multi-part key used as one sortable index, fetching related records in one query.
Hash partitioning	Each partition owns a range of **hash values**.
Consistent hashing	Karger et al.'s random-boundary method; the book prefers "hash partitioning" for databases.
Compound primary key	Cassandra key whose first part is hashed for the partition, rest used for sorting within it.
Secondary index	Finds records by a non-key value (e.g. all red cars).
Document-partitioned (local) / term-partitioned (global) index	Each partition indexes only its documents / one index over all partitions, itself partitioned by term.
Term	In full-text search a word; here a value like `color:red`.
Scatter/gather	Query all partitions and combine results.
Tail latency amplification	Waiting on many sub-requests, the slowest dictates total time.
Rebalancing	Moving load (data and requests) between nodes.
Hash mod N	Node = `hash(key) % N`; a bad rebalancing approach.
Fixed number of partitions / dynamic partitioning	Many more partitions than nodes, moved whole / split big partitions, merge small ones.
Pre-splitting	Configuring initial partitions on an empty database.
Partitioning proportionally to nodes	Fixed number of partitions per node (Cassandra, Ketama).
Cascading failure	Overload spreading from node to node, e.g. via automatic rebalancing.
Request routing / service discovery	Getting a request to the node owning its partition / the general problem of finding which address serves something.
Routing tier	Partition-aware load balancer that forwards but handles no requests.
ZooKeeper	Coordination service storing cluster metadata such as the partition-to-node map.
Gossip protocol	Nodes spreading cluster-state changes to each other directly.
MPP (massively parallel processing)	Analytic relational databases running complex query stages in parallel across nodes.

Partitioning and Replication

Partitioning is usually **combined with replication**: each record is in one partition, but that partition is stored on several nodes for fault tolerance. A node can hold many partitions. With leader–follower replication, each partition has its **leader on one node and followers on others**, so a node can be leader for some partitions and follower for others.

Figure 6-1 (redrawn): four nodes, four partitions, each partition with one leader and two followers. Every node leads one partition and follows two others; a client writing to partition 4 goes to its leader on node 4. Arrows are per-partition replication streams.



Chapter 5 applies equally to replicating partitions; since the partitioning scheme is mostly independent of replication, the rest of the chapter ignores replication.

Partitioning of Key-Value Data

Goal: spread data and query load **evenly**: 10 nodes could then handle 10x the data and throughput of one (ignoring replication).

Unfair partitioning is **skewed** and much less effective; in the extreme all load hits one partition, **9 of 10 nodes sit idle**, and the busy one is the bottleneck. A partition with disproportionate load is a **hot spot**.

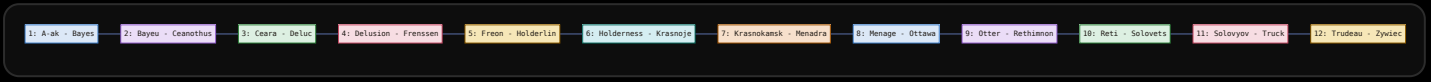
Why not random assignment? Data would be even, but you wouldn't know where an item lives and would have to **query every node in parallel**.

Better: assume a **key-value** model where records are accessed by primary key, like entries in a paper **encyclopedia** sorted by title.

Partitioning by Key Range

What it is. Each partition gets a **continuous key range** (min to max), like printed encyclopedia volumes. Knowing the boundaries tells you the partition; knowing the partition's node lets you go straight there.

Figure 6-2 (redrawn): a 12-volume print encyclopedia partitioned by key range; each volume covers an uneven span of titles.



Ranges need not be equal: data isn't evenly spread over the alphabet (volume 1 holds A and B; volume 12 holds T, U, V, X, Y, Z), so boundaries must **adapt to the data**, set by an administrator or automatically (see Rebalancing).

Used by: Bigtable, HBase (its open-source counterpart), RethinkDB, MongoDB before 2.4.

Benefit: sorted keys and range scans. Keys within a partition can be kept sorted (SSTables/LSM-trees, Chapter 3), so range scans are easy and the key can serve as a **concatenated index**. E.g. a **sensor network** keyed by timestamp (year-month-day-hour-minute-second): one month's readings are one range scan.

Downside: hot spots from access patterns. With timestamp keys, partitions correspond to time ranges (e.g. one per day), and since sensors write as they measure, **every write goes to today's partition** while others idle.

Fix: put something else first, e.g. **the sensor name before the timestamp**, partitioning by sensor then time; many active sensors spread writes. Cost: reading several sensors over a time range needs **one range query per sensor**. Illustrative layout (our example):

```
key = <device_id> + ":" + <timestamp>
therm-07:2024-03-01T10:00:00   -> partition holding therm-00 .. therm-09
therm-42:2024-03-01T10:00:00   -> partition holding therm-40 .. therm-49
```

Partitioning by Hash of Key

To avoid skew and hot spots, many distributed datastores choose the partition with a **hash function** of the key.

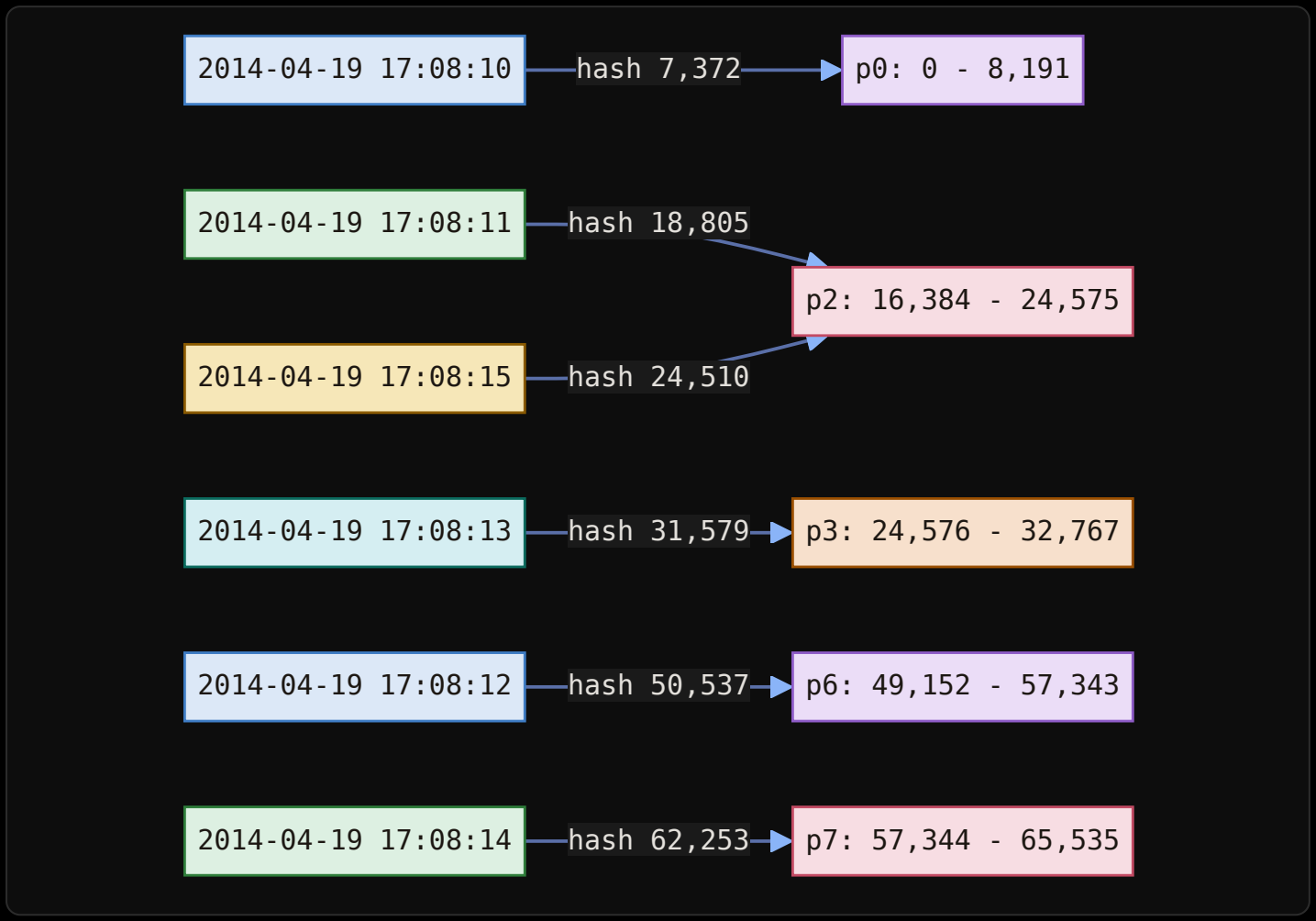
> **Key term — hash function:** turns any input into a fixed-size, random-looking number; the same input always gives the same number, but near-identical inputs give very different numbers, so similar keys no longer cluster.

A good hash makes skewed input uniform: a 32-bit hash maps any string to a seemingly random number in 0 to 2^32 − 1, evenly spread even for near-identical strings.

Choosing the hash function. It need not be cryptographically strong: **Cassandra and MongoDB use MD5, Voldemort uses Fowler–Noll–Vo**. Built-in language hashes (for hash tables) may be **unsuitable**: Java's `Object.hashCode()` and Ruby's `Object#hash` can give the same key different hashes in different processes.

How it works: give each partition a **range of hashes** and store each key in the partition whose range contains `hash(key)`.

Figure 6-3 (redrawn): six consecutive timestamp keys, one second apart, hash (first 2 bytes of MD5, so 0..65,535) to scattered values; the hash space is cut into eight equal ranges p0..p7, and each key lands in the partition whose range contains its hash.



Boundaries can be **evenly spaced** or **chosen pseudorandomly** — the latter sometimes called **consistent hashing**.

> **Consistent Hashing.** Karger et al. defined it for balancing load across internet-wide caches such as a **CDN** (content delivery network), using random boundaries so no central control or distributed consensus (Chapter 9) is needed. "Consistent" here is unrelated to replica consistency (Chapter 5) or ACID consistency (Chapter 7); it describes a rebalancing approach. It works poorly for databases and is rarely used in practice (docs that say "consistent hashing" are often inaccurate), so the book says **hash partitioning**.

The cost: no efficient range queries — adjacent keys are scattered. **MongoDB** with hash sharding sends range queries to **all** partitions; **Riak, Couchbase and Voldemort** don't support range queries on the primary key.

Cassandra's compromise: compound primary keys. Only the first column is hashed to pick the partition; the **remaining columns form a concatenated index** sorting data within SSTables. You can't range-scan the first column, but with it fixed you can range-scan the others efficiently.

This suits **one-to-many** data: with key `(user_id, update_timestamp)` you can fetch one user's updates in a time window, sorted; different users are on different partitions, but each user's updates are together in time order. Our CQL (Cassandra Query Language) illustration — `driver_id` is hashed, `event_time` sorts within the partition, so the query touches one partition and range-scans it (a time range without `driver_id` could not use this layout):

```
CREATE TABLE ride_events (
  driver_id  uuid,
  event_time timestamp,
  status     text,
  PRIMARY KEY ((driver_id), event_time)
);

SELECT status FROM ride_events
WHERE driver_id = 5b1c0f2e-7a3d-4c1e-9f00-1a2b3c4d5e6f
  AND event_time >= '2024-05-01' AND event_time < '2024-05-02';
```

Skewed Workloads and Relieving Hot Spots

Hashing reduces hot spots but **cannot eliminate them**: if all requests target the **same key**, they all hit one partition. E.g. a **celebrity** with millions of followers triggers a storm of writes to one key (their user ID, or the ID of the action being commented on); identical IDs hash identically. Most data systems **cannot compensate automatically**, so the application must.

Technique: split the hot key. For a known-hot key, **add a random number to the start or end**; a **two-digit decimal** (00–99) spreads writes over **100 different keys** on different partitions.

Costs: reads must fetch all 100 keys and combine; **bookkeeping** is needed to track which few keys are split (pointless overhead for low-traffic keys). Future systems may handle skew automatically; for now, weigh the trade-offs per application.

Sketch (Python, our example) — `HOT` is the bookkeeping, `read_keys` shows the read penalty:

```
import random
HOT = {"post:9001"}                    # keys known to be hot

def write_key(key):
    if key in HOT:
        return f"{key}#{random.randint(0, 99):02d}"   # one of 100 sub-keys
    return key

def read_keys(key):
    return [f"{key}#{i:02d}" for i in range(100)] if key in HOT else [key]
```

Partitioning and Secondary Indexes

Primary-key access identifies the partition directly. **Secondary indexes** usually don't identify a record uniquely; they find all records with a value: all actions by user 123, all articles containing *hogwash*, all red cars.

- The **bread and butter** of relational databases; common in document databases.
- Many key-value stores (**HBase, Voldemort**) avoid them due to complexity; others (**Riak**) are adding them for their modelling value.
- The **whole reason for existing** of search servers like **Solr and Elasticsearch**.

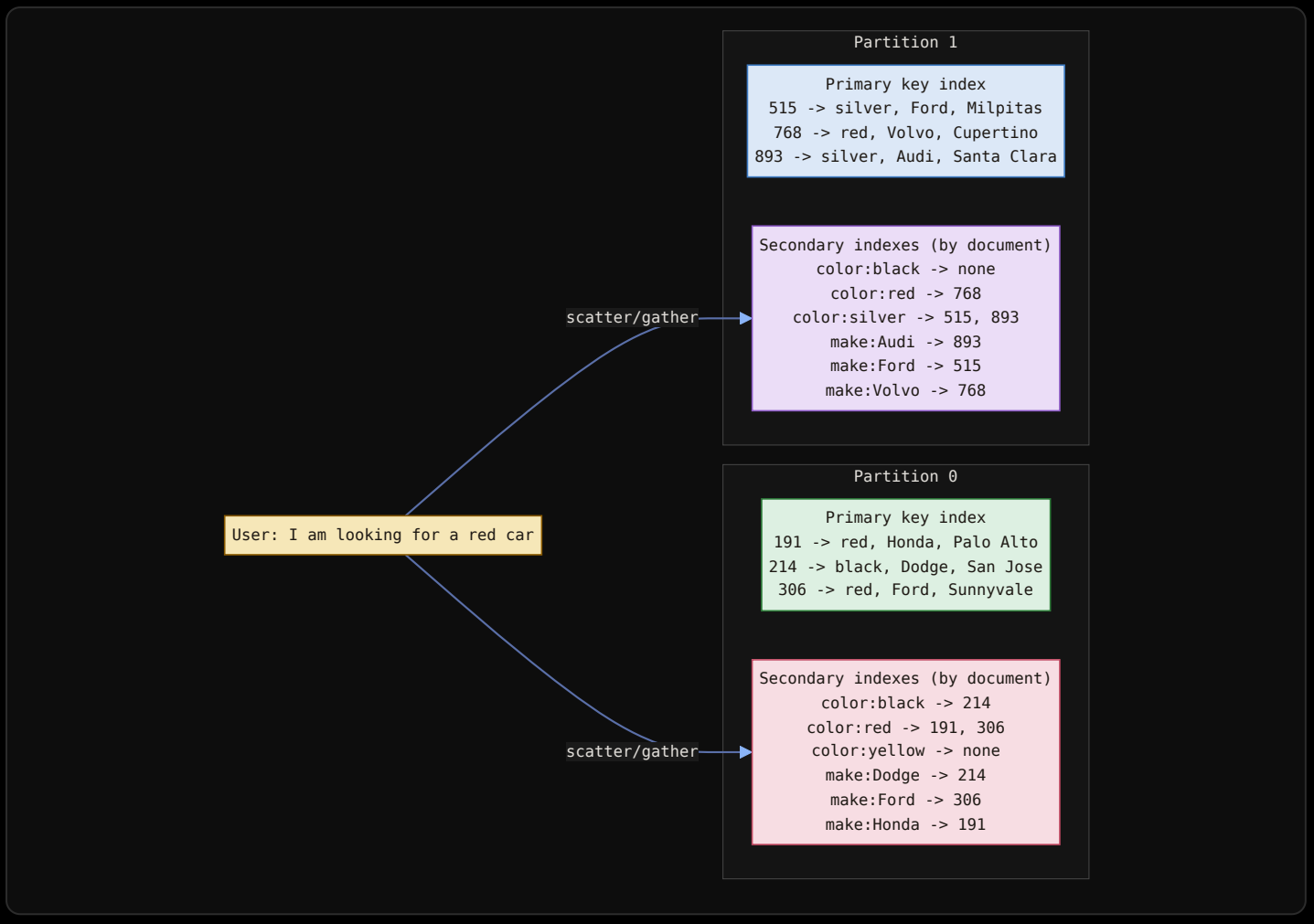
They **don't map neatly onto partitions**. Two approaches: **document-based** and **term-based** partitioning.

Partitioning Secondary Indexes by Document

Book example: a used-car website. Each listing has a **document ID**; the database is partitioned by ID (0–499 in partition 0, 500–999 in partition 1, …). Users filter by **color** and **make**, so secondary indexes exist on those fields; adding a red car makes its partition add its ID to `color:red`.

> **Rolling your own index.** In a pure key-value store you might map values to document IDs in application code, but race conditions and partial write failures easily leave the index out of sync (see multi-object transactions, Chapter 7).

Figure 6-4 (redrawn): each partition keeps a primary-key index and its own secondary indexes covering only its own cars, so a search for red cars must scatter to both partitions and gather the results.



How it works. Each partition is independent and indexes **only its own documents**; a write touches **only the partition holding that document**: a **local index**.

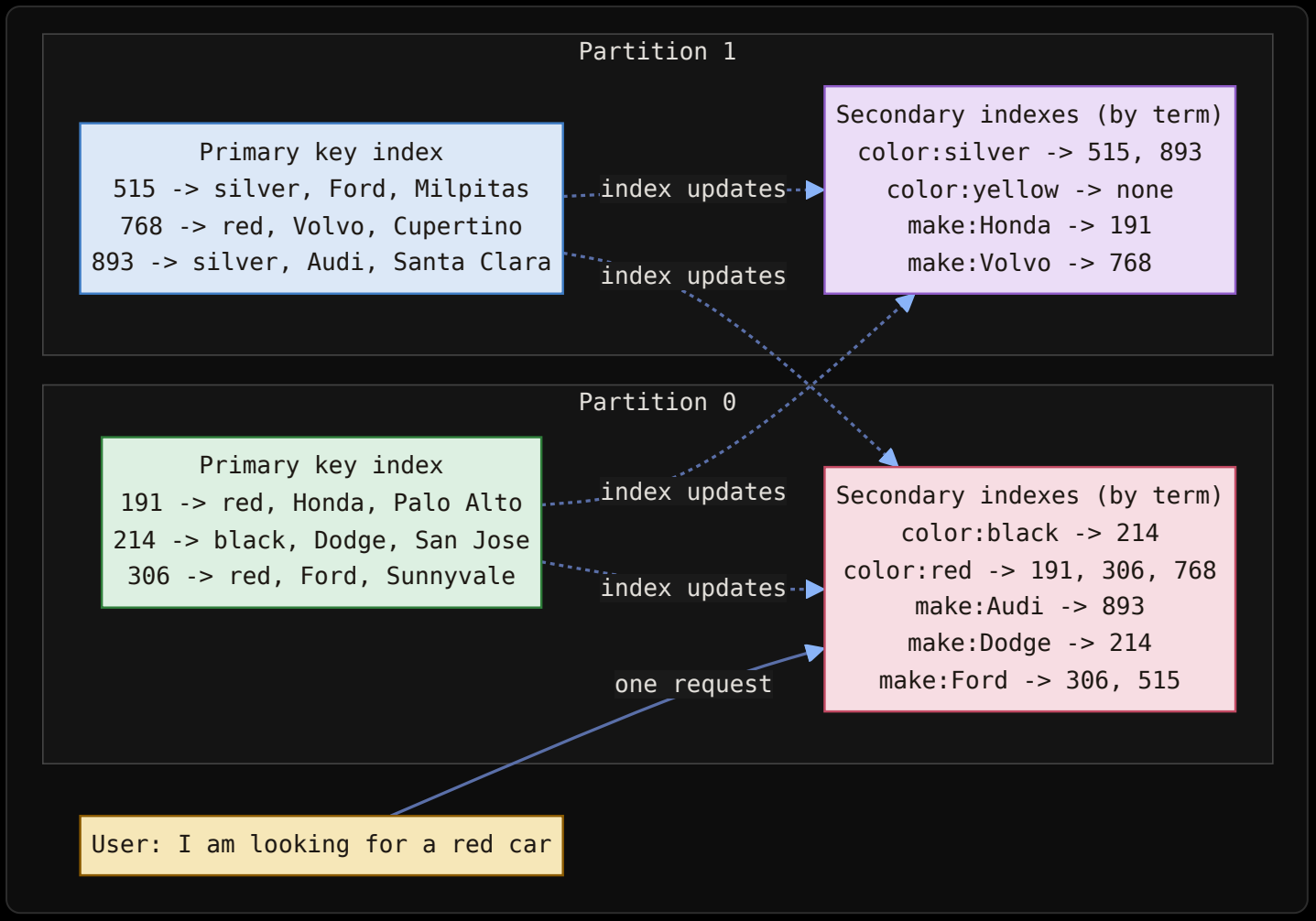
Reads: cars of one color are spread across partitions, so a query must go to **all partitions** and combine results: **scatter/gather**.

- Makes secondary-index reads **expensive**; even in parallel, **tail latency amplification** applies.
- **Widely used** anyway: MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB.
- Vendors advise partitioning so index queries hit **one partition**, often impossible with **several secondary indexes in one query** (color **and** make).

Partitioning Secondary Indexes by Term

Idea. A **global index** covering all partitions, which itself must be **partitioned** (one node would be a bottleneck), differently from the primary key.

Figure 6-5 (redrawn): the same cars, but the secondary indexes are global and partitioned by term: color:black, color:red and makes Audi to Ford live on partition 0; color:silver, color:yellow and makes Honda to Volvo on partition 1. Writing one car may update index entries on both partitions (dashed), while a search for red cars reads one index partition.



In the book's figure, colors **a to r** are in index partition 0 and **s to z** in partition 1; makes split between **f and h**. The searched **term** (e.g. `color:red`) decides the index partition (term comes from **full-text indexes**, where terms are words).

By the term itself helps **range scans** (e.g. asking price); **by hash of the term** spreads load **more evenly**.

	Document-partitioned (local)	Term-partitioned (global)
Where the index lives	With each data partition, covering only its documents	Separately, partitioned by indexed value
Writes	Touch one partition: fast and simple	One document may touch **several index partitions**, maybe on several nodes: slower, more complex
Reads	**Scatter/gather** to all partitions	Only the partition(s) holding the term
Freshness	Updated with the document	Often **asynchronous**; may lag
Examples	MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB	DynamoDB global secondary indexes, Riak search, Oracle data warehouse (choice of local or global)

Keeping a global index up to date. Immediate consistency would need a **distributed transaction** across all affected index partitions, which not every database supports (Chapters 7, 9). In practice updates are often **asynchronous**: a read right after a write may miss it. **Amazon DynamoDB** global secondary indexes normally update within **a fraction of a second**, longer during infrastructure faults. Also used by **Riak's search feature** and the **Oracle data warehouse** (local or global). Chapter 12 revisits implementation.

Rebalancing Partitions

Over time query throughput rises (**more CPUs**), data grows (**more disks and RAM**), and machines **fail** (others take over). Moving load between nodes is **rebalancing**. Minimum requirements:

1. Afterwards, load (storage, reads, writes) is **shared fairly**.
2. During it, the database **keeps accepting reads and writes**.
3. **No more data than necessary** moves, keeping it fast and limiting network and disk I/O.

Strategies for Rebalancing

How not to do it: hash mod N

Figure 6-3 assigned **ranges** of hashes to partitions (partition 0 if 0 ≤ hash(key) < b0, partition 1 if b0 ≤ hash(key) < b1, …). Why not **mod**? `hash(key) mod 10` gives 0–9, seemingly a neat way to pick among 10 nodes.

The problem: when **N** changes, **most keys move**. Book example, hash(key) = 123456:

Nodes	123456 mod N	Key lives on
10	6	node 6
11	3	node 3
12	0	node 0

That makes rebalancing far too expensive. Quick demonstration (Python, our numbers; Python randomises string hashes per process — the same pitfall as Java and Ruby — but the proportion moved is what matters):

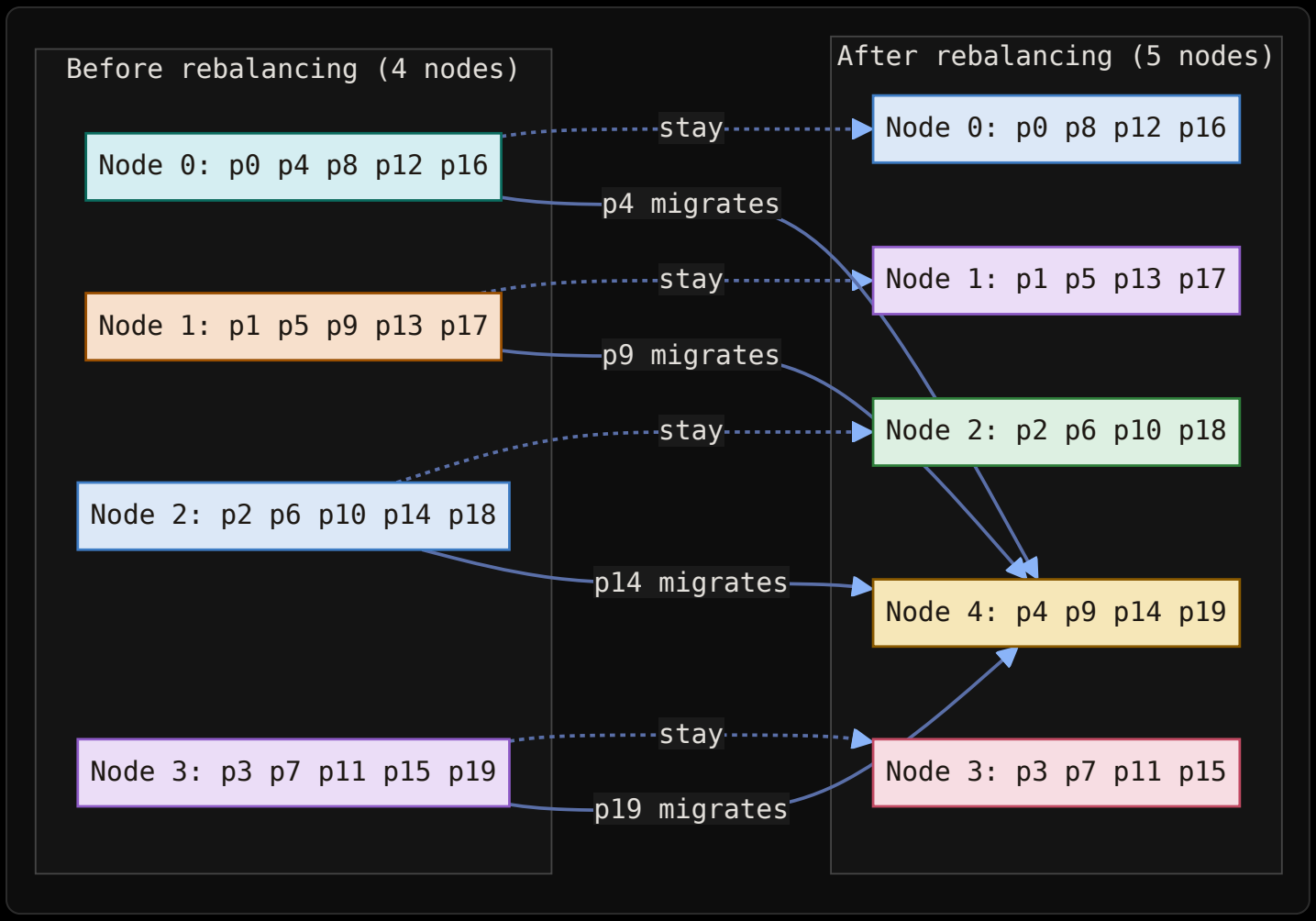
```
keys = range(100_000)
moved = sum(1 for k in keys if hash(str(k)) % 4 != hash(str(k)) % 5)
print(moved / len(keys))   # roughly 0.8: about 80% of keys change node going from 4 to 5 nodes
```

Fixed number of partitions

Idea: create **many more partitions than nodes** up front, several per node — e.g. a **10-node** cluster with **1,000 partitions**, **~100 per node**. A new node **takes a few partitions from each existing node** until even; removal reverses this.

Figure 6-6 (redrawn): 20 fixed partitions on a 4-node cluster; when node 4 joins, it takes one partition from each existing node (p4, p9, p14, p19). All other partitions stay where they were, and no

key changes partition.



- **Only whole partitions move**; the partition count and key-to-partition mapping never change, only the **partition-to-node** assignment.
- Transfer takes time, so the **old assignment serves** reads and writes meanwhile.
- **Uneven hardware**: give stronger nodes more partitions.
- **Used by:** Riak, Elasticsearch, Couchbase, Voldemort.

Choosing the number: usually **fixed at setup** (splitting/merging is possible but often not implemented, for simplicity), so it caps the **maximum number of nodes** — choose high enough for growth. But each partition has **management overhead**. With highly variable total data it is hard to size, since partition size grows with the dataset: **very large partitions** make rebalancing and recovery expensive, **very small** ones add overhead.

Dynamic partitioning

Fixed boundaries are awkward for **key-range** partitioning (bad boundaries could put all data in one partition; manual reconfiguration is tedious). So **HBase and RethinkDB** create partitions **dynamically**:

1. A partition exceeding a configured size (**10 GB by default in HBase**) **splits into two** halves.
2. One shrinking below a threshold (after heavy deletes) can **merge** with its neighbour.
3. Similar to the top level of a **B-tree** splitting full pages.
4. As with fixed partitioning, a node holds many partitions and a split half can **move to another node**; in HBase, partition files move via **HDFS** (Hadoop Distributed File System).

Advantage: partition count **adapts to data volume**: few partitions (low overhead) when small, size capped when large.

Caveat: an empty database starts with **one partition** (no information about boundaries), so **one node takes all writes** until the first split. **HBase and MongoDB** allow **pre-splitting** an empty database, which for key ranges requires knowing the key distribution in advance.

Dynamic partitioning also works for hashed data: **MongoDB since 2.4** supports key-range and hash partitioning, splitting dynamically in either.

Partitioning proportionally to nodes

- **Dynamic partitioning:** partition count proportional to **dataset size**.
- **Fixed number of partitions:** partition size proportional to **dataset size**.
- In both, partition count is **independent of node count**.

Third option (Cassandra, Ketama): a **fixed number of partitions per node**.

- Partition size grows with data at fixed node count and shrinks as nodes are added; since more data usually means more nodes, size stays **fairly stable**.
- A **new node** picks a fixed number of existing partitions **at random**, **splits** each, and takes **one half**.
- Random splits can be **unfair**, but averaged over many partitions (**256 per node by default in Cassandra**) the share is fair; **Cassandra 3.0** added an algorithm avoiding unfair splits.
- Random boundaries require **hash-based partitioning**; this is closest to the **original definition of consistent hashing**. Newer hash functions achieve similar effects with less metadata overhead.

Strategy	Number of partitions	Partition size	Who uses it	Watch out for
hash mod N	= nodes	varies	(avoid)	Most keys move when N changes
Fixed number of partitions	Fixed at setup	Grows with data	Riak, Elasticsearch, Couchbase, Voldemort	Choosing the count; caps node count
Dynamic partitioning	Grows with data	Bounded (e.g. HBase 10 GB split)	HBase, RethinkDB, MongoDB	Starts with one partition; pre-split
Proportional to nodes	Fixed per node	Stable-ish	Cassandra (256/node), Ketama	Random splits can be unfair

Operations: Automatic or Manual Rebalancing

From **fully automatic** (system moves partitions itself) to **fully manual** (admin configures the assignment). In between, **Couchbase, Riak, and Voldemort** generate a **suggested** assignment that an administrator must **commit**.

	Fully automatic	Human in the loop
Convenience	Less routine operational work	Slower
Predictability	Can be unpredictable	Helps avoid operational surprises

Why automation can hurt. Rebalancing is **expensive** (rerouting, moving lots of data) and can overload the network or nodes. Combined with **automatic failure detection**: an overloaded, temporarily slow node is declared dead; load is rebalanced away from it, adding load to it, the other nodes and the network — possibly a **cascading failure**. A **human in the loop** is slower but safer.

Request Routing

How does a client know **which node to contact** for key "foo" (which IP address and port), when assignments change with rebalancing? This is **service discovery**, common to any networked software, especially highly available redundant systems; many in-house and open-source tools exist.

Three approaches:

1. **Contact any node** (e.g. via a **round-robin load balancer**); if it owns the partition it handles the request, else it **forwards** to the right node and relays the reply.
2. **Routing tier**: requests first go to a **partition-aware load balancer** that forwards them and handles none itself.

3. **Partition-aware client**: connects **directly** to the right node.

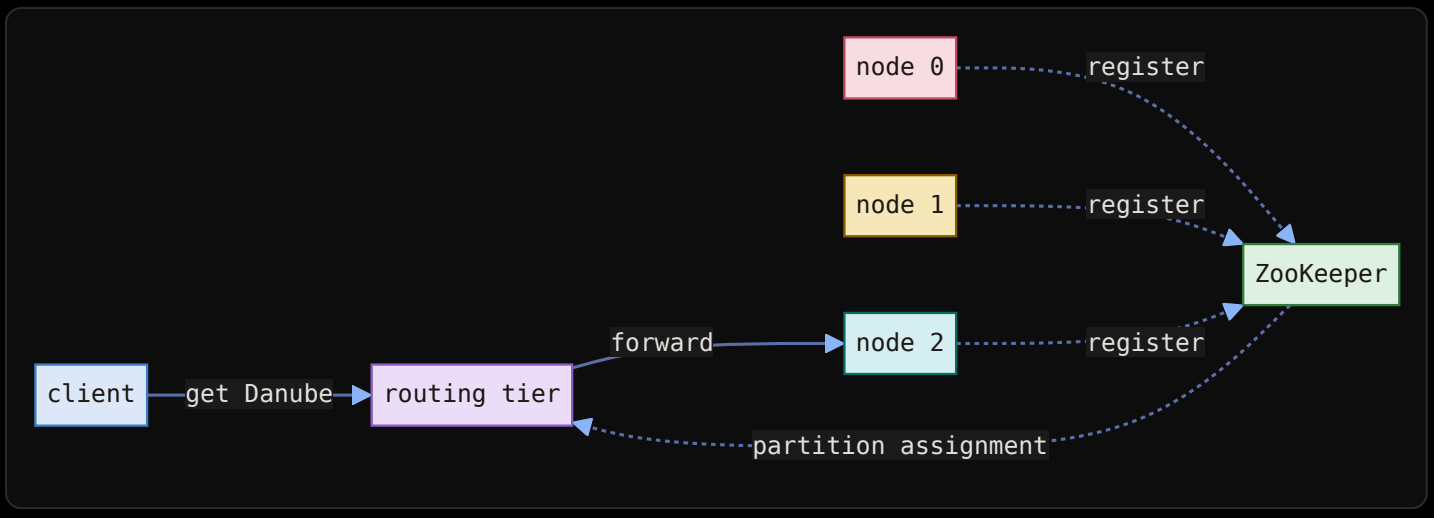
Figure 6-7 (redrawn): three ways to route a request for key "foo", which lives on node 2. The component that knows the partition-to-node assignment is, in turn, every node (1), a routing tier (2), or the client itself (3).



The core difficulty: how does the routing component **learn** of partition moves? All participants must agree or requests go astray; consensus protocols exist but are hard to implement (Chapter 9).

Using a coordination service (ZooKeeper). Each node **registers** in ZooKeeper, which keeps the **authoritative partition-to-node mapping**; routing tiers or partition-aware clients **subscribe**, and ZooKeeper **notifies** them when a partition changes owner or a node joins or leaves.

Figure 6-8 (redrawn): the nodes register with ZooKeeper, which holds the authoritative table of key range -> partition -> node -> IP address; the routing tier keeps its copy in sync with ZooKeeper and forwards a client's get "Danube" to node 2.



Key range	Partition	Node	IP address
A-ak - Bayes	partition 0	node 0	10.20.30.100
Bayeu - Ceanothus	partition 1	node 1	10.20.30.101
Ceara - Deluc	partition 2	node 2	10.20.30.102
Delusion - Frenssen	partition 3	node 0	10.20.30.100
Freon - Holderlin	partition 4	node 1	10.20.30.101
Holderness - Krasnoje	partition 5	node 2	10.20.30.102
Krasnokamsk - Menadra	partition 6	node 0	10.20.30.100
Menage - Ottawa	partition 7	node 1	10.20.30.101
Otter - Rethimnon	partition 8	node 2	10.20.30.102
Reti - Solovets	partition 9	node 0	10.20.30.100
Solovyov - Truck	partition 10	node 1	10.20.30.101
Trudeau - Zywiec	partition 11	node 2	10.20.30.102

System	Routing approach
LinkedIn Espresso	**Helix** for cluster management (on ZooKeeper), with a routing tier as in Figure 6-8
HBase, SolrCloud, Kafka	ZooKeeper tracks partition assignment
MongoDB	Similar, but its own **config server** implementation and **mongos** daemons as the routing tier
Cassandra, Riak	**Gossip protocol** spreads cluster-state changes; any node accepts and forwards requests (approach 1). More node complexity, but no external dependency like ZooKeeper
Couchbase	No automatic rebalancing (simpler); usually a routing tier called **moxi** that learns routing changes from cluster nodes

Finding IP addresses. Clients still need node IPs, but these change far less often than assignments, so **DNS** usually suffices.

Parallel Query Execution

So far: simple single-key queries (plus scatter/gather for local indexes), roughly what most **NoSQL** datastores support. **Massively parallel processing (MPP)** relational databases, used for **analytics**, support complex warehouse queries with **joins, filters, groupings and aggregations**: the MPP **query optimizer** splits them into **execution stages and partitions** run **in parallel on different nodes**, benefiting queries that scan large parts of the data. A specialised, commercially important area; Chapter 10 discusses some techniques.

Summary

- **Why partition:** too much data to store and process on one machine. **Goal:** spread data and load evenly, avoiding **hot spots**, with a suitable scheme and **rebalancing** as nodes change.

	Key range partitioning	Hash partitioning
How	Keys sorted; a partition owns all keys from some minimum to some maximum	Hash each key; a partition owns a range of hashes
Range queries	Efficient	Inefficient (key order destroyed)
Hot-spot risk	High if the app often accesses keys close together in sort order	Lower; load more evenly spread
Typical rebalancing	Dynamic: split a range in two when a partition grows too big	Fixed number of partitions created in advance, several per node, whole partitions moved; dynamic also possible

- **Hybrid:** a **compound key**, one part for the partition, another for sort order (Cassandra).
- **Secondary indexes: document-partitioned (local)** — stored with the data; writes update one partition, reads **scatter/gather**; **term-partitioned (global)** — partitioned by indexed value; writes may update **several** index partitions, reads hit **one**.
- **Routing:** from partition-aware load balancing to elaborate **parallel query execution** engines.
- Partitions operate mostly independently, which enables scaling; but writes spanning **several partitions** are hard to reason about (what if one succeeds and another fails?), addressed in later chapters.

Interview angle

1. Key-range vs hash partitioning? 30-second answer: Key-range assigns contiguous sorted ranges (Bigtable, HBase, RethinkDB, MongoDB before 2.4): cheap range scans, but sequential keys like timestamps create hot spots. Hash partitioning assigns hash ranges (MD5 in Cassandra and MongoDB,

Fowler–Noll–Vo in Voldemort): even load, no efficient range queries. Cassandra's compound key hashes the first column and sorts by the rest.

- *Follow-up: timestamp hot spot for sensor data?* Put the sensor name before the timestamp; reading many sensors then needs one range query each.
- *Follow-up: why not Java's `hashCode()`?* The same key can hash differently in different processes.

2. Why is hash mod N bad for rebalancing, and what do real systems do? 30-second answer: Changing N moves most keys (123456 → node 6, 3, 0 for N = 10, 11, 12). Instead: a fixed large partition count, moving whole partitions (Riak, Elasticsearch, Couchbase, Voldemort); dynamic split/merge (HBase at 10 GB default, RethinkDB, MongoDB); or fixed partitions per node (Cassandra 256/node, Ketama).

- *Follow-up: fixed count downside?* Caps node count; hard to size with variable data (too big = costly rebalancing/recovery, too small = overhead).
- *Follow-up: dynamic downside?* Starts with one partition, so one node takes all writes until the first split; pre-split (HBase, MongoDB) if you know the key distribution.

3. Local vs global secondary indexes? 30-second answer: Local (document-partitioned) indexes live with each partition: writes touch one partition, reads scatter/gather with tail-latency amplification (MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB). Global (term-partitioned) indexes are partitioned by term: reads hit one partition, writes touch several and are usually asynchronous (DynamoDB GSIs: a fraction of a second normally, longer during faults).

- *Follow-up: why not synchronous?* It would need a distributed transaction across affected index partitions, which many databases lack.

4. How do you handle a celebrity hot key? 30-second answer: Hashing can't help (same key, same hash). Append a random two-digit number to spread writes over 100 keys; reads fetch and combine all 100, and you must track which keys are split. Only for the few hot keys.

5. How does a request find the right partition? 30-second answer: Any node forwards (Cassandra, Riak with gossip), a routing tier (Espresso via Helix/ZooKeeper, MongoDB mongos with config servers, Couchbase moxi), or partition-aware clients. The hard part is agreement on the partition map; many systems keep it in ZooKeeper, which notifies routers. DNS suffices for node IPs.

- *Follow-up: automatic rebalancing?* Combined with automatic failure detection it can turn a slow node into a cascading failure; human-committed suggested plans (Couchbase, Riak, Voldemort) are slower but safer.

Self-check

1. **What is skew, and what is a hot spot?** Skew: some partitions get more data or queries than others. Hot spot: a partition with disproportionately high load; in the extreme 9 of 10 nodes idle while one is the bottleneck.
2. **Why not just assign records to random nodes?** Even load, but you wouldn't know where records live and would query all nodes for every read.
3. **Scenario: an IoT (Internet of Things: networked sensors and devices) system keys readings by timestamp in a key-range-partitioned store and one node is overloaded. Why, and what change fixes it?** All current writes fall in the current time range's partition. Prefix keys with the sensor name; reading many sensors then needs one range query per sensor.
4. **In Cassandra with primary key (user_id, update_timestamp), which queries are efficient?** Fixing user_id and scanning a timestamp range (one partition, sorted). A range over user_id is not efficient because it is hashed.
5. **Trade-off: document-partitioned vs term-partitioned index for a read-heavy search page with many filters?** Term-partitioned makes each term lookup a single-partition read instead of scatter/gather; costs are slower multi-partition writes and often asynchronous (briefly stale) index updates.
6. **With hash(key) = 123456, where does the key go with 10, 11, and 12 nodes under mod N?** Nodes 6, 3, and 0 — why mod N moves too much data.
7. **In a fixed-partition scheme, what changes when a node is added, and what never changes?** Only the partition-to-node assignment; the partition count and key-to-partition mapping never change. The

old assignment serves requests during transfer.

8. **What is pre-splitting and why is it needed?** Configuring initial partitions on an empty database (HBase, MongoDB); otherwise dynamic partitioning starts with one partition on one node. For key ranges you must know the key distribution.
9. **How do Cassandra and Riak learn about partition assignment changes, compared with HBase?** Gossip among nodes, with any node forwarding requests; HBase (like SolrCloud and Kafka) uses ZooKeeper.
10. **Scenario: an overloaded but alive node triggers automatic rebalancing. What can happen?** Others think it is dead and move its load, adding load to it, the other nodes and the network — a possible cascading failure. A human in the loop reduces this risk.